Computer Vision · Week 2

Beginner study guide: plain-language notes + worked lab notebooks with outputs

Part 1: notes (01 features · 02 matching · 03 tracking · 04 kNN/SVM · 05 Bag of Visual Words · 06 face detection · 07 pre-CNN foundations · 08 CNN basics)
Part 2: the course's lab notebooks (Days 8–15), fixed, explained and run

Plus the cheat sheet. Runnable notebooks are in the **labs/** folder next to this PDF.

01 · Feature detection: Harris, SIFT, ORB (+ panorama stitching)

Key Concepts covers: What features represent · Why features matter · Harris · SIFT · ORB · Detect-keypoints exercise · Panorama mini project · What are keypoints?

Lab notebook: `labs/day08_feature_detection_panorama.ipynb` (course lab `CV-Week2-Lecture1.ipynb`, Day 8)

> Week 1's Key Concepts already introduced corners, SIFT and optical flow in one paragraph each (`../week-1/notes/key_concepts.md`). Week 2 is where you actually **use** them.

---

1. What is a "feature"?

> A **feature** is a **distinctive, informative pattern** in an image that can be **detected** (found), **described** (turned into numbers) and **compared** (matched with another image).

Think of how you'd describe a building to a friend so they can find it. You wouldn't say "it has grey pixels". You'd say "the corner with the red door, the round window above it". Those are **features**: small, unusual, recognisable spots.

Kind	What it is	Example detector
Corners	intensity changes in **two** directions	Harris, Shi–Tomasi
Edges	boundaries between regions of different brightness or texture	Canny (Week 1)
Blobs / keypoints	regions that stand out from their surroundings	SIFT, ORB
Descriptors	a **vector of numbers** describing the neighbourhood around a feature	SIFT (128 floats), ORB (256 bits)

Why features matter (the three magic properties)

1. **Uniqueness:** distinctive enough to be matched across different images. A plain blue sky is not unique, but a window corner is.
2. **Repeatability:** the same feature is found again even if the image changes (scale, rotation, lighting).
3. **Compact representation:** instead of comparing millions of pixels, compare a few hundred features.

---

2. Keypoint vs descriptor (exam favourite: "detection vs description")

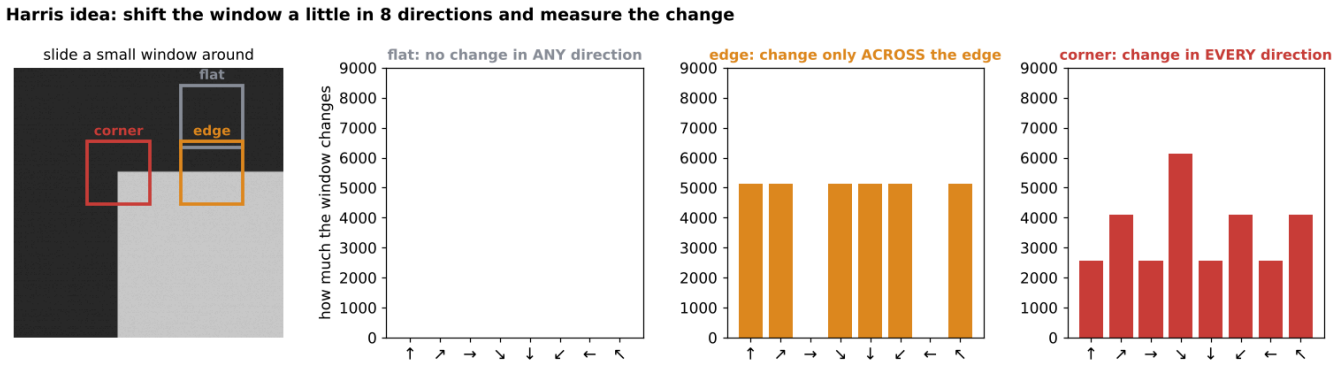
	Keypoint (detection)	Descriptor (description)
Answers	**where** is something interesting?	**what does it look like** there?
Is	a location + extra info	a vector of numbers
In OpenCV	`cv2.KeyPoint`: `.pt` (x, y), `.size` (scale), `.angle` (orientation), `.response` (strength)	a row of the `des` array

```
kp, des = sift.detectAndCompute(gray, None)   # detect AND describe in one call
print(len(kp), des.shape)                      # e.g. 1200 keypoints → des is (1200, 128)
print(kp[0].pt, kp[0].size, kp[0].angle, kp[0].response)
```

Analogy: a keypoint is a **pin on a map** (the location). A descriptor is the **written description** of that spot ("red door, brick wall, step on the left") that lets someone recognise it in another photo.

---

3. Harris corner detection



Concept: put a small window on the image and imagine sliding it a little in every direction. - **Flat region:** nothing changes, whichever way you slide. - **Edge:** changes only when you slide **across** the edge, not along it. - **Corner:** changes in **every** direction. That's what Harris looks for.

Method: it uses the image **gradients** (Week 1's Sobel) inside the window to compute a **corner response score R** for every pixel (the course says "corner response"). Big positive R means corner, negative R means edge, and R near 0 means flat.

```
gray = np.float32(gray)                          # cornerHarris needs float input
dst = cv2.cornerHarris(gray, 2, 3, 0.04)
#                            |  |  └─ k: sensitivity constant (0.04–0.06 typical)
#                            |  └──── ksize: Sobel kernel size for gradients
#                            └─────── blockSize: neighbourhood (window) size
```

```
img[dst > 0.01 * dst.max()] = [0, 0, 255]     # mark pixels whose score is > 1% of
the best one: red
```

Pros	Cons
simple and effective	**not scale-invariant**: a corner seen from far away can look like a smooth curve when zoomed in
good for rigid structures (buildings, checkerboards)	**not rotation-invariant** as a *descriptor* (it doesn't describe anything)
fast	sensitive to image transformations; only gives *locations*, no descriptor

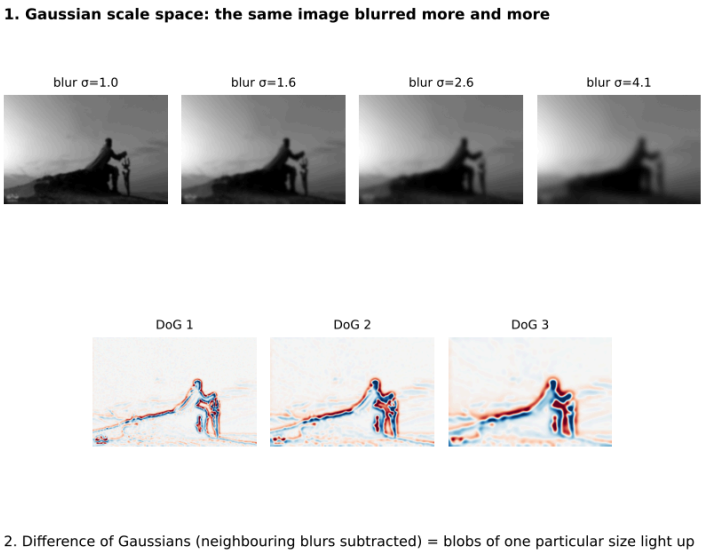
> Strictly speaking, the Harris *response* itself is fairly rotation-tolerant (a rotated corner is still a corner), but Harris has **no scale handling and no descriptor**, so it can't match features across zoom or rotation. The course's line "not invariant to scale or rotation" refers to using it for matching. The Day 8 lab measures this.

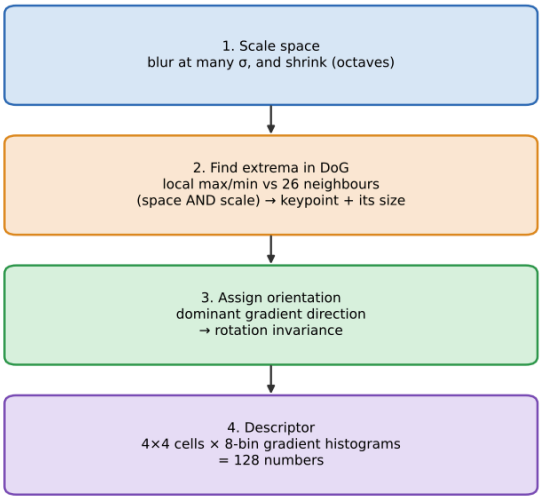
Lab outcome: red dots on the corners of the sample image.

---

4. SIFT: Scale-Invariant Feature Transform

Concept: detect **and describe** keypoints that survive **zoom, rotation and lighting changes**. It was invented by David Lowe (1999/2004).





The 4-step pipeline (learn it in order): 1. **Build a Gaussian scale space.** Blur the image more and more, and shrink it into "octaves". This lets SIFT look for features of **every size**. 2. **Detect extrema in the Difference of Gaussians (DoG).** Subtract neighbouring blur levels. A point is a keypoint if it is a local max or min compared with its **26 neighbours** (8 in the same level, 9 above, 9 below). It is found **at its own scale**, so the keypoint knows its **size**. That gives **scale invariance**. 3. **Assign an orientation.** Find the dominant gradient direction around the keypoint. Descriptors are measured **relative to it**, which gives **rotation invariance**. 4. **Create the descriptor.** Take a 16×16 area around the keypoint, split it into 4×4 cells, and make an 8-bin gradient-direction histogram in each: 4 × 4 × 8 = **128 numbers**. Normalising the vector gives some **illumination** robustness.

```
sift = cv2.SIFT_create()                          # free in OpenCV ≥ 4.4 (the patent
expired in 2020)
kp, des = sift.detectAndCompute(gray, None)
out = cv2.drawKeypoints(img, kp, None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
#                                                  └─ draws a circle sized by scale + a
line showing orientation
```

Pros	Cons
very robust: scale, rotation, illumination	**computationally heavy** (slow)
widely used in research and industry	**historically patented**, now free in OpenCV ≥ 4.4
rich 128-D descriptor → reliable matching	128 floats per keypoint = more memory

Lab outcome: "rich" keypoints drawn as circles (size = scale) with a line (orientation).

---

5. ORB: Oriented FAST and Rotated BRIEF

Concept: a **fast, free** alternative to SIFT, built from two older parts: - **FAST** (detector): looks at a ring of 16 pixels around a candidate. If enough of them are all much brighter or all much darker than the centre, it's a corner. Very quick. - **BRIEF** (descriptor): compares the brightness of **256 pairs** of pixels around the keypoint ("is A brighter than B?"). That gives a **256-bit binary string**. - ORB **adds orientation** (so it is rotation invariant), uses an **image pyramid** (some scale tolerance), and ranks corners by Harris score.

```
orb = cv2.ORB_create()                            # default nfeatures = 500 keypoints
kp_orb, des_orb = orb.detectAndCompute(gray, None)
print(des_orb.shape, des_orb.dtype)              # (500, 32) uint8 → 32 bytes = 256
bits
```

Binary descriptors are compared with **Hamming distance** (how many bits differ). A computer does that with one XOR and a bit count, which is extremely fast.

Pros	Cons
free, **fast**, efficient (real-time, phones, robots)	**less robust than SIFT for large scale changes**
rotation invariant	binary descriptor is less discriminative than SIFT's 128 floats

Harris vs SIFT vs ORB (the exercise's learning point)

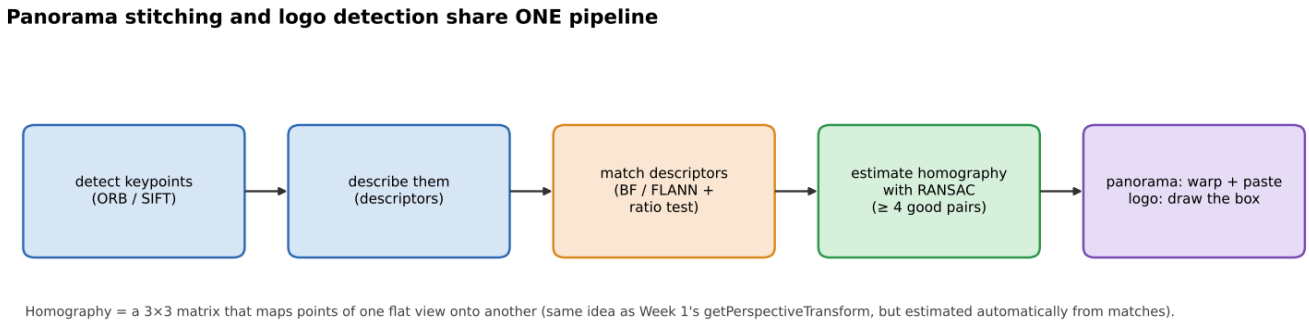
	Harris	SIFT	ORB
Finds	corners	blobs at any scale	FAST corners
Descriptor	**none**	128 floats	256 bits
Scale invariant	❌	✅	partly (pyramid)

	Harris	SIFT	ORB
Rotation invariant	❌ (for matching)	✅	✅
Speed	fast	slow	**very fast**
Course summary	finds corners but misses scale/rotation changes	finds **fewer but more reliable** keypoints	finds **many fast** keypoints, good for real time

> **What the Day 8 lab actually measured** (rotating and rescaling `building.jpg` and `graf1.png` by known amounts): - **SIFT matches** stayed **92–99% correct** at every rotation and 80–98% from 0.5× to 3× zoom. - **ORB** stayed 69–90% correct under rotation but dropped to about 50% at 3× zoom. - **Harris + a raw-patch descriptor** fell to ≤ 3% correct from 30° rotation onwards.
>
> Two honest corrections to the slogans: - Harris *detection* is fairly stable under rotation (64–98% of corners reappear). It is the matching without an orientation that fails. Under zoom the detection itself fails too. - With default settings SIFT found **more** keypoints than ORB (4,566 vs 500 on `building.jpg`), simply because `ORB_create()` stops at `nfeatures=500`. "SIFT finds fewer" is only true when ORB is given a higher limit. SIFT was 5–36× slower.
>
> For the viva, give the course's answer, and mention the measurement if asked to explain.

---

6. Mini project: panorama stitching



Workflow: 1. **Detect features** (ORB) in both overlapping photos. 2. **Match features** between them (Brute-Force matcher with Hamming distance; see chapter 02). 3. **Estimate a homography** with **RANSAC**. 4. **Warp** one image into the other's frame and **stitch** them together.

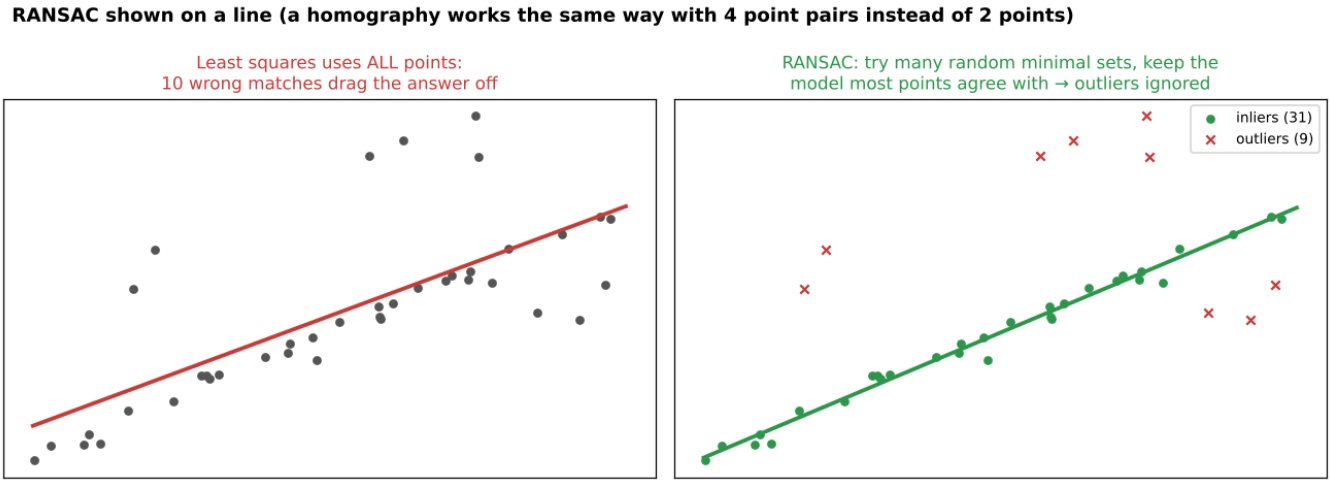
Homography

> A **homography** is a 3×3 matrix that maps points on one image plane to another. It is the same "perspective transform" as Week 1 Day 2 (`getPerspectiveTransform`), but here it is **estimated automatically from matched features** instead of 4 hand-picked corners.

It is exactly right when the camera **rotates without moving** (a panorama taken from one spot), or when the scene is **flat** (a poster, a logo, a document).

RANSAC (RANdom SAmple Consensus)

Some matches are always **wrong** (outliers). Normal fitting uses every point, so a few bad matches ruin the answer. RANSAC instead: 1. picks a **random minimal set** (4 matches for a homography); 2. computes the homography from just those; 3. counts how many **other** matches agree with it (within a threshold, e.g. 5 pixels, which are the **inliers**); 4. repeats many times and keeps the homography with the **most inliers**.




```
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
#                                                  └─ max reprojection error
(pixels) to count as an inlier
print("inliers:", mask.sum(), "of", len(mask))
result = cv2.warpPerspective(img1, M, (w1 + w2, h))   # warp img1 into img2's
coordinate frame
```

> ⚠️ **Watch out in the course code:** it warps `img1` (left) into `img2`'s (right) frame and then pastes `img2` at the left of the canvas. For a normal left/right pair, the left image lands at **negative x** (−215 px in the lab) and gets cut off. What survives is then covered by the pasted right image, so the "panorama" is just the right photo plus black. The course's own saved output shows exactly this. The lab notebook shows the bug and the fix: warp the **right** image into the **left** image's frame on a shifted canvas.

Real-world uses: phone panorama mode, satellite and drone image mosaics, Google Street View, medical slide scanning.

Check yourself

1. What three properties make a good feature?
2. What's the difference between a keypoint and a descriptor? What does a `cv2.KeyPoint` store?
3. How does Harris tell a corner from an edge?

4. List SIFT's 4 steps. Which step gives scale invariance and which gives rotation invariance?
5. Why is ORB so fast? How are ORB descriptors compared?
6. What does a homography do, and why do we need RANSAC to estimate it?

Answers

1. Uniqueness (distinctive), repeatability (found again after scale, rotation or lighting changes) and compactness (few numbers instead of all the pixels).
2. A keypoint is where something interesting is (x, y, size, angle, response). A descriptor is a vector describing what the area around it looks like.
3. Sliding a window over an edge changes it in one direction only. Over a corner, every direction changes. Harris turns this into a response score from the gradients.
4. Scale space → DoG extrema (scale invariance) → orientation assignment (rotation invariance) → 128-D descriptor.
5. FAST is a quick brightness test on a 16-pixel ring, and BRIEF is 256 simple brightness comparisons. Descriptors are binary and compared with Hamming distance (count of differing bits).
6. It maps points of one image plane onto another (warping, stitching, locating). Some matches are wrong, and RANSAC finds the homography most matches agree with, ignoring the outliers.

02 · Image matching: descriptors, FLANN, the ratio test (+ logo detector)

Key Concepts covers: Why image matching · Feature descriptors · FLANN · Match-logos exercise · Logo detector mini project · Tips

Lab notebook: `labs/day09_image_matching_logo_detector.ipynb` (course lab `CV-Week2-Lecture2.ipynb`, Day 9)

---

1. What is image matching?

> **Image matching** = finding **correspondences** between features in two or more images: "this point in image A is the same physical point as that point in image B".

Applications: - **logo detection** in adverts and media (brand monitoring: how often does our logo appear on TV?); - **object recognition** in robotics and surveillance; - **image retrieval** in search engines ("search by image"); - panoramas (chapter 01), augmented reality (stick a 3-D model on a recognised poster), visual SLAM for robots.

It always follows the same recipe: **detect → describe → match → verify geometry.**

---

2. Feature descriptors

> A **descriptor** encodes the neighbourhood around a keypoint into a **vector of numbers**. Two keypoints **match** if their descriptors are **similar** (a small distance).

Descriptor	Type	Size	Compare with	Notes
SIFT	float	128 numbers	**Euclidean (L2)** distance	robust to scale and rotation
ORB	binary	256 bits (32 bytes)	**Hamming** distance	fast, free
SURF	float	64 numbers	L2	faster than SIFT, but **patented** (not in standard OpenCV builds)

Distance in plain words: - **L2 (Euclidean):** the straight-line distance between two lists of numbers, $\sqrt{\Sigma(a_i - b_i)^2}$. - **Hamming:** the number of bits that differ. `10110` vs `10011` → 2.

---

3. Two ways to match

Brute-Force (BF) matcher

Compare **every** descriptor in image A with **every** descriptor in image B and take the closest. It is exact but slow when there are many features (N × M comparisons).

```
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)   # Hamming for ORB; use
cv2.NORM_L2 for SIFT
matches = sorted(bf.match(des1, des2), key=lambda m: m.distance)   # best (smallest
distance) first
```

`crossCheck=True` keeps a match only if **A's best is B and B's best is A** (they choose each other), which removes many wrong matches.

FLANN (Fast Library for Approximate Nearest Neighbours)

> A library optimised for **fast matching of high-dimensional descriptors**. It builds a smart **index** (like the index at the back of a book) so it doesn't have to compare everything.

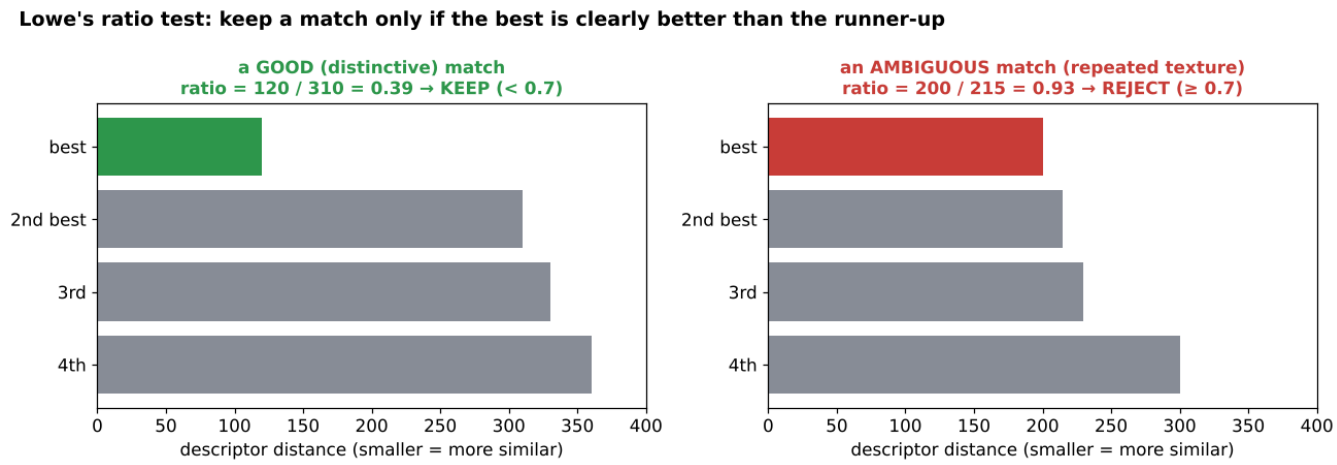
- For **SIFT** (floats) → a **KD-tree** index: `dict(algorithm=1, trees=5)`.
- For **ORB** (binary) → **LSH** (Locality Sensitive Hashing): `dict(algorithm=6, table_number=6, key_size=12, multi_probe_level=1)`. Hashing puts similar bit strings in the same "bucket".
- `search_params = dict(checks=50)` sets how many index entries to check. More checks are more accurate but slower.

Pros	Cons
efficient for large datasets	**approximate**: may miss the exact best match
scales well with many features	needs **parameter tuning**

---

4. Lowe's ratio test (the key technique)

```
matches = flann.knnMatch(des1, des2, k=2)    # for each feature, the BEST and SECOND-
BEST match
good = [m for m, n in matches if m.distance < 0.7 * n.distance]
```



Idea: a truly matching feature has **one** clearly closest partner. If the best and second-best candidates are almost equally close, the feature is **ambiguous** (repeated texture: windows on a building, letters in text), so throw it away.

- **Ratio 0.7–0.8** is typical (Lowe used 0.8, the course uses 0.7).
- **Lower ratio** → fewer matches, but more of them correct.
- **Higher ratio** → more matches, but more of them wrong.

  In the course code, `for m, n in matches` crashes if FLANN returns fewer than 2 neighbours for some feature, which can happen with LSH. The lab notebook guards against this.

---

5. Mini project: logo detector

Workflow: 1. **Detect features** in both the logo and the scene. 2. **Match descriptors** using FLANN. 3. **Apply Lowe's ratio test** to keep good matches. 4. **Estimate a homography** (logo → scene) with **RANSAC** (needs at least 4 pairs; the course requires > 10 to be safe). 5. **Draw a bounding box**: transform the logo's 4 corners with the homography and draw the resulting quadrilateral.

```
if len(good) > 10:
    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    h, w = logo.shape
    corners = np.float32([[0, 0], [0, h - 1], [w - 1, h - 1], [w - 1,
0]]).reshape(-1, 1, 2)
    box = cv2.perspectiveTransform(corners, M)                # where the logo's
corners land in the scene
    cv2.polylines(scene_color, [np.int32(box)], True, (0, 255, 0), 3)
else:
    print("Not enough matches - logo probably not in the scene")
```

> ⚠️ **What the Day 9 lab found:** - `len(good) > 10` alone is a weak "logo found" rule. At ratio 0.8 it wrongly reported the logo in **all 10** scenes that didn't contain it. - Counting **RANSAC inliers** and checking that the box is a sensible shape gave 0 false detections in 20 logo-free tests. - In scenes with the logo pasted at known positions, ORB found it 8 times out of 9 and SIFT 9 out of 9 (SIFT still worked at 0.35× zoom, where ORB failed).

The box is a **tilted quadrilateral**, not a rectangle, because the homography includes perspective: a logo seen at an angle.

Outcome: the logo localised in the scene, even when it is rotated, scaled, partly hidden or surrounded by clutter.

Evaluating matches (the exercise's discussion)

- **How many matches** were found?
- **Accuracy:** are the matches **clustered on the logo** or **scattered** across the scene? Scattered means false matches.
- **Inlier ratio** after RANSAC (`mask.sum() / len(mask)`) is a good confidence number.
- **ORB vs SIFT:** SIFT usually gives more correct matches when the logo is small or scaled. ORB is much faster.

Tips from the course

- Remember the difference between **detection and description** (keypoints vs descriptors).
- Matching quality depends on the **descriptor choice** and the **ratio-test threshold**.
- Real-world uses: **brand monitoring, AR, visual search**.
- Test with different logos and **cluttered scenes**. Plain, textureless logos (a single coloured circle) have almost no keypoints and **can't** be found this way.

Check yourself

1. Which distance measure do you use for SIFT descriptors, and which for ORB? Why?
2. What does `crossCheck=True` do?
3. Why is FLANN faster than brute force, and what's the trade-off?
4. Explain Lowe's ratio test in one sentence. What happens if you raise the ratio from 0.7 to 0.9?
5. Why does the logo detector need a homography and not just the matches?
6. Why can't this method find a plain, single-colour logo?

Answers

1. SIFT uses L2 (Euclidean), because it is float vectors. ORB uses Hamming, because it is binary strings, and counting differing bits is natural and very fast.
2. It keeps a match only if the two features are each other's best match (mutual).
3. It uses an index (KD-tree or LSH) to search only likely candidates. It is approximate, so it may miss the true best match, and it needs tuning.
4. Keep a match only if the best distance is clearly smaller than the second best (best $< 0.7 \times$ second). At 0.9 you get more matches but many more wrong ones.
5. Matches are just point pairs and some are wrong. The homography (with RANSAC) finds the one consistent geometric transformation, rejects outliers and tells you where the logo's outline is.
6. A plain shape has no corners or texture, so it has no keypoints and nothing to describe or match.

03 · Object tracking: optical flow, KLT, dense flow

Key Concepts covers: Why object tracking · Optical flow (dense vs sparse) · KLT tracker · Track-a-ball exercise · Motion tracker mini project (Farneback) · Tips

Lab notebook: `labs/day10_object_tracking_optical_flow.ipynb` (course lab `CV-Week2-Lecture3.ipynb`, Day 10)

---

1. What is object tracking?

> **Object tracking** = following a moving target across **consecutive video frames**.

Uses: surveillance (following a person across CCTV), sports analytics (ball and player paths), autonomous driving (where is that car going?), human–computer interaction (hand and eye tracking).

Why it's hard: objects **change appearance** (turn, deform, get lit differently), get **occluded** (hidden behind something) and **move quickly** (big jumps between frames, motion blur).

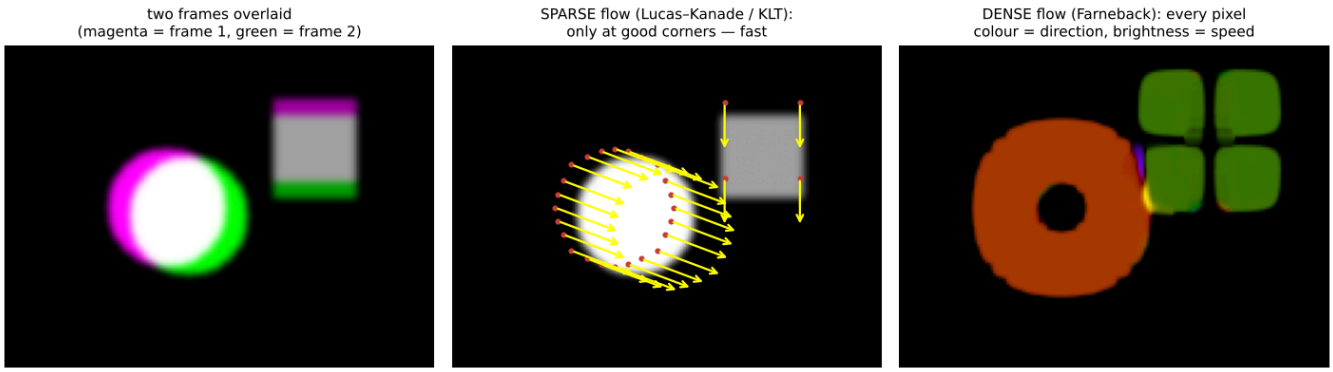
Detection vs tracking: detection finds objects in *one* frame from scratch. Tracking uses the *previous* frame's answer to find the object again, which is faster and keeps each object's **identity** (this is still car #3).

---

2. Optical flow

> **Optical flow** estimates how each pixel **moves** between two consecutive frames: a motion vector (dx, dy).

Key idea, "brightness constancy": a point keeps the same brightness as it moves, so if a bright spot was at (10, 20) and the same bright pattern is now at (13, 21), that point moved (+3, +1).



	Sparse optical flow	Dense optical flow
Computes motion for	only selected **keypoints** (good corners)	**every pixel**
Typical algorithm	**Lucas–Kanade** → KLT tracker	**Farneback**
Speed	**fast**, often used in real-time systems	slower
Gives	tracks of specific points	a full motion map, great for **visualisation**
OpenCV	`cv2.calcOpticalFlowPyrLK`	`cv2.calcOpticalFlowFarneback`

Limitations: sensitive to **large displacements** (fast motion), **lighting changes** (which break brightness constancy) and **occlusions** (the point disappears).

---

3. The KLT (Kanade–Lucas–Tomasi) tracker

> A widely used tracker based on **sparse** optical flow.

Two steps: 1. **Detect "good features to track"**, usually corners (the Shi–Tomasi corner detector, a refinement of Harris). You can only track points you can see moving: a flat wall has no texture to follow, and a point on an edge can slide along it (the "aperture problem"). 2. **Follow them with Lucas–Kanade optical flow**, frame by frame.

```
feature_params = dict(maxCorners=100,     # at most 100 points
                      qualityLevel=0.3,   # keep corners ≥ 30% as strong as the best one
                      minDistance=7,      # at least 7 px apart (spread them out)
                      blockSize=7)        # neighbourhood used to judge "corneriness"
p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)

lk_params = dict(winSize=(15, 15),        # search window around each point
                 maxLevel=2,              # pyramid levels: 0 = no pyramid; 2 = also search at 1/2 and 1/4 size
                 criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))
#                          stop after 10 iterations OR when the update is smaller than 0.03

p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_params)
good_new = p1[st == 1]                    # st = 1: this point was found again, st = 0: lost
```

Why the pyramid (`maxLevel`) matters: Lucas–Kanade assumes **small** motion, within the window. Shrinking the image makes a 40-pixel jump only 10 pixels at quarter size, so it can be found. Then the estimate is refined at full size. This is the fix for "**large motion jumps**".

Pros	Cons
efficient and relatively simple	struggles with **occlusion** (object hidden)
works well for **moderate motion and stable lighting**	struggles with **large motion jumps** (use pyramids and a bigger winSize)

Pros	Cons
	points get lost over time, so it **needs re-initialisation** (re-detect features every N frames)

> **What the Day 10 lab found with the course code:** - It **never re-detects** features. On a street video, only about 5 of its points per frame were still on moving things. Re-detecting every few frames (an improved tracker) raised that to about 48, and picked the ball up again after it passed behind a pillar. - `qualityLevel=0.3` returned only 56 corners, not the 100 that `maxCorners` allows. - On a synthetic ball, the course settings (with the pyramid) kept 100% of the ball's points up to 12 px/frame and 92% at 16 px/frame. Without the pyramid (`maxLevel=0`) the ball was lost at 12 px/frame. At 24 px/frame every setting failed.

Exercise: track a moving ball

- Run KLT on a ball video. The course's `ball.mp4` wasn't provided, so the lab generates one.
- **maxCorners:** more points means a sturdier track but more work.
- **winSize:** bigger windows cope with faster motion but blur detail and cost more. Too small and a fast ball is lost.

---

4. Mini project: motion tracker with dense optical flow (Farneback)

Workflow: 1. Compute dense optical flow with **Farneback's algorithm**. 2. Convert the flow vectors (dx, dy) into **magnitude** (speed) and **angle** (direction): `cv2.cartToPolar`. 3. Visualise with **colour coding**: put angle into **Hue**, speed into **Value (brightness)**, then convert **HSV → BGR**.

```
flow = cv2.calcOpticalFlowFarneback(prvs, next, None,
                                    0.5,   # pyr_scale: each pyramid level is half
the size
                                    3,     # levels: number of pyramid levels
                                    15,    # winsize: averaging window (bigger =
smoother, less detail)
                                    3,     # iterations at each level
                                    5,     # poly_n: neighbourhood for the polynomial
fit
                                    1.2,   # poly_sigma: Gaussian smoothing for that
fit
                                    0)     # flags
mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
hsv[..., 0] = ang * 180 / np.pi / 2                          # direction → hue
(OpenCV hue is 0–179)
hsv[..., 1] = 255                                            # full saturation
hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)   # speed →
brightness
bgr = cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)
```

How to read the picture: black means nothing moves. **Bright colour** means motion, and the **colour tells the direction** (e.g. red = moving right, cyan = moving left), the same HSV colour wheel as Week 1 Day 1.

Outcome: moving regions are highlighted, a clear visualisation of motion patterns.

Applications: - detecting **unusual movement** in surveillance (someone running against the crowd); - **player motion** in sports; - **traffic-flow** visualisation in driving research.

> The course code uses `cv2.imshow` in a `while` loop, which only works when run as a desktop script, not inside a notebook. The lab notebook shows frames with matplotlib instead and can write an output video.

---

5. Tips (from the course)

- **Sparse vs dense:** efficiency vs detail.
- KLT builds on optical flow by focusing on **good features to track**.
- Test with different videos: **fast motion, occlusion, multiple objects**.
- Real-world systems: **CCTV tracking, sports replay analysis, driver-assistance systems**.
- Week 1's **Kalman filter** (`../week-1/notes/key_concepts.md`) is the classic partner of a tracker: it **predicts** where the object should be next, which helps it survive short occlusions.

Check yourself

1. What assumption does optical flow rely on?
2. Sparse vs dense optical flow: which is faster? Which gives a motion map for every pixel? Name the OpenCV function for each.
3. What are the two steps of KLT? Why does it track corners rather than flat areas?
4. What do `winSize` and `maxLevel` control? How would you fix a tracker that loses a fast ball?
5. In the dense-flow visualisation, what do hue and brightness mean?
6. Give two situations where KLT fails and how to handle them.

Answers

1. Brightness constancy: a point keeps its brightness as it moves (and motion between frames is small).
2. Sparse (Lucas–Kanade, `calcOpticalFlowPyrLK`) is faster. Dense (Farneback, `calcOpticalFlowFarneback`) gives every pixel.
3. Detect good features to track (Shi–Tomasi corners), then follow them with Lucas–Kanade. Corners have texture in two directions, so their motion is unambiguous. Flat areas and edges are not.
4. `winSize` is the search window size. `maxLevel` is the number of pyramid levels, which lets it handle bigger motion. For a fast ball: increase `maxLevel` and/or `winSize` , and re-detect lost points.
5. Hue is the direction of motion. Brightness is the speed. Black means static.

6. Occlusion (the points vanish): re-detect features, or use prediction (Kalman). Large jumps: use pyramids and a bigger window. Lighting changes: brightness constancy breaks, so re-initialise.

04 · Machine learning in CV: kNN and SVM (+ digit recogniser)

Key Concepts covers: Why ML in computer vision · k-Nearest Neighbours · Support Vector Machines · Handwritten digit recogniser mini project

Lab notebook: `labs/day11_ml_knn_svm_digits.ipynb` (course lab `CV-Week2-Lecture4.ipynb`, Day 11)

---

1. Why machine learning in computer vision?

Week 1 and Days 8–10 used **hand-written rules**: "a corner is where the window changes in every direction", "red is hue 0–10". That works when **you** can describe the rule. But try writing rules that tell a handwritten **4** from a **9** for every person's handwriting. It's hopeless.

> **Machine learning (ML)** algorithms **learn patterns from examples (data)** and use them to classify or recognise new inputs. You provide labelled examples instead of rules.

Applications: handwritten digit recognition (**MNIST**: post codes, bank cheques), object classification in photos, medical image analysis.

The ML vocabulary you need

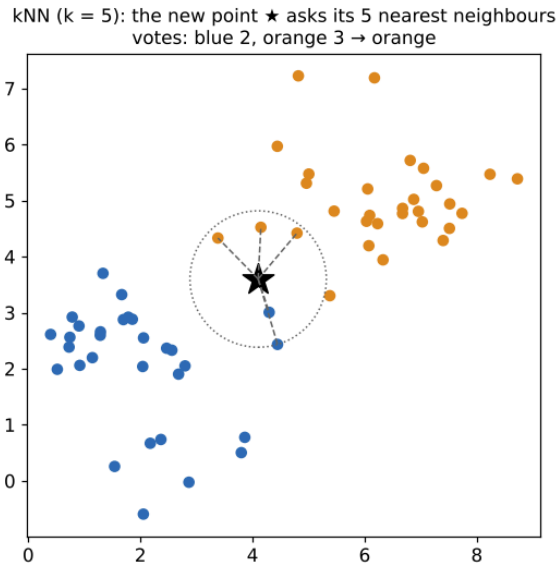
Word	Meaning	In the digits lab
Sample	one example	one 8×8 digit image
Features (X)	the numbers describing a sample	its 64 pixel values (flattened)
Label (y)	the correct answer	0–9
Training set	examples the model learns from	70% of the images
Test set	held-back examples used only to check it	the other 30%
Classifier	the model that predicts labels	kNN, SVM

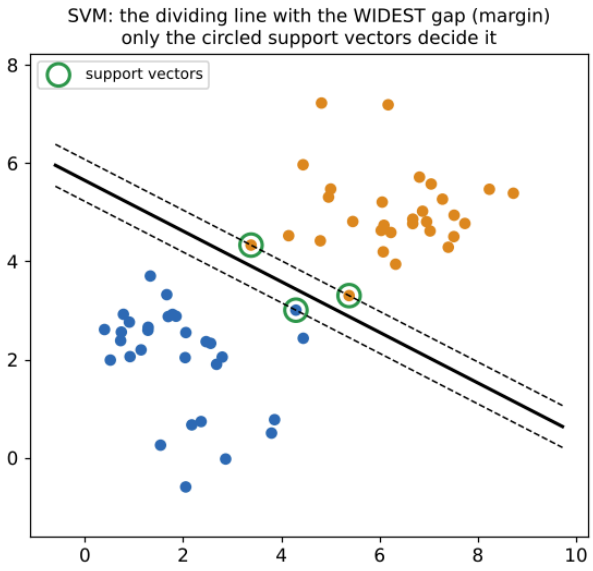
```
digits = datasets.load_digits()                          # 1,797 tiny 8×8 digits, pixel values 0–16
X = digits.images.reshape((len(digits.images), -1))   # 8×8 grid → 64 numbers per sample
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

> ⚠️ `load_digits` is **not** the real MNIST (which is 70,000 images of 28×28). It's a small 8×8 dataset that ships with scikit-learn, so nothing needs downloading. The Day 15 lab uses real MNIST.

---

2. k-Nearest Neighbours (kNN)





> **Classify a new sample by the majority label of its k closest training examples.** "Tell me who your neighbours are and I'll tell you who you are."

Steps: 1. Compute the **distance** (e.g. Euclidean) between the test sample and **every** training sample. 2. Pick the **k nearest**. 3. **Vote:** assign the most common label among them.

```
knn = KNeighborsClassifier(n_neighbors=5)   # k = 5
knn.fit(X_train, y_train)                   # "training" = just storing the data!
y_pred_knn = knn.predict(X_test)            # the real work (all the distances)
happens here
```

Pros	Cons
simple, intuitive	**slow at prediction for large datasets**: compares with *every* training sample
no training phase (it just memorises)	sensitive to **irrelevant features** and the **choice of distance metric**
works well for small datasets	needs features on the same scale; memory grows with the data

Choosing k: - **k = 1** follows every noisy point (overfits). - A **huge k** always votes for the most common class (underfits). - Use an **odd** k for two classes to avoid ties, and try several values on validation data.

---

3. Support Vector Machines (SVM)

> **Find the dividing line (in many dimensions, a hyperplane) that separates the classes with the maximum margin**, i.e. the widest possible "street" between them.

Steps (course version): 1. **Map** the data into a high-dimensional space (the **kernel** does this implicitly). 2. Find the separating **hyperplane** with the **largest margin**. 3. Classify new samples by **which side** of the hyperplane they fall on.

- **Support vectors** = the training points closest to the boundary (circled in the picture). **Only they** determine the line, and the rest could be deleted without changing it.
- **Why the widest margin?** A boundary that sits far from both classes is less likely to misclassify slightly different new samples, so it generalises better.

Kernels: what if no straight line separates the classes?

- **Linear kernel:** a flat hyperplane in the original space.
- **RBF (Gaussian) kernel**, the default in scikit-learn: implicitly maps the data into a very high-dimensional space where a flat cut becomes a **curved** boundary in the original space. This is the "kernel trick": it is computed without ever building that space.

```
svm = SVC(kernel='linear')        # course code also passes gamma='scale', which a
LINEAR kernel ignores
svm = SVC(kernel='rbf', C=1.0, gamma='scale')   # C: how hard to punish mistakes;
gamma: how "curvy" the boundary is
svm.fit(X_train, y_train)
```

Pros	Cons
effective in high-dimensional spaces (64 pixels, 128-D SIFT, BoVW histograms)	**computationally expensive** to train on large datasets
robust to overfitting when tuned (margin maximisation)	needs a careful choice of **kernel and parameters** (C, gamma)
fast prediction (only support vectors matter)	multi-class is done by combining many 2-class SVMs

kNN vs SVM

	kNN	SVM
Training	none (store the data)	solve an optimisation (slow on big data)
Prediction	slow (distances to everything)	fast
Memory	keeps all training data	keeps only the support vectors
Idea	majority vote of neighbours	best dividing line
Course one-liner	"majority vote of neighbours"	"find the best dividing line"

What the Day 11 lab measured

- **On the 8×8 digits (30% test split):** kNN (k = 5) **99.26%**, linear SVM 97.96%, RBF SVM 98.70%.
- **With 5-fold cross-validation:** the best k was 1 (98.65%), and a tuned RBF SVM reached 99.21%.
- **On real MNIST (20,000 training images):** kNN 94.0%, RBF SVM **95.9%**, linear SVM 88.2%.
- **Prediction speed:** kNN does get slower as the training set grows, which confirms the textbook. But an **RBF SVM was even slower** at prediction here (about 5 s vs 1.3–1.7 s), because it keeps thousands of support vectors. Only the *linear* SVM is really fast.

- **Your own drawn digits:** 0/30 correct when fed in directly, 21/30 after preprocessing them to look like the training data.

4. Reading `classification_report`

```
              precision    recall  f1-score   support
           3       0.98      1.00      0.99        54
```

Column	Plain meaning
precision	of everything predicted as "3", how many really were 3? (few false alarms)
recall	of all the real 3s, how many did we find? (few misses)
f1-score	one number balancing precision and recall (their harmonic mean)
support	how many real 3s are in the test set
accuracy	overall fraction correct

A **confusion matrix** shows *which* digits get mixed up (e.g. 8 ↔ 1 or 9 ↔ 3). It is more useful than a single accuracy number.

---

5. Mini project: handwritten digit recogniser

Workflow: 1. Train an SVM on the digits. 2. Write a function that predicts the label of a new digit image. 3. Test it with sample digits.

```
def recognize_digit(image):            # image must be 8×8 with values 0–16, like the training data
    return svm.predict(image.reshape(1, -1))[0]
```

> ⚠️ **Two traps the lab notebook demonstrates:** 1. The course tests on `digits.images[10]`, which may well be in the **training** set. Testing on data the model has seen proves nothing. 2. A **new** digit (your own drawing) must be **preprocessed to look like the training data**: 8×8, white-on-black, centred, values 0–16. Otherwise the predictions are garbage.

Real-world uses (course tips): postal-code recognition, bank-cheque processing, OCR systems.

Check yourself

1. What does "training" mean for kNN? Why is kNN slow at prediction time?
2. What happens with k = 1 vs a very large k?
3. What is the margin in an SVM, and what are support vectors?
4. When do you need a non-linear (RBF) kernel?
5. Precision vs recall: explain each with the digit "3".
6. Why must you test on data the model hasn't seen?

Answers

1. It just stores the training data. Prediction computes the distance to every stored sample.
2. k = 1 is noisy and overfits. A huge k is too smooth: it votes for the majority class and underfits.
3. The margin is the gap between the boundary and the nearest points of each class (SVM maximises it). Support vectors are those nearest points, and they alone define the boundary.
4. When the classes can't be separated by a straight line or flat plane in the original feature space.
5. Precision: of the predicted 3s, the fraction that were really 3. Recall: of the real 3s, the fraction found.
6. Accuracy on training data measures memorisation. Only unseen data shows how well it generalises to new inputs.

05 · Bag of Visual Words (BoVW) (+ texture classifier)

Key Concepts covers: Why BoVW · Feature clustering · Histogram representation · BoVW-for-textures exercise · Texture classifier mini project · Tips

Lab notebook: `labs/day12_bag_of_visual_words_textures.ipynb` (course lab `CV-Week2-Lecture5.ipynb`, Day 12)

---

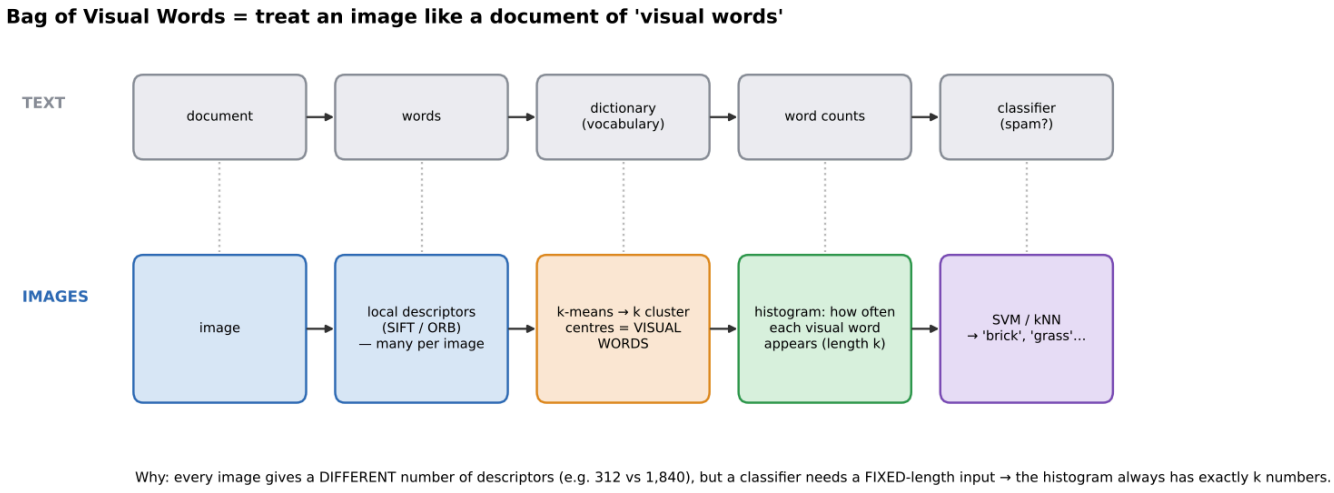
1. The problem BoVW solves

Chapters 01–02 give you **many descriptors per image**, e.g. 312 SIFT vectors for one image and 1,840 for another. Chapter 04's classifiers (SVM, kNN) need **one fixed-length vector per image**. How do you turn a variable-sized **bag** of descriptors into one fixed-size vector?

> **BoVW** clusters local descriptors into a vocabulary of **visual words**, then represents each image as a **histogram** counting how often each visual word appears. That gives a **fixed-length** vector, ready for standard ML.

---

2. The analogy with text (the course's main tip)



A spam filter doesn't understand sentences. It counts words: an email with "winner" ×3, "free" ×5 and "click" ×4 is probably spam. Word order is ignored, hence a "**bag** of words".

Text	Images
word	**visual word** = a typical small patch pattern (a cluster of similar descriptors)
dictionary / vocabulary	the **k cluster centres** from k-means
document	**image**

Text	Images
word-frequency histogram	**visual-word histogram**
spam / not spam classifier	SVM / kNN → "brick", "grass", "fabric"

---

3. Step 1: feature clustering (building the vocabulary)

1. **Extract local features** from all **training** images (e.g. SIFT descriptors, 128 numbers each).
2. **Cluster** all those descriptors with **k-means** into k groups.
3. Each **cluster centre** is a **visual word**.

k-means in plain words: 1. Drop k random "centres" among the points. 2. Assign every point to its nearest centre. 3. Move each centre to the average of its points. 4. Repeat 2–3 until nothing changes.

It is **unsupervised** (no labels needed, see chapter 07).

Analogy (course): like grouping **synonyms** in language. "Big", "large" and "huge" become one word. Slightly different-looking corner patches become one visual word.

```
k = 20                                          # vocabulary size = number of visual words
kmeans = KMeans(n_clusters=k, random_state=42).fit(all_train_descriptors)
```

4. Step 2: histogram representation

1. **Assign** each descriptor of an image to its **nearest** cluster (visual word): `kmeans.predict(des)`.
2. **Count** how often each word occurs, giving a histogram of length k.
3. That histogram **is** the image's representation.

```
def build_histogram(des):
    words = kmeans.predict(des)                 # e.g. [3, 17, 3, 0, 9, ...]
    hist = np.bincount(words, minlength=k).astype(float)
    return hist / hist.sum()                    # normalise: images with more keypoints aren't "bigger"
```

Benefit: a **fixed-length** vector (k numbers) no matter how many features the image had. It can be fed to **SVM or kNN**.

---

5. Mini project: texture classifier

Workflow: 1. Extract features from texture images. 2. Cluster the descriptors into visual words. 3. Build a histogram for each image. 4. Train an SVM on these histograms.

Why textures suit BoVW: texture is about **which small patterns appear and how often** (bricks: many horizontal and vertical edge corners; grass: many thin random strokes), not about where they are, and BoVW deliberately throws away *where*.

Applications: material recognition, surface inspection (defects on fabric or metal), texture-based retrieval, recognising fabrics in fashion, natural textures in environmental monitoring. It is also used for **scene recognition** and **object categorisation**.

Choosing k (vocabulary size)

- **Too small** (e.g. 5): very different patterns get lumped together, so the histograms can't tell classes apart.
- **Too large** (e.g. 5,000 with little data): words become so specific that each image's histogram is sparse and noisy.
- The exercise's discussion: **changing k changes the vocabulary size and the accuracy**. Try several values. The lab notebook sweeps k and plots accuracy.

  ⚠️ **The course notebook as given can't work.** It uses **3 images, one per class**, so the train/test split leaves 2 histograms to train on and 1 to test, which is why it reports **0%** (and k-means prints overflow warnings). A classifier needs **many** examples per class. The lab notebook rebuilds it on a real texture dataset with a proper train/test split.

What the Day 12 lab measured on a real 7-class texture set (tested on unseen regions): - **Course settings (k = 20, linear SVM): 79%.** - **k = 100 with an RBF SVM: 97.5%.** - **Normalising histograms matters** when images differ in size: raw counts fell to 61.5% on bigger patches, while normalised histograms held 95.8%. - On completely new photos of the same materials it managed only 33%. BoVW learns the training textures' specific look, not "brick" in general.

Important rule: build the vocabulary from **training images only**. If test images help choose the words, the test isn't fair ("data leakage").

---

6. Where BoVW sits in history

BoVW (around 2003–2012) was the **state of the art for image classification before CNNs**. It bridges **feature extraction** (SIFT) and **machine-learning classification** (SVM). Its weakness is that it throws away **spatial layout**: a face with the mouth above the eyes gets the same histogram. "Spatial pyramid" versions patched this. CNNs (Day 15 and Week 3) learn **features and classifier together** and keep spatial structure, which is why they replaced BoVW.

Check yourself

1. Why can't you feed SIFT descriptors straight into an SVM?
2. What is a visual word, and how is the vocabulary built?
3. Give the text ↔ image analogy for four terms.
4. What happens if k is too small? Too large?
5. Why is BoVW good for textures but weak for objects where layout matters?

6. Why did the course notebook report 0% accuracy?

Answers

1. Each image has a different number of descriptors, but an SVM needs one fixed-length vector per image.
2. A cluster centre of similar local descriptors. Run k-means on the descriptors from the training images, and the k centres are the vocabulary.
3. Word ↔ visual word, dictionary ↔ cluster centres, document ↔ image, word-frequency histogram ↔ visual-word histogram.
4. Too small: different patterns are merged and classes look alike. Too large: words are too specific and the histograms are sparse and noisy (needs more data and is slower).
5. Textures are defined by which patterns occur and how often, which is exactly what the histogram captures. Objects also depend on where parts are, and BoVW discards position.
6. Only 3 images (one per class): there is nothing to learn from and only 1 test image. You need many images per class and a proper split.

06 · Face detection: Haar cascades and HOG (+ face detector app)

Key Concepts covers: Why face detection · Haar cascades · HOG · Webcam exercise · Face detector app mini project · Tips

Lab notebook: `labs/day13_face_detection_haar_hog.ipynb` (course lab `CV-Week2-Lecture6.ipynb`, Day 13)

---

1. Detection vs recognition

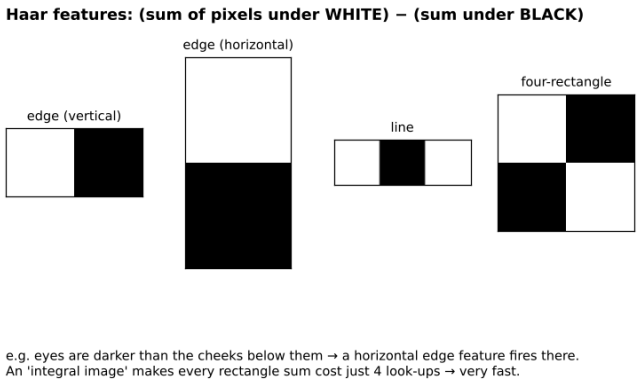
	Face detection	Face recognition
Question	**where** are the faces?	**whose** face is this?
Output	boxes	a name / identity
Order	first	second (needs the detected face)

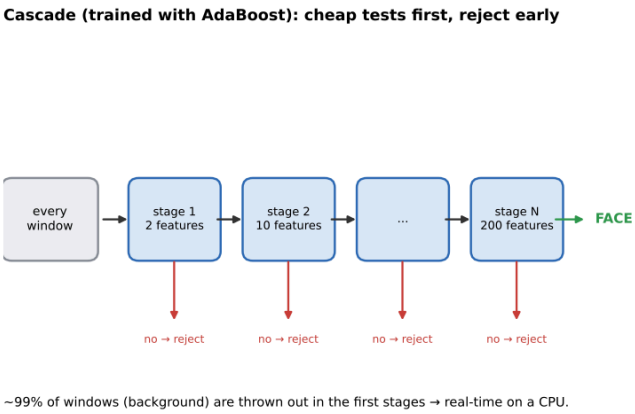
> **Face detection** = locating human faces in images or video. It is the **fundamental first step** before face recognition, emotion analysis or biometrics.

Applications: security and surveillance, human–computer interaction (face unlock), social-media filters and AR (they must find your face before adding dog ears), camera autofocus, counting people. The `../project/` notebooks use a modern deep-learning detector (YuNet) for the same job.

---

2. Haar cascades (Viola–Jones, 2001)





Haar features

> **Haar features** are simple **rectangular patterns** (like edge detectors): value = (sum of pixels under the white part) − (sum under the black part).

Faces have consistent light and dark patterns: the **eye region is darker than the cheeks** below it, and the **bridge of the nose is brighter than the eyes** on either side. A rectangle feature placed there gives a big value on faces and a small one elsewhere.

Speed trick, the integral image: pre-compute running sums once, and then **any** rectangle's sum costs just 4 look-ups, whatever its size.

Training with AdaBoost → a cascade

- A 24×24 window has **over 160,000** possible Haar features, and most are useless.
- **AdaBoost** selects the **most important** few and weights them. Each one alone is a weak classifier (a bit better than guessing), but together they are strong.
- They are arranged in a **cascade** of stages: stage 1 uses 2 cheap features, and later stages use more. A window is **rejected as soon as any stage says "no"**.
- Most windows in a photo are background and get thrown out in the first stages, so the expensive stages run rarely. That is why it is **real-time on a CPU**.

Using it in OpenCV

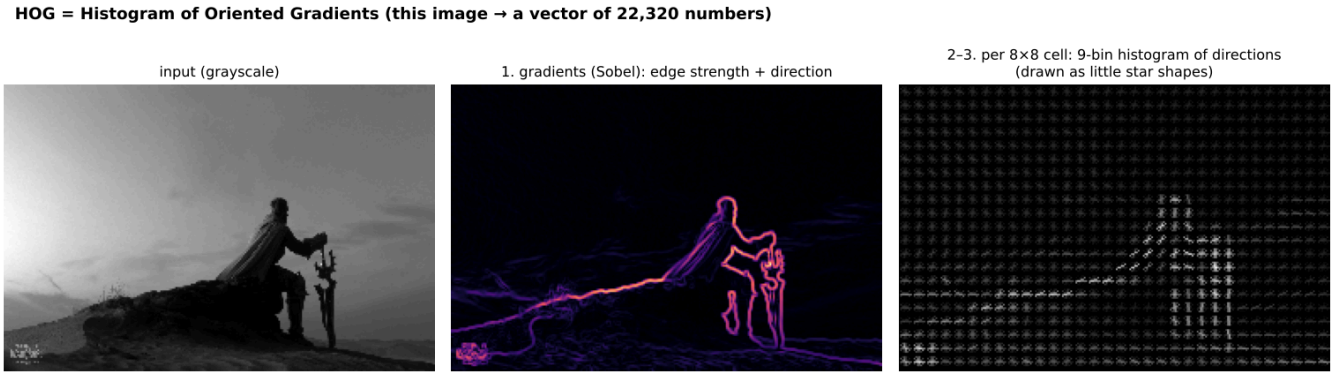
```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)          # Haar works on brightness only
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)
for (x, y, w, h) in faces:                            # each face = top-left corner + width + height
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

Parameter	Meaning	Turn it…
scaleFactor	how much the image is shrunk at each step of the search pyramid (1.3 = 30% each time), to find faces of different sizes	**lower** (1.05–1.1): finds more faces at more sizes, but slower and more false alarms. **Higher**: faster, may skip faces
minNeighbors	how many overlapping detections a spot needs before it counts as a face	**higher**: fewer false positives, may miss faces. **Lower**: catches more but adds junk
minSize	ignore detections smaller than this	raise it to skip tiny false positives

Pros	Cons
fast and lightweight	sensitive to **lighting, pose (turned or tilted heads) and occlusion** (hands, masks)
works well for **frontal faces** in controlled settings	**less robust in complex scenes**: false positives on textured backgrounds
ships with OpenCV, no download	one cascade per view (frontal and profile need separate files)

---

3. HOG: Histogram of Oriented Gradients (Dalal & Triggs, 2005)



> **HOG** describes shape by the **distribution of edge directions** in small regions of the image.

How it's computed: 1. Compute **gradients** (Sobel): edge strength and direction at each pixel. 2. Split the image into small **cells** (e.g. 8×8 pixels). In each, build a **histogram of gradient directions** (9 bins, 0–180°), weighted by strength. 3. Group cells into **blocks** (2×2 cells) and **normalise** each block. This is what makes HOG **robust to illumination**: brighter lighting scales all gradients equally, and normalising cancels that out. 4. Concatenate everything into one long feature vector → feed it to a **linear SVM** (chapter 04), sliding a window over the image.

Usage: the classical method for detecting **pedestrians** (OpenCV ships a HOG people detector) and **faces** (dlib's famous frontal-face detector is HOG + SVM).

Pros	Cons
robust to illumination changes (block normalisation)	**computationally heavier** than Haar cascades
captures **shape and edge information** effectively	may need more preprocessing (resizing, window scanning)
fewer false positives than Haar in cluttered scenes	still struggles with big pose changes and heavy occlusion

Haar vs HOG (the course's discussion question)

	Haar cascade	HOG + SVM
Features	rectangle brightness differences	histograms of edge directions
Speed	**faster** (integral image + early rejection)	slower
Robustness	sensitive to lighting and pose	**more robust**, especially to lighting
Typical use	quick frontal-face detection on weak hardware	pedestrians, faces in harder conditions
One-liner	**fast, lightweight**	**robust, heavier**

What the Day 13 lab measured (85 hand-checked faces in 9 photos): - **Course Haar settings (1.3, 5):** found 68. - **Haar with scaleFactor 1.1:** found 81, with 6 false alarms. - **dlib's HOG face detector:** found 79, with **0** false alarms. - **Rotation:** at 30° Haar found **4 of 83** faces and HOG **72 of 83** (dlib's detector includes filters trained on slightly tilted faces). -

Speed: HOG was **not** slower than Haar at similar accuracy (about 2 s vs 3–5 s on one core), against the textbook claim. Haar is only clearly faster with coarse settings like the course's 1.3.

For the viva, give the course's answer (Haar = fast, HOG = robust), and mention the measurement if asked to discuss.

> **OpenCV 5 note:** in OpenCV 5.0, `cv2.CascadeClassifier` and `HOGDescriptor` moved out of the main package. With OpenCV 4.x (the usual `pip install opencv-python`) the course code works as written. On 5.x, install `opencv-contrib-python`. The lab notebook explains this and downloads the cascade `.xml` file if it's missing.

Both are "classical". Modern systems use CNN detectors (e.g. YuNet, RetinaFace, YOLO-face), which handle pose, lighting and occlusion far better (Week 3 detection chapter).

---

4. Exercise: faces in a webcam feed

```
cap = cv2.VideoCapture(0)                    # 0 = the default webcam
while True:
    ret, frame = cap.read()                  # grab a frame
    if not ret:
        break                                # camera unplugged / no frame
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    for (x, y, w, h) in face_cascade.detectMultiScale(gray, 1.3, 5):
        cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
    cv2.imshow('Webcam Face Detection', frame)
    if cv2.waitKey(1) & 0xFF == ord('q'):  # press q to quit
        break
cap.release(); cv2.destroyAllWindows()
```

- **Observe:** it is real time, but watch the **false positives** (boxes on a patterned shirt) and the **missed detections** (turn your head, cover half your face, dim the lights).
- **Experiment:** change `scaleFactor` and `minNeighbors` and see the trade-off.
- This must run as a **desktop Python script** (`cv2.imshow` windows don't work inside Jupyter or Colab). The lab notebook keeps it behind a `RUN_WEBCAM = False` switch.

5. Mini project: face detector app

Workflow: load the Haar cascade → detect faces in a **static image or the webcam** → draw bounding boxes. **Outcome:** an interactive face-detection app, an entry-level prototype for **security systems, attendance monitoring or AR filters**. Compare `../project/project3_attendance_system.ipynb`, which goes further: it detects **and recognises** faces.

Check yourself

1. Detection vs recognition: which comes first, and what does each output?
2. What is a Haar feature? Why are Haar cascades so fast (two reasons)?
3. What does AdaBoost do in Viola–Jones?
4. What do `scaleFactor` and `minNeighbors` control? How do you reduce false positives?

5. List the HOG steps. Which step makes it robust to lighting?
6. Haar vs HOG: which is faster, which is more robust, and why?

Answers

1. Detection first (where the faces are, as boxes), then recognition (whose face, as an identity).
2. A difference of pixel sums between adjacent light and dark rectangles. Fast because of the integral image (any rectangle sum in 4 look-ups) and the cascade (most windows are rejected in the first cheap stages).
3. It picks the most useful few features out of more than 160,000 candidates, weights them into a strong classifier, and builds the stages of the cascade.
4. `scaleFactor` is the size step of the image pyramid (smaller steps find more sizes but are slower). `minNeighbors` is how many overlapping detections are needed to accept a face. To reduce false positives, raise `minNeighbors` (and/or `minSize`).
5. Gradients → per-cell orientation histograms → block normalisation → concatenate → SVM. Block normalisation gives the lighting robustness.
6. Haar is faster (simple sums, early rejection). HOG is more robust (normalised edge-direction histograms capture shape and are less sensitive to lighting).

07 · Pre-CNN module: maths, ML fundamentals, the artificial neuron

Key Concepts covers: Programming basics (Python + NumPy) · Mathematics foundations · Image basics · Machine-learning fundamentals (supervised vs unsupervised) · Artificial neuron · Neural network basics (ANN) · Bridge to CNNs

Lab notebook: `labs/day14_pre_cnn_foundations.ipynb`. There is no course lab file for this day. The notebook does the activities the Key Concepts sheet lists.

> This chapter is the "ladder" from Week 1 to deep learning. The course says to **build progressive complexity: pixels → features → ML → ANN → CNN.**

---

1. Programming basics (Python + NumPy)

You need variables, loops, functions, classes, NumPy arrays and Matplotlib plots. Week 1's `00_python_numpy_basics.md` covers exactly this.

Analogy (course): a NumPy array is like an **Excel sheet** where each cell holds a number, **with superpowers for maths**. You can add, multiply or average the whole sheet in one line.

---

2. Mathematics foundations

Linear algebra

- **Vectors and matrices:** lists and grids of numbers. An image is a matrix, and a flattened image or a descriptor is a vector.
- **Dot product → a similarity measure:** `a · b = a₁b₁ + a₂b₂ + …`. It is large when two vectors point the same way. `python np.dot([1, 2, 3], [1, 2, 3]) # 14 (similar → big) np.dot([1, 2, 3], [3, -2, 0]) # -1 (different → small or negative)`
- **Convolution = a sliding dot product.** At each position you take the dot product of the kernel with the patch under it (Week 1 Day 3). Every CNN layer is built on this one fact.

Calculus

- **Derivative** = the **slope** of a curve: how fast the output changes when you nudge the input.
- **Gradient** = the derivatives for every input together. It points in the **direction of steepest increase**.
- **Analogy (course):** the gradient is "**how steep the hill is**". Training a network = walking **downhill** on the error landscape by stepping **against** the gradient (gradient descent, see `../week-2-3/notes/00_deep_learning_basics.md`).
- You already met gradients in images: Sobel computes the brightness gradient, i.e. the edges.

Probability and statistics

- **Mean** (average) and **variance** (how spread out the values are), and **distributions** (bell curves etc.).
- **Why normalisation matters:** if one feature ranges 0–255 and another 0–1, the big one dominates distances (kNN) and gradients (neural nets). Rescaling to similar ranges, e.g. subtracting the mean and dividing by the standard deviation, makes learning fair and faster. That's what `transforms.Normalize((0.1307,), (0.3081,))` does in the Day 15 MNIST lab: 0.1307 and 0.3081 are the mean and standard deviation of MNIST's pixels.

---

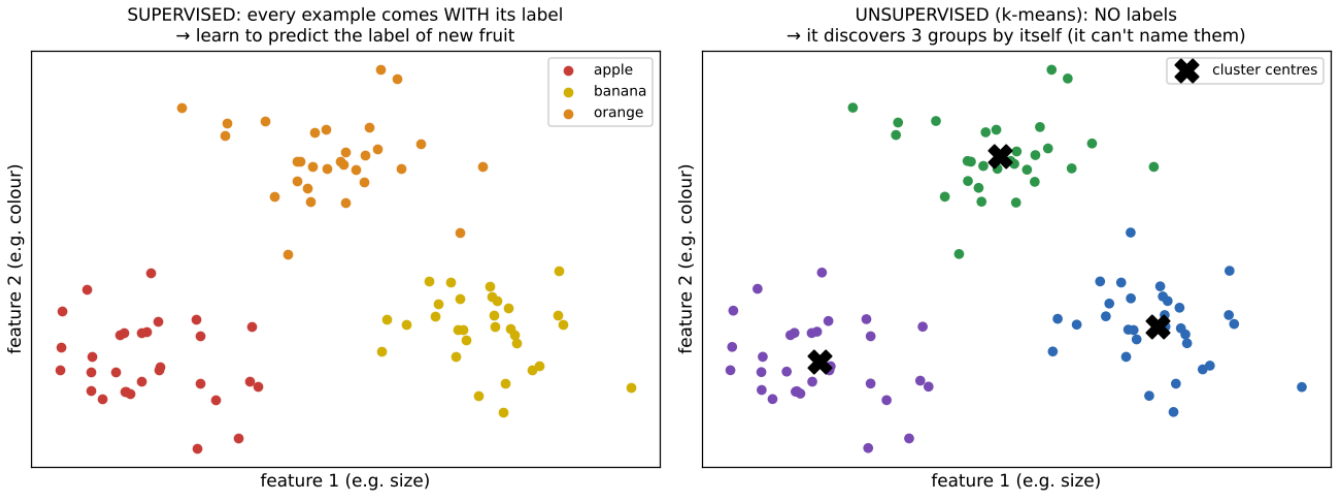
3. Image basics (recap)

- **Pixels and channels:** grayscale = **one channel**, RGB = **three** (red, green, blue).
- **Image as a matrix:** each pixel is a number (or 3 numbers).
- **Transformations:** scaling, rotation, cropping (Week 1 Day 2).
- **Analogy (course):** pixels are **Lego blocks**, and arranging them builds the image.
- **Activity:** rotate and flip an image in code, then discuss **how transformations affect recognition**. A digit "6" flipped upside down becomes a "9"! That's why data augmentation must be chosen carefully: flipping is fine for cats, but not for digits.

---

4. Machine-learning fundamentals

Machine learning lets computers **learn from data and improve without being explicitly programmed**: you give examples and it figures out the patterns.



Supervised learning: learning with guidance

- The data has **inputs (features) AND correct outputs (labels)**.
- Goal: learn the input → output relationship so it can **predict labels for new, unseen inputs**.
- **Fruit example (course):** features = colour, size, shape; labels = apple, banana, orange. After seeing many examples, it labels a new fruit.
- **Key points:** training data needs inputs **and** outputs. **Prediction** guesses the output for new inputs. **Evaluation** compares predictions with the true labels (accuracy).
- **Two task types:**
- **classification**: predict a **category** (spam / not spam, digit 0–9);

- **regression**: predict a **number** (a house price).
- **Basic models: kNN** = "majority vote of neighbours", **SVM** = "find the best dividing line" (chapter 04).

Unsupervised learning: learning without guidance

- Only **inputs, no labels**. The model must **discover structure by itself**.
- **Fruit example (course):** an unlabelled basket. It groups red round fruits together and long yellow ones together, without knowing the words "apple" or "banana".
- **Key techniques:**
- **clustering:** group similar points (**k-means**, as used in BoVW; hierarchical clustering);
- **dimensionality reduction:** simplify the data but keep the important patterns (**PCA**, **t-SNE**);
- **association rules:** "people who buy bread also buy butter".
- **Applications:** customer segmentation, organising photo collections, anomaly detection (fraud, unusual behaviour), recommendation systems.

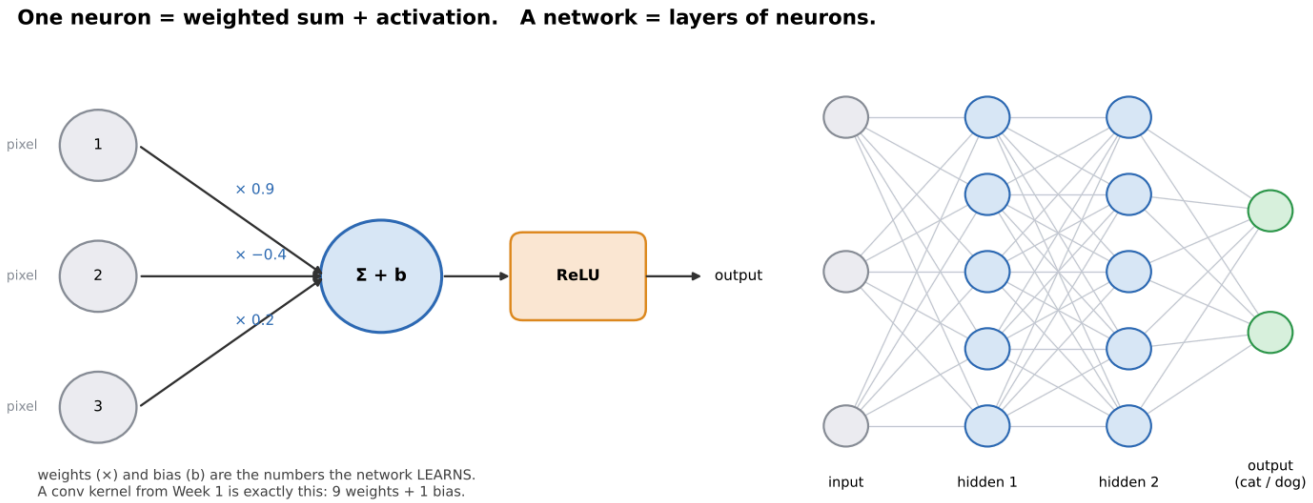
The comparison table (exam favourite)

Aspect	Supervised	Unsupervised
Definition	learns from **labelled** data (inputs + correct outputs)	learns from **unlabelled** data (inputs only)
Goal	**predict** outcomes for new data	**discover** hidden patterns, groups, structure
Data	labelled dataset (images with categories, emails marked spam)	raw unlabelled data (purchase records with no categories)
Techniques	classification, regression	clustering, dimensionality reduction, association
Output	a specific prediction (class label or number)	groups, clusters or a reduced representation
Analogy	a teacher giving questions **with answers**	exploring a new city **without a guide**
Examples	digit recognition (MNIST), medical diagnosis, sentiment analysis	customer segmentation, organising photos, anomaly detection

---

5. The artificial neuron

The basic computational unit of an **Artificial Neural Network (ANN)**, inspired by biological neurons but purely mathematical.



Structure: - **inputs** $x_1, x_2, \ldots, x_n$: features or signals (e.g. pixel values); - **weights** $w_1, \ldots, w_n$: how **important** each input is (learned); - **bias** b: a constant that shifts the result (learned); - **summation:** $z = \Sigma\, w_i x_i + b = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$; - **activation function** f(z): adds non-linearity and decides whether and how strongly the neuron "fires" (ReLU, sigmoid, tanh); - **output** f(z): passed on to the next layer.

Worked example: inputs x = (0.5, 0.8), weights w = (0.4, −0.2), bias b = 0.1. $z = 0.4 \cdot 0.5 + (-0.2) \cdot 0.8 + 0.1 = 0.2 - 0.16 + 0.1 =$ **0.14** → ReLU(0.14) = **0.14**, and sigmoid(0.14) ≈ **0.535**.

Analogy (course), a decision unit: - inputs = **evidence**; - weights = **importance** of each piece of evidence; - bias = **prior tendency** (how easily it says yes); - activation = the **final decision** (yes/no, strong/weak).

Digit example: inputs = pixel values of a handwritten "5". The weights learn the importance of certain strokes. The output is the probability that it's a "5".

---

6. Neural network basics (ANN)

- **Artificial neurons** = weighted sum + activation.
- **Layers:** input → hidden → output.
- **Forward pass:** data flows forward and predictions are made.
- **Backpropagation:** the error flows **backward** and the weights are updated (using the gradients from section 2).
- **Analogy (course):** each neuron is like a student giving a small opinion, and the network combines them into a final decision.
- **Activity:** build a small feed-forward network to classify digits and **watch the accuracy improve** epoch by epoch (done in the Day 14 notebook).

---

7. Bridge to CNNs

Problems with fully connected networks on images: 1. **Too many parameters:** a 224×224×3 image has 150,528 inputs. One layer of 1,000 neurons needs about **150 million** weights. 2. **They lose spatial structure:** the image is flattened, so pixels are treated independently and the network doesn't know which pixels were neighbours.

Motivation for CNNs: - **Convolutions preserve spatial locality:** each neuron looks at a small neighbourhood. - **Filters detect edges, textures and shapes**, reusing the same small kernel everywhere (few parameters). - **Analogy (course):** CNNs are like **detectives with magnifying glasses** (filters), spotting clues (features) in different parts of the image. - **Activity:** apply an edge-detector kernel **by hand** (a sliding dot product), then show that a CNN layer does the same thing but **learns** the kernel values.

Next chapter: CNN basics.

Check yourself

1. Why is the dot product a similarity measure? How does it relate to convolution?
2. What is a gradient, and how is it used in training?
3. Why normalise inputs?
4. Supervised vs unsupervised: definition, data, goal, one example each.
5. Classification vs regression?
6. Compute a neuron: x = (1, 2), w = (0.5, −1), b = 1, with ReLU.
7. Give two reasons fully connected networks are a poor fit for images.

Answers

1. It is large when vectors point the same way. Convolution is a dot product of the kernel with each image patch, slid across the image.
2. The direction and steepness of the error's increase. Training steps the weights against it (gradient descent).
3. So features on big scales don't dominate distances or gradients, and training is faster and more stable.
4. Supervised: labelled data, predict labels (digit recognition). Unsupervised: unlabelled data, find structure (customer segmentation or k-means clustering).
5. Classification predicts a category. Regression predicts a continuous number.
6. $z = 0.5 \cdot 1 + (-1) \cdot 2 + 1 = -0.5 \rightarrow \text{ReLU} = 0$.
7. A huge number of parameters, and the loss of spatial structure (neighbouring pixels are treated independently).

08 · CNN basics: convolution, pooling, activations, stride, padding, flatten (+ MNIST CNN)

Key Concepts covers: Why CNNs · Convolutions · Pooling · Activation functions · CNN in PyTorch for MNIST · Training & testing loop · Digit classifier mini project · What are feature maps / pooling / stride / padding / flatten

Lab notebook: `labs/day15_cnn_basics_mnist.ipynb` (course lab `CV-Week2-Lecture7.ipynb`, Day 15)

> Week 3 goes deeper into all of this (`../week-2-3/notes/00_deep_learning_basics.md` and `01_building_blocks.md`). This chapter is what the Day 15 lab and the Week 2 viva need.

---

1. Why CNNs?

> **Convolutional Neural Networks** are neural networks specialised for images. They **automatically learn a spatial hierarchy of features**: edges and textures in early layers, then shapes, then objects in deeper layers.

Applications: image classification, object detection, face recognition, medical imaging.

Compared with Day 11–13 methods: SIFT, HOG and Haar features were **designed by people**. A CNN **learns its own features** from the data, and learns the classifier at the same time.

---

2. The building blocks (Key Concepts definitions)

Convolution → feature maps

- Small **filters (kernels)** slide across the image to detect local patterns (Week 1 Day 3, but the kernel values are **learned**).
- A **feature map** is the 2-D grid of results: how strongly **one** filter responds at each position.
- A kernel (e.g. 3×3) slides over the image.
- At each position it computes a weighted sum (dot product) of the pixels under it.
- The result is stored in the feature map.
- `Conv2d(1, 32, 3)` = **32 different kernels**, giving **32 feature maps**.
- **Benefit:** far fewer parameters than fully connected layers (one small kernel reused everywhere), so CNNs are efficient and scalable.

Stride

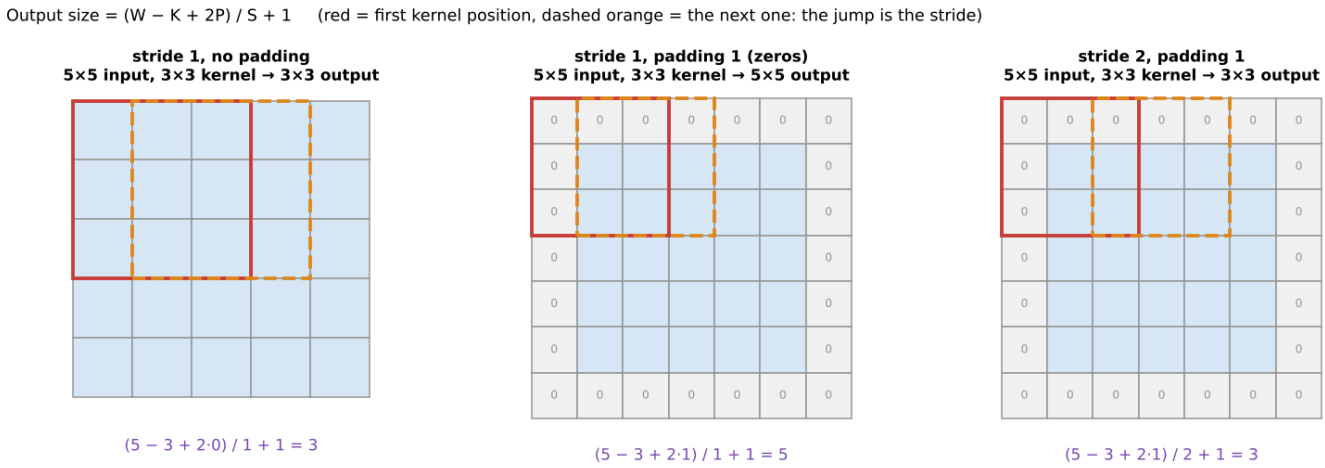
> The **step size** of the kernel as it slides.

Stride 1 moves one pixel at a time (overlapping windows). Stride 2 jumps two pixels, which roughly **halves** the output size. It controls how much the receptive fields overlap and the **output size**.

Padding

> Surrounding the input with extra pixels (usually **zeros**).

- **Prevents the image shrinking** after each convolution: a 3×3 kernel without padding loses 1 pixel on every side.
- Lets filters **cover the edge pixels** properly.
- Helps **control the output size**. `padding=1` with a 3×3 kernel keeps the size the same ("same" padding).



The output-size formula (learn it!)

Output = (W − K + 2P) / S + 1

where W = input size, K = kernel size, P = padding, S = stride.

Input	Kernel	Padding	Stride	Output
28	3	0	1	(28 − 3 + 0)/1 + 1 = **26**
28	3	1	1	(28 − 3 + 2)/1 + 1 = **28** (same size)
28	3	1	2	(28 − 3 + 2)/2 + 1 = **14** (rounded down)
32	5	0	1	28 (LeNet's first layer)

Pooling

> Pooling **summarises regions** of a feature map into single values (downsampling).

- **Max pooling:** take the **maximum** of each region, which captures the **strongest** response.
- **Average pooling:** take the **average**, which **smooths** the features.
- **Purpose:**
- **reduces computation** (smaller maps);
- **translation invariance**: robust to small shifts;

- **helps control overfitting** (fewer numbers downstream).
- No weights to learn. `max_pool2d(x, 2)` halves the height and width.

Activation functions

> Introduce **non-linearity** so the network can learn complex patterns. Without them, stacked layers collapse into one linear model.

Function	Output range	Notes
ReLU max(0, x)	0 → ∞	**fast, effective, the most widely used**
Sigmoid $1/(1 + e^{-x})$	0 → 1	useful for **probabilities** (e.g. yes/no outputs)
Tanh	−1 → 1	zero-centred, but less common today (vanishing gradients)

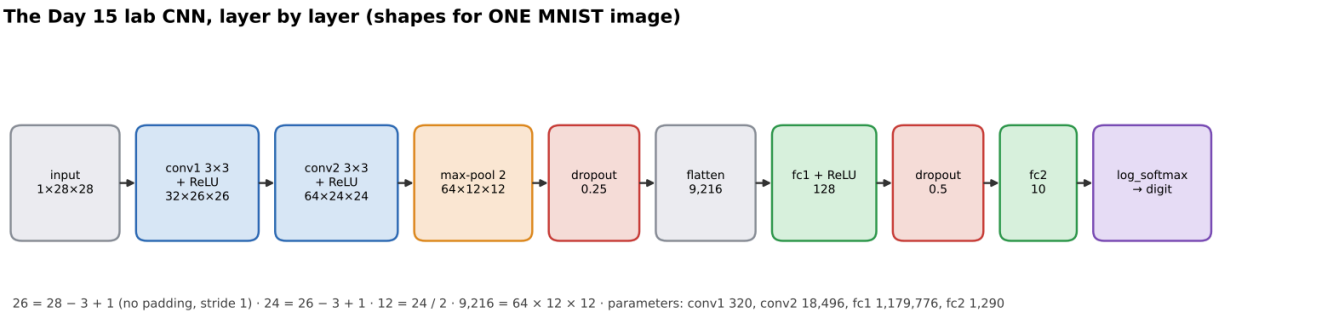
Flatten

> Reshapes the 3-D output of the conv and pool layers (channels × height × width) into **one long vector**.

Fully connected layers expect 1-D input, so **flatten bridges** spatial feature maps → classification layers.

---

3. The Day 15 lab CNN, shape by shape




```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)      # 1 input channel (grey) → 32 maps,
3×3 kernel, stride 1
        self.conv2 = nn.Conv2d(32, 64, 3, 1)     # 32 → 64 maps
        self.dropout1 = nn.Dropout(0.25)         # drop 25% of values (training only)
        self.dropout2 = nn.Dropout(0.5)          # drop 50%
        self.fc1 = nn.Linear(9216, 128)          # 9216 = 64 × 12 × 12 (see below)
        self.fc2 = nn.Linear(128, 10)            # 10 digits

    def forward(self, x):                        # x: (batch, 1, 28, 28)
        x = F.relu(self.conv1(x))                # → (batch, 32, 26, 26)   26 = 28 −
3 + 1
        x = F.relu(self.conv2(x))                # → (batch, 64, 24, 24)   24 = 26 −
3 + 1
```

```
        x = F.max_pool2d(x, 2)                    # → (batch, 64, 12, 12)   halve
        x = self.dropout1(x)
        x = torch.flatten(x, 1)                   # → (batch, 9216)         64 × 12 ×
12
        x = F.relu(self.fc1(x))                   # → (batch, 128)
        x = self.dropout2(x)
        x = self.fc2(x)                           # → (batch, 10)  one score per digit
        return F.log_softmax(x, dim=1)            # → log-probabilities
```

The famous viva question, "where does 9216 come from?": 28 → conv 3×3 → 26 → conv 3×3 → 24 → max-pool 2 → 12. There are 64 channels, so **64 × 12 × 12 = 9,216**. If you change the input size or a layer, this number changes and you get a "shape mismatch" error.

Parameter count: conv1 = 32·(1·3·3) + 32 = **320**, conv2 = 64·(32·3·3) + 64 = **18,496**, fc1 = 9216·128 + 128 = **1,179,776**, fc2 = 128·10 + 10 = **1,290**. The single dense layer fc1 holds about **98%** of all the weights. That is the "too many parameters" problem of dense layers, seen in real numbers.

---

4. Data preparation

```
transform = transforms.Compose([
    transforms.ToTensor(),                        # PIL image 0–255 → tensor 0.0–
1.0, shape (1, 28, 28)
    transforms.Normalize((0.1307,), (0.3081,))    # subtract MNIST's mean, divide by
its std
])
train_dataset = datasets.MNIST('./data', train=True, download=True,
transform=transform)   # 60,000 images
test_dataset  = datasets.MNIST('./data', train=False, transform=transform)
# 10,000 images
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)     # 64 images
per step, shuffled each epoch
```

---

5. Training and testing loop

Training: 1. **Forward pass** → predictions. 2. **Loss function: negative log-likelihood** (`F.nll_loss`). Combined with `log_softmax` in the model, this is exactly **cross-entropy**: a small loss when the correct digit gets a high probability. 3. **Backpropagation** (`loss.backward()`) → **update the weights** with the **Adam** optimiser (`optimizer.step()` , lr = 0.001). Adam is gradient descent with an automatically adapted step size for each weight.

```
model.train()                                     # dropout ON
for data, target in train_loader:
    optimizer.zero_grad()                        # clear last step's gradients (they
accumulate otherwise!)
    output = model(data)
    loss = F.nll_loss(output, target)
    loss.backward()
    optimizer.step()
```

Testing: evaluate on **unseen** data and report accuracy and loss.

```
model.eval()                                    # dropout OFF
with torch.no_grad():                           # no gradients needed → faster, less
memory
    pred = model(data).argmax(dim=1)            # the digit with the highest score
```

Outcome: accuracy improves with each epoch. The course's own run reached **98.79%** test accuracy after 2 epochs. Our lab notebook ran 1 epoch on a shared CPU and got **98.56%** (144 mistakes out of 10,000) in about 5 minutes. Setting `EPOCHS = 2` restores the course setting.

---

6. Mini project: digit classifier

1. Train the CNN on MNIST.
2. Test it with **custom digit images or drawings**.
3. Evaluate the predictions.

> ⚠️ Your own drawing must look like MNIST: **white digit on a black background, centred, 28×28, normalised** the same way. A black-on-white phone photo fed in directly gives nonsense. In the lab notebook, 30 drawn digits scored **2/30** fed in directly and **29/30** after full preprocessing.

Applications: OCR, postal-code recognition, bank-cheque processing, digit-recognition apps, face unlock, medical image classifiers.

Data flow to remember (course tip)

Input → Convolution → Pooling → Flatten → Fully connected → Output

---

Check yourself

1. What is a feature map? How many does `Conv2d(1, 32, 3)` produce?
2. Define stride and padding. What is the output size for input 32, kernel 3, padding 1, stride 2?
3. Max vs average pooling, and three purposes of pooling?
4. Why do we need activation functions? When would you use sigmoid?
5. What does flatten do and why is it needed?
6. Explain 9216 in the lab CNN.
7. What do `model.train()`, `model.eval()`, `optimizer.zero_grad()` and `torch.no_grad()` do?
8. `log_softmax` + `nll_loss` equals which loss?

Answers

1. A 2-D grid of how strongly one filter responds at each position. It produces 32.
2. Stride: the step of the kernel. Padding: extra border pixels (zeros). (32 − 3 + 2)/2 + 1 = 16 (15.5 rounded down).
3. Max keeps the strongest response, average smooths. Purposes: reduce computation, give translation invariance, help control overfitting.

4. Without them the network is linear and can't learn complex patterns. Sigmoid is used for probability outputs between 0 and 1 (e.g. binary yes/no).
5. It turns the C×H×W feature maps into one vector, because fully connected layers need 1-D input.
6. 28 → 26 (conv) → 24 (conv) → 12 (pool), and 64 channels × 12 × 12 = 9,216.
7. `train()` turns dropout on. `eval()` turns it off. `zero_grad()` clears the accumulated gradients. `no_grad()` skips gradient tracking during testing.
8. Cross-entropy loss.

Day 8 · Feature detection (Harris, SIFT, ORB) + mini project: panorama stitching

Course lab: `CV-Week2-Lecture1.ipynb` (Day 8) **Notes chapter:** `week-2/notes/01_feature_detection_harris_sift_orb.md` (read it for the full theory)

What you will learn

1. What a "feature" (keypoint) is and why corners make good ones.
2. How to run the three detectors from the lab (**Harris**, **SIFT**, **ORB**) and what *every* parameter means.
3. The lab exercise done properly: number of keypoints, speed, and *where* they land on several images.
4. A measured experiment that **proves** (or questions) the textbook claims "Harris is not scale/rotation invariant, SIFT is, ORB is rotation invariant but weaker on scale".
5. The mini project: **panorama stitching**. The course code has a real bug (the left image is thrown away); we show the bug with real photos and fix it.

Run time: about 20–30 seconds on a laptop CPU (it downloads ~3 MB of images the first time). Everything below was executed; all numbers in the text come from the outputs you see.

1 · Theory recap in plain language

A feature (keypoint) is a small spot in an image that you could find again in another photo of the same scene. Think of giving directions: "turn left at the *church tower*" works; "turn left at the *blue sky*" doesn't. Good features are **distinctive** (unlike their neighbours) and **repeatable** (found again after the camera moves, zooms or rotates).

Region type	What happens if you slide a small window over it	Good landmark?
Flat area (sky, wall)	nothing changes in any direction	❌
Edge (horizon, door frame)	changes across the edge, **not along it**	❌ you can't tell *where* along the edge you are
Corner	changes in **every** direction	✅ unique and easy to pin down

Detector vs descriptor (examiners love this distinction):

- a **detector** answers *where* are the interesting points? (x, y, and for SIFT/ORB also size and angle)
- a **descriptor** answers *what does the neighbourhood look like?* as a vector of numbers, so points can be **compared** between images.

Harris (1988): "does the window change in every direction?"

For every pixel, Harris looks at the image gradients (how fast brightness changes in x and y) inside a small window and builds a 2×2 matrix **M** (the *structure tensor*). Its two eigenvalues λ1, λ2 say how much the brightness changes in the two main directions:

- both small → flat; one big, one small → edge; **both big → corner**.

To avoid computing eigenvalues, Harris uses a score $\mathbf{R = det(M) - k \cdot trace(M)^2 = \lambda1 \cdot \lambda2 - k(\lambda1+\lambda2)^2}$. R is large and positive at corners, negative at edges, near 0 on flat areas. Harris gives **only a location and a score, no descriptor and no scale**.

SIFT (Lowe, 2004): "find blobs at every zoom level"

1. **Scale space:** blur the image more and more (Gaussian) and also shrink it (an image *pyramid*).
2. **Difference of Gaussians (DoG):** subtract neighbouring blur levels; keep points that are a maximum/minimum compared with their 26 neighbours in space *and* scale. This gives each keypoint its own **size** → scale invariance.
3. **Orientation:** the dominant gradient direction around the point → rotation invariance.
4. **Descriptor:** a 4×4 grid of 8-bin gradient histograms = **128 numbers** (float).

ORB (Rublee et al., 2011): "fast and free"

Oriented **F**AST and **R**otated **B**RIEF:

- **FAST** finds corners very quickly (is a ring of 16 pixels much brighter/darker than the centre?), ranked with the Harris score, on an **8-level pyramid** (scale factor 1.2) to cope with some zoom;
- an **orientation** from the intensity centroid (rotation invariance);
- **BRIEF** descriptor: 256 yes/no brightness comparisons → **32 bytes, binary**, compared with the **Hamming distance** (count of differing bits), which is very fast.

2 · Setup

The course notebook used `Ganesh3.jpg`, `blackdog.jpg`, `left.jpg` and `right.jpg`, but **those files were never given to students**, so the original cells crash with `None` errors (or "Image not found"). We download free, public OpenCV sample images instead. If you do have the course images, just change the file names.

Course file (missing)	Replacement used here	Why
`Ganesh3.jpg` (Harris)	`building.jpg` (OpenCV samples)	a building facade full of real corners
`blackdog.jpg` (SIFT/ORB)	`messi5.jpg` (OpenCV samples)	a natural photo (person, grass, crowd)
`left.jpg`, `right.jpg` (panorama)	`s1.jpg`, `s2.jpg` (OpenCV stitching test data)	a genuine left/right pair of the Pont du Gard

```
%matplotlib inline
%config InlineBackend.figure_formats = ["jpeg"]
import time
```

```
import numpy as np
import matplotlib.pyplot as plt
import cv2

plt.rcParams["figure.dpi"] = 80          # small JPEG figures keep the saved notebook small
print("OpenCV version:", cv2.__version__)   # SIFT_create needs OpenCV >= 4.4

import os, urllib.request

DATA, OUT = "data", "output"            # data/ and output/ sit next to this notebook (both are
git-ignored)
os.makedirs(DATA, exist_ok=True)
os.makedirs(OUT, exist_ok=True)

OPENCV_SAMPLES = "https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/"
OPENCV_STITCH  = "https://raw.githubusercontent.com/opencv/opencv_extra/4.x/testdata/stitching/"

def download(url, path):
    """Download url to path once; later runs reuse the cached file."""
    if os.path.exists(path) and os.path.getsize(path) > 0:
        return path
    print("downloading", url)
    tmp = path + ".part"
    try:
        urllib.request.urlretrieve(url, tmp)
        os.replace(tmp, path)
    except Exception as e:
        raise RuntimeError(
            f"Could not download {url}\n({e})\n"
            f"Are you offline? Open the URL in a browser, save the file as\n
{os.path.abspath(path)}\n"
            "and re-run this cell. (Or use any image/video of your own and change the file
name.)") from e
    return path

def to_rgb(img):
    """OpenCV stores colour as B,G,R; matplotlib expects R,G,B. Grey images are left alone."""
    return img if img.ndim == 2 else cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

def show(images, titles=None, cols=None, size=4.5, max_side=900):
    """Show one or more OpenCV images side by side (downscaled for a small notebook file)."""
    if not isinstance(images, (list, tuple)):
        images, titles = [images], [titles]
    cols = cols or len(images)
    rows = (len(images) + cols - 1) // cols
    aspect = max(im.shape[0] / im.shape[1] for im in images)          # height/width of the
tallest image
    fig, axes = plt.subplots(rows, cols, figsize=(size * cols, size * rows * aspect + 0.35 *
rows), squeeze=False)
    for ax in axes.ravel():
        ax.axis("off")
    for i, img in enumerate(images):
        s = min(1.0, max_side / max(img.shape[:2]))
        if s < 1:
            img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
        ax = axes.ravel()[i]
        ax.imshow(to_rgb(img), cmap="gray" if img.ndim == 2 else None)
        if titles and titles[i]:
            ax.set_title(titles[i], fontsize=10)
    plt.tight_layout()
    plt.show()

files = {
    "building.jpg": OPENCV_SAMPLES + "building.jpg",
    "messi5.jpg":   OPENCV_SAMPLES + "messi5.jpg",
    "graf1.png":    OPENCV_SAMPLES + "graf1.png",
    "blox.jpg":     OPENCV_SAMPLES + "blox.jpg",
    "s1.jpg":       OPENCV_STITCH + "s1.jpg",
    "s2.jpg":       OPENCV_STITCH + "s2.jpg",
```

```
    "boat1.jpg":    OPENCV_STITCH + "boat1.jpg",
    "boat2.jpg":    OPENCV_STITCH + "boat2.jpg",
}
for name, url in files.items():
    download(url, os.path.join(DATA, name))
print("ready:", ", ".join(files))
```

```
OpenCV version: 5.0.0
ready: building.jpg, messi5.jpg, graf1.png, blox.jpg, s1.jpg, s2.jpg, boat1.jpg, boat2.jpg
```

3 · Harris corners: the course code, explained line by line

Same calls and parameters as the course (`cv2.cornerHarris(gray, 2, 3, 0.04)` and the `0.01 * dst.max()` threshold); only the file name changed.

```
img = cv2.imread(os.path.join(DATA, 'building.jpg'))      # course: 'Ganesh3.jpg' (not provided)
if img is None:                                            # imread does NOT raise an error, it
silently returns None
    raise FileNotFoundError("Image not found: check the path")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)               # Harris works on one brightness
channel

gray = np.float32(gray)                                    # cornerHarris wants float32 input
dst = cv2.cornerHarris(gray, 2, 3, 0.04)                   # blockSize=2, ksize=3, k=0.04 -> R
score for EVERY pixel
img_harris = img.copy()
img_harris[dst > 0.01 * dst.max()] = [0, 0, 255]           # paint pixels with R > 1% of the best
score red (BGR!)

print("image size:", img.shape, "| dst (R map) shape:", dst.shape, dst.dtype)
print(f"R ranges from {dst.min():.3g} (edges, negative) to {dst.max():.3g} (strongest corner)")
print("red pixels:", int((dst > 0.01 * dst.max()).sum()))

plt.imshow(cv2.cvtColor(img_harris, cv2.COLOR_BGR2RGB))   # BGR -> RGB so red looks red
plt.title("Harris Corners")
plt.axis('off')
plt.show()
```

```
image size: (600, 868, 3) | dst (R map) shape: (600, 868) float32
R ranges from -4.13e+07 (edges, negative) to 2.98e+07 (strongest corner)
red pixels: 6548
```



What each Harris parameter means

Parameter	Course value	Meaning	Bigger value →
`blockSize`	2	size of the neighbourhood (window) used to build the matrix M	smoother response, finds larger corners, fewer tiny ones
`ksize`	3	size of the **Sobel** filter used to compute the gradients (odd: 1, 3, 5, 7)	gradients less noisy but blurrier
`k`	0.04	the "k" in R = det − k·trace² (typical 0.04–0.06)	**fewer** corners (edges are penalised more)
`0.01 * dst.max()`	1 %	not a Harris parameter: *our* threshold, relative to the strongest corner	fewer red pixels

Two things that the course output hides:

1. **Red pixels are not corners.** One physical corner usually produces a small *blob* of several pixels above the threshold. To count corners we group touching red pixels (connected components) or use non-maximum suppression.
2. The threshold is **relative** (1 % of the maximum), so one very strong corner can suppress everything else, and the count changes from image to image.

Let's measure the effect of each parameter (the count is the number of corner *blobs*).

```
def harris_count(gray, block=2, ksize=3, k=0.04, rel=0.01):
    R = cv2.cornerHarris(np.float32(gray), block, ksize, k)
    mask = (R > rel * R.max()).astype(np.uint8)
    n_blobs = cv2.connectedComponents(mask)[0] - 1          # label 0 is the background
    return R, mask, n_blobs

g = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
settings = [dict(), dict(rel=0.001), dict(rel=0.1), dict(k=0.06), dict(block=5), dict(ksize=7)]
panels, titles = [], []
print(f"{'setting':32s} red pixels   corners (blobs)")
for s in settings:
    R, mask, n = harris_count(g, **s)
    label = ", ".join(f"{a}={b}" for a, b in s.items()) or "course: block=2, ksize=3, k=0.04, 1%"
    print(f"{label:32s} {int(mask.sum()):10d}   {n:8d}")
    vis = img.copy()
    vis[cv2.dilate(mask, None) > 0] = (0, 0, 255)            # dilate = make the dots bigger so
you can see them
    panels.append(vis); titles.append(f"{label}\n{n} corners")
show(panels, titles, cols=3, size=4.3)
```

```
setting                          red pixels   corners (blobs)
course: block=2, ksize=3, k=0.04, 1%       6548       1152
rel=0.001                                 21262       2601
rel=0.1                                     682        296
k=0.06                                     4924       1086
block=5                                   27229        608
ksize=7                                    7478       1063
```



Reading the table: lowering the threshold to 0.1 % floods the image with weak corners (texture, noise); raising it to 10 % keeps only the very strongest. Increasing `k` removes edge-like responses. A larger `blockSize` merges nearby corners into fewer, larger blobs. A larger `ksize` (Sobel 7) changes the gradient scale and therefore which corners win.

4 · SIFT keypoints: the course code

Same code as the course (`cv2.SIFT_create()` , `detectAndCompute` , rich keypoints). `cv2.SIFT_create` is in the main OpenCV package since version 4.4 (the patent expired in March 2020). Older versions needed `cv2.xfeatures2d.SIFT_create()` from `opencv-contrib-python` .

```
# Load the image
img = cv2.imread(os.path.join(DATA, 'messi5.jpg'))   # course: 'blackdog.jpg' (not provided)
if img is None:
    raise ValueError("Image not found. Check the file path.")

# Convert to grayscale (uint8)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Create SIFT detector (default: keep all keypoints, 3 layers per octave, contrastThreshold 0.04,
edgeThreshold 10, sigma 1.6)
sift = cv2.SIFT_create()

# Detect keypoints and compute their 128-number descriptors in one call; None = no mask (search
the whole image)
kp, des = sift.detectAndCompute(gray, None)

# Draw keypoints: RICH = circle radius shows the keypoint's scale, the line shows its orientation
img_sift = cv2.drawKeypoints(img, kp, None,
                             flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

print("SIFT keypoints:", len(kp), "| descriptor array:", des.shape, des.dtype)
k0 = max(kp, key=lambda k: k.size)
print(f"largest keypoint: pt=({k0.pt[0]:.1f}, {k0.pt[1]:.1f}) size={k0.size:.1f}px angle=
{k0.angle:.1f} deg octave={k0.octave & 255}")

plt.figure(figsize=(8, 5))
```

```
plt.imshow(cv2.cvtColor(img_sift, cv2.COLOR_BGR2RGB))
plt.title("SIFT Keypoints")
plt.axis('off')
plt.show()
```

```
SIFT keypoints: 640 | descriptor array: (640, 128) float32
largest keypoint: pt=(96.3, 151.4) size=49.1px angle=78.8 deg octave=3
```



What a keypoint object holds: `kp[i].pt` (x, y), `.size` (diameter of the meaningful neighbourhood → the circle), `.angle` (orientation in degrees → the line), `.response` (strength), `.octave` (pyramid level). `des` has **one row of 128 float32 numbers per keypoint**.

Useful `SIFT_create` parameters: `nfeatures` (keep only the N best; 0 = all), `contrastThreshold` (higher → fewer, stronger keypoints in low-contrast regions), `edgeThreshold` (higher → keeps more edge-like points), `sigma` (blur of the first level).

5 · ORB keypoints: the course code

```
orb = cv2.ORB_create()                                    # defaults: nfeatures=500,
scaleFactor=1.2, nlevels=8, fastThreshold=20
kp_orb, des_orb = orb.detectAndCompute(gray, None)        # same grey image as the SIFT cell
img_orb = cv2.drawKeypoints(img, kp_orb, None, color=(0,255,0), flags=0)   # flags=0: just small
circles, no size/angle

print("ORB keypoints:", len(kp_orb), "| descriptor array:", des_orb.shape, des_orb.dtype, "= 32
bytes = 256 bits each")

plt.figure(figsize=(8, 5))
plt.imshow(cv2.cvtColor(img_orb, cv2.COLOR_BGR2RGB))
plt.title("ORB Keypoints")
plt.axis('off')
plt.show()
```

```
ORB keypoints: 500 | descriptor array: (500, 32) uint8 = 32 bytes = 256 bits each
```

ORB Keypoints



Why exactly 500? `ORB_create()` has `nfeatures=500` by default: ORB finds many FAST corners and then keeps the 500 with the best Harris score. That is a cap, not a property of the image; ask for more with `cv2.ORB_create(nfeatures=2000)`.

Other ORB parameters worth knowing: `scaleFactor=1.2` and `nlevels=8` (the pyramid: 8 levels, each 1.2× smaller, so it covers about a $1.2^7 \approx 3.6\times$ zoom range), `edgeThreshold=31` (border where no features are detected, because the descriptor patch needs room), `fastThreshold=20` (how much brighter/darker the ring must be), `WTA_K=2` (pixels compared per BRIEF test; 3 or 4 need `NORM_HAMMING2`).

Note how ORB keypoints **cluster** on the high-contrast parts of the player (face, shirt, arms), the ball and the watermark, and almost none land on the blurred crowd, whereas SIFT spreads over the whole image (compare with the SIFT picture above). ORB keeps the 500 strongest FAST corners, and they bunch together where the contrast is highest.

6 · Exercise: detect keypoints in different images, compare number, speed and distribution

The course exercise asks us to run all three detectors on different images (building facades, natural scenes) and compare. We do it on four images, time each detector (median of 3 runs, detection + description) and count:

- Harris: corner blobs with the course settings;
- SIFT: default settings (no cap);
- ORB: default (cap 500) **and** `nfeatures=5000` so the comparison is fair.

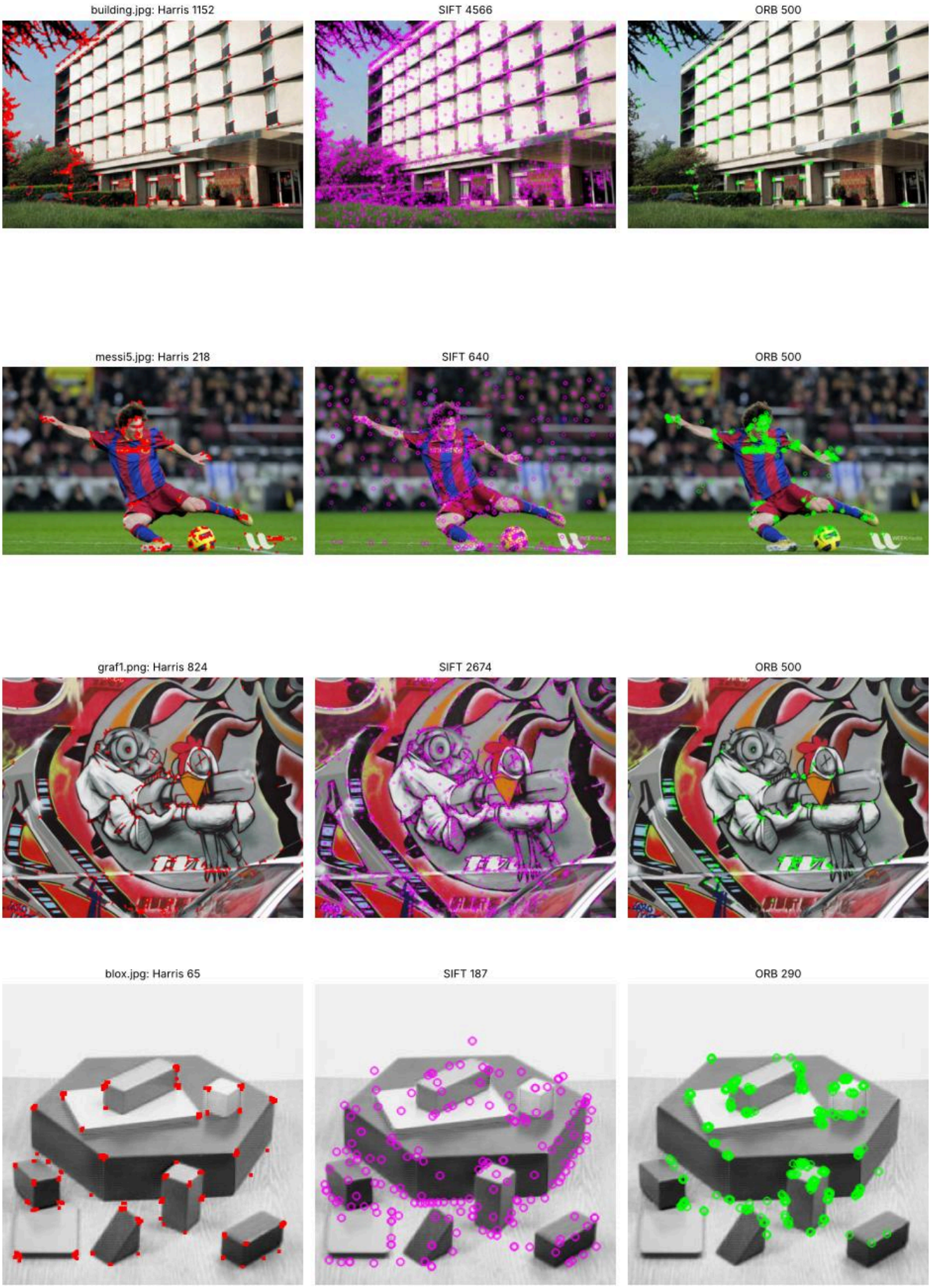
```
def timed(fn, reps=3):
    ts = []
    for _ in range(reps):
        t = time.perf_counter(); out = fn(); ts.append(time.perf_counter() - t)
    return out, 1000 * np.median(ts)

names = ["building.jpg", "messi5.jpg", "graf1.png", "blox.jpg"]
```

```
sift, orb500, orb5k = cv2.SIFT_create(), cv2.ORB_create(), cv2.ORB_create(nfeatures=5000)
rows, panels, titles = [], [], []
for name in names:
    im = cv2.imread(os.path.join(DATA, name))
    g = cv2.cvtColor(im, cv2.COLOR_BGR2GRAY)
    (R, mask, nh), th = timed(lambda: harris_count(g))
    (ks, _), ts = timed(lambda: sift.detectAndCompute(g, None))
    (ko, _), to = timed(lambda: orb500.detectAndCompute(g, None))
    (ko5, _), to5 = timed(lambda: orb5k.detectAndCompute(g, None))
    rows.append((name, f"{g.shape[1]}x{g.shape[0]}", nh, th, len(ks), ts, len(ko), to, len(ko5),
to5))
    vis_h = im.copy(); vis_h[cv2.dilate(mask, None) > 0] = (0, 0, 255)
    panels += [vis_h,
               cv2.drawKeypoints(im, ks, None, color=(255, 0, 255)),
               cv2.drawKeypoints(im, ko, None, color=(0, 255, 0))]
    titles += [f"{name}: Harris {nh}", f"SIFT {len(ks)}", f"ORB {len(ko)}"]

print(f"{'image':13s} {'size':>9s} | {'Harris':>6s} {'ms':>6s} | {'SIFT':>5s} {'ms':>6s} |
{'ORB500':>6s} {'ms':>5s} | {'ORB5000':>7s} {'ms':>5s}")
for r in rows:
    print(f"{r[0]:13s} {r[1]:>9s} | {r[2]:6d} {r[3]:6.1f} | {r[4]:5d} {r[5]:6.1f} | {r[6]:6d}
{r[7]:5.1f} | {r[8]:7d} {r[9]:5.1f}")
print("SIFT time / ORB-500 time:", ", ".join(f"{r[0]} {r[5] / r[7]:.0f}x" for r in rows))
show(panels, titles, cols=3, size=4.0, max_side=500)
```

```
image              size | Harris     ms |  SIFT     ms | ORB500    ms | ORB5000    ms
building.jpg    868x600 |   1152    8.0 |  4566  356.1 |    500  14.5 |    5000  25.3
messi5.jpg      548x342 |    218    1.4 |   640   59.1 |    500   3.2 |    2573  10.0
graf1.png       800x640 |    824    5.3 |  2674  273.1 |    500  12.1 |    5000  24.6
blox.jpg        256x256 |     65    0.4 |   187   16.1 |    290   1.5 |     290   1.4
SIFT time / ORB-500 time: building.jpg 25x, messi5.jpg 18x, graf1.png 23x, blox.jpg 11x
```



What the measurements show (this machine; your times will differ):

- **Speed.** Harris is always the fastest (a single filtering pass), but it only gives *locations*: no descriptor, no scale, no angle. ORB comes next. SIFT is the slowest by a wide margin: across

our runs it was **at least 5× and up to 36× slower than ORB-500** (see the printed ratios), and still several times slower than ORB asked for 5000 points. Absolute times vary a lot with machine load (SIFT on `building.jpg` took between about 130 and 510 ms in different runs), but the order **Harris < ORB < SIFT** never changed.

- **Count.** The lab's "learning point" says *SIFT finds fewer keypoints*. With default settings we saw the opposite on every image: SIFT found 4566 on `building.jpg` vs ORB's 500. The reason is simply that `ORB_create()` **stops at 500** (`nfeatures`), while `SIFT_create()` keeps everything above its contrast threshold. Asked for 5000, ORB returned 5000 on the two big images, but only 2573 on `messi5.jpg` . "Fewer/more" is a setting, not a property of the method.
- `blox.jpg` (a small 256×256 synthetic scene) shows the limits: ORB found only 290 because it ignores a 31-pixel border (`edgeThreshold`) and the smooth surfaces have few FAST corners. Harris found 65 corner blobs, almost all on the real block corners.
- **Distribution (look at the grid).** Harris marks building corners but also floods the fine tree branches (texture looks "cornery"). SIFT spreads over the whole image at many sizes, including blobs in foliage, grass and the crowd. ORB's 500 points **bunch together** on the highest-contrast spots (window corners, the player's shirt) and leave large areas empty. This matters for stitching and tracking, where you want points everywhere.

7 · Experiment: are they really scale and rotation invariant? Let's *prove* it

The lecture claims: *Harris is not scale/rotation invariant; SIFT is scale **and** rotation invariant; ORB is rotation invariant but weaker on scale.* Instead of believing it, we test it.

Method. Take an image, rotate or rescale it by a **known** amount, so we know exactly where every original pixel must land (ground truth). Then measure two things:

1. **Repeatability of the detector:** of the keypoints found in the original, what % are found again (within 2.5 px, at the coarser resolution) at the predicted place in the transformed image? We also report the **chance level**: the same score for random points, so we know what "luck" looks like.
2. **Correct matches (detector + descriptor):** match descriptors between the two images (brute force, cross-check) and count what % of the matches land at the right place. Harris has no descriptor, so we give it the simplest one: the raw 11×11 pixel patch around the corner (normalised for brightness). This is what "using Harris for matching" means in practice.

All detectors are limited to their **500 best** points so that no detector wins just by having more points.

```
N = 500
sift_n, orb_n = cv2.SIFT_create(nfeatures=N), cv2.ORB_create(nfeatures=N)

def harris_with_patch(g):
    """Harris corners (500 best, course's k=0.04, blockSize=2) + an 11x11 raw-pixel patch as a
naive descriptor."""
    p = cv2.goodFeaturesToTrack(g, N, 0.01, 5, blockSize=2, useHarrisDetector=True, k=0.04)
    p = np.zeros((0, 2), np.float32) if p is None else p.reshape(-1, 2)
    gp = cv2.copyMakeBorder(g, 8, 8, 8, 8, cv2.BORDER_REFLECT).astype(np.float32)
    d = []
```

```
    for x, y in p:
        x, y = int(round(x)) + 8, int(round(y)) + 8
        patch = gp[y-5:y+6, x-5:x+6].ravel()
        d.append((patch - patch.mean()) / (patch.std() + 1e-6))
    return p, np.float32(d)

def with_desc(det):
    def f(g):
        k, d = det.detectAndCompute(g, None)
        return np.float32([q.pt for q in k]), d
    return f

DETECTORS = {"Harris": (harris_with_patch, cv2.NORM_L2),
             "SIFT":   (with_desc(sift_n), cv2.NORM_L2),
             "ORB":    (with_desc(orb_n), cv2.NORM_HAMMING)}

def transform(g, angle=0, scale=1.0):
    """Rotate/scale about the centre on a canvas big enough to keep the whole image. Returns
image and the 2x3 matrix."""
    h, w = g.shape
    M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, scale)
    c, s = abs(M[0, 0]), abs(M[0, 1])
    nw, nh = int(w * c + h * s), int(h * c + w * s)
    M[0, 2] += nw / 2 - w / 2; M[1, 2] += nh / 2 - h / 2
    return cv2.warpAffine(g, M, (nw, nh)), M

rng = np.random.default_rng(0)
def evaluate(g, name, angle=0, scale=1.0):
    f, norm = DETECTORS[name]
    p, dp = f(g)
    g2, M = transform(g, angle, scale)
    q, dq = f(g2)
    p_true = p @ M[:, :2].T + M[:, 2]                # where each original keypoint MUST be in
the new image
    eps = 2.5 * max(1.0, scale)                      # 2.5 px at the coarser of the two
resolutions
    dist = np.linalg.norm(p_true[:, None] - q[None], axis=2)
    repeat = 100 * (dist.min(1) < eps).mean()
    q_rand = np.c_[rng.uniform(0, g2.shape[1], len(q)), rng.uniform(0, g2.shape[0], len(q))]
    chance = 100 * (np.linalg.norm(p_true[:, None] - q_rand[None], axis=2).min(1) < eps).mean()
    matches = cv2.BFMatcher(norm, crossCheck=True).match(dp, dq)
    ok = [np.linalg.norm(p_true[m.queryIdx] - q[m.trainIdx]) < eps for m in matches]
    return repeat, chance, 100 * np.mean(ok) if ok else 0.0, int(np.sum(ok))
```

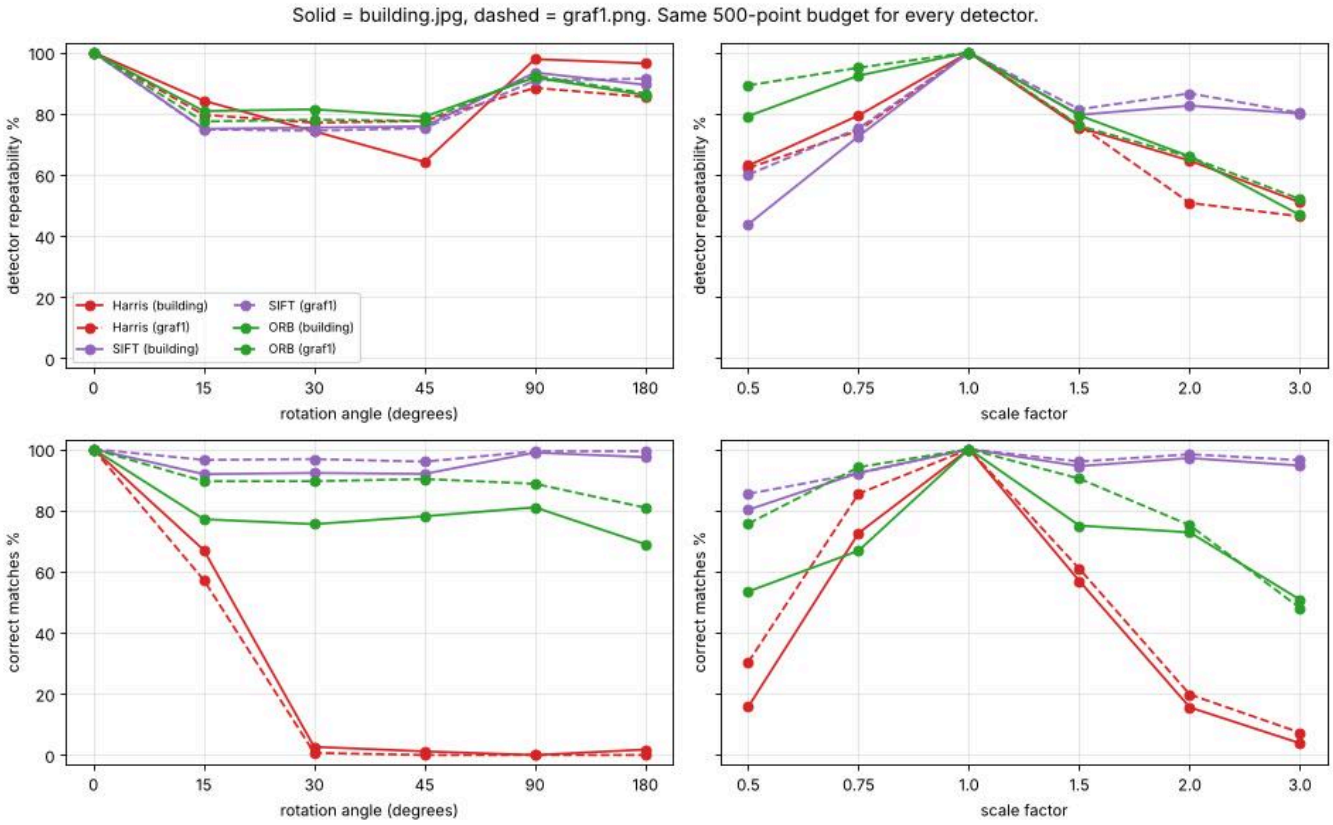
```
angles, scales = [0, 15, 30, 45, 90, 180], [0.5, 0.75, 1.0, 1.5, 2.0, 3.0]
results = {}
for im_name in ["building.jpg", "graf1.png"]:
    g = cv2.imread(os.path.join(DATA, im_name), cv2.IMREAD_GRAYSCALE)
    s = 640 / max(g.shape); g = cv2.resize(g, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    for det in DETECTORS:
        results[(im_name, det, "rot")] = [evaluate(g, det, angle=a) for a in angles]
        results[(im_name, det, "scale")] = [evaluate(g, det, scale=sc) for sc in scales]

for kind, xs, unit in [("rot", angles, "deg"), ("scale", scales, "x")]:
    print(f"\n=== {'ROTATION' if kind == 'rot' else 'SCALE'}: repeatability% (chance%) | correct
matches% ===")
    for im_name in ["building.jpg", "graf1.png"]:
        print(im_name)
        print("  " + " " * 7 + "".join(f"{str(x) + unit:>18s}" for x in xs))
        for det in DETECTORS:
            cells = [f"{r:3.0f} ({c:2.0f}) | {m:3.0f}%" for r, c, m, _ in results[(im_name, det,
kind)]]
            print(f"  {det:7s}" + "".join(f"{x:>18s}" for x in cells))
```

```
=== ROTATION: repeatability% (chance%) | correct matches% ===
building.jpg
                      0deg             15deg             30deg             45deg             90d
eg            180deg
  Harris    100 ( 4) | 100%    84 ( 1) |  67%    74 ( 1) |   3%    64 ( 1) |   1%    98 ( 3) |
0%    96 ( 3) |   2%
  SIFT      100 ( 5) | 100%    75 ( 2) |  92%    75 ( 1) |  92%    76 ( 3) |  92%    93 ( 4) |  9
9%    89 ( 3) |  97%
  ORB       100 ( 0) | 100%    81 ( 3) |  77%    81 ( 3) |  75%    79 ( 3) |  78%    92 ( 3) |  8
1%    86 ( 4) |  69%
graf1.png
                      0deg             15deg             30deg             45deg             90d
eg            180deg
  Harris    100 ( 2) | 100%    80 ( 3) |  57%    77 ( 1) |   1%    78 ( 1) |   0%    88 ( 3) |
0%    85 ( 3) |   0%
  SIFT      100 ( 6) | 100%    75 ( 3) |  96%    74 ( 3) |  97%    75 ( 1) |  96%    91 ( 3) |  9
9%    91 ( 3) |  99%
  ORB       100 ( 2) | 100%    77 ( 3) |  89%    78 ( 1) |  90%    78 ( 1) |  90%    92 ( 2) |  8
9%    87 ( 4) |  81%

=== SCALE: repeatability% (chance%) | correct matches% ===
building.jpg
                      0.5x             0.75x              1.0x              1.5x              2.
0x             3.0x
  Harris     63 ( 7) |  16%    79 ( 7) |  73%   100 ( 5) | 100%    75 ( 3) |  57%    65 ( 4) |  1
6%    51 ( 3) |   4%
  SIFT       44 (12) |  80%    73 ( 6) |  92%   100 ( 3) | 100%    80 ( 4) |  94%    83 ( 5) |  9
7%    80 ( 3) |  95%
  ORB        79 ( 9) |  53%    92 ( 5) |  67%   100 ( 6) | 100%    79 ( 3) |  75%    66 ( 3) |  7
3%    47 ( 3) |  51%
graf1.png
                      0.5x             0.75x              1.0x              1.5x              2.
0x             3.0x
  Harris     62 ( 6) |  30%    74 ( 3) |  85%   100 ( 3) | 100%    76 ( 3) |  61%    51 ( 2) |  2
0%    46 ( 3) |   7%
  SIFT       60 ( 9) |  85%    75 ( 3) |  92%   100 ( 3) | 100%    81 ( 3) |  96%    87 ( 3) |  9
8%    80 ( 4) |  96%
  ORB        89 ( 9) |  76%    95 ( 5) |  94%   100 ( 3) | 100%    76 ( 2) |  90%    66 ( 3) |  7
5%    52 ( 2) |  48%
```

```
fig, axes = plt.subplots(2, 2, figsize=(12, 7.5), sharey=True)
colors = {"Harris": "tab:red", "SIFT": "tab:purple", "ORB": "tab:green"}
for col, (kind, xs, lab) in enumerate([("rot", angles, "rotation angle (degrees)"), ("scale",
scales, "scale factor")]):
    for row, (idx, what) in enumerate([(0, "detector repeatability %"), (2, "correct matches
%")]):
        ax = axes[row, col]
        for det in DETECTORS:
            for im_name, ls in [("building.jpg", "-"), ("graf1.png", "--")]:
                ys = [r[idx] for r in results[(im_name, det, kind)]]
                ax.plot(range(len(xs)), ys, ls, marker="o", color=colors[det],
                        label=f"{det} ({im_name.split('.')[0]})")
        ax.set_xticks(range(len(xs))); ax.set_xticklabels(xs); ax.set_xlabel(lab);
ax.set_ylabel(what)
        ax.set_ylim(-3, 103); ax.grid(alpha=.3)
axes[0, 0].legend(fontsize=8, ncol=2)
fig.suptitle("Solid = building.jpg, dashed = graf1.png. Same 500-point budget for every
detector.")
plt.tight_layout(); plt.show()
```



What the experiment really shows (be ready to say this in the viva)

Correct matches (bottom row: the result that matters for matching and stitching):

- **Harris + raw patch collapses under rotation:** 57–67 % correct at 15°, then **≤ 3 % from 30° onwards** on both images. It also collapses under zoom: 4–7 % correct at 3×. A raw pixel patch has no idea the image turned or grew.
- **SIFT stays high everywhere:** 92–99 % correct at every rotation, and 80–98 % at every scale from 0.5× to 3×. This is the scale **and** rotation invariance the lecture promises.
- **ORB is rotation invariant** (69–90 % correct at all angles) but **weaker on scale:** it is fine from 0.75× to 1.5× (67–94 %) and drops to about **50 % at 3×** (51 % and 48 %). Its 8-level, 1.2× pyramid only covers about 3.6×, and each level is coarse.

Detector repeatability (top row) has a surprise, and it is worth knowing for the viva:

- **Harris *detection* is NOT badly hurt by rotation.** 64–98 % of Harris corners reappear at the right place after rotation, similar to SIFT and ORB (chance level 1–4 %). This is expected from the maths: R depends on the eigenvalues of M, and eigenvalues don't change when you rotate the image. The textbook phrase "Harris is not rotation invariant" is really about using Harris *for matching*: it gives no orientation, so the patch you compare is not rotated.
- **Scale is where Harris detection really fails:** when the image is enlarged 3×, only 46–51 % of the corners reappear (SIFT: 80 %), because a sharp corner at one zoom becomes a rounded curve at another, and the fixed 2×2 window sees a different structure.
- SIFT's detector repeatability is "only" 74–93 % under rotation, because we kept only its 500 best points and the ranking shuffles slightly after resampling. Its descriptor makes up for it (≥ 92 % correct matches).

- At 0.5× all detectors lose points (and the chance level rises to 6–12 %, because the image is smaller and the same number of points fills more of it), so compare the curves against chance, not against 100 %.

One-line summary: *Harris = rotation-stable detector, but not scale invariant, and has no descriptor → useless for matching under rotation or zoom. SIFT = robust to both. ORB = robust to rotation, moderate scale range, much faster.*

8 · Mini project: panorama stitching

Workflow (from the lecture): detect ORB features → match them → estimate a **homography** with RANSAC → warp one image onto the other → paste.

- A **homography** is a 3×3 matrix H that maps points of one image plane to another: `[x', y', 1]ᵀ ∝ H·[x, y, 1]ᵀ`. It is exact when the scene is (almost) flat **or** the camera only rotates about its centre, which is the situation for a panorama taken from one spot. It has 8 degrees of freedom, so it needs at least **4 point pairs**.
- **RANSAC** (RANdom SAmple Consensus): pick 4 random matches, compute H, count how many other matches agree with it (within the *reprojection threshold*, 5 px here) → these are **inliers**. Repeat many times; keep the H with the most inliers. Wrong matches (outliers) simply don't vote for the right answer.

8a · The course code, run on a real left/right pair

We run the course cell **unchanged** except for the file names: `img1` = left photo, `img2` = right photo.

```
def load_small(name, scale=0.5):
    im = cv2.imread(os.path.join(DATA, name))
    if im is None:
        raise FileNotFoundError(name)
    return cv2.resize(im, None, fx=scale, fy=scale, interpolation=cv2.INTER_AREA)

left_img, right_img = load_small("s1.jpg"), load_small("s2.jpg")     # same scale for both
show([left_img, right_img], ["left photo (s1.jpg)", "right photo (s2.jpg)"], size=5)
```




```
# ---- course code (only the file names changed) ----
img1 = left_img.copy()     # course: cv2.imread('left.jpg')
img2 = right_img.copy()    # course: cv2.imread('right.jpg')

# Convert to grayscale
```

```python
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Detect ORB keypoints and descriptors
orb = cv2.ORB_create()
kp1, des1 = orb.detectAndCompute(gray1, None)
kp2, des2 = orb.detectAndCompute(gray2, None)

# Match features: Hamming distance for binary ORB descriptors; crossCheck keeps i<->j only if each is the other's best match
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x:x.distance)            # best (smallest distance) first

# Draw matches (flags=2 = NOT_DRAW_SINGLE_POINTS: don't draw unmatched keypoints)
img_matches = cv2.drawMatches(img1, kp1, img2, kp2, matches[:20], None, flags=2)
plt.figure(figsize=(12, 4))
plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))
plt.title("Feature Matches")
plt.axis('off')
plt.show()

# Estimate homography: maps points of img1 (left) INTO img2's (right) coordinate frame
src_pts = np.float32([ kp1[m.queryIdx].pt for m in matches ]).reshape(-1,1,2)
dst_pts = np.float32([ kp2[m.trainIdx].pt for m in matches ]).reshape(-1,1,2)
M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC,5.0)

# Warp and stitch
h,w = img1.shape[:2]
result = cv2.warpPerspective(img1, M, (w + img2.shape[1], h))   # canvas starts at x=0 of img2's frame
result[0:img2.shape[0], 0:img2.shape[1]] = img2                 # paste img2 at the left of the canvas

plt.figure(figsize=(12, 4))
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Panorama Result (course code)")
plt.axis('off')
plt.show()
print(f"{len(kp1)} / {len(kp2)} keypoints, {len(matches)} cross-checked matches, "
      f"{int(mask.sum())} RANSAC inliers ({100 * mask.mean():.0f}%)")
```



Panorama Result (course code)



```
500 / 500 keypoints, 235 cross-checked matches, 206 RANSAC inliers (88%)
```

8b · Why is the left half of the panorama missing? Diagnosing the bug

The "panorama" above is just the **right photo followed by black**; the left photo has vanished. Let's ask the homography where the corners of `img1` (the left photo) end up:

```
corners1 = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
where = cv2.perspectiveTransform(corners1, M).reshape(-1, 2)
for name, (x, y) in zip(["top-left", "top-right", "bottom-right", "bottom-left"], where):
    print(f"img1 {name:12s} -> x = {x:7.1f}, y = {y:6.1f}")
print(f"canvas covers x = 0 ... {w + img2.shape[1]}, img2 is pasted over x = 0 ...
{img2.shape[1]}")
```

```
img1 top-left     -> x =  -214.8, y =    0.0
img1 top-right    -> x =   409.5, y =   -0.2
img1 bottom-right -> x =   407.6, y =  349.5
img1 bottom-left  -> x =  -214.4, y =  349.6
canvas covers x = 0 ... 1315, img2 is pasted over x = 0 ... 692
```

The bug, in one sentence: `M` maps the *left* image into the *right* image's frame, and in that frame the left image lies at **negative x** (to the left of the right photo). `warpPerspective` only draws pixels with x ≥ 0, so the whole strip from x ≈ −215 to 0 (a third of the left photo, including the river bank on the left) is simply lost. The part that does land at x ≥ 0 is the overlap, and it is then **overwritten** when `img2` is pasted at `[0:h, 0:w]`. Result: the left photo contributes nothing. (The course notebook's own saved output shows exactly this: its "Panorama Result" is the second image followed by a dark smear.)

The course code only works if `img1` is the **right** image and `img2` the **left** one: then H pushes `img1` to positive x, into the empty space on the right. That is why many online tutorials *seem* to work: they name the files the other way round.

The fix (two parts):

1. Compute the homography that maps the **right image into the left image's frame** (`findHomography(dst_pts, src_pts)`), warp the right image, and paste the left image at the origin.
2. More generally (any order, any direction): warp the corners first, compute the bounding box of *both* images, and add a **translation** so that nothing has negative coordinates. Then crop the black border.

We also improve the matches: instead of using *all* cross-checked matches (many are wrong; RANSAC copes, but only just), use the **Lowe ratio test** (Day 9) and report the inlier ratio.

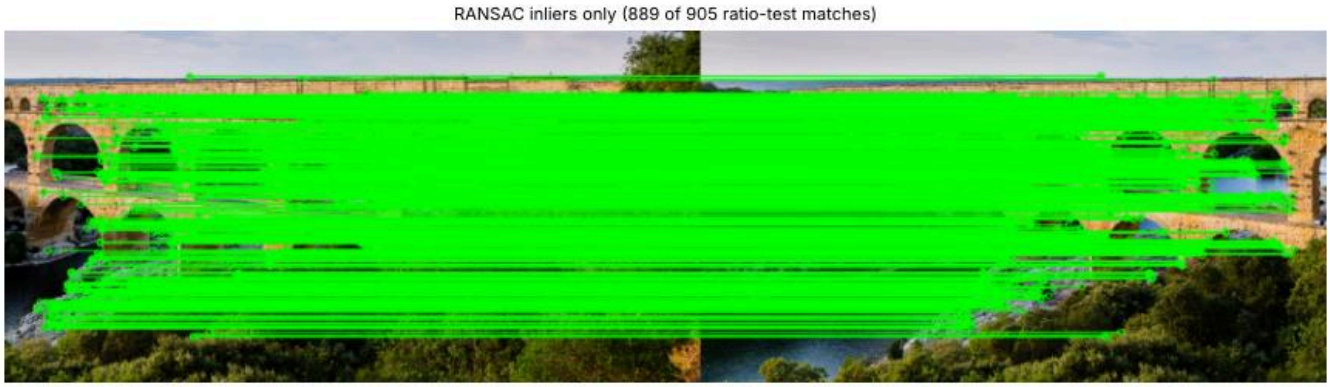
```
def stitch_pair(img_left, img_right, ratio=0.75, nfeatures=2000, show_matches=False):
    """Stitch two overlapping photos. Works whichever image is 'left': the canvas is translated
to fit both."""
    g1 = cv2.cvtColor(img_left, cv2.COLOR_BGR2GRAY)
    g2 = cv2.cvtColor(img_right, cv2.COLOR_BGR2GRAY)
    orb = cv2.ORB_create(nfeatures=nfeatures)
    k1, d1 = orb.detectAndCompute(g1, None)
    k2, d2 = orb.detectAndCompute(g2, None)
    if d1 is None or d2 is None:
        raise RuntimeError("No features found in one of the images")

    # 1) match the RIGHT image to the LEFT image (query = right, train = left) and apply Lowe's
ratio test
    knn = cv2.BFMatcher(cv2.NORM_HAMMING).knnMatch(d2, d1, k=2)
    good = [p[0] for p in knn if len(p) == 2 and p[0].distance < ratio * p[1].distance]
    if len(good) < 4:
        raise RuntimeError(f"Only {len(good)} good matches: need at least 4 for a homography")
    src = np.float32([k2[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)   # points in the RIGHT
image
    dst = np.float32([k1[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)   # same points in the
LEFT image
    H, inl = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)                  # H: right -> left
frame
    if H is None:
        raise RuntimeError("findHomography failed")

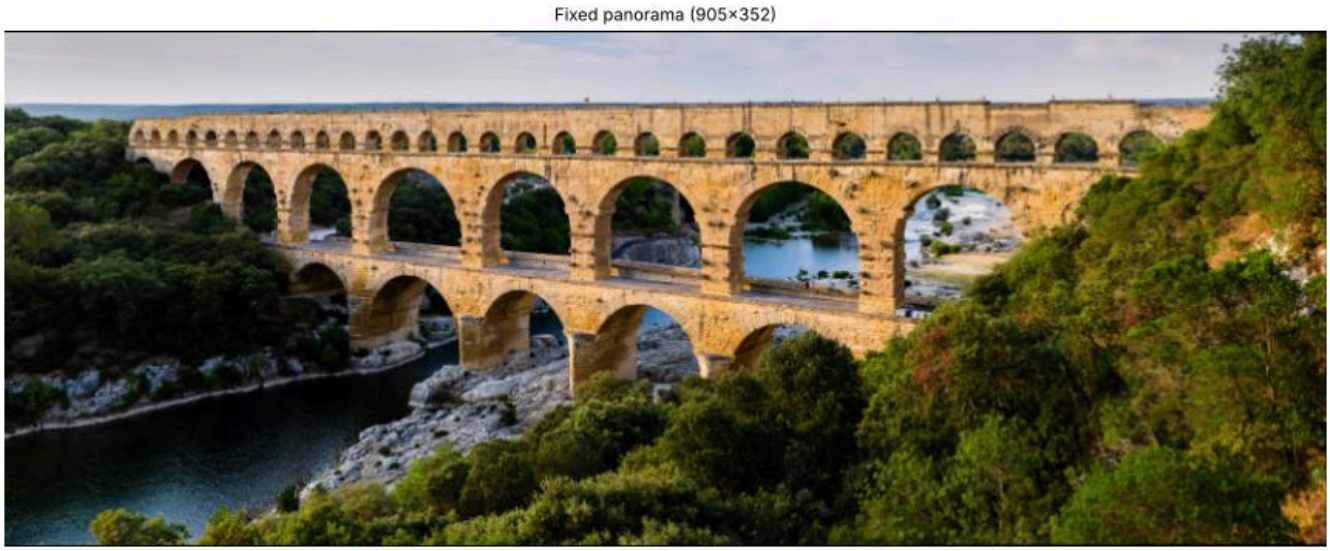
    # 2) bounding box of both images in the left image's frame, then shift so that nothing is
negative
    h1, w1 = img_left.shape[:2]; h2, w2 = img_right.shape[:2]
    c2 = cv2.perspectiveTransform(np.float32([[0, 0], [w2, 0], [w2, h2], [0, h2]]).reshape(-1, 1,
2), H)
    allc = np.vstack([c2.reshape(-1, 2), [[0, 0], [w1, 0], [w1, h1], [0, h1]]])
    xmin, ymin = np.floor(allc.min(0)).astype(int); xmax, ymax = np.ceil(allc.max(0)).astype(int)
    T = np.array([[1, 0, -xmin], [0, 1, -ymin], [0, 0, 1]], dtype=np.float64)   # translation
matrix
    pano = cv2.warpPerspective(img_right, T @ H, (xmax - xmin, ymax - ymin))
    pano[-ymin:-ymin + h1, -xmin:-xmin + w1] = img_left                          # paste the left
image (shifted)

    # 3) crop the black border (rows/columns that are entirely empty)
    nz = np.argwhere(pano.max(axis=2) > 0)
    (y0, x0), (y1, x1) = nz.min(0), nz.max(0) + 1
    info = dict(kp=(len(k1), len(k2)), good=len(good), inliers=int(inl.sum()),
inlier_ratio=float(inl.mean()), H=H)
    if show_matches:
        vis = cv2.drawMatches(img_right, k2, img_left, k1, [m for m, i in zip(good, inl.ravel())
if i], None,
                              matchColor=(0, 255, 0),
flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
        show(vis, f"RANSAC inliers only ({info['inliers']} of {len(good)} ratio-test matches)",
size=11)
    return pano[y0:y1, x0:x1], info

pano, info = stitch_pair(left_img, right_img, show_matches=True)
print(f"keypoints {info['kp']}, good matches after ratio test: {info['good']}, "
      f"RANSAC inliers: {info['inliers']} ({100 * info['inlier_ratio']:.0f}%)")
show(pano, f"Fixed panorama ({pano.shape[1]}x{pano.shape[0]})", size=11)
```




```
keypoints (1994, 2000), good matches after ratio test: 905, RANSAC inliers: 889 (98%)
```



8c · How much does filtering the matches help? (inlier ratio)

RANSAC can find the right homography even when many matches are wrong, but the more outliers there are, the more random samples it needs and the more fragile it gets. We compare the **inlier ratio** (fraction of matches that agree with the final H) for the course's "all cross-checked matches" and for the ratio test.

```
g1, g2 = cv2.cvtColor(left_img, cv2.COLOR_BGR2GRAY), cv2.cvtColor(right_img, cv2.COLOR_BGR2GRAY)
for nf in [500, 2000]:
    orb = cv2.ORB_create(nfeatures=nf)
    k1, d1 = orb.detectAndCompute(g1, None); k2, d2 = orb.detectAndCompute(g2, None)
    cc = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True).match(d2, d1)
    knn = cv2.BFMatcher(cv2.NORM_HAMMING).knnMatch(d2, d1, k=2)
    for label, ms in [("cross-check (course)", cc)] + [
            (f"ratio test {r}", [p[0] for p in knn if len(p) == 2 and p[0].distance < r *
p[1].distance]) for r in (0.7, 0.8)]:
        s = np.float32([k2[m.queryIdx].pt for m in ms]).reshape(-1, 1, 2)
        d = np.float32([k1[m.trainIdx].pt for m in ms]).reshape(-1, 1, 2)
        H, inl = cv2.findHomography(s, d, cv2.RANSAC, 5.0)
        print(f"nfeatures={nf:4d}  {label:22s} matches={len(ms):4d}  inliers={int(inl.sum()):4d}
inlier ratio={100 * inl.mean():5.1f}%")
```

```
nfeatures= 500  cross-check (course)   matches= 235  inliers= 206  inlier ratio= 87.7%
nfeatures= 500  ratio test 0.7         matches= 197  inliers= 194  inlier ratio= 98.5%
nfeatures= 500  ratio test 0.8         matches= 220  inliers= 205  inlier ratio= 93.2%
nfeatures=2000  cross-check (course)   matches=1026  inliers= 926  inlier ratio= 90.3%
nfeatures=2000  ratio test 0.7         matches= 874  inliers= 867  inlier ratio= 99.2%
nfeatures=2000  ratio test 0.8         matches= 944  inliers= 915  inlier ratio= 96.9%
```

Honest result: on this easy pair the course's cross-checked matches are already fairly clean (**88 % inliers** with 500 features, 90 % with 2000), so RANSAC finds the right homography anyway; the stitch failed only because of the warping bug, not because of bad matches. The ratio test raises the inlier ratio to **98.5 % (0.7)** and 93 % (0.8). On harder pairs (repetitive texture, small overlap) the difference grows, and a low inlier ratio (say < 30 %) is your warning sign that the homography may be wrong.

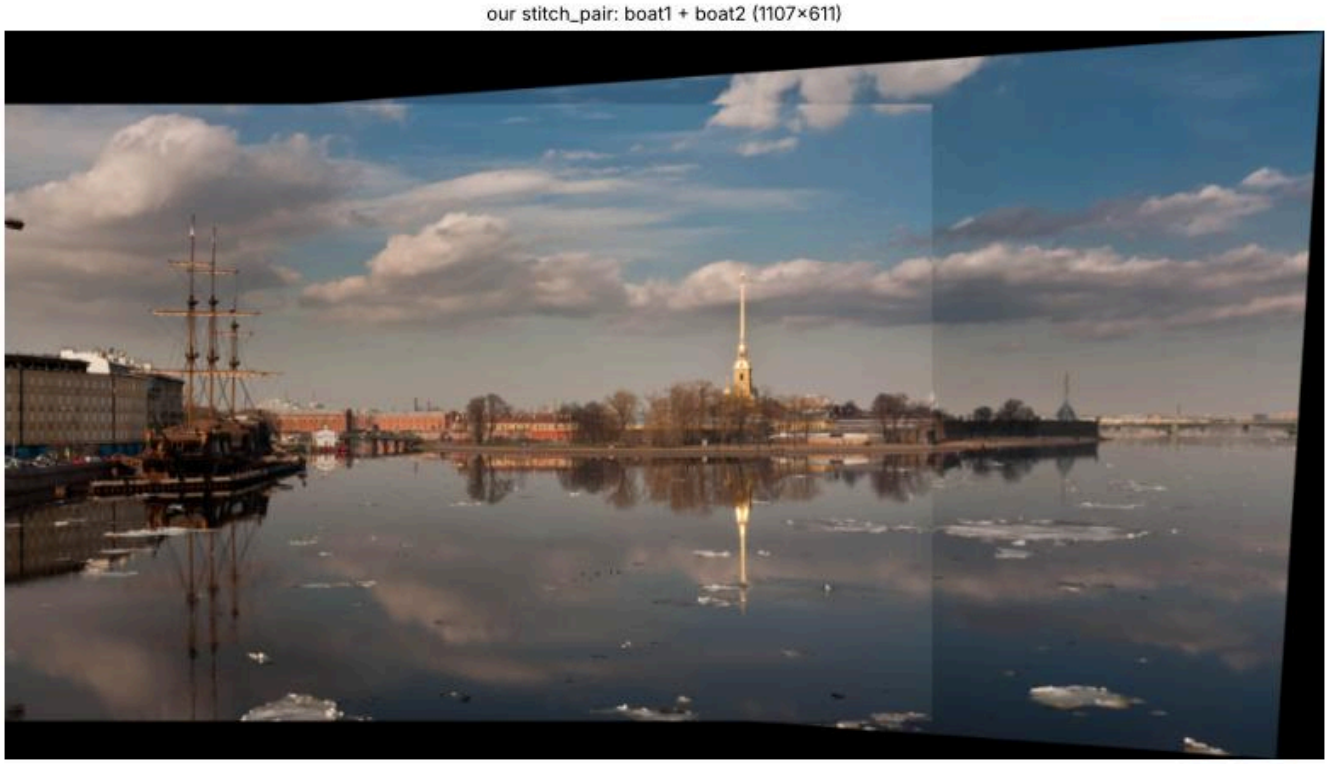
8d · A second pair, and a comparison with OpenCV's built-in `Stitcher`

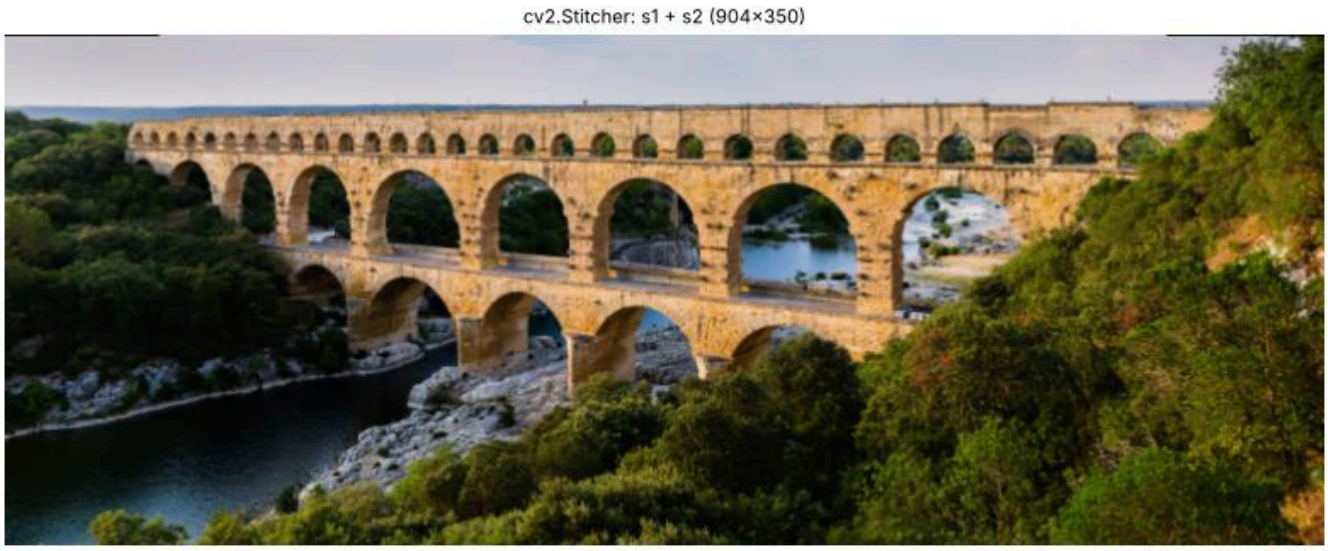
`boat1.jpg` / `boat2.jpg` (St Petersburg waterfront) are 3888×2592 photos; we shrink them to 20 %. OpenCV's `cv2.Stitcher` is a full pipeline (feature matching, bundle adjustment, cylindrical/spherical warping, exposure compensation, seam finding, multi-band blending), which is what you would use in a real project.

```
bl, br = load_small("boat1.jpg", 0.2), load_small("boat2.jpg", 0.2)
pano_b, info_b = stitch_pair(bl, br)
print(f"boat pair: good matches {info_b['good']}, inliers {info_b['inliers']} ({100 *
info_b['inlier_ratio']:.0f}%)")

t = time.perf_counter()
status, pano_cv = cv2.Stitcher_create(cv2.Stitcher_PANORAMA).stitch([left_img, right_img])
t_cv = time.perf_counter() - t
print("cv2.Stitcher status:", status, "(0 = OK)", f"in {t_cv:.1f}s")
show(pano_b, f"our stitch_pair: boat1 + boat2 ({pano_b.shape[1]}x{pano_b.shape[0]})", size=10)
if status == 0:
    show(pano_cv, f"cv2.Stitcher: s1 + s2 ({pano_cv.shape[1]}x{pano_cv.shape[0]})", size=10)
cv2.imwrite(os.path.join(OUT, "panorama_s1_s2.jpg"), pano)
print("saved", os.path.join(OUT, "panorama_s1_s2.jpg"))
```

```
boat pair: good matches 501, inliers 484 (97%)
cv2.Stitcher status: 0 (0 = OK) in 0.7s
```






```
saved output/panorama_s1_s2.jpg
```

What to notice:

- Our fixed `stitch_pair` produces a complete Pont du Gard panorama (left photo intact, right photo warped onto it); 889 of 905 ratio-test matches (98 %) are RANSAC inliers.
- The boat panorama is geometrically right (the spire and the buildings line up), but you can see a **vertical seam**, because the two photos have slightly different exposure, and black **wedges** where the warped right image does not fill the rectangle. Real stitchers fix both with exposure compensation and blending, and crop to the largest inner rectangle.
- `cv2.Stitcher` (about a second here) gives a seamless result on s1+s2 thanks to blending. For a lab, our simple version shows the idea; for a product, use `cv2.Stitcher`.

9 · 🔧 What we changed from the course notebook, and why

#	Course code	Problem (verified)	What we did
1	`cv2.imread('Ganesh3.jpg')`, `'blackdog.jpg'`, `'left.jpg'`, `'right.jpg'`	The images were never provided. `imread` returns `None` silently and the next line crashes with a confusing `cvtColor` assertion error	Download public OpenCV sample images with a cached `download()` helper; check `img is None` with a clear message
2	Panorama: `H` maps **left→right**, then `warpPerspective(img1, M, (w+w2, h))` and `result[0:h2, 0:w2] = img2`	With a real left/right pair the left image is warped to **negative x** and cut off, and the part that survives is overwritten by `img2`. The output is just the right photo plus black (also visible in the course's own saved output)	Estimate H **right→left**, compute the bounding box of both images, add a **translation** so nothing is negative, crop black borders
3	Panorama uses **all** cross-checked matches	Many are wrong; RANSAC hides it. No one looks at the inlier ratio	Lowe ratio test + print inliers / inlier ratio; draw only the inliers
4	Canvas size `(w + img2.shape[1], h)`	Assumes the images line up horizontally at the same height;	Canvas size computed from the warped corners

#	Course code	Problem (verified)	What we did
		anything warped above/below y=0 or below h is cut off	
5	Harris: `img[dst > 0.01*dst.max()] = red`	Marks *pixels*, not corners; dots are 1 px wide and nearly invisible on big images	Count corners as connected blobs; dilate the mask for display; show the effect of every parameter
6	Exercise: "try different images" with no measurement	—	Counted and timed all three detectors on 4 images; ORB also with a fair `nfeatures`
7	Claims "Harris not scale/rotation invariant…" with no evidence	Partly misleading: Harris *detection* is fairly rotation-stable (see §7); what fails is matching without a scale/orientation-aware descriptor	Measured repeatability and correct-match rate under known rotations and scalings
8	`plt.imshow` with default figure size	Small, hard-to-see results	Bigger figures, downscaled images (keeps the notebook small)

10 · Results, honestly

- **Harris / SIFT / ORB all work** on the replacement images; the course code for these three cells was correct apart from the missing files.
- **The panorama code in the course notebook is genuinely broken** for a left/right pair: it outputs only the right photo plus black (we reproduced it, and the course's own saved output shows the same thing). The fix is to warp the right image into the left image's frame on a translated canvas.
- **The lecture's invariance claims are confirmed by the matching test**, with one important nuance: Harris *detection* is rotation-stable; Harris fails under rotation only when you match raw patches. Under zoom Harris fails at both detection and matching.
- **"SIFT finds fewer keypoints than ORB" was not true here** with default settings: ORB's default cap of 500 decides the count.
- Timing numbers come from one shared CPU and varied between runs by a factor of up to about 3; the ranking (Harris < ORB < SIFT) never changed.
- Limitations: our simple stitcher pastes without blending, so exposure seams remain; a single homography is only correct for (near-)pure camera rotation or planar scenes.

11 · Common errors and how to fix them

Error / symptom	Cause	Fix
`error: (-215:Assertion failed) !_src.empty() in function 'cvtColor'`	`cv2.imread` returned `None` : wrong path or file missing (imread never raises)	Check `os.path.exists(path)` ; print `os.getcwd()` ; test `if img is None`
`AttributeError: module 'cv2' has no attribute 'SIFT_create'`	OpenCV older than 4.4	`pip install -U opencv-python` (or use `cv2.xfeatures2d.SIFT_create()` from `opencv-contrib-python` on old versions)
`cornerHarris` assertion about type	Input not `float32` / `uint8` single-channel	`np.float32(gray)` on a **grey** image
`des is None` / `bf.match` error "query descriptors empty"	No keypoints found (blank, tiny or very blurry image, or ORB's `edgeThreshold` border bigger than the image)	Check `len(kp)` ; use a larger or more textured image; lower `fastThreshold`
`bf.match` error about type/size	Mixing descriptor types, e.g. SIFT (float32) with `NORM_HAMMING`	SIFT/SURF → `NORM_L2` ; ORB/BRIEF/BRISK → `NORM_HAMMING`
`findHomography` returns `None`	Fewer than 4 matches, or all points collinear	Check `len(matches) >= 4` ; more features; better overlap
Panorama shows only one image / black half	Homography direction vs paste position (§8b)	Map the right image into the left frame, translated canvas
Panorama is badly skewed or "folded"	Most matches are wrong (little overlap, repetitive texture) or the scene is not planar and the camera moved	Ratio test, check inlier ratio, take photos by rotating the camera about one spot with 30–50 % overlap
Colours look blue/orange-swapped	Showing a BGR image with matplotlib	`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

12 · Try this yourself

1. Take two photos of your room with your phone, rotating (not walking) about 20° between them, with about 40 % overlap. Stitch them with `stitch_pair` . Then try **walking** sideways between shots; why is the result worse? (Hint: parallax; a single homography only fits a plane or a pure rotation.)
2. In §7, add a **blur** or **brightness** change (`cv2.GaussianBlur` , `cv2.convertScaleAbs(g, alpha=1.5)`) instead of rotation. Which detector is most robust?
3. Replace ORB by SIFT in `stitch_pair` (remember `NORM_L2`). Compare the inlier count and run time.
4. Run Harris with `cv2.goodFeaturesToTrack(gray, 100, 0.01, 10, useHarrisDetector=True)` : you get a clean list of 100 corners with non-maximum suppression built in.
5. Stitch three images (`boat1` , `boat2` , `boat3`) by stitching the first two, then stitching the result with the third. What goes wrong as the panorama grows?

6. Add simple blending: in the overlap, average the two images (or use a linear "feather" weight) and compare the seam.

13 · Viva questions (with short model answers)

1. **What is a feature/keypoint, and what makes a good one?** A distinctive, repeatable point (corner/blob) that can be found again in another image of the same scene; good = distinctive, repeatable under viewpoint/scale/lighting change, well localised.
2. **Detector vs descriptor?** Detector finds *where* (x, y, scale, angle); descriptor encodes *what the neighbourhood looks like* as a vector so points can be compared. Harris is only a detector; SIFT/ORB are both.
3. **Why are corners better than edges?** At a corner the image changes in every direction, so the position is unique; along an edge it looks the same, so you can't tell where you are (the *aperture problem*).
4. **Explain the Harris response.** Build the 2×2 structure tensor M from gradients in a window; $R = \det(M) - k\cdot\mathrm{trace}(M)^2 = \lambda1\lambda2 - k(\lambda1+\lambda2)^2$. $R \gg 0$ corner, $R < 0$ edge, $|R|$ small flat.
5. **What do `blockSize`, `ksize` and `k` in `cornerHarris(gray, 2, 3, 0.04)` mean?** Window size for M (2), Sobel aperture for gradients (3), the empirical constant k (0.04–0.06; higher → fewer corners).
6. **Why `dst > 0.01*dst.max()`?** It keeps pixels whose response is at least 1 % of the strongest; it is a *relative* threshold, so lower → more corners. It marks pixels, not individual corners.
7. **Is Harris rotation invariant?** The corner *response* uses eigenvalues, which do not change when the image rotates, so detection is largely rotation-stable (we measured this). But Harris has no orientation or scale, so a raw-patch comparison fails under rotation, and corners appear/disappear when the image is zoomed: it is **not scale invariant**.
8. **How does SIFT achieve scale and rotation invariance?** Scale: searches for DoG extrema across a Gaussian scale space and assigns each keypoint a characteristic scale. Rotation: assigns a dominant gradient orientation and computes the descriptor relative to it.
9. **What is in a SIFT descriptor?** 16×16 region → 4×4 cells × 8-bin orientation histograms = 128 floats, normalised (illumination robustness).
10. **What does ORB stand for and why is it fast?** Oriented FAST and Rotated BRIEF: FAST corners (simple pixel-ring test), BRIEF binary descriptor (256 intensity comparisons, 32 bytes) compared with Hamming distance (XOR + bit count).
11. **Why does `ORB_create()` always give 500 keypoints?** `nfeatures=500` is the default cap; change it with `ORB_create(nfeatures=N)`.
12. **ORB vs SIFT?** ORB: free, much faster (5–36× in our runs), binary, rotation invariant, limited scale range (8-level pyramid). SIFT: slower, float descriptors, more robust to large scale changes (our test: correct matches stay ≈ 95 % at 3× zoom, ORB ≈ 50 %).
13. **What is a homography and when is it valid for stitching?** A 3×3 projective transform (8 DOF) between two image planes; valid for a planar scene or a camera rotating about its optical centre. Needs ≥ 4 point pairs.
14. **What does RANSAC do and what is the 5.0 in `findHomography(..., cv2.RANSAC, 5.0)`?** Repeatedly fits H to 4 random matches and keeps the H with most inliers; 5.0 is the reprojection threshold in pixels for a match to count as an inlier.

15. **What was wrong with the course panorama code?** H mapped the left image into the right image's frame, where it lies at negative x and is clipped; the remainder is overwritten by pasting img2. Fix: warp the right image into the left frame (or translate the canvas).
16. **What is `crossCheck=True`?** Keep match i↔j only if j is i's best match *and* i is j's best match; removes many one-sided wrong matches. Cannot be combined with `knnMatch` ratio test (k=2).

Day 9 · Image matching (descriptors, FLANN, Lowe's ratio test) + mini project: logo detector

Course lab: `CV-Week2-Lecture2.ipynb` (Day 9) **Notes chapter:** `week-2/notes/02_image_matching_flann_logo_detector.md` **Builds on:** Day 8 (`day08_feature_detection_panorama.ipynb`): keypoints, SIFT, ORB, homography, RANSAC.

What you will learn

1. What a descriptor is, how two descriptors are compared (L2 vs Hamming distance) and how matches are found (brute force vs **FLANN**).
2. **Lowe's ratio test**: why `m.distance < 0.7 * n.distance` throws away most wrong matches.
3. The course code with a real logo/scene pair, every parameter explained, and a crash in the course code that appears with small images (and how to guard against it).
4. Measured experiments: ratio threshold sweep, ORB vs SIFT, brute force vs FLANN speed, what happens when the logo is **not** in the scene.
5. Mini project: a **logo detector** that draws a box around the logo, tested with known ground truth (rotation, zoom, perspective) and on scenes that do *not* contain the logo.

Run time: about 15–30 seconds on a laptop CPU. All numbers in the text come from the outputs below.

1 · Theory recap in plain language

Matching = finding the same physical point in two images. Day 8 gave us keypoints (*where*) and descriptors (*what it looks like*). Matching compares descriptors: two keypoints "match" if their descriptors are close.

How close? It depends on the descriptor type:

Descriptor	Type	Size	Distance	OpenCV norm
SIFT	128 floats	512 bytes	Euclidean (L2): $\sqrt{\Sigma(a_i-b_i)^2}$	`cv2.NORM_L2`
ORB / BRIEF	256 bits	32 bytes	**Hamming**: number of bits that differ (XOR + count)	`cv2.NORM_HAMMING`

Finding the nearest descriptor:

- **Brute force (`BFMatcher`)**: compare every descriptor in image A with every descriptor in image B. Exact, simple, cost grows as N×M.
- **FLANN** (*Fast Library for Approximate Nearest Neighbours*): first builds an **index** (a search structure), then answers queries quickly but **approximately** (it may occasionally miss the true nearest neighbour). Like looking a word up in a dictionary by its first letters rather than reading every page.

- for float descriptors (SIFT): **KD-trees** (`algorithm=1`): repeatedly split the 128-D space in half along one dimension; several randomised trees are searched in parallel.
- for binary descriptors (ORB): **LSH**, *Locality Sensitive Hashing* (`algorithm=6`): hash random subsets of bits into buckets, so similar bit strings land in the same bucket; only that bucket is searched.

Lowe's ratio test (SIFT paper, 2004). For each descriptor, find its **2** nearest neighbours m (best) and n (second best). Keep the match only if `m.distance < 0.7 × n.distance`. *Analogy:* you're looking for your friend in a crowd. If one face looks clearly more like them than any other face, it's probably them. If two faces look almost equally similar, you can't be sure, so you don't wave at either. Repetitive patterns (windows, bricks, text) produce exactly these ambiguous "two similar candidates" cases.

Homography + RANSAC (from Day 8): a flat logo seen from any viewpoint is related to its image in the scene by a 3×3 homography H. With ≥ 4 good matches, RANSAC finds the H that most matches agree with, ignoring wrong matches (outliers). Projecting the logo's 4 corners with H gives the box to draw.

2 · Setup and data

The course code loads `left.jpg` and `right.jpg` (two photos of books, *not* a logo and a scene), and the exercise text mentions `logo.jpg` and `scene.jpg`; **none of these files were provided**. We use the classic OpenCV pair made for exactly this task: `box.png` (a cereal box, our "logo") and `box_in_scene.png` (the box in a cluttered scene, rotated and partly hidden). We also use the OpenCV logo and several images that do **not** contain the box.

```
%matplotlib inline
%config InlineBackend.figure_formats = ["jpeg"]
import time
import numpy as np
import matplotlib.pyplot as plt
import cv2

plt.rcParams["figure.dpi"] = 80
print("OpenCV version:", cv2.__version__)

import os, urllib.request

DATA, OUT = "data", "output"            # data/ and output/ sit next to this notebook (both are
git-ignored)
os.makedirs(DATA, exist_ok=True)
os.makedirs(OUT, exist_ok=True)

OPENCV_SAMPLES = "https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/"
OPENCV_STITCH  = "https://raw.githubusercontent.com/opencv/opencv_extra/4.x/testdata/stitching/"

def download(url, path):
    """Download url to path once; later runs reuse the cached file."""
    if os.path.exists(path) and os.path.getsize(path) > 0:
        return path
    print("downloading", url)
    tmp = path + ".part"
    try:
        urllib.request.urlretrieve(url, tmp)
        os.replace(tmp, path)
    except Exception as e:
```

```
        raise RuntimeError(
            f"Could not download {url}\n({e})\n"
            f"Are you offline? Open the URL in a browser, save the file as\n
{os.path.abspath(path)}\n"
            "and re-run this cell. (Or use any image/video of your own and change the file
name.)") from e
    return path

def to_rgb(img):
    """OpenCV stores colour as B,G,R; matplotlib expects R,G,B. Grey images are left alone."""
    return img if img.ndim == 2 else cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

def show(images, titles=None, cols=None, size=4.5, max_side=900):
    """Show one or more OpenCV images side by side (downscaled for a small notebook file)."""
    if not isinstance(images, (list, tuple)):
        images, titles = [images], [titles]
    cols = cols or len(images)
    rows = (len(images) + cols - 1) // cols
    aspect = max(im.shape[0] / im.shape[1] for im in images)          # height/width of the
tallest image
    fig, axes = plt.subplots(rows, cols, figsize=(size * cols, size * rows * aspect + 0.35 *
rows), squeeze=False)
    for ax in axes.ravel():
        ax.axis("off")
    for i, img in enumerate(images):
        s = min(1.0, max_side / max(img.shape[:2]))
        if s < 1:
            img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
        ax = axes.ravel()[i]
        ax.imshow(to_rgb(img), cmap="gray" if img.ndim == 2 else None)
        if titles and titles[i]:
            ax.set_title(titles[i], fontsize=10)
    plt.tight_layout()
    plt.show()

for name in ["box.png", "box_in_scene.png", "opencv-logo.png", "messi5.jpg", "building.jpg",
             "home.jpg", "left01.jpg", "graf1.png", "graf3.png"]:
    download(OPENCV_SAMPLES + name, os.path.join(DATA, name))

def imread_checked(path, flags=cv2.IMREAD_COLOR):
    img = cv2.imread(path, flags)
    if img is None:
        raise FileNotFoundError(f"cv2.imread could not read {path}")
    return img

show([imread_checked(os.path.join(DATA, "box.png")), imread_checked(os.path.join(DATA,
"box_in_scene.png"))],
     ["logo (box.png)", "scene (box_in_scene.png)"], size=5)
```

```
OpenCV version: 5.0.0
```





3 · The course code: ORB + FLANN (LSH) + ratio test

Same calls, same parameters and the same variable names as the course; only the file names changed, and the display line is corrected (see the comment).

```
# Load two images (logo and scene) in greyscale: descriptors are computed on brightness only
logo = cv2.imread(os.path.join(DATA, 'box.png'), cv2.IMREAD_GRAYSCALE)            # course:
'left.jpg'
scene = cv2.imread(os.path.join(DATA, 'box_in_scene.png'), cv2.IMREAD_GRAYSCALE)  # course:
'right.jpg'

# Initialize ORB detector (default nfeatures=500)
orb = cv2.ORB_create()
kp1, des1 = orb.detectAndCompute(logo, None)     # kp1: keypoints of the logo, des1: (N1, 32)
uint8
kp2, des2 = orb.detectAndCompute(scene, None)    # kp2/des2: same for the scene

# FLANN parameters for ORB (using LSH index, because ORB descriptors are binary)
FLANN_INDEX_LSH = 6
index_params = dict(algorithm=FLANN_INDEX_LSH,
                    table_number=6,       # number of hash tables (more = more accurate, more
memory)
                    key_size=12,          # bits per hash key (longer = smaller, purer buckets)
                    multi_probe_level=1)  # also search neighbouring buckets (0 = only the exact
bucket)
search_params = dict(checks=50)          # how many candidates/leaves to examine per query
(accuracy vs speed)

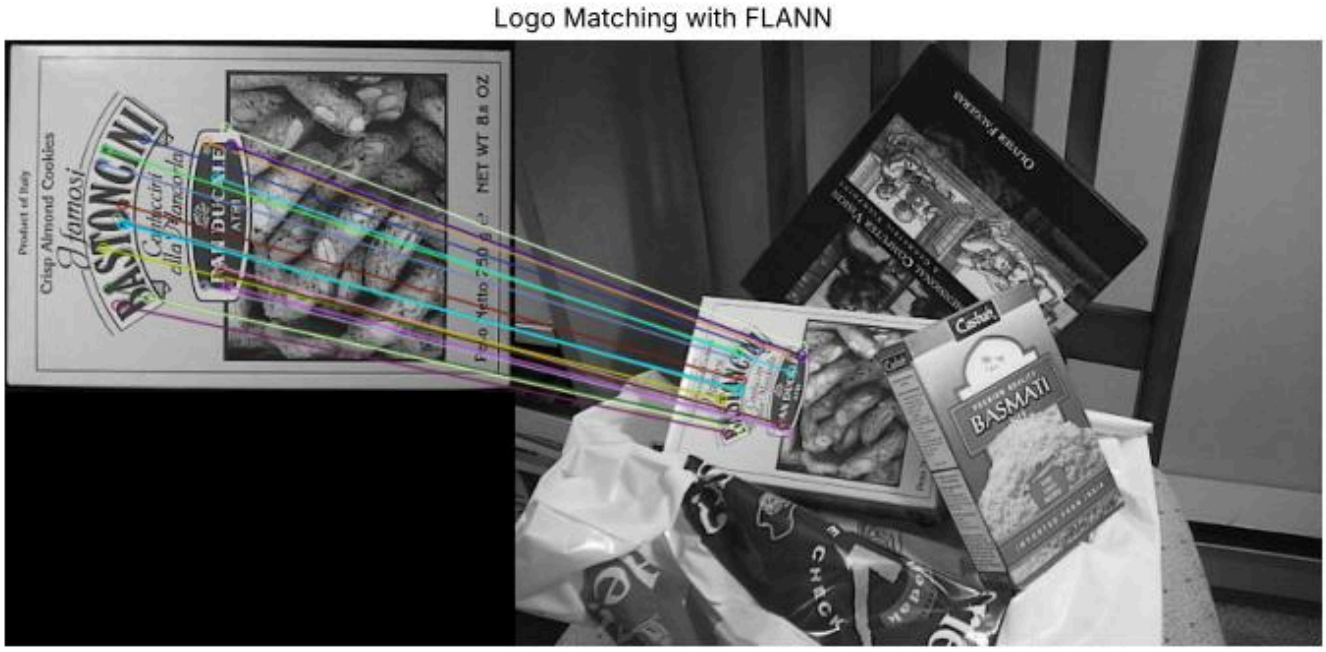
flann = cv2.FlannBasedMatcher(index_params, search_params)
matches = flann.knnMatch(des1, des2, k=2)   # for each logo descriptor: its 2 nearest scene
descriptors

# Apply Lowe's ratio test
good = []
for m,n in matches:                          # (!) crashes if a descriptor got fewer than 2
neighbours, see section 4
    if m.distance < 0.7*n.distance:          # best clearly better than second best -> keep
        good.append(m)

img_matches = cv2.drawMatches(logo, kp1, scene, kp2, good, None, flags=2)
print(f"keypoints: logo {len(kp1)}, scene {len(kp2)} | knn pairs {len(matches)} | good after
ratio test: {len(good)}")
print("drawMatches output:", img_matches.shape, "-> a 3-channel BGR image, even though the inputs
were grey")

plt.figure(figsize=(11, 5))
plt.imshow(cv2.cvtColor(img_matches, cv2.COLOR_BGR2RGB))   # course: plt.imshow(img_matches)
(BGR shown as RGB)
plt.title("Logo Matching with FLANN")
plt.axis('off')
plt.show()
```

```
keypoints: logo 453, scene 500 | knn pairs 453 | good after ratio test: 20
drawMatches output: (384, 836, 3) -> a 3-channel BGR image, even though the inputs were grey
```



The parameters, one by one

Parameter	Value	What it does
`algorithm=6` (`FLANN_INDEX_LSH`)	LSH	index for **binary** descriptors (ORB, BRIEF, BRISK). For SIFT use `algorithm=1` (KD-tree) with `trees=5`
`table_number=6`	6 hash tables	each table hashes a different random subset of bits; more tables → a true neighbour is less likely to be missed. OpenCV's tutorial suggests 6 (or 12)
`key_size=12`	12 bits	length of each hash key (out of 256 bits); suggested 12–20
`multi_probe_level=1`	1	also look in buckets whose key differs by 1 bit; 0 = exact bucket only (faster, misses more); suggested 1–2
`checks=50`	50	how many candidates to check per query; larger → closer to brute-force accuracy, slower
`k=2`	2 neighbours	needed for the ratio test (best *and* second best)
`0.7`	ratio	Lowe used 0.8 for SIFT; 0.7 is stricter (fewer but cleaner matches). We sweep it in §6
`flags=2`	`NOT_DRAW_SINGLE_POINTS`	draw only matched keypoints

About the display line: `drawMatches` always returns a **BGR colour** image (it converts grey inputs so it can draw coloured lines). `plt.imshow(img_matches)` shows it as if it were RGB, so the red and blue channels are swapped. Here the photo is grey and the line colours are random, so you can't see the mistake, but with colour inputs the colours are wrong. Always convert with `cv2.cvtColor(..., cv2.COLOR_BGR2RGB)` before `plt.imshow`.

4 · A crash hidden in `for m, n in matches`

`knnMatch(k=2)` returns a list with one entry per query descriptor, and each entry is *supposed* to hold 2 matches. With **brute force** that is always true (if the scene has ≥ 2 descriptors). With **FLANN-LSH** it is **not guaranteed**: if the hash buckets around a descriptor contain fewer than 2 candidates, you get 1 or even 0 neighbours, and `for m, n in matches` raises `ValueError: not enough values to unpack`. This happens most easily when the scene has **few descriptors** (a small or low-texture image). Let's provoke it with a small (100×80) version of the scene:

```
from collections import Counter

small_scene = cv2.resize(scene, (100, 80))
kp_s, des_s = orb.detectAndCompute(small_scene, None)
matches_small = flann.knnMatch(des1, des_s, k=2)
print("scene descriptors:", len(kp_s))
print("neighbours returned per logo descriptor:", dict(sorted(Counter(len(p) for p in
matches_small).items())))
print("on the full scene:                     ", dict(Counter(len(p) for p in matches)))

try:
    good_small = []
    for m, n in matches_small:                      # the course loop
        if m.distance < 0.7 * n.distance:
            good_small.append(m)
except ValueError as e:
    print("\nCourse loop crashes ->", type(e).__name__ + ":", e)

# The fix: only unpack pairs that really have 2 neighbours
good_small = [p[0] for p in matches_small if len(p) == 2 and p[0].distance < 0.7 * p[1].distance]
print("guarded loop works -> good matches:", len(good_small))
```

```
scene descriptors: 59
neighbours returned per logo descriptor: {0: 76, 1: 115, 2: 262}
on the full scene:                      {2: 453}

Course loop crashes -> ValueError: not enough values to unpack (expected 2, got 0)
guarded loop works -> good matches: 10
```

Two more defensive checks belong in every matching script:

- `des is None` when an image has **no** keypoints at all (blank image, tiny image). `knnMatch(None, ...)` raises an error, so check first.
- FLANN with `NORM`-mismatched index: SIFT descriptors with an LSH index (or ORB with a KD-tree) fail with an error about the descriptor type. KD-tree needs `float32`, LSH needs `uint8`.

From now on we use this helper, which does everything above safely:

```
FLANN_INDEX_KDTREE, FLANN_INDEX_LSH = 1, 6

def make_detector(method="orb", nfeatures=500):
    return cv2.SIFT_create() if method == "sift" else cv2.ORB_create(nfeatures=nfeatures)

def make_flann(method="orb"):
    if method == "sift":   # float descriptors -> randomised KD-trees
        return cv2.FlannBasedMatcher(dict(algorithm=FLANN_INDEX_KDTREE, trees=5),
dict(checks=50))
    return cv2.FlannBasedMatcher(dict(algorithm=FLANN_INDEX_LSH, table_number=6, key_size=12,
```

```
                                    multi_probe_level=1), dict(checks=50))

def ratio_matches(des_a, des_b, method="orb", ratio=0.7):
    """FLANN knnMatch + Lowe's ratio test, safe for None descriptors and <2 neighbours."""
    if des_a is None or des_b is None or len(des_a) == 0 or len(des_b) < 2:
        return []
    knn = make_flann(method).knnMatch(des_a, des_b, k=2)
    return [p[0] for p in knn if len(p) == 2 and p[0].distance < ratio * p[1].distance]

def ransac_homography(kp_a, kp_b, good, thresh=5.0):
    """Returns H (or None), the inlier mask and the number of inliers."""
    if len(good) < 4:
        return None, None, 0
    src = np.float32([kp_a[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp_b[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, mask = cv2.findHomography(src, dst, cv2.RANSAC, thresh)
    if H is None:
        return None, None, 0
    return H, mask.ravel().astype(bool), int(mask.sum())
```

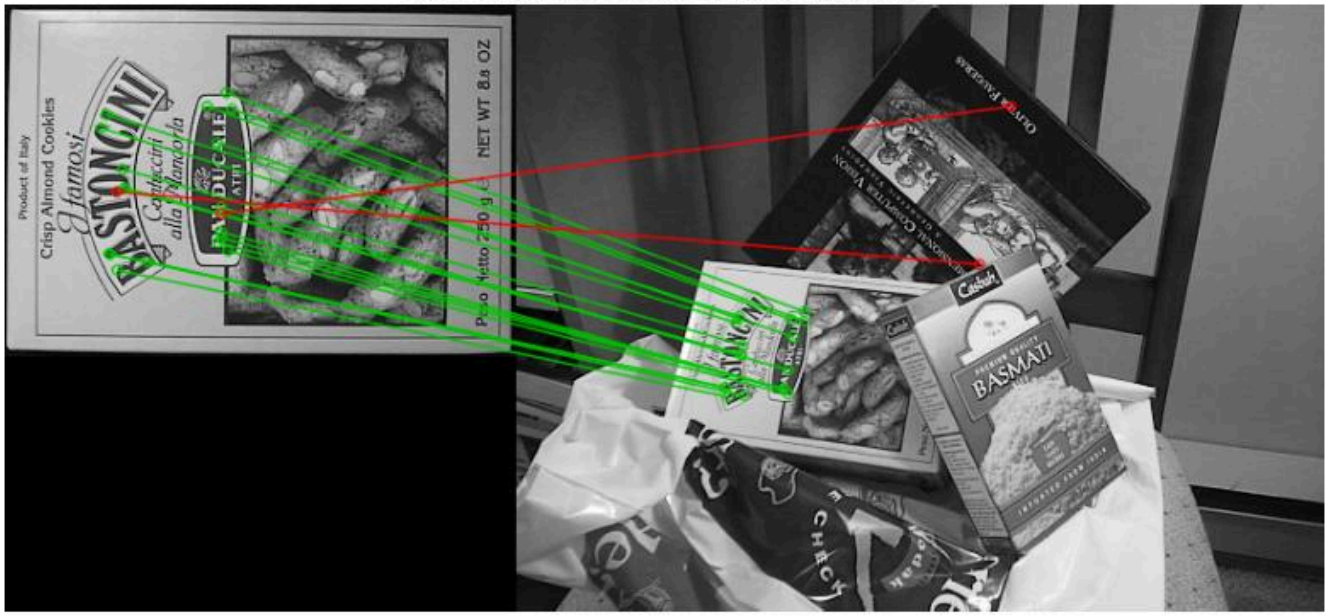
5 · Exercise: match the logo, count the matches, judge their accuracy

The course exercise: *observe how many matches are found and how accurate they are*. "Accurate" needs a definition, so we use the RANSAC **inliers**: matches that agree with one common homography. A match that lands somewhere else in the scene cannot agree with it. Green = inlier, red = outlier.

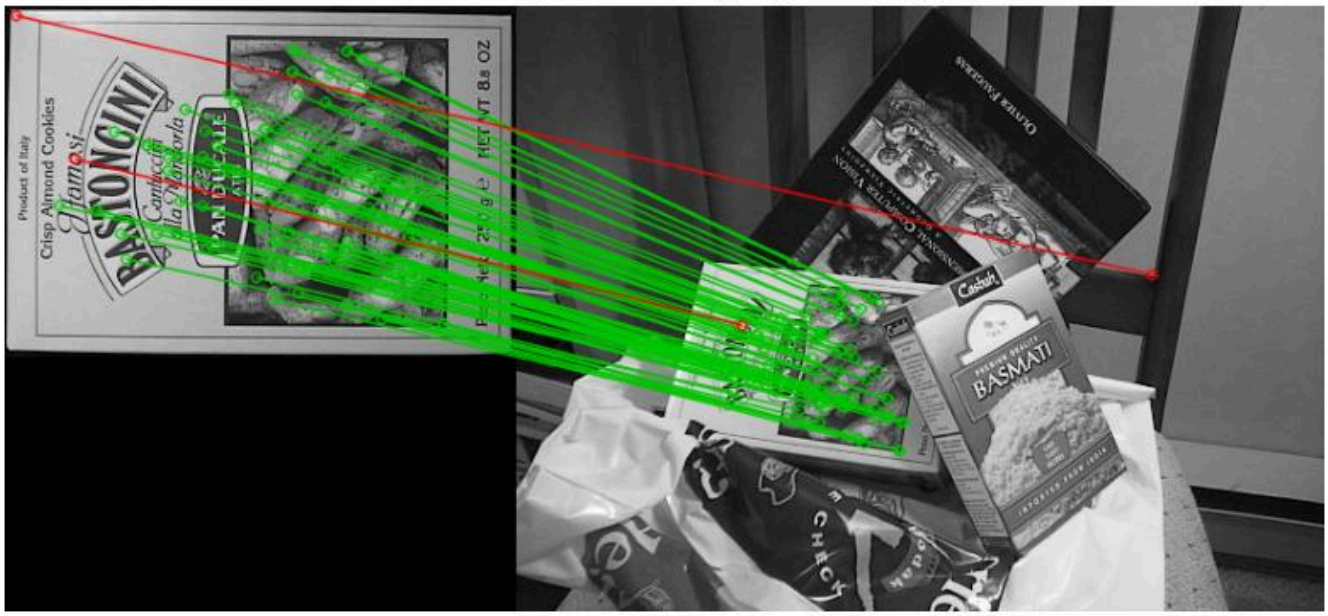
```
logo_c, scene_c = imread_checked(os.path.join(DATA, "box.png")),
imread_checked(os.path.join(DATA, "box_in_scene.png"))
panels, titles = [], []
for method in ["orb", "sift"]:
    det = make_detector(method)
    t = time.perf_counter()
    ka, da = det.detectAndCompute(logo, None)
    kb, db = det.detectAndCompute(scene, None)
    t_det = time.perf_counter() - t
    t = time.perf_counter()
    good_m = ratio_matches(da, db, method, 0.7)
    t_match = time.perf_counter() - t
    H, inl, n_in = ransac_homography(ka, kb, good_m)
    print(f"{method.upper():4s}: keypoints {len(ka)}/{len(kb)}, good {len(good_m)}, inliers
{n_in} "
          f"({100 * n_in / max(1, len(good_m)):.0f}%), detect+describe {1000 * t_det:.0f} ms,
match {1000 * t_match:.1f} ms")
    vis = cv2.drawMatches(logo_c, ka, scene_c, kb, [m for m, i in zip(good_m, inl) if i], None,
                          matchColor=(0, 200, 0), flags=2)
    vis = cv2.drawMatches(logo_c, ka, scene_c, kb, [m for m, i in zip(good_m, inl) if not i],
vis,
                          matchColor=(0, 0, 255), flags=2 |
cv2.DrawMatchesFlags_DRAW_OVER_OUTIMG)
    panels.append(vis); titles.append(f"{method.upper()} + FLANN, ratio 0.7: {n_in} inliers
(green), {len(good_m) - n_in} outliers (red)")
show(panels, titles, cols=1, size=10)
```

```
ORB : keypoints 453/500, good 20, inliers 18 (90%), detect+describe 7 ms, match 1.6 ms
SIFT: keypoints 604/969, good 75, inliers 73 (97%), detect+describe 83 ms, match 23.4 ms
```

ORB + FLANN, ratio 0.7: 18 inliers (green), 2 outliers (red)



SIFT + FLANN, ratio 0.7: 73 inliers (green), 2 outliers (red)



5a · How the ratio threshold changes the matches (0.5 … 0.95)

For each threshold we count the good matches and how many survive RANSAC. The **inlier %** tells us how clean the set is.

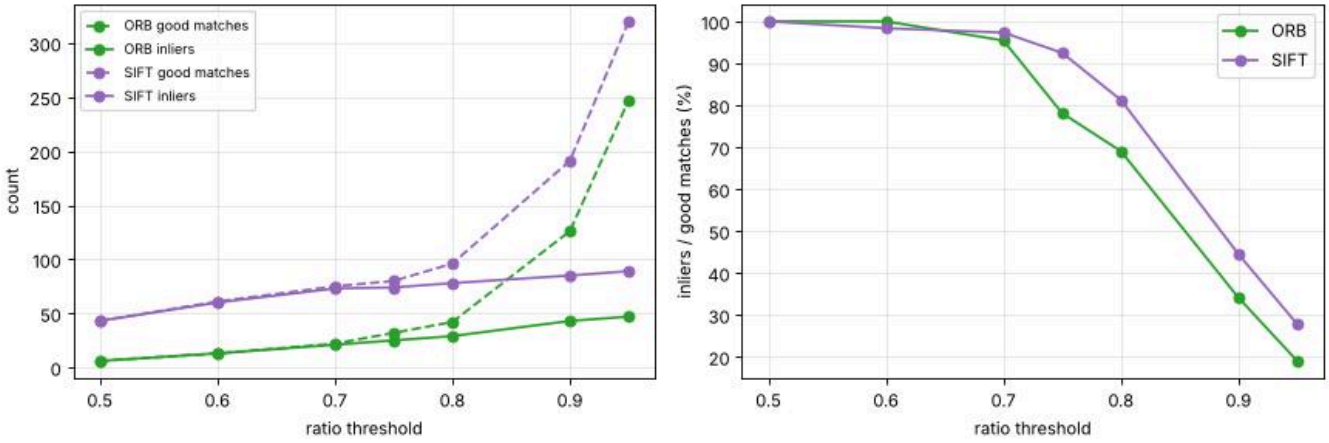
```
ratios = [0.5, 0.6, 0.7, 0.75, 0.8, 0.9, 0.95]
sweep = {}
for method in ["orb", "sift"]:
    det = make_detector(method)
    ka, da = det.detectAndCompute(logo, None); kb, db = det.detectAndCompute(scene, None)
    print(f"\n{method.upper()}   ratio   good  inliers  inlier%")
    for r in ratios:
        g = ratio_matches(da, db, method, r)
        _, _, n_in = ransac_homography(ka, kb, g)
        sweep[(method, r)] = (len(g), n_in)
        print(f"        {r:4.2f}  {len(g):5d}  {n_in:7d}  {100 * n_in / max(1, len(g)):6.0f}%")

fig, ax = plt.subplots(1, 2, figsize=(11, 3.8))
for method, c in [("orb", "tab:green"), ("sift", "tab:purple")]:
    g = [sweep[(method, r)][0] for r in ratios]; i = [sweep[(method, r)][1] for r in ratios]
    ax[0].plot(ratios, g, "--o", color=c, label=f"{method.upper()} good matches")
```

```
    ax[0].plot(ratios, i, "-o", color=c, label=f"{method.upper()} inliers")
    ax[1].plot(ratios, [100 * b / max(1, a) for a, b in zip(g, i)], "-o", color=c,
label=method.upper())
ax[0].set_xlabel("ratio threshold"); ax[0].set_ylabel("count"); ax[0].legend(fontsize=8);
ax[0].grid(alpha=.3)
ax[1].set_xlabel("ratio threshold"); ax[1].set_ylabel("inliers / good matches (%)");
ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

```
ORB    ratio   good  inliers  inlier%
         0.50      6        6     100%
         0.60     13       13     100%
         0.70     22       21      95%
         0.75     32       25      78%
         0.80     42       29      69%
         0.90    126       43      34%
         0.95    247       47      19%

SIFT   ratio   good  inliers  inlier%
         0.50     43       43     100%
         0.60     61       60      98%
         0.70     75       73      97%
         0.75     80       74      92%
         0.80     96       78      81%
         0.90    191       85      45%
         0.95    320       89      28%
```



Reading the sweep:

- At **0.5–0.6** almost every surviving match is correct (98–100 % inliers), but there are very few (ORB: 6–13).
- At **0.7** (course value) ORB keeps 22 matches with 95 % inliers; SIFT keeps 75 with 97 %.
- From **0.8** upwards the number of "good" matches shoots up (ORB 42 → 126 → 247) but the number of *inliers* barely grows (29 → 43 → 47): **the extra matches are mostly wrong**. At 0.95 only 19 % (ORB) and 28 % (SIFT) of the matches are correct.
- So the ratio test is a trade-off: low ratio = few, clean matches; high ratio = many, dirty matches. **0.7–0.8 is the sweet spot**, which is why Lowe suggested 0.8 and the course uses 0.7.
- SIFT gives **3–4× more inliers** than ORB on this pair at every ratio. The box in the scene is smaller and rotated, and SIFT handles scale change better (see Day 8).
- **FLANN is randomised.** The course cell in §3 found 20 good ORB matches at 0.7, and the sweep found 22 with the same images and settings, because each FLANN-LSH index is built with random hash functions. Re-running a cell can change the counts slightly; the conclusions don't change.

5b · Brute force vs FLANN: is FLANN really faster?

FLANN's advantage appears with **many** descriptors. We match SIFT descriptors of two 800×640 graffiti photos (`graf1.png` , `graf3.png` , a strong viewpoint change) with both matchers. We also check how many of FLANN's nearest neighbours are the true nearest neighbour (its *approximation* error).

```
g1 = imread_checked(os.path.join(DATA, "graf1.png"), cv2.IMREAD_GRAYSCALE)
g3 = imread_checked(os.path.join(DATA, "graf3.png"), cv2.IMREAD_GRAYSCALE)
sift = cv2.SIFT_create()
k1, d1 = sift.detectAndCompute(g1, None); k3, d3 = sift.detectAndCompute(g3, None)
print(f"SIFT descriptors: {len(d1)} x {len(d3)}  -> brute force computes {len(d1) * len(d3) /
1e6:.1f} million distances")

def time_it(fn, reps=3):
    ts = []
    for _ in range(reps):
        t = time.perf_counter(); out = fn(); ts.append(time.perf_counter() - t)
    return out, 1000 * min(ts)

bf_knn, t_bf = time_it(lambda: cv2.BFMatcher(cv2.NORM_L2).knnMatch(d1, d3, k=2))
fl_knn, t_fl = time_it(lambda: make_flann("sift").knnMatch(d1, d3, k=2))
same = np.mean([a[0].trainIdx == b[0].trainIdx for a, b in zip(bf_knn, fl_knn)])
print(f"BFMatcher (exact):     {t_bf:6.0f} ms")
print(f"FLANN KD-tree (5 trees, checks=50): {t_fl:6.0f} ms  (includes building the index)")
print(f"FLANN found the same nearest neighbour as brute force for {100 * same:.0f}% of
descriptors")
for name, knn in [("BF", bf_knn), ("FLANN", fl_knn)]:
    gm = [p[0] for p in knn if len(p) == 2 and p[0].distance < 0.7 * p[1].distance]
    _, _, n_in = ransac_homography(k1, k3, gm)
    print(f"{name:5s}: {len(gm)} good matches after ratio test, {n_in} RANSAC inliers")

kb_o, db_o = cv2.ORB_create(nfeatures=5000).detectAndCompute(g1, None)
kc_o, dc_o = cv2.ORB_create(nfeatures=5000).detectAndCompute(g3, None)
_, t_bf_o = time_it(lambda: cv2.BFMatcher(cv2.NORM_HAMMING).knnMatch(db_o, dc_o, k=2))
_, t_fl_o = time_it(lambda: make_flann("orb").knnMatch(db_o, dc_o, k=2))
print(f"\nORB {len(db_o)} x {len(dc_o)}:  BF Hamming {t_bf_o:.0f} ms | FLANN LSH {t_fl_o:.0f}
ms")
```

```
SIFT descriptors: 2665 x 3498  -> brute force computes 9.3 million distances
BFMatcher (exact):        208 ms
FLANN KD-tree (5 trees, checks=50):     56 ms  (includes building the index)
FLANN found the same nearest neighbour as brute force for 82% of descriptors
BF   : 378 good matches after ratio test, 344 RANSAC inliers
FLANN: 395 good matches after ratio test, 357 RANSAC inliers
ORB 5000 x 5000:  BF Hamming 209 ms | FLANN LSH 67 ms
```

Reading the timings:

- With ~2,700 × ~3,500 SIFT descriptors, FLANN (KD-tree) was several times faster than brute force (see the printed times; they vary from run to run with machine load). For ORB with 5000 × 5000 descriptors, LSH was also several times faster than brute-force Hamming.
- FLANN is **approximate**: only 82 % of its nearest neighbours were the true nearest neighbour. After the ratio test and RANSAC this hardly matters (395 vs 378 good matches, 357 vs 344 inliers). The neighbours it gets "wrong" are mostly ambiguous descriptors that the ratio test would reject anyway.
- For a small logo (a few hundred descriptors) matching is quick either way (§5: 1–2 ms for ORB and a few tens of milliseconds for SIFT, including building the FLANN index). FLANN pays off for **large** descriptor sets or databases of many images. Detection and description usually cost

more than matching: in §5, SIFT needed more than 10× longer than ORB to detect and describe the box and the scene (we measured 82–202 ms for SIFT and 7–9 ms for ORB across runs).

5c · What if the logo is NOT in the scene?

A detector that always "finds" something is useless. We match the box against five scenes that do **not** contain it and look at the numbers the course code uses to decide (`len(good) > 10`).

```
negatives = ["messi5.jpg", "building.jpg", "home.jpg", "left01.jpg", "graf1.png"]
neg_rows = []
print(f"{'scene':13s} {'method':6s} {'ratio':>5s} {'good':>5s} {'inliers':>7s} {'inlier%':>7s}
course rule (good > 10)")
for name in negatives:
    sc = imread_checked(os.path.join(DATA, name), cv2.IMREAD_GRAYSCALE)
    for method in ["orb", "sift"]:
        det = make_detector(method)
        ka, da = det.detectAndCompute(logo, None); kb, db = det.detectAndCompute(sc, None)
        for r in [0.7, 0.8, 0.9]:
            g = ratio_matches(da, db, method, r)
            _, _, n_in = ransac_homography(ka, kb, g)
            neg_rows.append((name, method, r, len(g), n_in))
            print(f"{name:13s} {method:6s} {r:5.1f} {len(g):5d} {n_in:7d} {100 * n_in / max(1,
len(g)):6.0f}%  "
                  f"{'FALSE DETECTION' if len(g) > 10 else 'not found (correct)'}")
```

```
scene         method ratio  good inliers inlier%  course rule (good > 10)
messi5.jpg    orb      0.7     0       0      0%  not found (correct)
messi5.jpg    orb      0.8    17       5     29%  FALSE DETECTION
messi5.jpg    orb      0.9   116      11      9%  FALSE DETECTION
messi5.jpg    sift     0.7     3       0      0%  not found (correct)
messi5.jpg    sift     0.8    22       8     36%  FALSE DETECTION
messi5.jpg    sift     0.9   123      11      9%  FALSE DETECTION
building.jpg  orb      0.7     6       0      0%  not found (correct)
building.jpg  orb      0.8    12       4     33%  FALSE DETECTION
building.jpg  orb      0.9    93       6      6%  FALSE DETECTION
building.jpg  sift     0.7     1       0      0%  not found (correct)
building.jpg  sift     0.8    11       4     36%  FALSE DETECTION
building.jpg  sift     0.9   120       7      6%  FALSE DETECTION
home.jpg      orb      0.7     4       4    100%  not found (correct)
home.jpg      orb      0.8    18       5     28%  FALSE DETECTION
home.jpg      orb      0.9   115       9      8%  FALSE DETECTION
home.jpg      sift     0.7     1       0      0%  not found (correct)
home.jpg      sift     0.8    15       6     40%  FALSE DETECTION
home.jpg      sift     0.9    98      10     10%  FALSE DETECTION
left01.jpg    orb      0.7     3       0      0%  not found (correct)
left01.jpg    orb      0.8    17       5     29%  FALSE DETECTION
left01.jpg    orb      0.9    60       5      8%  FALSE DETECTION
left01.jpg    sift     0.7     2       0      0%  not found (correct)
left01.jpg    sift     0.8    12       4     33%  FALSE DETECTION
left01.jpg    sift     0.9    91       7      8%  FALSE DETECTION
graf1.png     orb      0.7     1       0      0%  not found (correct)
graf1.png     orb      0.8    25      10     40%  FALSE DETECTION
graf1.png     orb      0.9   120       9      8%  FALSE DETECTION
graf1.png     sift     0.7     2       0      0%  not found (correct)
graf1.png     sift     0.8    14       5     36%  FALSE DETECTION
graf1.png     sift     0.9   117       9      8%  FALSE DETECTION
```

This is important. At the course's ratio of **0.7**, every negative scene gives at most 6 good matches, so `len(good) > 10` correctly says "not found". But the rule only works because the ratio is strict. At **0.8**, every one of the 10 negative tests (5 scenes × ORB/SIFT) has 11–25 "good" matches, and the course rule **falsely reports the logo in all of them**; at 0.9 there are 60–123.

What *does* separate positives from negatives is **geometry**: in the negative scenes at most 11 matches agree with any single homography (RANSAC inliers), while the real box gives 18–21 (ORB) and 73 (SIFT) inliers in `box_in_scene.png`. Note `home.jpg`, ORB, 0.7: 4 good matches, 4 inliers, "100 %". **Any 4 matches fit a homography perfectly**, so an inlier ratio is meaningless when there are only a handful of matches; you need a minimum inlier *count* as well.

6 · Mini project: logo detector

6a · The course's mini-project cell, explained

This is the course code (it continues from the `good` matches of §3), with comments. It works on our real logo/scene pair.

```
# Extract matched keypoints
if len(good) > 10:                                    # need enough matches to trust a
homography
    src_pts = np.float32([ kp1[m.queryIdx].pt for m in good ]).reshape(-1,1,2)   # points in the
logo
    dst_pts = np.float32([ kp2[m.trainIdx].pt for m in good ]).reshape(-1,1,2)   # where they are
in the scene

    # Find homography: logo plane -> scene, RANSAC with 5-pixel reprojection threshold
    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    h, w = logo.shape
    pts = np.float32([[0,0],[0,h-1],[w-1,h-1],[w-1,0]]).reshape(-1,1,2)   # the logo's 4 corners
    dst = cv2.perspectiveTransform(pts, M)                                 # ...projected into
the scene

    scene_color = cv2.cvtColor(scene, cv2.COLOR_GRAY2BGR)                  # colour copy so we
can draw in green
    cv2.polylines(scene_color, [np.int32(dst)], True, (0,255,0), 3)        # closed polygon,
green, 3 px

    print(f"{len(good)} good matches, {int(mask.sum())} RANSAC inliers")
    plt.figure(figsize=(6, 4.5))
    plt.imshow(cv2.cvtColor(scene_color, cv2.COLOR_BGR2RGB))             # course:
plt.imshow(scene_color)
    plt.title("Detected Logo")
    plt.axis('off')
    plt.show()
else:
    print("Not enough matches found - try another image or adjust parameters.")
```

```
20 good matches, 20 RANSAC inliers
```



The RANSAC reprojection threshold `5.0`: after fitting H, a match counts as an inlier if the logo point, projected with H, lands within 5 pixels of its matched scene point. Smaller → stricter (fewer inliers, more precise H); larger → tolerates noisy keypoints but also lets some wrong matches in. 3–5 px is typical.

What the course cell does not check: whether `M` is `None` (it is when RANSAC fails), how many matches are *inliers* (it only counts ratio-test matches), and whether the box is a sensible shape (a twisted "bow-tie" polygon means H is nonsense). The course notebook's own saved output shows this failure: matching `left.jpg` against `right.jpg` drew a green line running off the image, because the whole "logo" image is not a flat object.

6b · An improved, reusable detector

Same pipeline (detect → FLANN → ratio test → RANSAC homography → project corners), plus three sanity checks that decide **found / not found**:

1. at least `min_inliers` (12) RANSAC inliers;
2. at least 25 % of the good matches are inliers;
3. the projected box is **convex** and not tiny (no twisted or collapsed polygons).

```
def detect_logo(logo_gray, scene_gray, method="orb", ratio=0.7, nfeatures=1000, min_inliers=12,
min_inlier_ratio=0.25):
    det = make_detector(method, nfeatures)
    ka, da = det.detectAndCompute(logo_gray, None)
    kb, db = det.detectAndCompute(scene_gray, None)
    good = ratio_matches(da, db, method, ratio)
    H, inl, n_in = ransac_homography(ka, kb, good)
    res = dict(method=method, good=len(good), inliers=n_in, H=H, found=False, box=None,
reason="")
    if H is None:
        res["reason"] = f"only {len(good)} good matches"
        return res
    h, w = logo_gray.shape[:2]
    box = cv2.perspectiveTransform(np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h -
1]]).reshape(-1, 1, 2), H)
    res["box"] = box.reshape(-1, 2)
    area = cv2.contourArea(box)
    if n_in < min_inliers:
        res["reason"] = f"{n_in} inliers < {min_inliers}"
```

```
    elif n_in / len(good) < min_inlier_ratio:
        res["reason"] = f"inlier ratio {n_in / len(good):.0%} too low"
    elif not cv2.isContourConvex(np.int32(box)) or area < 0.002 * scene_gray.size:
        res["reason"] = "box not a sensible shape"
    else:
        res["found"], res["reason"] = True, f"{n_in} inliers / {len(good)} good"
    return res

def draw_result(scene_bgr, res, gt=None):
    vis = scene_bgr.copy()
    if gt is not None:
        cv2.polylines(vis, [np.int32(gt)], True, (255, 0, 0), 2)                    # ground
truth in blue
    if res["box"] is not None:
        cv2.polylines(vis, [np.int32(res["box"])], True, (0, 255, 0) if res["found"] else (0, 0,
255), 3)
    return vis

res = detect_logo(logo, scene, "orb")
print(res["found"], "|", res["reason"])
show(draw_result(scene_c, res), "box.png found in box_in_scene.png (ORB)", size=6)
```

```
True | 29 inliers / 32 good
```

box.png found in box_in_scene.png (ORB)



6c · Measured robustness: logo pasted into a scene with a KNOWN transformation

For `box_in_scene.png` we don't know the exact true position, so we can't measure accuracy. So we make our own test scenes: we warp the logo with a known homography (rotation, zoom, perspective tilt) and paste it into a cluttered photo (`messi5.jpg`, enlarged to 800×500). Because we know where the corners *should* be, we can measure the **corner error** in pixels. Blue = ground truth, green = detected.

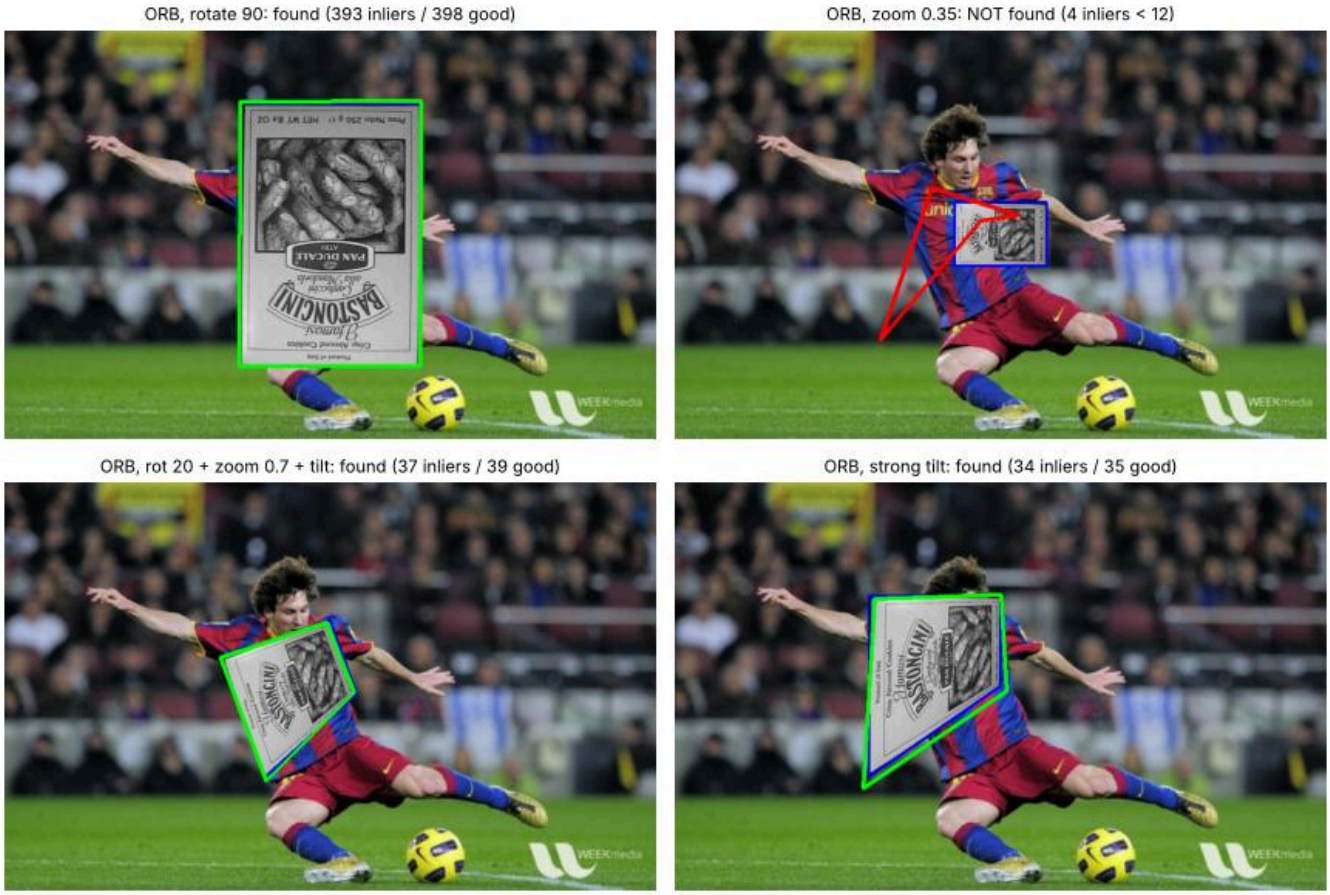
```
bg = cv2.resize(imread_checked(os.path.join(DATA, "messi5.jpg")), (800, 500))

def make_test_scene(logo_bgr, angle=0, scale=1.0, tilt=0.0, centre=(400, 250)):
    """Warp the logo (rotate/scale about its centre, optional perspective tilt) and paste it into
bg."""
    h, w = logo_bgr.shape[:2]
```

```
    R = np.vstack([cv2.getRotationMatrix2D((w / 2, h / 2), angle, scale), [0, 0, 1]])
    T = np.array([[1, 0, centre[0] - w / 2], [0, 1, centre[1] - h / 2], [0, 0, 1]], dtype=float)
    P = np.array([[1, 0, 0], [0, 1, 0], [tilt / w, 0, 1]], dtype=float)           # perspective:
right side further away
    H = T @ R @ P
    warped = cv2.warpPerspective(logo_bgr, H, (bg.shape[1], bg.shape[0]))
    m = cv2.warpPerspective(np.full(logo_bgr.shape[:2], 255, np.uint8), H, (bg.shape[1],
bg.shape[0]))
    sc = bg.copy(); sc[m > 0] = warped[m > 0]
    gt = cv2.perspectiveTransform(np.float32([[0, 0], [w - 1, 0], [w - 1, h - 1], [0, h -
1]]).reshape(-1, 1, 2), H)
    return sc, gt.reshape(-1, 2)

cases = [("identity", 0, 1.0, 0.0), ("rotate 30", 30, 1.0, 0.0), ("rotate 90", 90, 1.0, 0.0),
("rotate 180", 180, 1.0, 0.0),
         ("zoom 0.5", 0, 0.5, 0.0), ("zoom 0.35", 0, 0.35, 0.0), ("zoom 1.5", 0, 1.5, 0.0),
         ("rot 20 + zoom 0.7 + tilt", 20, 0.7, 0.6), ("strong tilt", 0, 1.0, 1.0)]
print(f"{'case':26s} | {'ORB: found  inliers  corner err':32s} | {'SIFT: found  inliers  corner
err'}")
panels, titles = [], []
for name, a, s, t in cases:
    sc, gt = make_test_scene(logo_c, a, s, t)
    sg = cv2.cvtColor(sc, cv2.COLOR_BGR2GRAY)
    line = f"{name:26s}"
    for method in ["orb", "sift"]:
        r = detect_logo(logo, sg, method)
        err = np.linalg.norm(r["box"] - gt, axis=1).mean() if r["box"] is not None else
float("nan")
        line += f" | {str(r['found']):>11s} {r['inliers']:8d} {err:9.1f} px      "
        if method == "orb" and name in ("rotate 90", "zoom 0.35", "rot 20 + zoom 0.7 + tilt",
"strong tilt"):
            panels.append(draw_result(sc, r, gt)); titles.append(f"ORB, {name}: {'found' if
r['found'] else 'NOT found'} ({r['reason']})")
    print(line)
show(panels, titles, cols=2, size=5.5)
```

```
case                       | ORB: found  inliers  corner err  | SIFT: found  inliers  corner err
identity                   |        True      388       0.7 px       |        True      417
0.1 px
rotate 30                  |        True      378       0.7 px       |        True      414
0.2 px
rotate 90                  |        True      393       1.1 px       |        True      409
0.5 px
rotate 180                 |        True      398       1.4 px       |        True      412
0.7 px
zoom 0.5                   |        True       57       1.2 px       |        True      111
0.4 px
zoom 0.35                  |       False        4      93.5 px       |        True       67
0.8 px
zoom 1.5                   |        True      269       2.1 px       |        True      399
0.2 px
rot 20 + zoom 0.7 + tilt   |        True       37       3.9 px       |        True       77
0.5 px
strong tilt                |        True       34       8.7 px       |        True       61
0.6 px
```



Reading the table:

- **Rotation is no problem** for either method (0–180°): both found the box every time; ORB corner error 0.7–1.4 px, SIFT 0.1–0.7 px.
- **Zoom:** at 0.5× both still work (ORB with 57 inliers instead of ~390). At **0.35×** (the box is only about 110 px wide) ORB finds just 4 inliers; RANSAC returns a nonsense box (93.5 px corner error, the red triangle above) and our sanity checks **correctly reject it**. SIFT still finds it (67 inliers, 0.8 px error). This is the "ORB is weaker on scale" claim, measured.
- **Perspective tilt:** both found it; ORB's corners are less precise (3.9 px and 8.7 px error) than SIFT's (0.5–0.6 px). SIFT's sub-pixel keypoint localisation and richer descriptor give more, and more precise, inliers.
- Overall: ORB found the logo in 8 of 9 cases, SIFT in 9 of 9. SIFT's corner errors stayed at or below 0.8 px; ORB's ranged from 0.7 to 8.7 px.

6d · A real logo: the OpenCV logo pasted into a scene

Real logos are often **flat colour shapes**: few textures, many smooth edges, repeated elements (the OpenCV logo has three similar rings). That is a harder case for feature matching than a richly printed cereal box.

```
L = imread_checked(os.path.join(DATA, "opencv-logo.png"), cv2.IMREAD_UNCHANGED)      # 4 channels:
B, G, R, alpha
alpha = L[..., 3:4] / 255.0
logo2 = (L[..., :3] * alpha + 255 * (1 - alpha)).astype(np.uint8)                   # put it on a
white background
logo2 = cv2.resize(logo2, None, fx=0.4, fy=0.4, interpolation=cv2.INTER_AREA)
logo2_g = cv2.cvtColor(logo2, cv2.COLOR_BGR2GRAY)
print("logo size", logo2.shape[:2], "| ORB keypoints", len(cv2.ORB_create(1000).detect(logo2_g,
```

```
None)),
      "| SIFT keypoints", len(cv2.SIFT_create().detect(logo2_g, None)))

panels, titles = [], []
for name, a, s, t in [("rotate 20, zoom 0.8", 20, 0.8, 0.0), ("zoom 0.6", 0, 0.6, 0.0), ("zoom
0.4 + tilt", 0, 0.4, 0.5)]:
    sc, gt = make_test_scene(logo2, a, s, t, centre=(560, 220))
    sg = cv2.cvtColor(sc, cv2.COLOR_BGR2GRAY)
    for method in ["orb", "sift"]:
        r = detect_logo(logo2_g, sg, method)
        err = np.linalg.norm(r["box"] - gt, axis=1).mean() if r["box"] is not None else
float("nan")
        print(f"{name:20s} {method.upper():4s}: found={r['found']!s:5s} {r['reason']:28s} corner
error {err:.1f} px")
        if method == "orb":
            panels.append(draw_result(sc, r, gt)); titles.append(f"ORB, {name}: {'found' if
r['found'] else 'NOT found'}")
show(panels, titles, cols=3, size=4.5)
```

```
logo size (318, 240) | ORB keypoints 255 | SIFT keypoints 97
rotate 20, zoom 0.8  ORB : found=True  34 inliers / 44 good         corner error 9.2 px
rotate 20, zoom 0.8  SIFT: found=True  48 inliers / 55 good         corner error 0.5 px
zoom 0.6             ORB : found=True  18 inliers / 28 good         corner error 2.5 px
zoom 0.6             SIFT: found=True  38 inliers / 43 good         corner error 0.9 px
zoom 0.4 + tilt      ORB : found=False only 0 good matches          corner error nan px
zoom 0.4 + tilt      SIFT: found=False only 3 good matches          corner error nan px
```



Reading the result:

- The flat OpenCV logo gives only **97 SIFT keypoints** (flat colour has few blob-like structures). ORB finds 255, mostly on the text "OpenCV" and on the ring edges.
- At 0.8× and 0.6× both methods find it. ORB's box is less precise (9.2 px and 2.5 px corner error, against SIFT's 0.5 and 0.9 px).
- At **0.4× with tilt** (logo ≈ 95 px wide) **both fail** (0 and 3 good matches) and the detector honestly says "not found". Small, flat, low-texture logos are the weak spot of keypoint matching. Remedies: a larger template, matching at several template scales, or a trained detector (CNN, Week 3).

6e · The detector on scenes without the logo

```
print("ratio 0.7 (default) and the looser 0.9:")
for name in negatives:
    sg = imread_checked(os.path.join(DATA, name), cv2.IMREAD_GRAYSCALE)
    out = []
    for method in ["orb", "sift"]:
        for r in [0.7, 0.9]:
            res = detect_logo(logo, sg, method, ratio=r)
            out.append(f"{method.upper()}@{r}: {'FOUND' if res['found'] else 'no'}
({res['reason']})")
    print(f"{name:13s} " + " | ".join(out))
```

```
ratio 0.7 (default) and the looser 0.9:
```

```
messi5.jpg    ORB@0.7: no (only 0 good matches) | ORB@0.9: no (10 inliers < 12) | SIFT@0.7: no (o
nly 3 good matches) | SIFT@0.9: no (inlier ratio 12% too low)
building.jpg  ORB@0.7: no (only 1 good matches) | ORB@0.9: no (9 inliers < 12) | SIFT@0.7: no (on
ly 1 good matches) | SIFT@0.9: no (6 inliers < 12)
home.jpg      ORB@0.7: no (only 1 good matches) | ORB@0.9: no (11 inliers < 12) | SIFT@0.7: no (o
nly 2 good matches) | SIFT@0.9: no (0 inliers < 12)
left01.jpg    ORB@0.7: no (4 inliers < 12) | ORB@0.9: no (8 inliers < 12) | SIFT@0.7: no (only 2
good matches) | SIFT@0.9: no (6 inliers < 12)
graf1.png     ORB@0.7: no (4 inliers < 12) | ORB@0.9: no (9 inliers < 12) | SIFT@0.7: no (only 1
good matches) | SIFT@0.9: no (9 inliers < 12)
```

With the improved decision rule (inliers ≥ 12, inlier ratio ≥ 25 %, convex box), there were **0 false detections in 20 negative tests**, even at the loose ratio 0.9 where the course rule failed every time. **Honest caveat:** at ratio 0.9 the margin is thin. ORB on `home.jpg` reached 11 inliers, just below our threshold of 12. With ratio 0.7 the negatives never exceeded 4 inliers, while true detections had ≥ 18. So: keep the ratio at 0.7–0.8 **and** decide on inliers.

7 · 🔧 What we changed from the course notebook, and why

#	Course code	Problem (verified)	What we did
1	`cv2.imread('left.jpg')`, `'right.jpg'` (and `logo.jpg` / `scene.jpg` in the exercise)	Files not provided → `None` → crash in `detectAndCompute`. Also, `left/right` are two views of a scene, **not a logo and a scene**, so the "detected logo" box in the course's saved output is meaningless (a line running off the image)	Real logo/scene pair `box.png` / `box_in_scene.png`, the OpenCV logo, and synthetic scenes with known ground truth
2	`for m,n in matches:`	**Crashes** with `ValueError: not enough values to unpack` when FLANN-LSH returns < 2 neighbours (reproduced in §4 with a small scene)	`[p[0] for p in matches if len(p) == 2 and ...]`
3	`plt.imshow(img_matches)`, `plt.imshow(scene_color)`	`drawMatches` / `cvtColor(GRAY2BGR)` give **BGR** images; matplotlib expects RGB. Invisible here (grey photo, random/green lines) but wrong colours with colour inputs	`cv2.cvtColor(..., cv2.COLOR_BGR2RGB)` before `plt.imshow`
4	Decision rule `len(good) > 10`	Counts ratio-test matches, not geometric agreement. At ratio 0.8 or 0.9 it reports the logo in every negative scene we tried (§5c)	Decide on RANSAC **inliers**, inlier ratio and a convex box
5	`M` from `findHomography` used without checking	It is `None` when RANSAC fails → `perspectiveTransform` crashes	Check `H is None`
6	No check for `des is None`	Images without keypoints crash `knnMatch`	Guard in `ratio_matches`
7	Exercise "observe how many matches… how accurate"	No measurement	Inlier counts, ratio sweep, ORB vs SIFT, BF vs FLANN timing, ground-truth corner errors, negatives

8 · Results, honestly

- The course pipeline (ORB → FLANN-LSH → ratio 0.7 → RANSAC homography → polygon) **works** once it is given a real logo and scene: it located the box in `box_in_scene.png` with 20 good matches, all RANSAC inliers.
- It has a **real crash** (`for m, n in matches` with FLANN-LSH, reproduced) and a **weak decision rule** (`len(good) > 10` , which falsely detects the logo in every negative scene once the ratio is relaxed to 0.8).
- **SIFT beat ORB on accuracy** in every test (more inliers, corner errors ≤ 0.9 px against ORB's up to 9.2 px, and it survived 0.35× zoom); **ORB was more than 10× faster** to detect and describe (§5). For real-time use ORB is still the practical choice.
- Lowering the ratio threshold trades quantity for quality; 0.7–0.8 was the sweet spot on our data.
- FLANN's speed advantage only matters for thousands of descriptors; its approximation did not hurt the final result.
- Limitations: our synthetic tests paste a perfectly flat, sharp logo with no blur, lighting change or occlusion, so real photos will be harder. Timings depend on the machine.

9 · Common errors and how to fix them

Error / symptom	Cause	Fix
`ValueError: not enough values to unpack (expected 2, got 1)` in `for m, n in matches`	FLANN-LSH returned < 2 neighbours	`if len(p) == 2` guard (§4)
`error: (-215) ... in function 'knnMatch'` / "unsupported format"	Descriptors are `None` , or type does not fit the index: SIFT (float32) needs KD-tree / `NORM_L2` , ORB (uint8) needs LSH / `NORM_HAMMING`	Check `des is None` ; pick the matching index/norm; never `astype` ORB descriptors to float for a KD-tree
`crossCheck=True` + `knnMatch(k=2)` gives empty/odd results	Cross-check and ratio test are alternative filters	Use `BFMatcher(norm)` (crossCheck False) for the ratio test
Very few good matches	Logo too small in the scene, blurry, low texture, ratio too strict, ORB's 500-point cap	Use more features (`ORB_create(2000)`), SIFT, ratio 0.75–0.8, larger logo image
Box drawn as a twisted "bow tie" or a huge shape	Homography fitted to wrong matches (logo not present or too few inliers)	Require ≥ 10–15 inliers, inlier ratio, convex box (`cv2.isContourConvex`)
`findHomography` returns `None`	< 4 matches or degenerate points	Check before using
Box slightly off for non-flat objects	A homography only describes **flat** objects	Use a planar logo; for 3-D objects use pose estimation (`solvePnP`)
Colours swapped in plots	BGR vs RGB	`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`

10 · Try this yourself

1. Photograph a book cover or a cereal box (the "logo") and then a messy desk with it lying at an angle (the "scene"). Run `detect_logo` with ORB and SIFT. At which angle/distance does each one fail?
2. Change the RANSAC threshold (1, 3, 5, 10 px) in `ransac_homography` and watch the inlier count and the corner error in §6c.
3. Replace RANSAC with `cv2.USAC_MAGSAC` (a modern robust estimator in OpenCV ≥ 4.5): `cv2.findHomography(src, dst, cv2.USAC_MAGSAC, 5.0)`. Is it more accurate?
4. Make the logo detector find **two copies** of the logo: after the first detection, remove its inlier matches and run RANSAC again on the rest.
5. Change `table_number`, `key_size`, `multi_probe_level` and `checks` for LSH. How do the number of neighbours returned and the match quality change?
6. Use `cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)` instead of the ratio test. Which gives a cleaner match set on `box_in_scene.png`?

11 · Viva questions (with short model answers)

1. **What is a feature descriptor?** A vector that summarises the image patch around a keypoint (gradients for SIFT, binary intensity comparisons for ORB) so keypoints in different images can be compared.
2. **SIFT vs ORB descriptor?** SIFT: 128 floats, L2 distance, very distinctive, slow. ORB: 256 bits (32 bytes), Hamming distance (XOR + popcount), very fast, a little less distinctive.
3. **Why Hamming distance for ORB?** The descriptor is a bit string; the natural dissimilarity is the number of differing bits, computed with one XOR and a bit count.
4. **Brute force vs FLANN?** BF compares all pairs (exact, O(N·M)); FLANN builds an index (KD-trees / LSH) and finds *approximate* nearest neighbours much faster for large sets. For a few hundred descriptors BF is just as fast.
5. **Why does ORB need an LSH index in FLANN?** KD-trees work on real-valued vectors with Euclidean distance; binary strings with Hamming distance need hashing (LSH: `algorithm=6`, `table_number`, `key_size`, `multi_probe_level`).
6. **Explain Lowe's ratio test.** For each descriptor take the two nearest neighbours; accept the best only if `d1 < ratio·d2` (0.7–0.8). Ambiguous matches (two similar candidates, e.g. repetitive texture) are rejected; Lowe showed this removes ~90 % of false matches while losing < 5 % of correct ones.
7. **Effect of the ratio value?** Lower (0.5) → few but very reliable matches; higher (0.9) → many matches with many outliers. Our sweep: the inlier percentage dropped as the ratio rose (see §5a).
8. **Why `k=2` in `knnMatch`?** The ratio test needs the best and the second-best neighbour.
9. **What is a homography and why can it locate a logo?** A 3×3 projective transform between two planes; a flat logo seen from any viewpoint maps to the scene by a homography, so projecting the logo's corners gives its outline.
10. **Minimum matches for a homography? Why use more?** 4 (8 unknowns, 2 equations per point). More, plus RANSAC, to average out noise and reject wrong matches.

11. **What does RANSAC do and what is the threshold 5.0?** It repeatedly fits H to 4 random matches and keeps the model with most inliers (matches whose reprojection error is < 5 px).
12. **How do you know the logo is NOT in the scene?** Too few RANSAC inliers, a low inlier ratio, or an implausible (non-convex, tiny) box. Counting ratio-test matches alone is not enough: at ratio 0.9 every negative scene we tried passed the course's `len(good) > 10` rule.
13. **What bug can `for m, n in matches` cause?** With FLANN-LSH some queries get fewer than 2 neighbours → `ValueError`. Guard with `len(p) == 2`.
14. **Which is more robust to scale change, ORB or SIFT? Evidence?** SIFT: in our ground-truth test it still found the box at zoom 0.35 while ORB did not (see §6c).
15. **Real-world uses?** Brand monitoring (logos in broadcast/social media), augmented reality (marker-less tracking of a magazine page), visual search, document alignment, robot object recognition.

Day 10 · Object tracking: optical flow and the KLT tracker + mini project: dense-flow motion tracker

Course lab: `CV-Week2-Lecture3.ipynb` (Day 10) **Notes chapter:** `week-2/notes/03_object_tracking_optical_flow_klt.md` **Builds on:** Day 8 (corners: Harris and Shi-Tomasi "good features to track").

What you will learn

1. What optical flow is, its key assumption (brightness constancy), and **sparse** (Lucas–Kanade) vs **dense** (Farneback) flow.
2. The **KLT tracker** from the course, made to run inside a notebook (no `cv2.imshow`), with every parameter of `goodFeaturesToTrack` and `calcOpticalFlowPyrLK` explained.
3. The exercise "track a moving ball" done for real on a **synthetic ball video with known ground truth**, with measured experiments: `winSize`, `maxLevel` (pyramids) vs ball speed, `maxCorners`, occlusion.
4. What goes wrong with the course tracker over time (features are never re-detected) and a fixed tracker.
5. Mini project: a **dense optical-flow motion tracker** (Farneback + HSV colour coding) on a real street video, plus turning the flow into **bounding boxes** around moving people, and a sparse vs dense speed comparison.

Run time: about 1 minute on a laptop CPU (downloads one 8 MB video the first time).

1 · Theory recap in plain language

Optical flow = for each pixel (or chosen point), *how far and in which direction did it move between two frames?* The result is a motion vector (u, v) in pixels per frame.

The key assumption, brightness constancy: a point keeps the same brightness as it moves, I(x, y, t) = I(x+u, y+v, t+1). A first-order Taylor expansion turns this into one equation per pixel, $\mathbf{I_x \cdot u + I_y \cdot v + I_t = 0}$ (I_x, I_y: spatial gradients; I_t: change over time). That is **one equation with two unknowns**, so a single pixel cannot tell you its motion. This is the **aperture problem**: through a small hole, a moving edge only shows motion *across* the edge.

Lucas–Kanade (1981): "assume the neighbours move together". Take a small window (e.g. 15×15 = 225 pixels), assume they all share the same (u, v), and solve the 225 equations by least squares. This works well only where the window contains gradients in two directions, i.e. at **corners**. (Same 2×2 matrix as Harris, Day 8.)

Pyramids, for large motion: LK's Taylor approximation only holds for small motions (about 1–2 px). Solution: shrink the image several times (a pyramid). A 16-pixel jump becomes a 2-pixel jump at 1/8 size. Estimate there, then refine level by level. `maxLevel` sets how many levels.

KLT tracker = Kanade–Lucas–Tomasi:

1. pick **good features to track** with Shi–Tomasi (score = min(λ1, λ2) of the same 2×2 matrix: a corner is good if even its *weaker* direction has strong gradient);
2. follow each one frame by frame with pyramidal Lucas–Kanade.

Sparse vs dense flow:

	Sparse (Lucas–Kanade / KLT)	Dense (Farneback)
Computes motion for	a few hundred chosen points	**every** pixel
Speed	very fast	slower
Good for	tracking objects/points, real time	motion maps, segmentation of moving regions, visualisation
OpenCV	`cv2.calcOpticalFlowPyrLK`	`cv2.calcOpticalFlowFarneback`

Farneback (2003): approximates each neighbourhood with a quadratic polynomial and works out how the polynomial shifted between frames, again on a pyramid.

2 · Setup and data

The course code opens `chaplin.mp4` and the exercise uses `ball.mp4`; **neither video was provided**. We use:

- `vtest.avi` from the OpenCV samples (a fixed camera filming pedestrians in a street, 768×576, 10 fps), in place of `chaplin.mp4`;
- a **synthetic ball video we generate ourselves** in place of `ball.mp4`. Because we draw it, we know exactly where the ball is in every frame (ground truth), so we can *measure* how well the tracker works.

`cv2.imshow` and `cv2.waitKey` open desktop windows; inside Jupyter they either do nothing, freeze the kernel or crash (and on a server there is no screen at all). We collect frames in a list and show a grid with matplotlib instead, and also write an `.mp4` to `output/` that you can play.

```
%matplotlib inline
%config InlineBackend.figure_formats = ["jpeg"]
import time
import numpy as np
import matplotlib.pyplot as plt
import cv2

plt.rcParams["figure.dpi"] = 80
print("OpenCV version:", cv2.__version__)
USE_WEBCAM = False      # set True only on your own laptop with a camera (see the last code cell)

import os, urllib.request

DATA, OUT = "data", "output"            # data/ and output/ sit next to this notebook (both are
git-ignored)
os.makedirs(DATA, exist_ok=True)
os.makedirs(OUT, exist_ok=True)

OPENCV_SAMPLES = "https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/"
```

```
OPENCV_STITCH  = "https://raw.githubusercontent.com/opencv/opencv_extra/4.x/testdata/stitching/"

def download(url, path):
    """Download url to path once; later runs reuse the cached file."""
    if os.path.exists(path) and os.path.getsize(path) > 0:
        return path
    print("downloading", url)
    tmp = path + ".part"
    try:
        urllib.request.urlretrieve(url, tmp)
        os.replace(tmp, path)
    except Exception as e:
        raise RuntimeError(
            f"Could not download {url}\n({e})\n"
            f"Are you offline? Open the URL in a browser, save the file as\n
{os.path.abspath(path)}\n"
            "and re-run this cell. (Or use any image/video of your own and change the file
name.)") from e
    return path

def to_rgb(img):
    """OpenCV stores colour as B,G,R; matplotlib expects R,G,B. Grey images are left alone."""
    return img if img.ndim == 2 else cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

def show(images, titles=None, cols=None, size=4.5, max_side=900):
    """Show one or more OpenCV images side by side (downscaled for a small notebook file)."""
    if not isinstance(images, (list, tuple)):
        images, titles = [images], [titles]
    cols = cols or len(images)
    rows = (len(images) + cols - 1) // cols
    aspect = max(im.shape[0] / im.shape[1] for im in images)          # height/width of the
tallest image
    fig, axes = plt.subplots(rows, cols, figsize=(size * cols, size * rows * aspect + 0.35 *
rows), squeeze=False)
    for ax in axes.ravel():
        ax.axis("off")
    for i, img in enumerate(images):
        s = min(1.0, max_side / max(img.shape[:2]))
        if s < 1:
            img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
        ax = axes.ravel()[i]
        ax.imshow(to_rgb(img), cmap="gray" if img.ndim == 2 else None)
        if titles and titles[i]:
            ax.set_title(titles[i], fontsize=10)
    plt.tight_layout()
    plt.show()

VIDEO = download(OPENCV_SAMPLES + "vtest.avi", os.path.join(DATA, "vtest.avi"))

cap = cv2.VideoCapture(VIDEO)
if not cap.isOpened():
    raise IOError("Cannot open video file")
print("frames:", int(cap.get(cv2.CAP_PROP_FRAME_COUNT)), "| fps:", cap.get(cv2.CAP_PROP_FPS),
      "| size:", int(cap.get(cv2.CAP_PROP_FRAME_WIDTH)), "x",
int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT)))
cap.release()

def read_frames(path, max_frames=150):
    """Read up to max_frames frames into a list (fine for short clips; stream for long ones)."""
    cap = cv2.VideoCapture(path)
    frames = []
    while len(frames) < max_frames:
        ret, f = cap.read()
        if not ret:
            break
        frames.append(f)
    cap.release()
    return frames
```

```python
def write_mp4(frames, path, fps=10):
    """Save frames as an .mp4 you can play outside the notebook (output/ is git-ignored)."""
    h, w = frames[0].shape[:2]
    vw = cv2.VideoWriter(path, cv2.VideoWriter_fourcc(*"mp4v"), fps, (w, h))
    for f in frames:
        vw.write(f)
    vw.release()
    return path

street = read_frames(VIDEO, 150)
print(len(street), "frames loaded (15 s of video)")
show([street[0], street[75], street[149]], ["frame 0", "frame 75", "frame 149"], size=4.5)
```

```
OpenCV version: 5.0.0
frames: 795 | fps: 10.0 | size: 768 x 576
150 frames loaded (15 s of video)
```



The synthetic "ball.mp4"

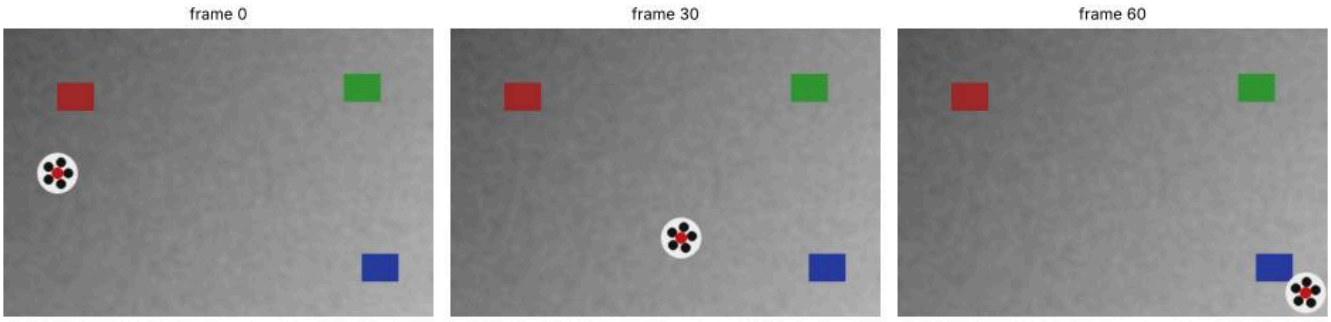
A textured ball (white with dark spots, so it has corners) bounces around a background with a soft gradient, mild blur-noise texture and three static coloured boxes (static corners, like a real background). We record the ball's true centre for every frame. Parameters: `speed` (pixels per frame), an optional grey **pillar** (to create occlusion), and `bounce=False` for a straight path (used in the parameter experiment).

```python
def make_ball_video(n=90, W=480, H=320, speed=7.0, r=22, occluder=False, bounce=True, start=
(60.0, 160.0), angle_deg=20, seed=0):
    rng = np.random.default_rng(seed)
    yy, xx = np.mgrid[0:H, 0:W]
    bg = (90 + 60 * xx / W + 30 * yy / H).astype(np.float32)
    bg += cv2.GaussianBlur(rng.normal(0, 25, (H, W)).astype(np.float32), (0, 0), 3)     # soft
texture
    bg = cv2.cvtColor(np.clip(bg, 0, 255).astype(np.uint8), cv2.COLOR_GRAY2BGR)
    for (x, y, c) in [(60, 60, (40, 40, 160)), (W - 80, H - 70, (160, 60, 40)), (W - 100, 50,
(50, 150, 50))]:
        cv2.rectangle(bg, (x, y), (x + 40, y + 30), c, -1)                              #
static boxes
    frames, gts = [], []
    x, y = start
    a = np.deg2rad(angle_deg); vx, vy = speed * np.cos(a), speed * np.sin(a)
    for i in range(n):
        f = bg.copy()
        c = (int(round(x)), int(round(y)))
        cv2.circle(f, c, r, (240, 240, 240), -1, cv2.LINE_AA)
        for k in range(5):                                                              # 5
spots, slowly rotating
            t = i * 0.08 + k * 2 * np.pi / 5
            cv2.circle(f, (int(round(x + 0.55 * r * np.cos(t))), int(round(y + 0.55 * r *
np.sin(t)))), 5, (20, 20, 20), -1, cv2.LINE_AA)
        cv2.circle(f, c, 6, (20, 20, 200), -1, cv2.LINE_AA)
        if occluder:
            cv2.rectangle(f, (W // 2 - 20, 0), (W // 2 + 20, H), (70, 70, 70), -1)         #
pillar in front of the ball
```

```
        frames.append(f); gts.append((x, y))
        x += vx; y += vy
        if bounce:
            if not r <= x <= W - r: vx = -vx; x = float(np.clip(x, r, W - r))
            if not r <= y <= H - r: vy = -vy; y = float(np.clip(y, r, H - r))
    return frames, np.array(gts)

ball_frames, ball_gt = make_ball_video()
os.makedirs(OUT, exist_ok=True)
write_mp4(ball_frames, os.path.join(OUT, "ball.mp4"), fps=25)
print(len(ball_frames), "frames,", ball_frames[0].shape, "| saved output/ball.mp4 (our stand-in
for the course's ball.mp4)")
show([ball_frames[0], ball_frames[30], ball_frames[60]], ["frame 0", "frame 30", "frame 60"],
size=4)
```

```
90 frames, (320, 480, 3) | saved output/ball.mp4 (our stand-in for the course's ball.mp4)
```



3 · The course KLT tracker, made to run in a notebook

Below is the course code, **line for line**, with these changes only:

- frames come from a `VideoCapture` as before, but instead of `cv2.imshow` / `cv2.waitKey` we keep a few **snapshots** for a matplotlib grid, write every drawn frame to an `.mp4`, and stop after `MAX_FRAMES`;
- we record how many points are alive in each frame, so we can see what happens over time;
- it is wrapped in a function so we can run it on both videos.

```
def course_klt(video_path, MAX_FRAMES=150, snap_at=(1, 50, 100, 149), out_mp4=None,
               feature_params=None, lk_params=None):
    # Load video
    cap = cv2.VideoCapture(video_path)  # course: 'chaplin.mp4' (not provided)
    if not cap.isOpened():
        raise IOError("Cannot open video file")

    # Parameters for ShiTomasi corner detection
    feature_params = feature_params or dict(maxCorners=100,     # keep at most the 100 best
corners
                                            qualityLevel=0.3,   # reject corners weaker than 30%
of the best one
                                            minDistance=7,      # corners at least 7 px apart
                                            blockSize=7)        # 7x7 window for the corner score

    # Parameters for Lucas-Kanade optical flow
    lk_params = lk_params or dict(winSize=(15,15),             # 15x15 window assumed to move
together
                                  maxLevel=2,                  # pyramid levels 0,1,2 (full,
1/2, 1/4 size)
                                  criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10,
0.03))  # stop after 10 iterations or when the update < 0.03 px

    # Take first frame and find corners in it
    ret, old_frame = cap.read()
    if not ret:
```

```
        raise IOError("Cannot read first frame from video")

    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)
    p0 = cv2.goodFeaturesToTrack(old_gray, mask=None, **feature_params)   # (N,1,2) float32
corner positions

    # Create mask image for drawing (the trails accumulate here)
    mask = np.zeros_like(old_frame)

    snapshots, alive, moving, drawn = {}, [len(p0)], [0], []
    frame_idx = 0
    while frame_idx < MAX_FRAMES:
        ret, frame = cap.read()
        if not ret:
            break
        frame_idx += 1
        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Calculate optical flow: where did each point of p0 move to in this frame?
        p1, st, err = cv2.calcOpticalFlowPyrLK(old_gray, frame_gray, p0, None, **lk_params)

        if p1 is None:          # happens when p0 is empty: every point has been lost
            break

        # Select good points (st == 1: flow was found for that point)
        good_new = p1[st == 1]
        good_old = p0[st == 1]

        # Draw tracks
        for new, old in zip(good_new, good_old):
            a, b = new.ravel()
            c, d = old.ravel()

            # Cast to int for drawing
            a, b, c, d = int(a), int(b), int(c), int(d)

            mask = cv2.line(mask, (a, b), (c, d), (0, 255, 0), 2)     # trail segment (never
erased)
            frame = cv2.circle(frame, (a, b), 5, (0, 0, 255), -1)     # current position

        img = cv2.add(frame, mask)

        # cv2.imshow('Tracking', img); cv2.waitKey(30)  -> replaced (no GUI windows in a
notebook):
        drawn.append(img)
        if frame_idx in snap_at:
            snapshots[frame_idx] = img
        alive.append(len(good_new))
        moving.append(int((np.linalg.norm(good_new - good_old, axis=1) > 0.5).sum()))   # points
that moved > 0.5 px

        # Update previous frame and points
        old_gray = frame_gray.copy()
        p0 = good_new.reshape(-1, 1, 2)

    cap.release()
    # cv2.destroyAllWindows()  -> not needed
    if out_mp4 and drawn:
        write_mp4(drawn, out_mp4)
    return snapshots, np.array(alive), np.array(moving)

t = time.perf_counter()
snaps, alive_course, moving_course = course_klt(VIDEO, out_mp4=os.path.join(OUT,
"klt_course_street.mp4"))
print(f"tracked {len(alive_course) - 1} frames in {time.perf_counter() - t:.1f}s (including
drawing)")
print("points alive at frames 0, 25, 50, 100, 149:", alive_course[[0, 25, 50, 100, 149]])
print("of these, points that moved > 0.5 px in that frame:", moving_course[[0, 25, 50, 100,
149]])
```

```
show([snaps[k] for k in sorted(snaps)], [f"course KLT, frame {k}" for k in sorted(snaps)],
cols=2, size=5.5)
```

```
tracked 150 frames in 1.0s (including drawing)
points alive at frames 0, 25, 50, 100, 149: [56 56 56 56 56]
of these, points that moved > 0.5 px in that frame: [0 4 6 6 9]
```



The parameters, explained

`goodFeaturesToTrack` (Shi–Tomasi)

Parameter	Course	Meaning	Effect of increasing
`maxCorners`	100	return at most this many corners (the strongest)	more points to track; slower
`qualityLevel`	0.3	minimum score as a fraction of the best corner's score	**fewer** corners (0.3 is strict; 0.01 is common)
`minDistance`	7	minimum pixel distance between returned corners	sparser, more evenly spread points
`blockSize`	7	window size for the corner score (the 2×2 gradient matrix)	favours larger-scale corners
`mask`	`None`	optional 8-bit image: only search where mask ≠ 0	(use it to restrict detection to a region)

`calcOpticalFlowPyrLK` **(pyramidal Lucas–Kanade)** returns `p1` (new positions), `st` (status: 1 = found, 0 = lost) and `err` (matching error per point).

Parameter	Course	Meaning	Trade-off
`winSize`	(15, 15)	window whose pixels are assumed to move together	bigger: more robust to noise and larger motion, but blurs motion boundaries and is slower
`maxLevel`	2	extra pyramid levels (0 = no pyramid; 2 = full, ½, ¼ size)	each level doubles the motion it can handle
`criteria`	`(EPS | COUNT, 10, 0.03)`	LK refines iteratively; stop after **10** iterations **or** when the step is smaller than **0.03 px**	more iterations = more accurate, slower

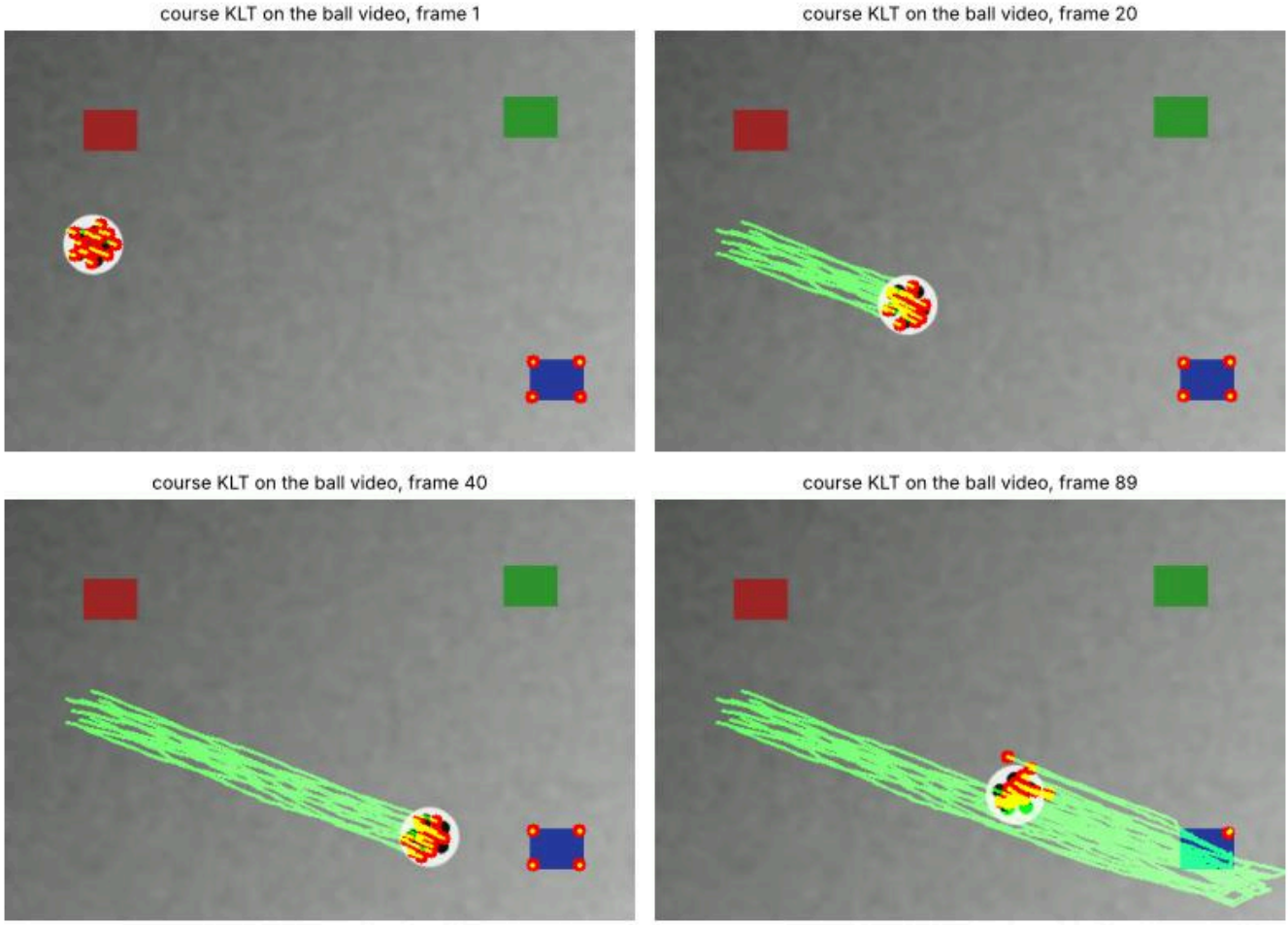
What the street result shows

- With the course settings (`qualityLevel=0.3`), `goodFeaturesToTrack` returns only **56 corners** in the first frame, not 100: `maxCorners` is just an upper limit. Most of them sit on **static** things (the building, the van, the tripod, the yellow object on the grass), and only a handful are on people.
- All 56 points stay "alive" (`st == 1`) for all 150 frames, but only **0–9 of them actually move** in any frame (5.0 on average). A static background corner is trivially easy to track, so it never gets lost.
- People who enter the scene later **never get any points**, because the code never looks for new corners. The green trails show a few early walkers, and the trails are never erased.
- So the tracker "works" in the sense that nothing crashes, but after a few seconds it is mostly tracking the background. Section 5 fixes this.

4 · Exercise: track the moving ball

The course exercise: *run the tracker on `ball.mp4`, observe how it follows the ball, try adjusting `maxCorners` or `winSize`*. We run the **same course function** on our synthetic ball video, and because we know the true ball position we can count how many of the points that started on the ball are still on it.

```
ball_path = os.path.join(OUT, "ball.mp4")
snaps_b, alive_b, _ = course_klt(ball_path, MAX_FRAMES=89, snap_at=(1, 20, 40, 89))
show([snaps_b[k] for k in sorted(snaps_b)], [f"course KLT on the ball video, frame {k}" for k in
sorted(snaps_b)], cols=2, size=5)
print("points alive per frame (every 10th):", alive_b[::10])
```




```
points alive per frame (every 10th): [16 16 16 16 16 16 16 16 16]
```

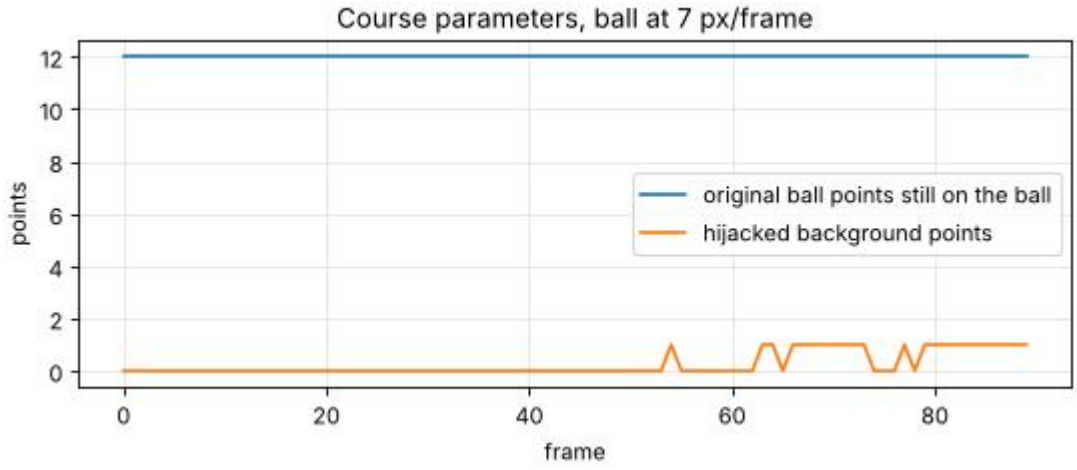
A helper that runs KLT on a list of frames and keeps the **identity** of every point, so we can ask: *of the points that started on the ball, how many are still on the ball?* (A point is "on the ball" if it is within the ball's radius + 2 px of the true centre.) It also counts **hijacked** points: background points that end up on the ball. Those appear when the ball rolls over a background corner and "carries it away", a classic optical-flow error at occlusion boundaries.

```
def klt_on_frames(frames, gts, r=22, winSize=(15, 15), maxLevel=2, maxCorners=100,
qualityLevel=0.3):
    fp = dict(maxCorners=maxCorners, qualityLevel=qualityLevel, minDistance=7, blockSize=7)
    lk = dict(winSize=winSize, maxLevel=maxLevel, criteria=(cv2.TERM_CRITERIA_EPS |
cv2.TERM_CRITERIA_COUNT, 10, 0.03))
    old = cv2.cvtColor(frames[0], cv2.COLOR_BGR2GRAY)
    p0 = cv2.goodFeaturesToTrack(old, mask=None, **fp)
    started_on_ball = np.linalg.norm(p0.reshape(-1, 2) - gts[0], axis=1) < r + 2
    ids = np.arange(len(p0))
    on_ball, hijacked = [int(started_on_ball.sum())], [0]
    for i in range(1, len(frames)):
        g = cv2.cvtColor(frames[i], cv2.COLOR_BGR2GRAY)
        if len(p0):
            p1, st, err = cv2.calcOpticalFlowPyrLK(old, g, p0, None, **lk)
            keep = st.ravel() == 1
            p0, ids = p1[keep].reshape(-1, 1, 2), ids[keep]
        near = np.linalg.norm(p0.reshape(-1, 2) - gts[i], axis=1) < r + 2 if len(p0) else
np.zeros(0, bool)
        on_ball.append(int((near & started_on_ball[ids]).sum()))
        hijacked.append(int((near & ~started_on_ball[ids]).sum()))
        old = g
    return int(started_on_ball.sum()), np.array(on_ball), np.array(hijacked)

n0, on_ball, hij = klt_on_frames(ball_frames, ball_gt)
print(f"{n0} of the initial corners are on the ball")
```

```
print("still on the ball (every 10th frame):", on_ball[::10])
print("hijacked background points on the ball:", hij[::10])
plt.figure(figsize=(8, 2.8))
plt.plot(on_ball, label="original ball points still on the ball"); plt.plot(hij, label="hijacked background points")
plt.xlabel("frame"); plt.ylabel("points"); plt.legend(); plt.grid(alpha=.3); plt.title("Course parameters, ball at 7 px/frame"); plt.show()
```

```
12 of the initial corners are on the ball
still on the ball (every 10th frame): [12 12 12 12 12 12 12 12 12]
hijacked background points on the ball: [0 0 0 0 0 0 0 1 1]
```



- The course function found 16 corners in the first frame: **12 on the ball** (the dark spots and the red centre on white make strong corners) and 4 on the static boxes.
- At 7 px/frame with the course parameters (`winSize=15` , `maxLevel=2`), **all 12 ball points stayed on the ball for all 89 frames**, including the bounces off the walls. The ball moves smoothly, and the pyramid handles 7 px easily.
- Towards the end, **1 background point was hijacked**: the ball rolled over a box corner and carried it along. LK sees a corner-like patch moving and cannot tell that it belonged to the background (the occlusion boundary problem).

4a · Experiment: ball speed vs `winSize` and `maxLevel` (pyramids)

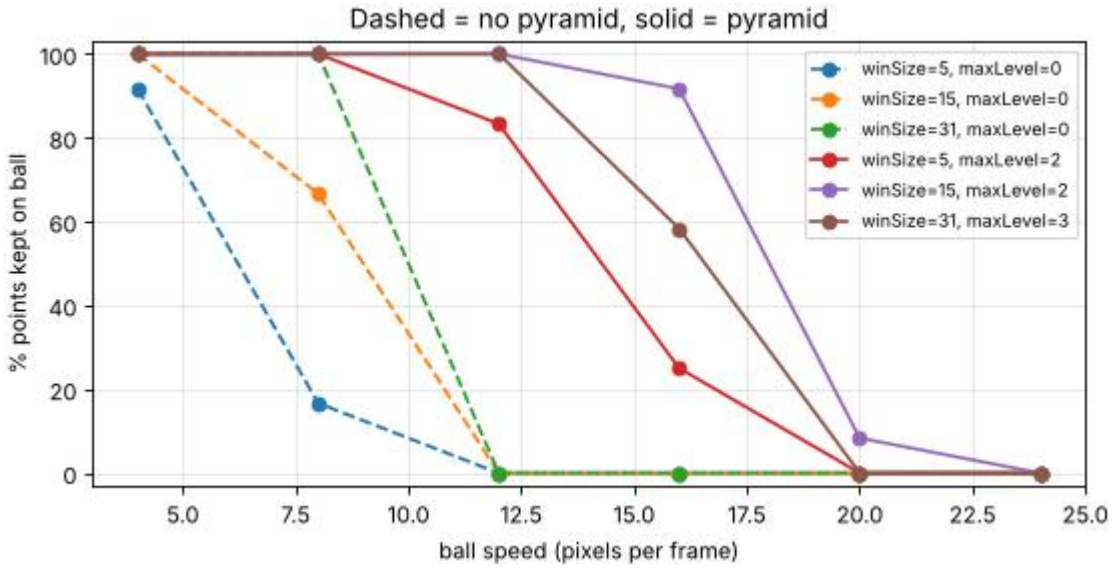
The textbook says *"KLT struggles with large motion; pyramids help"*. We test it: a ball moving in a **straight line** (no bounces, so only speed matters) for 20 frames at 4 to 24 px/frame, tracked with different window sizes, with and without pyramids. Score = % of the ball's original points still on the ball after 20 frames.

```
speeds = [4, 8, 12, 16, 20, 24]
configs = [((5, 5), 0), ((15, 15), 0), ((31, 31), 0), ((5, 5), 2), ((15, 15), 2), ((31, 31), 3)]
table = np.zeros((len(configs), len(speeds)))
for j, sp in enumerate(speeds):
    fr, gt = make_ball_video(n=21, W=40 + 21 * sp + 60, H=200, speed=sp, bounce=False, start=(40.0, 100.0), angle_deg=0)
    for i, (win, lvl) in enumerate(configs):
        n0_, ob, _ = klt_on_frames(fr, gt, winSize=win, maxLevel=lvl)
        table[i, j] = 100 * ob[-1] / n0_

print("% of ball points still on the ball after 20 frames")
print(f"{'winSize, maxLevel':20s}" + "".join(f"{s:>6d}px" for s in speeds))
for (win, lvl), row in zip(configs, table):
    print(f"{str(win) + ', ' + str(lvl):20s}" + "".join(f"{v:7.0f}%" for v in row))
```

```
plt.figure(figsize=(8, 3.6))
for (win, lvl), row in zip(configs, table):
    plt.plot(speeds, row, "-o" if lvl else "--o", label=f"winSize={win[0]}, maxLevel={lvl}")
plt.xlabel("ball speed (pixels per frame)"); plt.ylabel("% points kept on ball"); plt.ylim(-3,
103)
plt.title("Dashed = no pyramid, solid = pyramid"); plt.legend(fontsize=8); plt.grid(alpha=.3);
plt.show()
```

```
% of ball points still on the ball after 20 frames
winSize, maxLevel        4px     8px    12px    16px    20px    24px
(5, 5), 0                92%     17%      0%      0%      0%      0%
(15, 15), 0             100%     67%      0%      0%      0%      0%
(31, 31), 0             100%    100%      0%      0%      0%      0%
(5, 5), 2               100%    100%     83%     25%      0%      0%
(15, 15), 2             100%    100%    100%     92%      8%      0%
(31, 31), 3             100%    100%    100%     58%      0%      0%
```



Reading the table (this proves the "large motion" limitation and the value of pyramids):

- **Without a pyramid** (`maxLevel=0`), tracking breaks down at **8–12 px/frame**. A bigger window helps a little (31×31 still worked at 8 px, 5×5 kept only 17 %), but at 12 px/frame every no-pyramid setting lost the ball completely (0 %).
- **With a pyramid** the course setting (`15×15` , `maxLevel=2`) kept **100 % up to 12 px/frame and 92 % at 16 px/frame**. This is the reason `maxLevel` exists.
- **Bigger is not always better:** `31×31` with `maxLevel=3` did *worse* at 16 px/frame (58 %) than `15×15` / `maxLevel=2` . The ball is only 44 px wide; a 31×31 window at coarse levels contains mostly **static background**, which pulls the estimate towards zero motion.
- **Every setting failed at 24 px/frame** (and almost every one at 20). The ball moves more than its own radius (22 px) per frame. At the coarse pyramid levels needed for such a jump, the ball is only a few pixels wide and is lost in the background. Pyramids extend LK's range, but they do not make it unlimited. For very fast objects you need a higher frame rate, a motion model (e.g. a Kalman filter predicting where to look), or a detector.

4b · Experiment: `maxCorners` (and `qualityLevel`) on the street video

More corners = more information but more work. We count how many corners `goodFeaturesToTrack` actually returns and time tracking them over 50 frames.

```
gray_street = [cv2.cvtColor(f, cv2.COLOR_BGR2GRAY) for f in street]
lk = dict(winSize=(15, 15), maxLevel=2, criteria=(cv2.TERM_CRITERIA_EPS |
cv2.TERM_CRITERIA_COUNT, 10, 0.03))
print(f"{'maxCorners':>10s} {'quality':>8s} {'found':>6s} {'alive@50':>9s} {'ms/frame':>9s}")
for mc, q in [(10, 0.3), (100, 0.3), (500, 0.3), (100, 0.01), (500, 0.01), (2000, 0.01)]:
    p = cv2.goodFeaturesToTrack(gray_street[0], maxCorners=mc, qualityLevel=q, minDistance=7,
blockSize=7)
    n_found = len(p)
    t = time.perf_counter()
    for i in range(1, 51):
        p1, st, _ = cv2.calcOpticalFlowPyrLK(gray_street[i - 1], gray_street[i], p, None, **lk)
        p = p1[st.ravel() == 1].reshape(-1, 1, 2)
    ms = 1000 * (time.perf_counter() - t) / 50
    print(f"{mc:10d} {q:8.2f} {n_found:6d} {len(p):9d} {ms:9.2f}")
```

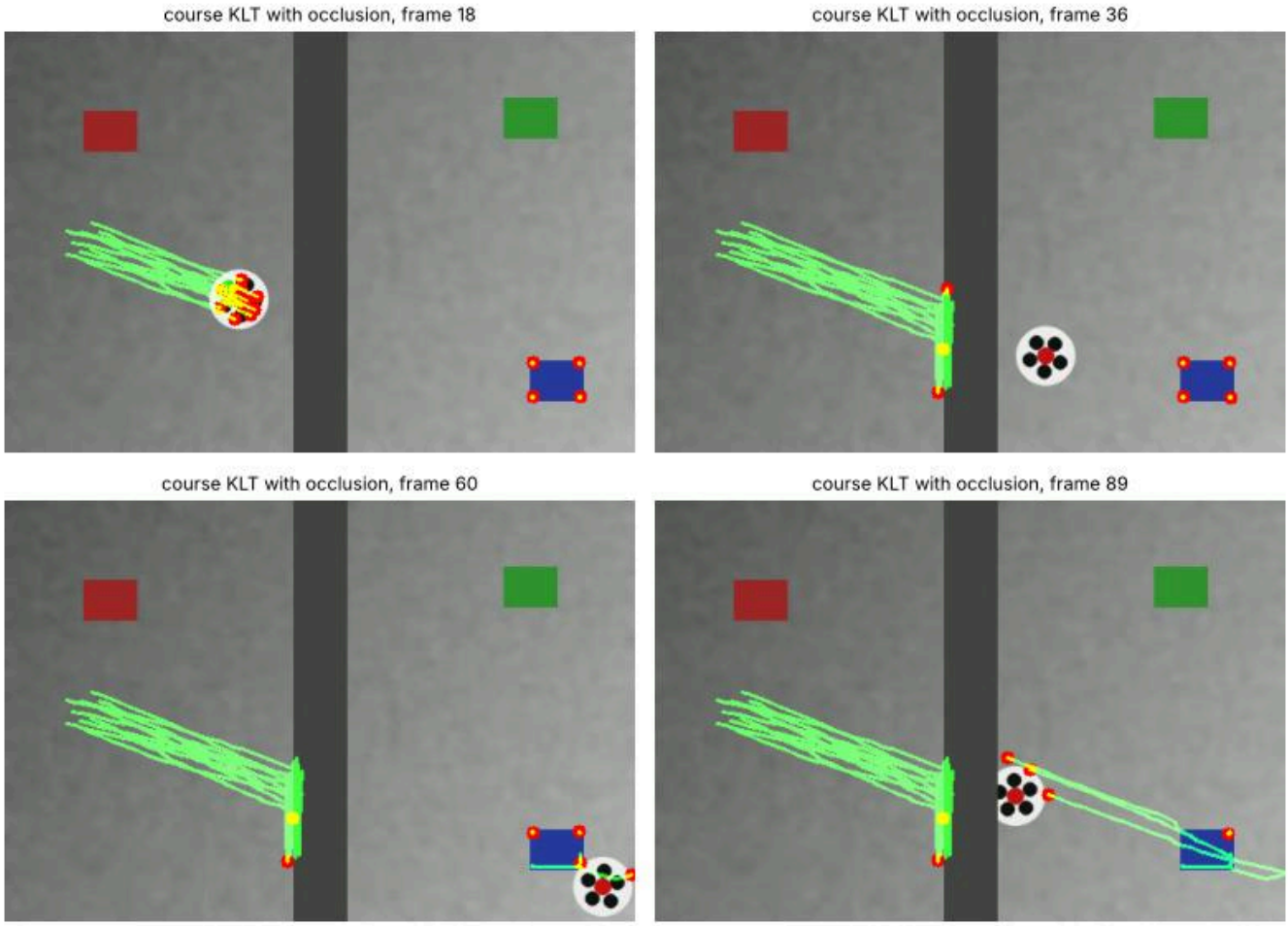
```
maxCorners  quality  found  alive@50  ms/frame
        10     0.30     10        10      0.82
       100     0.30     56        56      1.21
       500     0.30     56        56      1.20
       100     0.01    100       100      1.64
       500     0.01    500       500      4.18
      2000     0.01    619       619      4.90
```

- `maxCorners=100` and `500` with `qualityLevel=0.3` both return the same **56** corners: **the quality level is the real limit**, not `maxCorners`. Only with `qualityLevel=0.01` do we get 100, 500, and finally 619 (with `maxCorners=2000`, `minDistance=7` and the quality threshold limit it).
- Tracking cost grows with the number of points but stays tiny: under 10 ms per frame even for 600 points (exact times vary between runs). Sparse LK is cheap; the bigger cost in practice is often drawing and I/O.
- None of the points were lost within 50 frames ("alive@50" = "found"), again because most of them are on the static background.

4c · Occlusion: the ball passes behind a pillar

```
occ_frames, occ_gt = make_ball_video(occluder=True)
n0o, ob_o, hij_o = klt_on_frames(occ_frames, occ_gt)
covered = [abs(x - 240) < 20 + 22 for x, y in occ_gt]          # pillar spans x = 220..260, ball
radius 22
first = covered.index(True)
last = first + covered[first:].index(False) - 1                  # end of the FIRST pass behind
the pillar
behind = list(range(first, last + 1))
print(f"first pass behind the pillar: frames {first} ... {last} (the ball bounces and passes it
again later)")
print("original ball points still on the ball (every 5th frame):", ob_o[::5])
snaps_o = {}
_ = write_mp4(occ_frames, os.path.join(OUT, "ball_occlusion.mp4"), fps=25)
s_o, alive_o, _ = course_klt(os.path.join(OUT, "ball_occlusion.mp4"), MAX_FRAMES=89, snap_at=
(behind[0] - 3, behind[-1] + 3, 60, 89))
show([s_o[k] for k in sorted(s_o)], [f"course KLT with occlusion, frame {k}" for k in
sorted(s_o)], cols=2, size=5)
```

```
first pass behind the pillar: frames 21 ... 33 (the ball bounces and passes it again later)
original ball points still on the ball (every 5th frame): [12 12 12 12 12 12  0  0  0  0  0  0  0
0  0  0  0  0]
```



What happens: before the pillar all 12 ball points are on the ball (frame 18). When the ball goes behind the pillar (frames 21–33), the points do **not** disappear: they **stop at the pillar's edge** and stay there (frame 36: a column of points stuck on the edge while the ball has re-appeared on the right). From then on **0** original points are on the ball. Later, the ball rolls over the blue box and **drags some of its corners along** (frame 89: trails running from the blue box to the ball). Those are hijacked points, not real ball points.

This is exactly the textbook statement *"KLT struggles with occlusion and requires reinitialisation if features are lost"*. Notice that `st` stayed 1 for most of these points, so the status flag did not warn us.

5 · A better KLT tracker: re-detection + forward-backward check

The lecture notes say it plainly: KLT *"requires reinitialisation if features are lost"*. The course code never does this. Our improved tracker adds three standard ideas:

1. **Re-detection:** every `redetect_every` frames, or when fewer than `min_points` remain, detect new corners **away from existing ones** (using the `mask` argument of `goodFeaturesToTrack`) and add them.
2. **Forward-backward check** (Kalal et al., 2010): track a point forward (frame t → t+1), then track the result backward (t+1 → t). If it does not come back to within 1 px of where it started, the

track is unreliable, so drop it. This removes points that slid along edges or were hijacked at occlusions.

3. **Short, fading trails** (last 15 positions) instead of a mask that fills up forever.

```
class KLTTracker:
    def __init__(self, maxCorners=200, qualityLevel=0.01, minDistance=7, blockSize=7,
                 winSize=(15, 15), maxLevel=2, fb_thresh=1.0, min_points=40, redetect_every=10,
trail=15):
        self.fp = dict(maxCorners=maxCorners, qualityLevel=qualityLevel, minDistance=minDistance,
blockSize=blockSize)
        self.lk = dict(winSize=winSize, maxLevel=maxLevel,
                       criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03))
        self.fb_thresh, self.min_points, self.redetect_every, self.trail = fb_thresh, min_points,
redetect_every, trail
        self.tracks, self.prev, self.frame_idx = [], None, 0          # each track = list of (x,
y)

    def _detect(self, gray):
        mask = np.full(gray.shape, 255, np.uint8)
        for tr in self.tracks:                                         # don't detect next to
existing points
            cv2.circle(mask, tuple(int(v) for v in tr[-1]), self.fp["minDistance"], 0, -1)
        p = cv2.goodFeaturesToTrack(gray, mask=mask, **self.fp)
        if p is not None:
            self.tracks += [[tuple(pt)] for pt in p.reshape(-1, 2)]

    def update(self, frame):
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        if self.prev is not None and self.tracks:
            p0 = np.float32([tr[-1] for tr in self.tracks]).reshape(-1, 1, 2)
            p1, st1, _ = cv2.calcOpticalFlowPyrLK(self.prev, gray, p0, None, **self.lk)      #
forward
            p0r, st2, _ = cv2.calcOpticalFlowPyrLK(gray, self.prev, p1, None, **self.lk)     #
backward
            fb_err = np.linalg.norm((p0 - p0r).reshape(-1, 2), axis=1)
            good = (st1.ravel() == 1) & (st2.ravel() == 1) & (fb_err < self.fb_thresh)
            new_tracks = []
            for tr, (x, y), ok in zip(self.tracks, p1.reshape(-1, 2), good):
                if ok:
                    new_tracks.append((tr + [(x, y)])[-self.trail:])
            self.tracks = new_tracks
        if len(self.tracks) < self.min_points or self.frame_idx % self.redetect_every == 0:
            self._detect(gray)
        self.prev, self.frame_idx = gray, self.frame_idx + 1
        return self.tracks

    def draw(self, frame):
        vis = frame.copy()
        for tr in self.tracks:
            pts = np.int32(tr)
            moving = len(tr) > 1 and np.linalg.norm(np.subtract(tr[-1], tr[0])) > 2
            if moving:
                cv2.polylines(vis, [pts], False, (0, 255, 0), 2)
                cv2.circle(vis, tuple(pts[-1]), 3, (0, 0, 255), -1)
            else:
                cv2.circle(vis, tuple(pts[-1]), 1, (0, 140, 255), -1)       # static point: small
orange dot
        return vis
```

```
tracker = KLTTracker()
alive_fixed, moving_fixed, drawn, snaps_f = [], [], [], {}
t = time.perf_counter()
for i, f in enumerate(street):
    tracks = tracker.update(f)
    alive_fixed.append(len(tracks))
    moving_fixed.append(sum(1 for tr in tracks if len(tr) > 1 and
np.linalg.norm(np.subtract(tr[-1], tr[-2])) > 0.5))
```

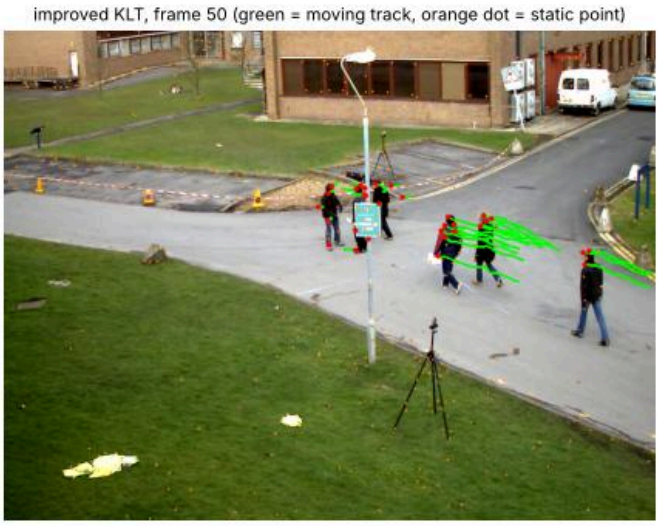
```
    vis = tracker.draw(f); drawn.append(vis)
    if i in (1, 50, 100, 149):
        snaps_f[i] = vis
print(f"{len(street)} frames in {time.perf_counter() - t:.1f}s")
write_mp4(drawn, os.path.join(OUT, "klt_fixed_street.mp4"))
show([snaps_f[k] for k in sorted(snaps_f)], [f"improved KLT, frame {k} (green = moving track,
orange dot = static point)" for k in sorted(snaps_f)], cols=2, size=5.5)

# Fair check: the course tracker with the SAME detection settings as the improved one (200
corners, qualityLevel 0.01),
# but still without re-detection, so we can see how much of the gain is due to re-detection
alone.
_, alive_c2, moving_c2 = course_klt(VIDEO, snap_at=(), feature_params=dict(maxCorners=200,
qualityLevel=0.01, minDistance=7, blockSize=7))

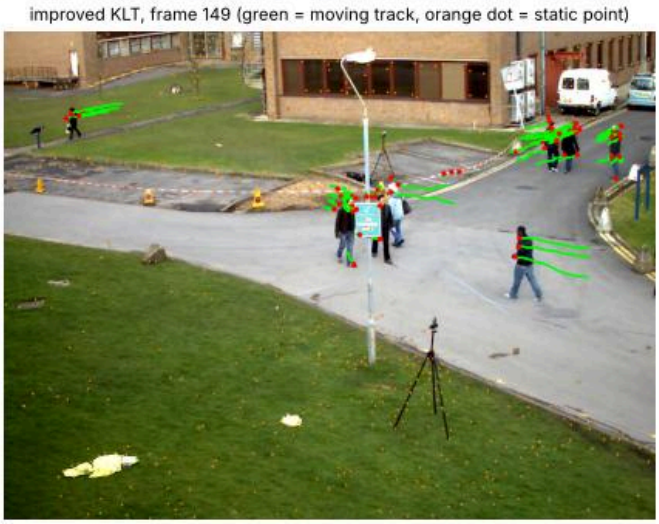
fig, ax = plt.subplots(1, 2, figsize=(12, 3.3))
ax[0].plot(alive_course, label="course KLT (never re-detects)")
ax[0].plot(alive_c2, label="course KLT, 200 corners @ quality 0.01")
ax[0].plot(alive_fixed, label="improved KLT")
ax[0].set_xlabel("frame"); ax[0].set_ylabel("points tracked (all)"); ax[0].legend();
ax[0].grid(alpha=.3)
ax[1].plot(moving_course, label="course KLT")
ax[1].plot(moving_c2, label="course KLT, 200 @ 0.01")
ax[1].plot(moving_fixed, label="improved KLT")
ax[1].set_xlabel("frame"); ax[1].set_ylabel("points that moved > 0.5 px");
ax[1].set_title("points on MOVING things (people)"); ax[1].legend(); ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
print("improved tracker: points at frames 0, 50, 100, 149 ->", [alive_fixed[i] for i in (0, 50,
100, 149)])
print(f"mean MOVING points per frame, frames 1-149: course {moving_course[1:].mean():.1f} | "
      f"course with 200 @ 0.01 {moving_c2[1:].mean():.1f} | improved
{np.mean(moving_fixed[1:]):.1f}")
print(f"mean MOVING points per frame, last 50 frames: course {moving_course[100:].mean():.1f} | "
      f"course with 200 @ 0.01 {moving_c2[100:].mean():.1f} | improved
{np.mean(moving_fixed[100:]):.1f}")
```

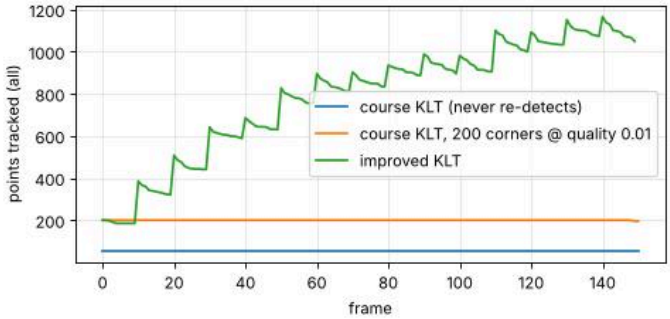
```
150 frames in 4.7s
```

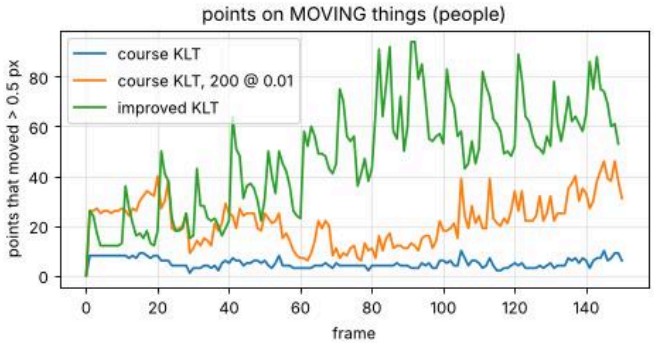








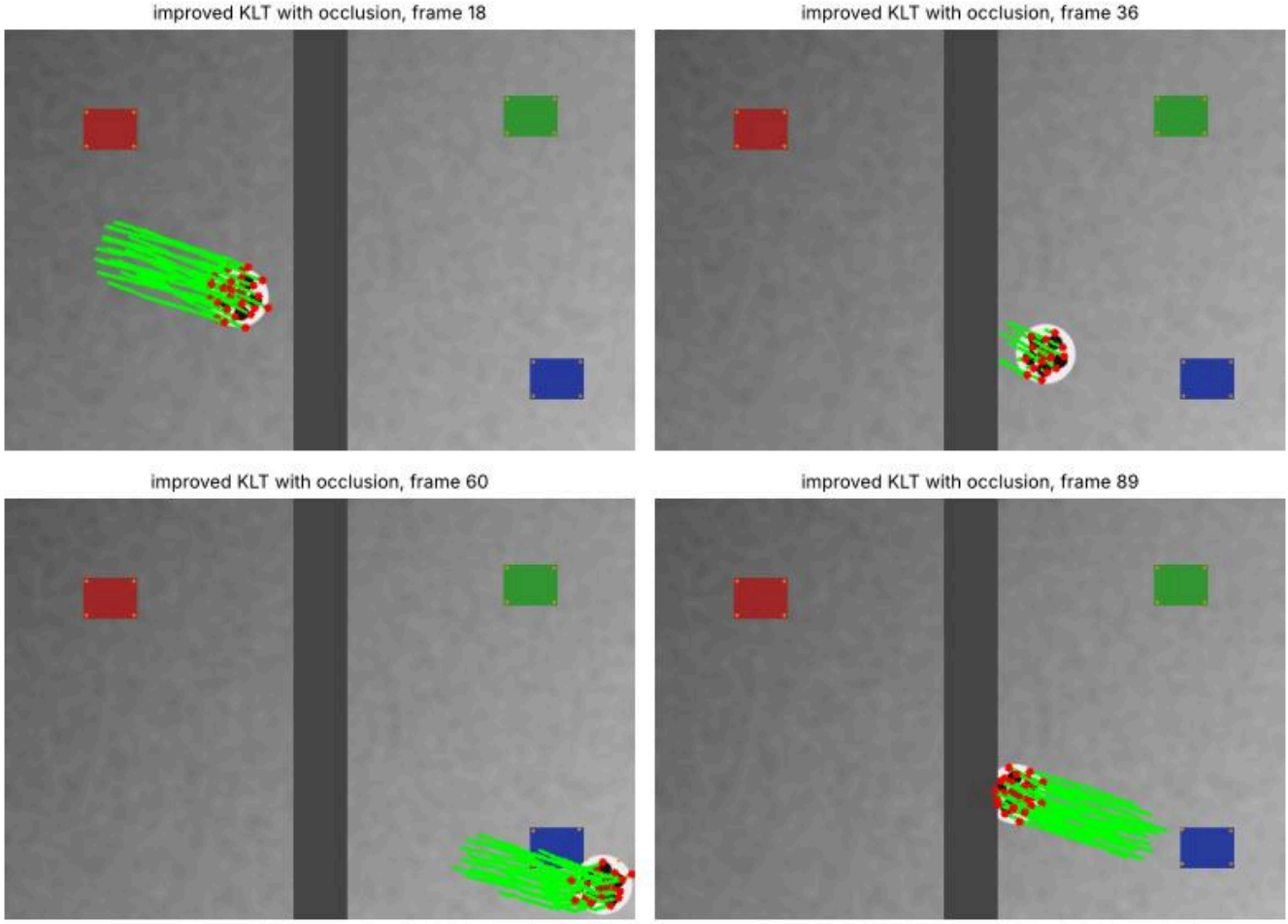





```
improved tracker: points at frames 0, 50, 100, 149 -> [200, 827, 981, 1049]
mean MOVING points per frame, frames 1-149: course 5.0 | course with 200 @ 0.01 21.8 | improved 4
7.5
mean MOVING points per frame, last 50 frames: course 5.1 | course with 200 @ 0.01 28.1 | improved
62.5
```

```
# Same improved tracker on the occlusion video: does the ball get picked up again after the
pillar?
tr_occ = KLTTracker(min_points=10, redetect_every=5)
on_ball_fixed, snaps_of = [], {}
for i, f in enumerate(occ_frames):
    tracks = tr_occ.update(f)
    on_ball_fixed.append(sum(1 for tr in tracks if np.linalg.norm(np.subtract(tr[-1], occ_gt[i]))
< 24))
    if i in (behind[0] - 3, behind[-1] + 3, 60, 89):
        snaps_of[i] = tr_occ.draw(f)
print("points on the ball, course tracker  (every 5th frame):", ob_o[::5] + hij_o[::5])
print("points on the ball, improved tracker (every 5th frame):", np.array(on_ball_fixed)[::5])
show([snaps_of[k] for k in sorted(snaps_of)], [f"improved KLT with occlusion, frame {k}" for k in
sorted(snaps_of)], cols=2, size=5)
```

```
points on the ball, course tracker  (every 5th frame): [12 12 12 12 12 12  0  0  0  0  0  0  0  0
1  0  1  1]
points on the ball, improved tracker (every 5th frame): [18 22 22 24 26 13  6 14 16 17 21 23 23 2
3 23 25 28 30]
```



Street video:

- The improved tracker follows **many more moving points**: on average **47.5 moving points per frame** against **5.0** for the course tracker (62.5 vs 5.1 in the last 50 frames), and people who enter later get tracked too (green tracks in the snapshots).
- **Fair comparison:** part of that gain comes simply from asking for more corners (`qualityLevel=0.01`). The course tracker with the *same* 200 corners at quality 0.01 but no re-detection reaches 21.8 moving points per frame (28.1 in the last 50). So re-detection plus the forward-backward check roughly **doubles** the useful points again, on top of the better detection settings.
- The total number of points grows (200 → 1049) because static background points are kept and new ones are added around them; `minDistance` stops them from piling up. In a real system you would remove points that have been static for a long time, or restrict detection to moving regions (§6d).

Occlusion video: the course tracker has 0 ball points after the pillar (the 1s near the end are the hijacked box corners). The improved tracker drops to **6** points on the ball while it is behind the pillar (frame 30), then **re-acquires** it within a few frames (14, 16, 17, 21, … up to 30 points). Re-detection finds the ball's corners again, and the forward-backward check removes the points that would otherwise stick to the pillar edge.

6 · Mini project: dense optical-flow motion tracker (Farneback)

6a · Sanity check: does Farneback measure the right motion?

Before trusting the colours, test with a motion we know: shift a street frame by exactly (+3, −2) pixels and see what flow Farneback reports.

```
g0 = gray_street[0]
M = np.float32([[1, 0, 3], [0, 1, -2]])                        # move 3 px right, 2 px up
g_shift = cv2.warpAffine(g0, M, (g0.shape[1], g0.shape[0]), borderMode=cv2.BORDER_REFLECT)
flow = cv2.calcOpticalFlowFarneback(g0, g_shift, None, 0.5, 3, 15, 3, 5, 1.2, 0)
inner = flow[20:-20, 20:-20]                                    # ignore the borders
print("flow shape:", flow.shape, "(H x W x 2: u = flow[...,0], v = flow[...,1])")
print(f"median flow: u = {np.median(inner[..., 0]):.2f} px, v = {np.median(inner[..., 1]):.2f} px
(true: +3, -2)")
print(f"pixels within 0.5 px of the truth: {100 * np.mean(np.hypot(inner[..., 0] - 3, inner[...,
1] + 2) < 0.5):.0f}%")
```

```
flow shape: (576, 768, 2) (H x W x 2: u = flow[...,0], v = flow[...,1])
median flow: u = 2.97 px, v = -1.98 px  (true: +3, -2)
pixels within 0.5 px of the truth: 82%
```

6b · The course motion tracker, explained

`cv2.calcOpticalFlowFarneback(prev, next, flow, pyr_scale, levels, winsize, iterations, poly_n, poly_sigma, flags)`. The course passes 10 positional arguments:

Position	Name	Course	Meaning
1, 2	`prev`, `next`	grey frames	the two consecutive frames (8-bit, single channel)
3	`flow`	`None`	output array (None = allocate a new one)
4	`pyr_scale`	0.5	each pyramid level is half the size of the previous one
5	`levels`	3	number of pyramid levels (1 = no pyramid); more → handles larger motion
6	`winsize`	15	averaging window; bigger = smoother, more robust to noise, blurrier motion edges
7	`iterations`	3	iterations at each pyramid level
8	`poly_n`	5	neighbourhood size for the polynomial fit (5 or 7)
9	`poly_sigma`	1.2	Gaussian smoothing for the polynomial fit (≈1.1 for poly_n=5, 1.5 for 7)
10	`flags`	0	e.g. `cv2.OPTFLOW_FARNEBACK_GAUSSIAN` (Gaussian window: slower, more accurate)

Visualising flow with HSV colours. Each pixel's flow vector (u, v) is converted to polar form with `cv2.cartToPolar`: **magnitude** (speed) and **angle** (direction, in radians 0…2π). Then:

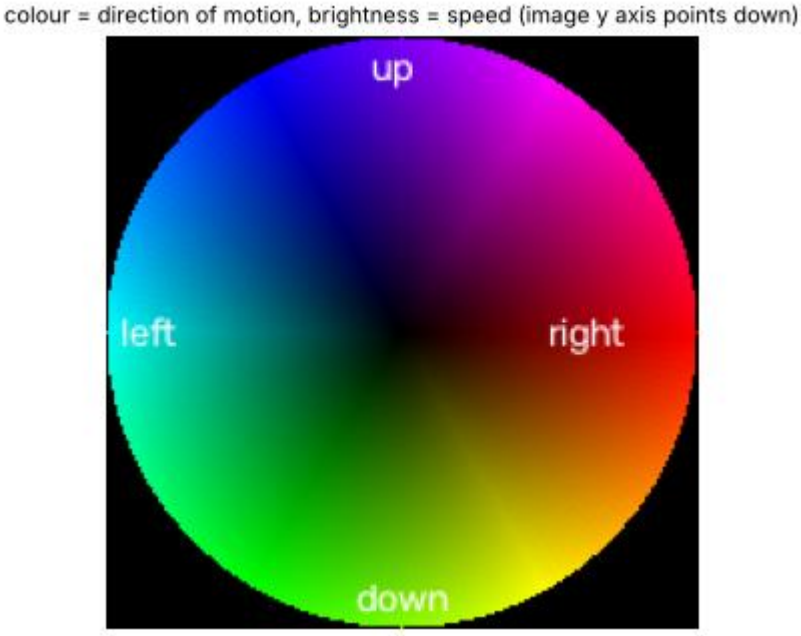
- **Hue = direction.** OpenCV stores 8-bit hue as 0…179 (degrees ÷ 2, so it fits in a byte), hence `ang*180/np.pi/2`.

- **Saturation = 255** (fully saturated colours).
- **Value (brightness) = speed**, scaled to 0…255 with `cv2.normalize(..., NORM_MINMAX)`.

So *black = not moving, colour = moving, and the colour tells the direction*. This colour wheel (made with the same code) is the legend:

```
def flow_to_bgr(flow, max_mag=None):
    """Course HSV encoding. max_mag=None -> course behaviour (per-frame NORM_MINMAX); a number ->
fixed scale."""
    mag, ang = cv2.cartToPolar(flow[..., 0], flow[..., 1])
    hsv = np.zeros((*flow.shape[:2], 3), np.uint8)
    hsv[..., 0] = ang * 180 / np.pi / 2
    hsv[..., 1] = 255
    if max_mag is None:
        hsv[..., 2] = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)
    else:
        hsv[..., 2] = np.clip(mag / max_mag * 255, 0, 255)
    return cv2.cvtColor(hsv, cv2.COLOR_HSV2BGR)

yy, xx = np.mgrid[-100:101, -100:101].astype(np.float32)
wheel = flow_to_bgr(np.dstack([xx, yy]) * (np.hypot(xx, yy) <= 100)[..., None], max_mag=100)
for txt, pos in [("right", (150, 105)), ("left", (5, 105)), ("down", (85, 195)), ("up", (90,
15))]:
    cv2.putText(wheel, txt, pos, cv2.FONT_HERSHEY_SIMPLEX, 0.45, (255, 255, 255), 1)
show(wheel, "colour = direction of motion, brightness = speed (image y axis points down)",
size=4)
```




```
cap = cv2.VideoCapture(VIDEO)  # course: 'chaplin.mp4'
ret, frame1 = cap.read()
if not ret:
    raise IOError("Cannot read first frame")
prvs = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
hsv = np.zeros_like(frame1)            # H, S, V image with the same size as the frame
hsv[...,1] = 255                       # saturation always full

MAX_FRAMES, snap_at = 100, (10, 40, 70, 100)
snaps_flow, vis_frames, scale_used, t_flow = {}, [], [], []
i = 0
while i < MAX_FRAMES:
    ret, frame2 = cap.read()
    if not ret:
        break
    i += 1
```

```
    next_gray = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)   # course called this variable `next`
(hides Python's built-in next())
    t = time.perf_counter()
    flow = cv2.calcOpticalFlowFarneback(prvs, next_gray, None, 0.5, 3, 15, 3, 5, 1.2, 0)
    t_flow.append(time.perf_counter() - t)
    mag, ang = cv2.cartToPolar(flow[...,0], flow[...,1])
    hsv[...,0] = ang*180/np.pi/2
    hsv[...,2] = cv2.normalize(mag,None,0,255,cv2.NORM_MINMAX)
    bgr = cv2.cvtColor(hsv,cv2.COLOR_HSV2BGR)
    # cv2.imshow('Motion Tracker', bgr); cv2.waitKey(30)  -> replaced:
    scale_used.append(mag.max())
    vis_frames.append(np.hstack([frame2, bgr]))
    if i in snap_at:
        snaps_flow[i] = np.hstack([frame2, bgr])
    prvs = next_gray

cap.release()
write_mp4(vis_frames, os.path.join(OUT, "farneback_street.mp4"))
print(f"{i} flow fields, {1000 * np.mean(t_flow):.0f} ms per frame on
{frame1.shape[1]}x{frame1.shape[0]} "
      f"({1 / np.mean(t_flow):.1f} frames per second)")
show([snaps_flow[k] for k in sorted(snaps_flow)], [f"frame {k}: video | course HSV flow" for k in
sorted(snaps_flow)], cols=1, size=10)
```

```
100 flow fields, 168 ms per frame on 768x576 (6.0 frames per second)
```

frame 10: video | course HSV flow



frame 40: video | course HSV flow

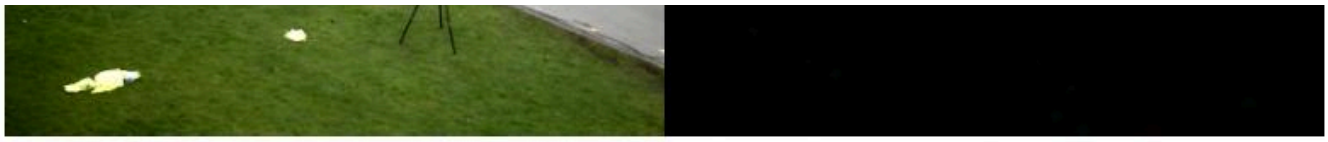


frame 70: video | course HSV flow



frame 100: video | course HSV flow



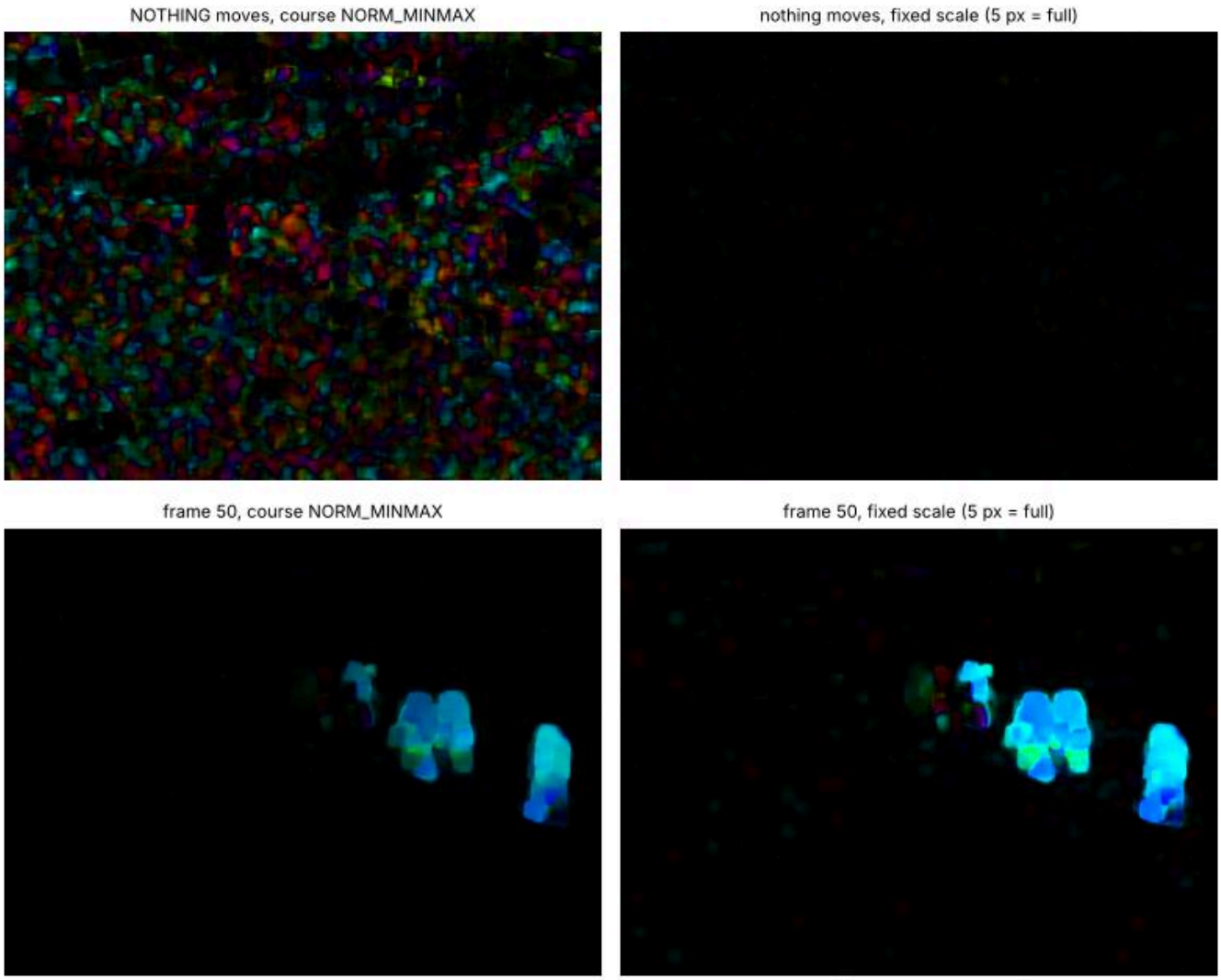


6c · A hidden problem with `NORM_MINMAX`: the brightness means something different in every frame

`cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX)` maps the **largest** motion **in that frame** to full brightness. So "bright" means "fast" only relative to the current frame. When nothing moves, tiny noise gets stretched to full brightness. Let's measure the scale used per frame, and show the extreme case: two frames that differ only by camera noise.

```
print(f"max flow magnitude per frame (= what 'full brightness' means): min {min(scale_used):.1f},
"
      f"max {max(scale_used):.1f} px/frame")
rng = np.random.default_rng(0)
noisy = np.clip(gray_street[0].astype(np.int16) + rng.normal(0, 2, gray_street[0].shape), 0,
255).astype(np.uint8)
flow_static = cv2.calcOpticalFlowFarneback(gray_street[0], noisy, None, 0.5, 3, 15, 3, 5, 1.2, 0)
flow_real = cv2.calcOpticalFlowFarneback(gray_street[49], gray_street[50], None, 0.5, 3, 15, 3,
5, 1.2, 0)
print(f"static scene + noise: largest 'motion' {np.hypot(*flow_static.transpose(2, 0,
1)).max():.2f} px")
show([flow_to_bgr(flow_static), flow_to_bgr(flow_static, max_mag=5), flow_to_bgr(flow_real),
flow_to_bgr(flow_real, max_mag=5)],
     ["NOTHING moves, course NORM_MINMAX", "nothing moves, fixed scale (5 px = full)",
      "frame 50, course NORM_MINMAX", "frame 50, fixed scale (5 px = full)"], cols=2, size=5)
```

```
max flow magnitude per frame (= what 'full brightness' means): min 4.3, max 32.4 px/frame
static scene + noise: largest 'motion' 0.41 px
```



- In the street video the value used as "full brightness" ranged from **4.3 to 32.4 px/frame** between frames, so the same colour brightness means a speed up to ~8× different from one frame to the next. That is why the course output flickers.
- **Extreme case:** two frames that differ only by a little random noise (σ = 2 grey levels). The largest "motion" is only **0.41 px**, but `NORM_MINMAX` stretches it to full brightness and the static scene looks like a storm of motion (top left). With a **fixed scale** (5 px/frame = full brightness), it is correctly almost black (top right).
- For real motion (frame 50) both look similar, because the walkers dominate.

Fix: use a fixed `max_mag` (as in `flow_to_bgr(flow, max_mag=5)`), so brightness always means the same speed.

6d · From colours to a motion *tracker*: boxes around moving objects

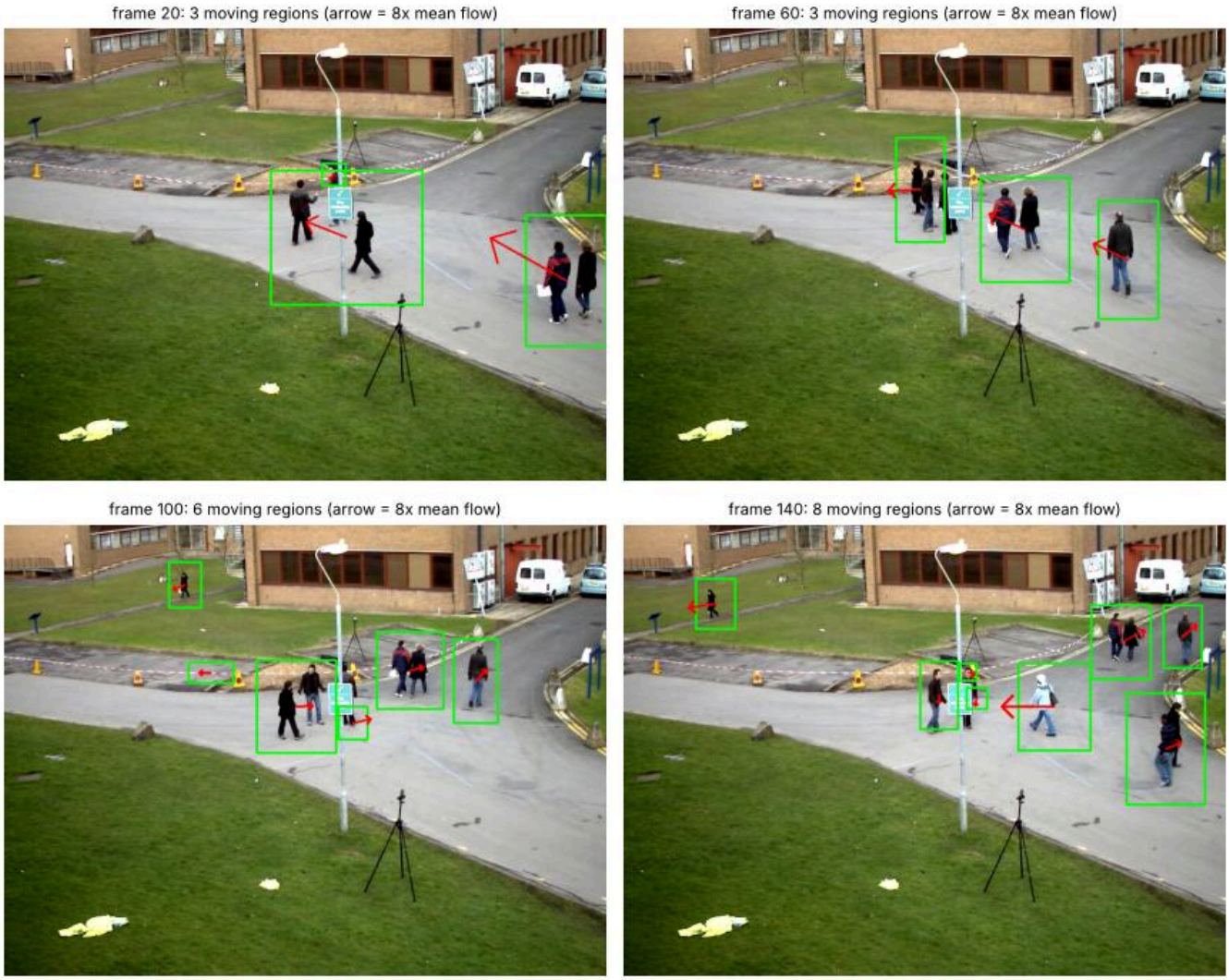
The course visualises motion but does not actually *track* anything. A simple extension turns dense flow into detections:

1. **threshold** the flow magnitude (pixels moving faster than 1 px/frame are "moving");
2. clean the mask with **morphology** (opening removes specks, closing/dilation fills holes);
3. find **connected regions** (contours) and keep those larger than 400 px²;

4. draw a **bounding box** and the region's average motion direction.

```
def moving_boxes(prev_gray, gray, mag_thresh=1.0, min_area=400):
    flow = cv2.calcOpticalFlowFarneback(prev_gray, gray, None, 0.5, 3, 15, 3, 5, 1.2, 0)
    mag = np.hypot(flow[..., 0], flow[..., 1])
    moving = (mag > mag_thresh).astype(np.uint8) * 255
    moving = cv2.morphologyEx(moving, cv2.MORPH_OPEN, np.ones((5, 5), np.uint8))
    moving = cv2.dilate(moving, np.ones((9, 9), np.uint8))
    contours, _ = cv2.findContours(moving, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    boxes = []
    for c in contours:
        if cv2.contourArea(c) < min_area:
            continue
        x, y, w, h = cv2.boundingRect(c)
        region = mag[y:y + h, x:x + w] > mag_thresh
        u, v = flow[y:y + h, x:x + w, 0][region].mean(), flow[y:y + h, x:x + w, 1][region].mean()
        boxes.append((x, y, w, h, u, v))
    return boxes, moving

panels, n_boxes = [], []
for i in range(1, len(street)):
    boxes, moving = moving_boxes(gray_street[i - 1], gray_street[i])
    n_boxes.append(len(boxes))
    if i in (20, 60, 100, 140):
        vis = street[i].copy()
        for x, y, w, h, u, v in boxes:
            cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cx, cy = x + w // 2, y + h // 2
            cv2.arrowedLine(vis, (cx, cy), (int(cx + 8 * u), int(cy + 8 * v)), (0, 0, 255), 2,
tipLength=0.3)
        panels.append(vis)
show(panels, [f"frame {k}: {n_boxes[k - 1]} moving regions (arrow = 8x mean flow)" for k in (20,
60, 100, 140)], cols=2, size=5.5)
print("moving regions per frame: min", min(n_boxes), "| median", int(np.median(n_boxes)), "|
max", max(n_boxes))
```




```
moving regions per frame: min 2 | median 5 | max 9
```

- The boxes follow the walkers and the arrows point in their walking direction: a working, if simple, **motion tracker** built from dense flow.
- We found 2–9 moving regions per frame (median 5). **Honest limitations:** people walking close together merge into **one box** (frame 20: two people in one large box); a person who stands still disappears (no motion → no box); the thresholds (1 px/frame, 400 px²) were picked by eye for this video and will need tuning for others. This method also has no identity: box #1 in one frame is not linked to box #1 in the next (that would need data association, e.g. matching boxes by overlap, or a Kalman filter).

6e · Sparse vs dense: how much slower is dense flow?

Same pair of frames, timed (median of 5 runs). `DIS` (Dense Inverse Search, in OpenCV since 4.0) is included as a modern fast dense method.

```
def med_ms(fn, reps=5):
    ts = []
    for _ in range(reps):
        t = time.perf_counter(); fn(); ts.append(time.perf_counter() - t)
    return 1000 * np.median(ts)

a, b = gray_street[50], gray_street[51]
a2, b2 = cv2.resize(a, None, fx=0.5, fy=0.5), cv2.resize(b, None, fx=0.5, fy=0.5)
p100 = cv2.goodFeaturesToTrack(a, 100, 0.3, 7, blockSize=7)
```

```
p1000 = cv2.goodFeaturesToTrack(a, 1000, 0.01, 7, blockSize=7)
dis = cv2.DISOpticalFlow_create(cv2.DISOPTICAL_FLOW_PRESET_FAST)
rows = [
    (f"Sparse LK, {len(p100)} points (course params)", med_ms(lambda: cv2.calcOpticalFlowPyrLK(a,
b, p100, None, **lk)), len(p100)),
    (f"Sparse LK, {len(p1000)} points", med_ms(lambda: cv2.calcOpticalFlowPyrLK(a, b, p1000,
None, **lk)), len(p1000)),
    ("Dense Farneback 768x576 (course params)", med_ms(lambda: cv2.calcOpticalFlowFarneback(a, b,
None, 0.5, 3, 15, 3, 5, 1.2, 0)), a.size),
    ("Dense Farneback 384x288 (half size)", med_ms(lambda: cv2.calcOpticalFlowFarneback(a2, b2,
None, 0.5, 3, 15, 3, 5, 1.2, 0)), a2.size),
    ("Dense DIS (fast preset) 768x576", med_ms(lambda: dis.calc(a, b, None)), a.size),
]
for name, ms, n in rows:
    print(f"{name:42s} {ms:8.2f} ms  ({1000 / ms:7.0f} fps)  vectors: {n:,}")
print(f"\nFarneback (full) / sparse LK ({len(p100)} pts): {rows[2][1] / rows[0][1]:.0f}x slower")
```

```
Sparse LK, 66 points (course params)           1.04 ms  (    957 fps)  vectors: 66
Sparse LK, 681 points                          7.97 ms  (    125 fps)  vectors: 681
Dense Farneback 768x576 (course params)      158.71 ms  (      6 fps)  vectors: 442,368
Dense Farneback 384x288 (half size)           34.10 ms  (     29 fps)  vectors: 110,592
Dense DIS (fast preset) 768x576               13.66 ms  (     73 fps)  vectors: 442,368

Farneback (full) / sparse LK (66 pts): 152x slower
```

Reading the timings (exact values vary with machine load; the order does not):

- **Sparse LK on ~66 points takes only 1–2 ms**: real time with room to spare, even at 681 points.
- **Dense Farneback at full 768×576 resolution took well over 100 ms per frame** in every run (about 5–8 fps), i.e. **more than 100× slower than sparse LK**. It computes 442,368 vectors instead of 66.
- **Halving the resolution makes Farneback about 4–5× faster** (4× fewer pixels). This is the first thing to try if you need dense flow in real time.
- **DIS** (fast preset) produced dense flow for the full frame in about 15 ms, much faster than Farneback.

This is the efficiency vs detail trade-off from the lecture: *sparse = efficient, dense = detailed*.

Optional: live webcam (only on your own computer)

Inside Jupyter we cannot open a window with `cv2.imshow`. If you want to try your webcam, this cell grabs a few frames, computes Farneback flow and shows them with matplotlib. It is switched off (`USE_WEBCAM = False`) so the notebook runs anywhere, including on servers without a camera.

```
if USE_WEBCAM:
    cam = cv2.VideoCapture(0)
    ok, f0 = cam.read()
    if not ok:
        print("No webcam found")
    else:
        prev = cv2.cvtColor(f0, cv2.COLOR_BGR2GRAY)
        shots = []
        for _ in range(30):                          # about 1 second: wave your hand!
            ok, f = cam.read()
            if not ok:
                break
            g = cv2.cvtColor(f, cv2.COLOR_BGR2GRAY)
            shots.append(np.hstack([f, flow_to_bgr(cv2.calcOpticalFlowFarneback(prev, g, None,
0.5, 3, 15, 3, 5, 1.2, 0), max_mag=10)]))
            prev = g
```

```
        cam.release()
        show(shots[::10], None, cols=1, size=9)
else:
    print("Webcam demo skipped (USE_WEBCAM = False).")
```

```
Webcam demo skipped (USE_WEBCAM = False).
```

7 · 🔧 What we changed from the course notebook, and why

#	Course code	Problem (verified)	What we did
1	`cv2.VideoCapture('chaplin.mp4')`, exercise `ball.mp4`	Videos not provided → `IOError("Cannot open video file")`	`vtest.avi` (OpenCV samples) downloaded automatically; a synthetic ball video with **known ground truth** generated in code
2	`cv2.imshow` / `cv2.waitKey(30)` / `destroyAllWindows` loops	Do not work in Jupyter (no GUI window; on servers they crash or hang)	Store frames, show snapshot grids with matplotlib, write `.mp4` files to `output/`
3	KLT never re-detects features	Points that are lost are never replaced; new objects entering the scene are never tracked; once every point is lost, `p1 is None` and the loop silently **stops**	Re-detection (periodic or when too few points), away from existing points
4	Only `st == 1` filters bad points	Points that slide along edges or get dragged by a passing object (hijacked) keep status 1	Forward-backward consistency check (1 px)
5	Trail `mask` accumulates forever	After a while the image is covered with old green lines	Trails of the last 15 positions only
6	`qualityLevel=0.3`	Very strict: on the street video only 56 corners pass in frame 0, few of them on people	Explained; the improved tracker uses 0.01
7	Farneback: `hsv[...,2] = cv2.normalize(mag, ..., NORM_MINMAX)`	Brightness is rescaled every frame: a static scene with only noise looks as bright as real motion (§6c)	Optional fixed scale (`max_mag`)
8	Variable named `next`	Hides Python's built-in `next()` (harmless here, a bad habit)	Renamed `next_gray`
9	Mini project "highlights moving objects" only as colours	No actual tracking output	Threshold + morphology + contours → bounding boxes and direction arrows

#	Course code	Problem (verified)	What we did
10	No check that `goodFeaturesToTrack` found anything	It returns `None` on a blank/flat first frame → `calcOpticalFlowPyrLK` raises an assertion error	`_detect` checks for `None`

8 · Results, honestly

- **The course KLT code is correct as far as it goes:** on our synthetic ball (7 px/frame) it kept all 12 ball points for all 89 frames. Its problems are long-term ones: no re-detection (on the street video only 0–9 of 56 points move in any frame, and new people are never tracked) and no defence against occlusion (points stick to the pillar edge, and the status flag does not notice).
- **Pyramids matter, and the measurements show it:** without a pyramid the ball was lost at 12 px/frame; with the course's `maxLevel=2` it was tracked up to 16 px/frame (92 %). **Nothing worked at 24 px/frame**, and the biggest window/pyramid was not the best, because the window fills with background.
- **Re-detection and the forward-backward check** gave many more useful (moving) points than simply lowering `qualityLevel` (47.5 vs 21.8 per frame), and they re-acquired the ball after the occlusion.
- **The Farneback mini project works** (moving people clearly coloured by direction; the synthetic shift test gave a median flow of (2.97, −1.98) against the true (3, −2)). But the course's `NORM_MINMAX` brightness is misleading, and dense flow was over 100× slower than sparse LK on this machine.
- The motion boxes are a simple, honest baseline: they merge nearby people and lose anyone who stops.
- Limitation of our synthetic test: a perfect, noise-free ball on a simple background is easier than real footage (motion blur, lighting changes, camera shake).

9 · Common errors and how to fix them

Error / symptom	Cause	Fix
`IOError: Cannot open video file` / `cap.isOpened()` is False	Wrong path, or the codec is not supported by your OpenCV build	Check the path; try another container (`.avi` with MJPG, `.mp4` with mp4v); `pip install opencv-python` (includes FFmpeg)
`ret` is False on the first `cap.read()`	Video is empty or unreadable	Check `cap.get(cv2.CAP_PROP_FRAME_COUNT)` ; re-encode the file
Kernel hangs or crashes at `cv2.imshow`	No GUI available inside Jupyter or on a server	Use matplotlib or write a video file (this notebook)

Error / symptom	Cause	Fix
`(-215:Assertion failed) (npoints = prevPtsMat.checkVector(2, CV_32F, true)) >= 0`	`p0` is `None` (no corners found) or not `float32` of shape (N, 1, 2)	Check `p0 is not None`; `p0 = np.float32(p0).reshape(-1, 1, 2)`
Tracking just stops after a while	All points lost and the course loop `break`s on `p1 is None`	Re-detect features (§5)
Points stick to the background or slide off the object	Large motion, occlusion, aperture problem	Increase `maxLevel` / `winSize`, forward-backward check, re-detect
`cv2.add(frame, mask)` error about sizes/types	`mask` created from a different frame size	Create it with `np.zeros_like(first_frame)` from the same video
Farneback very slow	Full-resolution frames	Downscale (half size ≈ 4× faster), or use DIS optical flow
Flow colours flicker frame to frame	`NORM_MINMAX` rescales each frame	Fixed magnitude scale

10 · Try this yourself

1. In §4a, add `maxLevel=4` and `winSize=(51, 51)`. Can you track the 24 px/frame ball now? Then make the ball bigger (`r=40`). Why does the size of the ball matter?
2. Change `qualityLevel` in the course KLT from 0.3 to 0.01 on the street video. How many more people get tracked?
3. Replace the pillar with a **semi-transparent** one (`cv2.addWeighted`). Does KLT survive partial occlusion better?
4. Use the forward-backward error to **colour** each track by its reliability.
5. In §6d, use the **direction** of each moving region to count people walking left vs right.
6. Record a 5-second phone video of a ball rolling across a table, load it with `read_frames`, and run both trackers.
7. Compare Farneback with DIS on the street video. Which gives cleaner outlines of the people?

11 · Viva questions (with short model answers)

1. **What is optical flow?** The apparent motion of brightness patterns between two frames: a 2-D vector (u, v) per pixel or per point.
2. **What is the brightness constancy assumption, and when does it fail?** A moving point keeps its brightness: I(x, y, t) = I(x+u, y+v, t+1). It fails with lighting changes, shadows, reflections, and occlusion.
3. **What is the aperture problem?** One pixel gives one equation ($I_x u + I_y v + I_t = 0$) for two unknowns; through a small window, an edge only reveals motion perpendicular to it. Corners solve it, because they have gradients in two directions.

4. **How does Lucas–Kanade solve it?** It assumes all pixels in a small window share the same motion and solves the over-determined system by least squares (a 2×2 system, the same matrix as Harris/Shi–Tomasi).
5. **Why pyramids (`maxLevel`)?** LK only works for small displacements (~1–2 px). On a pyramid, large motions become small at coarse levels; estimates are refined level by level. In our test the 16 px/frame ball was lost without a pyramid and tracked with `maxLevel=2` .
6. **What does KLT stand for and what are its steps?** Kanade–Lucas–Tomasi: (1) detect good features to track (Shi–Tomasi corners), (2) track them with pyramidal Lucas–Kanade frame by frame.
7. **Shi–Tomasi vs Harris?** Both use the 2×2 gradient matrix; Harris scores R = det − k·trace², Shi–Tomasi uses min(λ1, λ2), which is better suited to tracking.
8. **Explain `qualityLevel=0.3` , `minDistance=7` , `maxCorners=100` .** Keep corners whose score is ≥ 30 % of the best, at least 7 px apart, at most 100.
9. **Explain `winSize` , `maxLevel` , `criteria` in `lk_params` .** Window assumed to move together (15×15); number of extra pyramid levels (2); stop iterating after 10 iterations or when the update < 0.03 px.
10. **What do `st` and `err` mean?** `st[i] = 1` if the flow for point i was found, 0 if lost; `err` is the residual matching error.
11. **Why does the course tracker degrade over time, and how do you fix it?** Lost points are never replaced and new objects are never detected. Fix: periodic re-detection, forward-backward check.
12. **Sparse vs dense flow?** Sparse: a few points, fast (KLT). Dense: every pixel (Farneback), slower (on our machine dense was well over 10× slower than sparse with 100 points; see §6e), used for motion maps and segmentation.
13. **How is dense flow visualised?** Convert (u, v) to magnitude and angle; hue = direction (angle/2 to fit OpenCV's 0–179 hue), saturation = 255, value = normalised magnitude; convert HSV → BGR.
14. **What is wrong with `NORM_MINMAX` for the brightness?** It rescales every frame to its own maximum, so the same colour means different speeds in different frames, and noise in a static frame looks like strong motion.
15. **How does occlusion affect KLT?** Points on the hidden object are lost or jump onto the occluder/background; KLT has no memory of the object, so it needs re-detection (or a motion model such as a Kalman filter) to recover.
16. **Real-world uses?** Video stabilisation, sports analytics, traffic monitoring, driver-assistance (ego-motion), action recognition, SLAM/visual odometry.

Day 11 · Machine learning in computer vision: kNN and SVM on handwritten digits

Course lab: `CV-Week2-Lecture4.ipynb` (Day 11: Introduction to ML in CV) **Notes chapter:** `week-2/notes/04_ml_in_cv_knn_svm.md` **Run time:** about 2 minutes on a laptop CPU (the MNIST timing experiment in section 7 is the slow part, about 1 minute).

What you will learn

- Why we *learn* a classifier from examples instead of writing rules by hand.
- How **k-Nearest Neighbours (kNN)** and **Support Vector Machines (SVM)** classify an image.
- How to read a `classification_report` (precision, recall, f1-score, support) and a confusion matrix.
- What `n_neighbors`, `kernel`, `C` and `gamma` actually do, with experiments.
- How to build a digit recogniser that works on a digit **you draw yourself**, and the preprocessing that makes it work.

Every code cell from the course notebook is here, with the same names and calls, explained line by line. Where the course code had a bug or a misleading test, we fix it and say why (section 9).

1 · The idea in plain language

Hand-written rules do not scale. Try writing an `if` statement that recognises a "7" from pixel values: every person writes it differently. Machine learning flips the job around: we give the computer **many labelled examples** ("this picture is a 7") and it works out the rule itself. This is **supervised learning**: input → label → prediction.

To a classical ML model an image is just a **list of numbers** (a *feature vector*). An 8×8 grey image becomes 64 numbers, one per pixel. Two images of the same digit should give "nearby" lists of numbers.

kNN: "ask your neighbours"

> You move to a new town and want to know which football team to support. You ask your **5 nearest neighbours** and go with the majority.

kNN does exactly that. To classify a new image it:

1. measures the **distance** (usually Euclidean, straight-line distance in 64-D space) from the new image to **every** training image,
2. picks the **k** closest ones (`n_neighbors=k`),
3. takes a **majority vote** of their labels.

There is no real training: `fit()` just stores the data. All the work happens at prediction time, which is why kNN gets slower as the training set grows.

SVM: "draw the widest street between the classes"

> Imagine red and blue marbles on a table. An SVM lays a ruler (a line, or in many dimensions a *hyperplane*) between them so that the **gap (margin) on each side is as wide as possible**. The marbles that touch the edge of the street are the **support vectors**; they alone decide where the street goes.

- `kernel='linear'` : the street is straight.
- `kernel='rbf'` : the **kernel trick** lets the boundary curve around groups of points, by implicitly mapping the data into a higher-dimensional space.
- `C` = how strict the SVM is about training mistakes. Large `C` → tries hard to classify every training point correctly (narrow street, risk of overfitting). Small `C` → allows some mistakes for a wider, smoother street.
- `gamma` (RBF only) = how far the influence of one training point reaches. Large `gamma` → very local, wiggly boundary; small `gamma` → smooth boundary. **A linear kernel has no `gamma` .**

SVMs handle 10 classes by training many two-class SVMs (scikit-learn's `SVC` uses **one-vs-one**: 45 little SVMs, one per pair of digits, which then vote).

2 · Setup

Install once from a terminal: `pip install scikit-learn matplotlib opencv-python torch torchvision` (torch/torchvision are only needed for the MNIST experiment in section 7; the rest works without them).

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import cv2
import sklearn
from threadpoolctl import threadpool_limits

threadpool_limits(2)          # optional: use at most 2 CPU cores so the laptop stays responsive
np.random.seed(0)
plt.rcParams["figure.dpi"] = 80
print("scikit-learn", sklearn.__version__, "| OpenCV", cv2.__version__)
```

```
scikit-learn 1.9.1 | OpenCV 5.0.0
```

3 · The course code: load the digits, split, look at one

This is the course cell, unchanged apart from comments. One correction to the course comment: `datasets.load_digits()` is **not MNIST**. It is the small **UCI optical digits** set that ships inside scikit-learn: 1,797 images of **8×8 pixels**, with values from **0 to 16** (how many "ink" pixels fell into each 4×4 block of an original 32×32 scan). Real MNIST is 70,000 images of 28×28 pixels with values 0–255; we use it in section 7 and on Day 15.

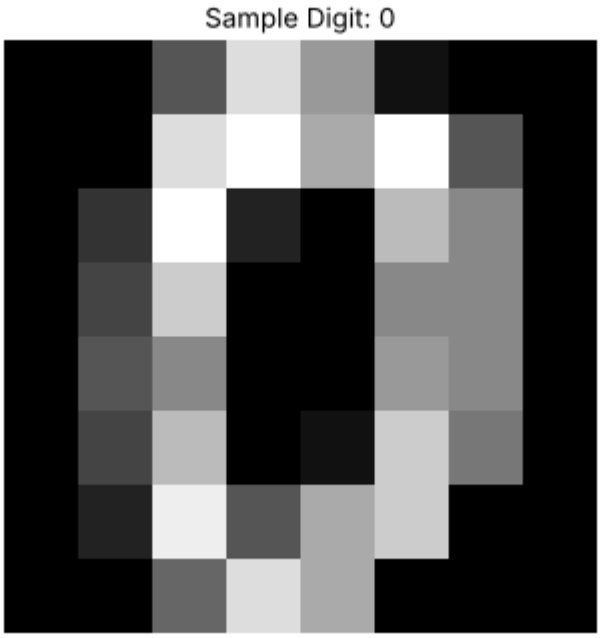
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix

# Load the scikit-learn digits dataset (8x8 images, values 0-16; NOT the 28x28 MNIST)
digits = datasets.load_digits()
X = digits.images.reshape((len(digits.images), -1))   # (1797, 8, 8) -> (1797, 64): each image
becomes a row of 64 numbers
y = digits.target                                      # the true label 0-9 of each image

# Split into train and test: 70% to learn from, 30% kept aside to measure honest accuracy
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Show sample digit
plt.imshow(digits.images[0], cmap='gray')
plt.title(f"Sample Digit: {digits.target[0]}")
plt.axis('off')
plt.show()
```

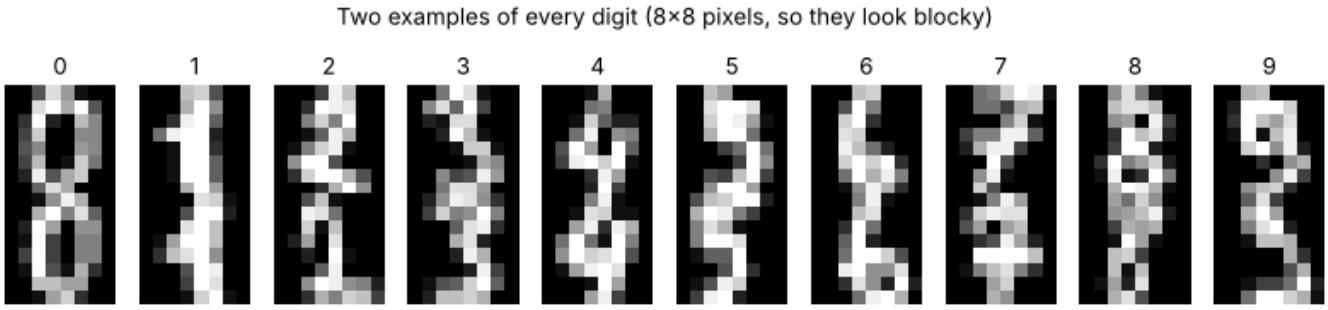



```
print("images:", digits.images.shape, "-> feature matrix X:", X.shape)
print("pixel value range:", X.min(), "to", X.max())
print("train:", X_train.shape, " test:", X_test.shape)
print("images per class in the test set:", np.bincount(y_test))

fig, axes = plt.subplots(2, 10, figsize=(10, 2.4))
for d in range(10):
    for r in range(2):
        idx = np.where(y == d)[0][r]
        axes[r, d].imshow(digits.images[idx], cmap='gray'); axes[r, d].axis('off')
    axes[0, d].set_title(str(d))
plt.suptitle("Two examples of every digit (8x8 pixels, so they look blocky)");
plt.tight_layout(); plt.show()

print("The first digit as the 8x8 grid of numbers the classifier sees:")
print(digits.images[0].astype(int))
```

```
images: (1797, 8, 8) -> feature matrix X: (1797, 64)
pixel value range: 0.0 to 16.0
train: (1257, 64)  test: (540, 64)
images per class in the test set: [53 50 47 54 60 66 53 55 43 59]
```




```
The first digit as the 8x8 grid of numbers the classifier sees:
[[ 0  0  5 13  9  1  0  0]
 [ 0  0 13 15 10 15  5  0]
 [ 0  3 15  2  0 11  8  0]
 [ 0  4 12  0  0  8  8  0]
 [ 0  5  8  0  0  9  8  0]
 [ 0  4 11  0  1 12  7  0]
 [ 0  2 14  5 10 12  0  0]
 [ 0  0  6 13 10  0  0  0]]
```

Line by line

- `reshape((len(digits.images), -1))` flattens each 8×8 image into 64 numbers. `-1` means "work this size out for me". kNN and SVM need one flat row per sample.
- `train_test_split(..., test_size=0.3, random_state=42)` shuffles and holds back 30% (540 images) for testing. The model never sees them during training, so the test score estimates how it does on **new** images. `random_state=42` makes the shuffle repeatable.
- In the grid of numbers, 0 = background and 16 = full ink. With `cmap='gray'` that shows as a **white digit on a black background**; remember this for the mini project.

4 · Exercise: classify digits with kNN (course code)

```python
# Train kNN classifier
knn = KNeighborsClassifier(n_neighbors=5)   # vote among the 5 closest training images
knn.fit(X_train, y_train)                   # "training" = just storing X_train and y_train
y_pred_knn = knn.predict(X_test)            # for each test image: distances to all 1257 training
images, top-5 vote

print("kNN Classification Report:")
print(classification_report(y_test, y_pred_knn))
```

```
kNN Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        53
           1       1.00      1.00      1.00        50
           2       1.00      1.00      1.00        47
           3       0.98      1.00      0.99        54
           4       0.98      1.00      0.99        60
           5       0.98      0.98      0.98        66
           6       1.00      1.00      1.00        53
           7       1.00      1.00      1.00        55
           8       1.00      1.00      1.00        43
           9       0.98      0.95      0.97        59

    accuracy                           0.99       540
   macro avg       0.99      0.99      0.99       540
weighted avg       0.99      0.99      0.99       540
```

How to read a `classification_report` (examiners love this)

Take one row, say digit **9**. Out of all test images:

- **precision** = of the images the model *called* "9", what fraction really were 9? ("When it says 9, can I trust it?") precision = TP / (TP + FP)
- **recall** = of the images that really *are* 9, what fraction did the model find? ("Did it miss any 9s?") recall = TP / (TP + FN)
- **f1-score** = the harmonic mean of the two, 2·P·R / (P + R). It is high only if **both** are high.
- **support** = how many test images of that class there are (it is a count, not a score).

The bottom rows: **accuracy** = correct / total over all classes; **macro avg** = plain average over the 10 classes (each class counts equally); **weighted avg** = average weighted by support (bigger classes count more).

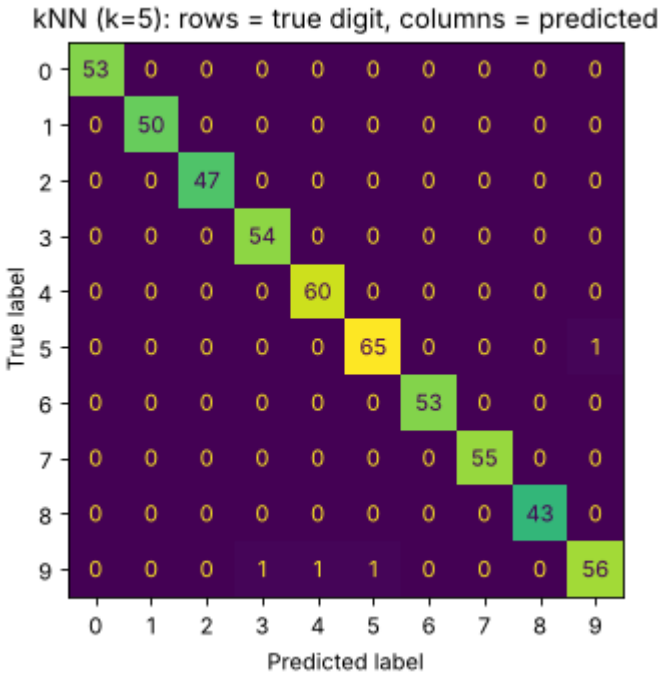
TP = true positives, FP = false positives (wrongly called 9), FN = false negatives (a real 9 called something else).

A **confusion matrix** shows the same information as a grid: row = true digit, column = predicted digit. Everything off the diagonal is a mistake.

```
from sklearn.metrics import ConfusionMatrixDisplay, accuracy_score

print(f"kNN test accuracy: {accuracy_score(y_test, y_pred_knn):.4f}  "
      f"({(y_pred_knn != y_test).sum()} mistakes out of {len(y_test)})")
fig, ax = plt.subplots(figsize=(5, 4.5))
ConfusionMatrixDisplay(confusion_matrix(y_test, y_pred_knn)).plot(ax=ax, colorbar=False)
ax.set_title("kNN (k=5): rows = true digit, columns = predicted"); plt.tight_layout(); plt.show()
```

```
kNN test accuracy: 0.9926  (4 mistakes out of 540)
```



5 · Exercise: classify digits with SVM (course code, fixed)

The course wrote `SVC(kernel='linear', gamma='scale')`. **`gamma` does nothing for a linear kernel**: it is a parameter of the RBF, polynomial and sigmoid kernels only. scikit-learn

silently ignores it, so the code runs, but it suggests a setting that is not being used. We prove that below, then use the clean version.

```
# Proof that gamma is ignored by a linear kernel: wildly different gamma, identical model
a = SVC(kernel='linear', gamma='scale').fit(X_train, y_train)
b = SVC(kernel='linear', gamma=1000.0).fit(X_train, y_train)
print("Same predictions with gamma='scale' and gamma=1000?", np.array_equal(a.predict(X_test),
b.predict(X_test)))
print("Same support vectors?", np.array_equal(a.support_, b.support_))
```

```
Same predictions with gamma='scale' and gamma=1000? True
Same support vectors? True
```

```
# Train SVM classifier (course code with the meaningless gamma removed)
svm = SVC(kernel='linear')        # straight-line (hyperplane) boundaries, C=1.0 by default
svm.fit(X_train, y_train)         # finds the widest-margin boundary for every pair of digits
(one-vs-one)
y_pred_svm = svm.predict(X_test)

print("SVM Classification Report:")
print(classification_report(y_test, y_pred_svm))
print(f"linear SVM test accuracy: {accuracy_score(y_test, y_pred_svm):.4f}  "
      f"| support vectors kept: {svm.n_support_.sum()} of {len(X_train)} training images")
```

```
SVM Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        53
           1       0.98      0.98      0.98        50
           2       0.98      1.00      0.99        47
           3       1.00      0.96      0.98        54
           4       0.98      0.98      0.98        60
           5       0.97      0.97      0.97        66
           6       1.00      1.00      1.00        53
           7       0.96      0.98      0.97        55
           8       0.95      0.98      0.97        43
           9       0.97      0.95      0.96        59

    accuracy                           0.98       540
   macro avg       0.98      0.98      0.98       540
weighted avg       0.98      0.98      0.98       540

linear SVM test accuracy: 0.9796  | support vectors kept: 374 of 1257 training images
```

`n_support_` shows the "widest street" idea in action: only the training images near a boundary are kept as support vectors; the rest could be deleted without changing the model.

Linear vs RBF kernel

The RBF kernel is scikit-learn's default (`SVC()` with no arguments). Let us compare it fairly with the linear one on the same split.

```
svm_rbf = SVC(kernel='rbf', gamma='scale', C=1.0).fit(X_train, y_train)   # gamma DOES matter
here
y_pred_rbf = svm_rbf.predict(X_test)
for name, pred in [("kNN (k=5)", y_pred_knn), ("SVM linear", y_pred_svm), ("SVM RBF",
y_pred_rbf)]:
    print(f"{name:12s} accuracy = {accuracy_score(y_test, pred):.4f}   mistakes = {(pred !=
y_test).sum()}")
print("gamma='scale' works out to", round(1 / (X_train.shape[1] * X_train.var()), 5), "for this
data")
```

```
kNN (k=5)     accuracy = 0.9926   mistakes = 4
SVM linear    accuracy = 0.9796   mistakes = 11
SVM RBF       accuracy = 0.9870   mistakes = 7
gamma='scale' works out to 0.00043 for this data
```

Careful: with only 540 test images, a difference of 2–3 mistakes is about 0.5% and could flip with a different random split. To compare models properly we use **cross-validation** (next section): split the training data 5 different ways and average.

6 · Experiments: what do the parameters do?

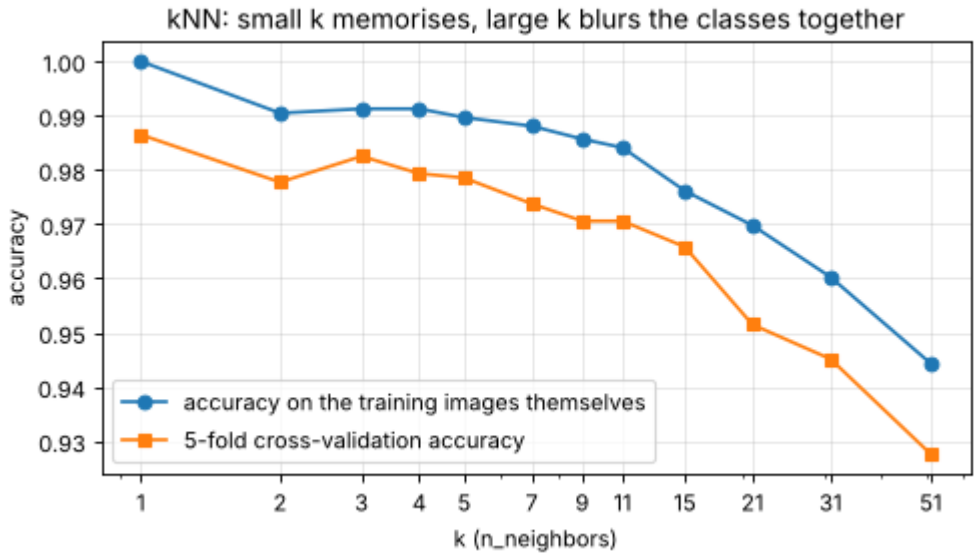
6a · Choosing k for kNN

We pick `k` using **5-fold cross-validation on the training set only**, so the test set stays untouched. (Choosing k by looking at test accuracy would be cheating: the test set would no longer be "unseen".)

```
from sklearn.model_selection import cross_val_score

ks = [1, 2, 3, 4, 5, 7, 9, 11, 15, 21, 31, 51]
cv_acc = [cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train, y_train, cv=5).mean() for
k in ks]
train_acc = [KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train).score(X_train, y_train)
for k in ks]

plt.figure(figsize=(7, 3.5))
plt.plot(ks, train_acc, "o-", label="accuracy on the training images themselves")
plt.plot(ks, cv_acc, "s-", label="5-fold cross-validation accuracy")
plt.xscale("log"); plt.xticks(ks, ks); plt.xlabel("k (n_neighbors)"); plt.ylabel("accuracy")
plt.title("kNN: small k memorises, large k blurs the classes together"); plt.legend();
plt.grid(alpha=.3); plt.show()
for k, a in zip(ks, cv_acc):
    print(f"k={k:>2}: CV accuracy {a:.4f}")
best_k = ks[int(np.argmax(cv_acc))]
print("best k by cross-validation:", best_k)
```



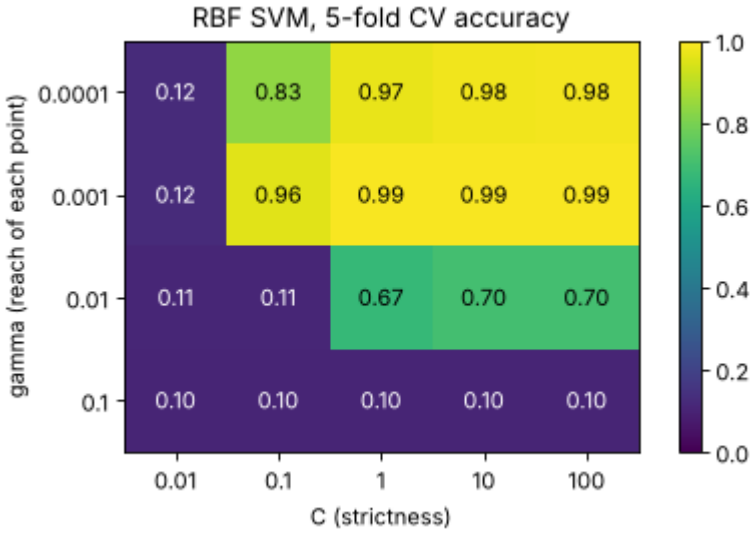
```
k= 1: CV accuracy 0.9865
k= 2: CV accuracy 0.9777
k= 3: CV accuracy 0.9825
k= 4: CV accuracy 0.9793
k= 5: CV accuracy 0.9785
k= 7: CV accuracy 0.9738
k= 9: CV accuracy 0.9706
k=11: CV accuracy 0.9706
k=15: CV accuracy 0.9658
k=21: CV accuracy 0.9515
k=31: CV accuracy 0.9451
k=51: CV accuracy 0.9276
best k by cross-validation: 1
```

- **k = 1** gets 100% on the training images (each image's nearest neighbour is itself), which tells you nothing. It is also the most sensitive to a single badly written or mislabelled neighbour.
- **Large k** (51) lets far-away images vote, so classes blur together and accuracy drops.
- On this small, clean dataset cross-validation actually prefers **k = 1** (98.65%), with k = 3 (98.25%) and k = 5 (97.85%) close behind. That happens when the classes are well separated and there is little label noise. On noisier real-world data a larger k usually wins. The usual advice is an **odd** k (fewer ties), tuned by cross-validation, not by the test set.
- Note that the course's k = 5 scored 99.26% on the *test* split but 97.85% in cross-validation: one 540-image split can easily be 1% "lucky". That is why we do not compare models on a single split.

6b · SVM: `C` and `gamma` for the RBF kernel

```
Cs = [0.01, 0.1, 1, 10, 100]
gammas = [1e-4, 1e-3, 1e-2, 1e-1]
grid = np.zeros((len(gammas), len(Cs)))
for i, g in enumerate(gammas):
    for j, C in enumerate(Cs):
        grid[i, j] = cross_val_score(SVC(kernel='rbf', C=C, gamma=g), X_train, y_train,
cv=5).mean()

fig, ax = plt.subplots(figsize=(6, 3.6))
im = ax.imshow(grid, cmap="viridis", vmin=0, vmax=1)
ax.set_xticks(range(len(Cs)), Cs); ax.set_yticks(range(len(gammas)), gammas)
ax.set_xlabel("C (strictness)"); ax.set_ylabel("gamma (reach of each point)")
for i in range(len(gammas)):
    for j in range(len(Cs)):
        ax.text(j, i, f"{grid[i, j]:.2f}", ha="center", va="center", color="w" if grid[i, j] < .6
else "k")
ax.set_title("RBF SVM, 5-fold CV accuracy"); plt.colorbar(im); plt.tight_layout(); plt.show()
i, j = np.unravel_index(grid.argmax(), grid.shape)
print(f"best: gamma={gammas[i]}, C={Cs[j]}, CV accuracy {grid[i, j]:.4f}")
print("note: gamma='scale' for this data is", round(1 / (X_train.shape[1] * X_train.var()), 5))
```




```
best: gamma=0.001, C=10, CV accuracy 0.9921
note: gamma='scale' for this data is 0.00043
```

How to read the grid:

- **gamma too large (0.1)**: each training image only influences a tiny bubble around itself, so the SVM memorises the training set and fails on new images (accuracy collapses).
- **gamma tiny and C tiny** (bottom-left): the boundary is so smooth and so tolerant of mistakes that it underfits.
- There is a broad "good region" (gamma 0.0001–0.001 with C ≥ 1, all about 97–99%); `gamma='scale'` (≈0.00043 here) lands inside it, which is why the default usually works.

7 · Is kNN really "slow for large datasets"? A timing experiment on real MNIST

The Key Concepts say kNN is *slow for large datasets because it compares with all samples*, and SVM is *computationally expensive for large datasets*. Both claims are about **different phases**, so we time **training** and **prediction** separately as the training set grows. 1,797 digits is too small to show anything, so we use real **MNIST** (28×28 = 784 features) downloaded by `torchvision` into `data/`. We predict the same 2,000 test images each time.

We add `LinearSVC`: a linear SVM that stores just one weight vector per class, so prediction is a single matrix multiplication.

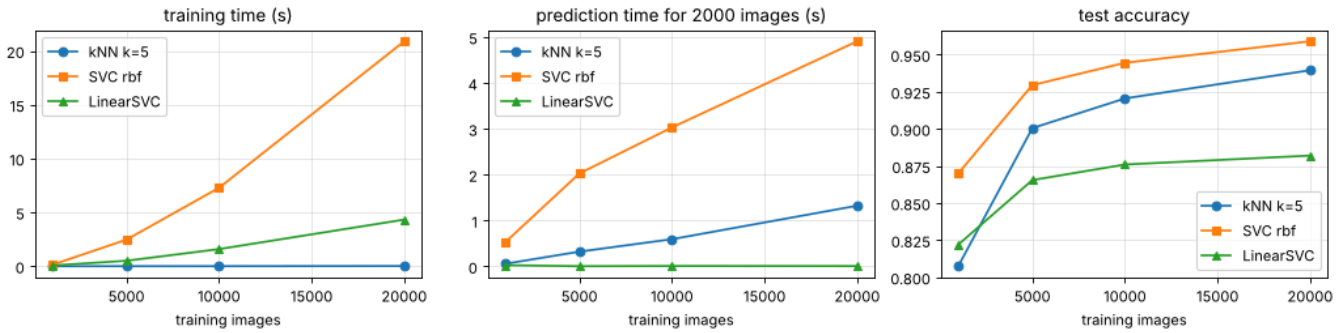
```
from pathlib import Path
from sklearn.svm import LinearSVC

DATA = Path("data")
try:
    import io, contextlib
    from torchvision import datasets as tv_datasets
    with contextlib.redirect_stderr(io.StringIO()):     # hide the download progress bars
        mn_train = tv_datasets.MNIST(str(DATA), train=True, download=True)
        mn_test = tv_datasets.MNIST(str(DATA), train=False, download=True)
    HAVE_MNIST = True
except Exception as e:   # no torchvision, or offline
    print("Could not load MNIST (need torchvision and an internet connection the first time):",
e)
    HAVE_MNIST = False
```

```python
if HAVE_MNIST:
    Xm = mn_train.data.numpy().reshape(-1, 784).astype(np.float32) / 255   # scale 0-255 -> 0-1
    ym = mn_train.targets.numpy()
    Xq = mn_test.data.numpy().reshape(-1, 784).astype(np.float32)[:2000] / 255
    yq = mn_test.targets.numpy()[:2000]
    sizes = [1000, 5000, 10000, 20000]
    rows = []
    for n in sizes:
        for name, model in [("kNN k=5", KNeighborsClassifier(n_neighbors=5)),
                            ("SVC rbf", SVC(kernel='rbf')),
                            ("LinearSVC", LinearSVC(C=0.1))]:
            t0 = time.perf_counter(); model.fit(Xm[:n], ym[:n]); t_fit = time.perf_counter() - t0
            t0 = time.perf_counter(); acc = (model.predict(Xq) == yq).mean(); t_pred =
time.perf_counter() - t0
            rows.append((n, name, t_fit, t_pred, acc))
            print(f"n_train={n:>6}  {name:10s} fit {t_fit:6.2f}s  predict 2000 {t_pred:5.2f}s
accuracy {acc:.3f}")
```

```
n_train=  1000  kNN k=5     fit   0.00s  predict 2000   0.06s  accuracy 0.807
n_train=  1000  SVC rbf     fit   0.13s  predict 2000   0.53s  accuracy 0.870
n_train=  1000  LinearSVC   fit   0.07s  predict 2000   0.03s  accuracy 0.822
n_train=  5000  kNN k=5     fit   0.00s  predict 2000   0.32s  accuracy 0.900
n_train=  5000  SVC rbf     fit   2.46s  predict 2000   2.03s  accuracy 0.929
n_train=  5000  LinearSVC   fit   0.51s  predict 2000   0.01s  accuracy 0.866
n_train= 10000  kNN k=5     fit   0.01s  predict 2000   0.59s  accuracy 0.920
n_train= 10000  SVC rbf     fit   7.28s  predict 2000   3.03s  accuracy 0.945
n_train= 10000  LinearSVC   fit   1.59s  predict 2000   0.01s  accuracy 0.876
n_train= 20000  kNN k=5     fit   0.02s  predict 2000   1.32s  accuracy 0.940
n_train= 20000  SVC rbf     fit  20.89s  predict 2000   4.91s  accuracy 0.959
n_train= 20000  LinearSVC   fit   4.33s  predict 2000   0.01s  accuracy 0.882
```

```python
if HAVE_MNIST:
    fig, ax = plt.subplots(1, 3, figsize=(13, 3.4))
    for name, mk in [("kNN k=5", "o-"), ("SVC rbf", "s-"), ("LinearSVC", "^-")]:
        r = [row for row in rows if row[1] == name]
        ax[0].plot(sizes, [x[2] for x in r], mk, label=name)
        ax[1].plot(sizes, [x[3] for x in r], mk, label=name)
        ax[2].plot(sizes, [x[4] for x in r], mk, label=name)
    for a, t in zip(ax, ["training time (s)", "prediction time for 2000 images (s)", "test
accuracy"]):
        a.set_title(t); a.set_xlabel("training images"); a.grid(alpha=.3); a.legend()
    plt.tight_layout(); plt.show()
```



What the experiment shows (see the printed numbers above):

- **kNN training is instant** (it only stores data), but its **prediction time grows with the training set**: it must compare every query with every stored image. That confirms the Key Concepts claim.
- **RBF SVM training grows much faster than linearly** (roughly quadratic) with the number of training images: that is the "computationally expensive for large datasets" con.
- **Surprise:** on this machine the RBF SVM was also **slower than kNN at prediction**. An RBF SVM compares each query with all its **support vectors** (thousands of them here), and kNN's brute-force search is done with fast matrix maths. So "kNN is slow, SVM is fast at prediction"

is only true for a **linear** SVM stored as one weight vector (LinearSVC), which predicts almost instantly.
- **Accuracy:** more data helps every model; the RBF SVM is the most accurate of the three at every size.

Exact timings depend on your CPU, but the trends are what to remember.

8 · Mini project: handwritten digit recogniser

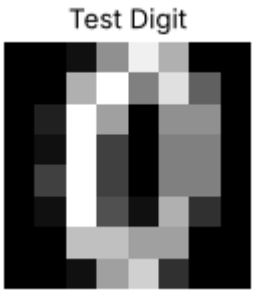
8a · The course version, and why its test proves nothing

The course function and test:

```
def recognize_digit(image):
    image = image.reshape(1, -1)          # 8x8 -> one row of 64 numbers, as the model expects
    prediction = svm.predict(image)
    return prediction[0]

# Test with a sample digit (course code)
sample_image = digits.images[10]
plt.figure(figsize=(2, 2)); plt.imshow(sample_image, cmap='gray'); plt.title("Test Digit");
plt.axis('off'); plt.show()
print("Predicted Label:", recognize_digit(digits.images[10]))

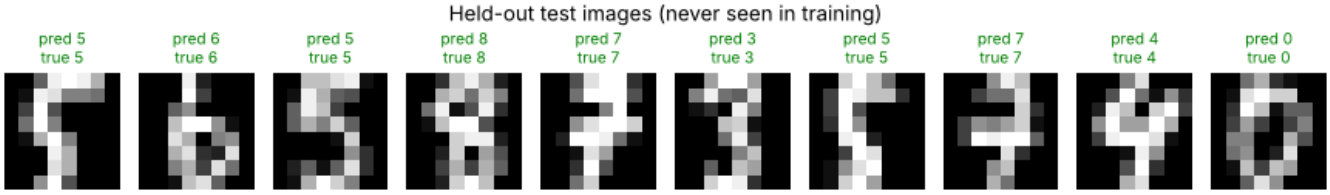
# Was image 10 one of the training images?
train_idx, test_idx = train_test_split(np.arange(len(X)), test_size=0.3, random_state=42)
print("Is digits.images[10] in the TRAINING set?", 10 in set(train_idx))
```




```
Predicted Label: 0
Is digits.images[10] in the TRAINING set? True
```

Image 10 **was in the training set**, so the model has already seen this exact image with its answer. Getting it right proves nothing (like testing a student on a question they were given the answer to). A fair test uses images the model **never saw**: the test split, or better still, a digit we draw ourselves.

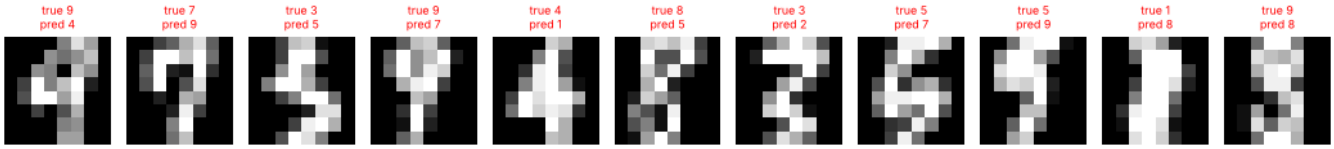
```
# Fair test: 10 random images from the held-out TEST split
rng = np.random.default_rng(1)
pick = rng.choice(len(X_test), 10, replace=False)
fig, axes = plt.subplots(1, 10, figsize=(11, 1.8))
for a, i in zip(axes, pick):
    p = recognize_digit(X_test[i])
    a.imshow(X_test[i].reshape(8, 8), cmap='gray'); a.axis('off')
    a.set_title(f"pred {p}\ntrue {y_test[i]}", color="green" if p == y_test[i] else "red",
fontsize=9)
plt.suptitle("Held-out test images (never seen in training)"); plt.tight_layout(); plt.show()
```



8b · The mistakes: which digits get confused?

```
wrong = np.where(y_pred_svm != y_test)[0]
print(f"linear SVM got {len(wrong)} of {len(y_test)} test images wrong")
fig, axes = plt.subplots(1, len(wrong), figsize=(1.2 * len(wrong), 1.8))
for a, i in zip(np.atleast_1d(axes), wrong):
    a.imshow(X_test[i].reshape(8, 8), cmap='gray'); a.axis('off')
    a.set_title(f"true {y_test[i]}\npred {y_pred_svm[i]}", fontsize=8, color="red")
plt.tight_layout(); plt.show()
cm = confusion_matrix(y_test, y_pred_svm)
np.fill_diagonal(cm, 0)
pairs = sorted(((cm[i, j], i, j) for i in range(10) for j in range(10) if cm[i, j]),
reverse=True)
print("most common confusions (count, true -> predicted):", [(c, f"{i}->{j}") for c, i, j in
pairs[:5]])
```

```
linear SVM got 11 of 540 test images wrong
```




```
most common confusions (count, true -> predicted): [(np.int64(1), '9->8'), (np.int64(1), '9->7'),
(np.int64(1), '9->4'), (np.int64(1), '8->5'), (np.int64(1), '7->9')]
```

The 11 mistakes are spread out (no pair of digits is confused more than once), and several are hard even for a human at 8×8 resolution. This is the real limit of pixel-based classifiers: they compare raw pixel patterns and have no idea about strokes or shapes.

8c · A digit *you* draw: the preprocessing that makes it work

A new image must look like the training data **in every way**, or the model fails:

Training data (`load_digits`)	A typical drawing
8×8 pixels	e.g. 200×200 pixels
values **0–16**	values 0–255
bright ink on dark background (0 = background)	dark ink on white paper
digit **cropped and centred**, filling the box	digit anywhere on the page, any size

The original dataset was made by scanning digits at 32×32 (black/white), then **counting ink pixels in each 4×4 block**, giving 8×8 blocks with counts 0–16. We copy that recipe exactly.

```
def draw_digit_font(d, font=cv2.FONT_HERSHEY_SIMPLEX, thickness=16, scale=6):
    """Black digit on white 200x200 'paper', drawn with an OpenCV font."""
    canvas = np.full((200, 200), 255, np.uint8)
    cv2.putText(canvas, str(d), (40, 165), font, scale, 0, thickness, cv2.LINE_AA)
    return canvas

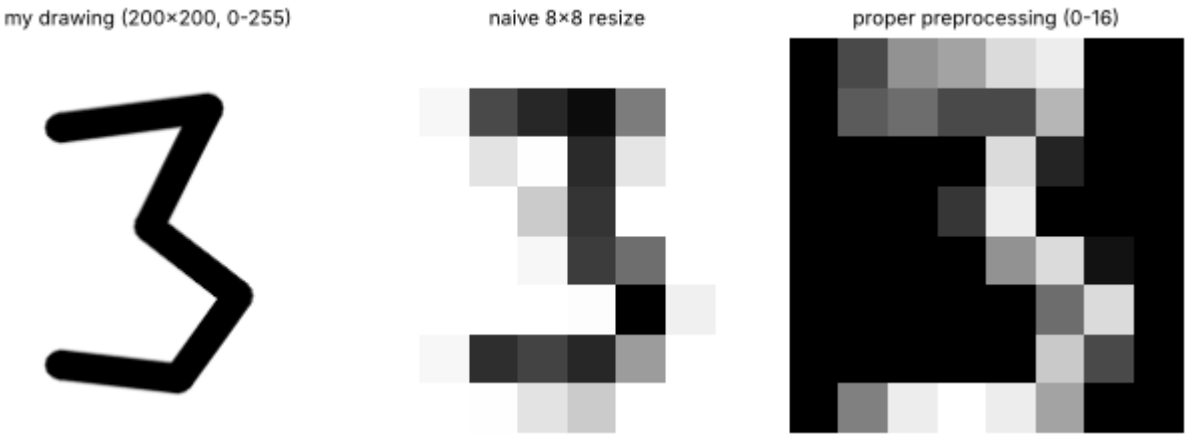
def draw_digit_strokes(d):
    """A rough 'hand-drawn' digit made from thick pen strokes (polylines), black on white."""
```

```
    strokes = {
        0: [[(100, 30), (60, 60), (55, 120), (80, 170), (120, 170), (145, 120), (140, 60), (100,
30)]],
        1: [[(80, 60), (110, 30), (110, 175)]],
        2: [[(55, 65), (90, 30), (135, 45), (140, 85), (60, 170), (150, 170)]],
        3: [[(55, 45), (130, 35), (100, 95), (145, 130), (115, 172), (55, 165)]],
        4: [[(120, 175), (120, 30), (50, 125), (155, 125)]],
        5: [[(140, 30), (70, 30), (62, 95), (125, 95), (140, 140), (105, 172), (55, 160)]],
        6: [[(130, 30), (75, 80), (60, 140), (95, 172), (138, 145), (118, 105), (65, 125)]],
        7: [[(50, 35), (150, 35), (90, 175)]],
        8: [[(100, 100), (60, 60), (100, 28), (140, 60), (100, 100), (55, 140), (100, 175), (145,
140), (100, 100)]],
        9: [[(135, 70), (100, 30), (60, 65), (95, 100), (135, 70), (125, 175)]],
    }
    canvas = np.full((200, 200), 255, np.uint8)
    for s in strokes[d]:
        cv2.polylines(canvas, [np.array(s, np.int32)], False, 0, 14, cv2.LINE_AA)
    return canvas

def preprocess_like_sklearn_digits(img):
    """Any grey image of ONE dark digit on light paper -> 8x8, values 0-16, like load_digits."""
    ink = (img < 128).astype(np.uint8)                # 1) binarise and INVERT: ink = 1, paper =
0
    ys, xs = np.nonzero(ink)                          # 2) crop to the digit's bounding box
    ink = ink[ys.min():ys.max() + 1, xs.min():xs.max() + 1]
    h, w = ink.shape                                  # 3) pad to a square, keeping the aspect
ratio, centred
    side = max(h, w)
    sq = np.zeros((side, side), np.uint8)
    sq[(side - h) // 2:(side - h) // 2 + h, (side - w) // 2:(side - w) // 2 + w] = ink
    sq32 = cv2.resize(sq, (32, 32), interpolation=cv2.INTER_NEAREST)   # 4) 32x32 black/white,
like the original scans
    return sq32.reshape(8, 4, 8, 4).sum(axis=(1, 3)).astype(np.float64)  # 5) count ink in each
4x4 block -> 0..16

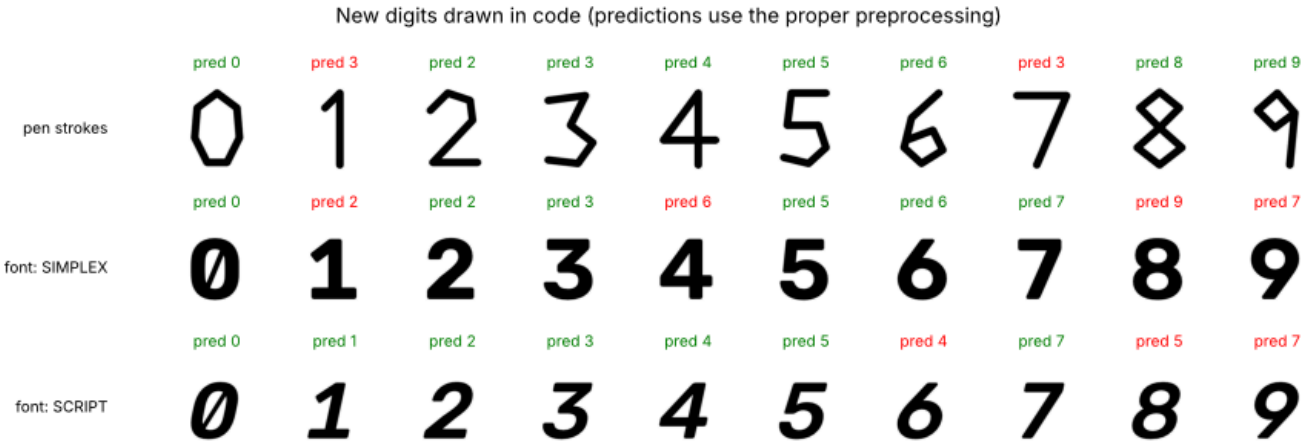
def naive_resize(img):
    """What beginners often do: just shrink the picture to 8x8. Wrong colours and wrong range."""
    return cv2.resize(img, (8, 8), interpolation=cv2.INTER_AREA).astype(np.float64)

raw = draw_digit_strokes(3)
steps = [raw, naive_resize(raw), preprocess_like_sklearn_digits(raw)]
titles = ["my drawing (200x200, 0-255)", "naive 8x8 resize", "proper preprocessing (0-16)"]
fig, ax = plt.subplots(1, 3, figsize=(8, 2.8))
for a, im, t in zip(ax, steps, titles):
    a.imshow(im, cmap='gray'); a.set_title(t, fontsize=9); a.axis('off')
plt.tight_layout(); plt.show()
print("naive  -> predicted", recognize_digit(naive_resize(raw)), "| value range",
naive_resize(raw).min(), "-", naive_resize(raw).max())
print("proper -> predicted", recognize_digit(preprocess_like_sklearn_digits(raw)))
```




```
naive  -> predicted 2 | value range 68.0 - 255.0
proper -> predicted 3
```

```python
# Now all ten digits, three drawing styles, with and without preprocessing
styles = {
    "pen strokes": draw_digit_strokes,
    "font: SIMPLEX": lambda d: draw_digit_font(d, cv2.FONT_HERSHEY_SIMPLEX),
    "font: SCRIPT": lambda d: draw_digit_font(d, cv2.FONT_HERSHEY_SCRIPT_SIMPLEX, thickness=14),
}
fig, axes = plt.subplots(len(styles), 10, figsize=(11, 4))
score = {}
for r, (sname, fn) in enumerate(styles.items()):
    ok_naive = ok_proper = 0
    for d in range(10):
        img = fn(d)
        p_proper = recognize_digit(preprocess_like_sklearn_digits(img))
        p_naive = recognize_digit(naive_resize(img))
        ok_proper += p_proper == d; ok_naive += p_naive == d
        axes[r, d].imshow(img, cmap='gray'); axes[r, d].axis('off')
        axes[r, d].set_title(f"pred {p_proper}", fontsize=9, color="green" if p_proper == d else
"red")
    axes[r, 0].text(-120, 100, sname, fontsize=9, ha="right", va="center")
    score[sname] = (ok_naive, ok_proper)
plt.suptitle("New digits drawn in code (predictions use the proper preprocessing)");
plt.tight_layout(); plt.show()
for s, (n, p) in score.items():
    print(f"{s:14s}: naive resize {n}/10 correct   |   proper preprocessing {p}/10 correct")
```




```
pen strokes   : naive resize 0/10 correct   |   proper preprocessing 8/10 correct
font: SIMPLEX : naive resize 0/10 correct   |   proper preprocessing 6/10 correct
font: SCRIPT  : naive resize 0/10 correct   |   proper preprocessing 7/10 correct
```

What this shows: without preprocessing the model got **0 of 30** right, worse than random guessing (the naive image has the wrong colours: a white background reads as "ink everywhere", and the values are 0–255 instead of 0–16). With the dataset's own recipe (invert, crop, centre, 32×32, count ink per 4×4 block), most new digits are recognised. The ones that still fail are drawn in a style that differs from the 1990s handwriting in the dataset (e.g. font shapes and the script font's slanted strokes); more varied training data, or a CNN (Day 15), handles this better.

To try your own photo of a handwritten digit: load it with `cv2.imread(path, cv2.IMREAD_GRAYSCALE)`, make sure it contains a single dark digit on light paper (crop it), and pass it through `preprocess_like_sklearn_digits` before `recognize_digit`. For uneven lighting, replace the fixed `img < 128` with Otsu's threshold: `cv2.threshold(img, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)`.

9 · 🔧 What we changed from the course notebook, and why

#	Course code	Problem	What we did
1	`SVC(kernel='linear', gamma='scale')`	`gamma` is only used by RBF/poly/sigmoid kernels. With a linear kernel it is **silently ignored** (we proved identical predictions with `gamma=1000`). Misleading, not a crash.	Used `SVC(kernel='linear')`, and showed where `gamma` really matters (RBF grid in 6b).
2	Mini project tests `digits.images[10]`	That image is **in the training split** (we checked), so a correct answer proves nothing.	Tested on held-out test images and on brand-new digits drawn in code.
3	Comment "Load MNIST digits dataset"	`load_digits()` is the 8×8 UCI digits set (1,797 images, values 0–16), not 28×28 MNIST.	Corrected the comment; used real MNIST for the timing experiment.
4	`recognize_digit` takes any image	It silently gives nonsense on any image not already in 8×8, 0–16, white-on-black format.	Added `preprocess_like_sklearn_digits` and showed the naive resize failing.
5	Only the report is printed	No confusion matrix, no look at the mistakes, `k` and `C` / `gamma` never tuned.	Added confusion matrices, misclassified examples, cross-validated k sweep and C/gamma grid.

10 · Results, honestly

(All numbers are from the run above.)

- On the 540-image test split: **kNN (k=5) 99.26% (4 mistakes)**, **linear SVM 97.96% (11 mistakes)**, **RBF SVM 98.70% (7 mistakes)**. All three are excellent because this dataset is small, clean and low-resolution; the gap between them is a handful of images, which is within the noise of one random split.
- Cross-validation picked **k = 1** for kNN (98.65%); the RBF SVM's best cross-validated setting (gamma = 0.001, C = 10) reached 99.21%. In cross-validation the tuned RBF SVM is the best model, even though kNN won on the single test split.
- On real MNIST with 20,000 training images: kNN 94.0%, RBF SVM 95.9%, LinearSVC 88.2% on 2,000 test images. kNN predicted more slowly as the training set grew (confirming the textbook), but the RBF SVM was even slower at prediction on this CPU (about 5 s vs under 2 s for 2,000 queries); only LinearSVC is truly fast at prediction (about 0.01 s). RBF SVM training rose from about 0.1 s (1,000 images) to about 20 s (20,000 images). Exact times vary from run to run and machine to machine.
- On real MNIST (784 features), kNN predicts more slowly as the training set grows (confirming the textbook), but the RBF SVM was even slower at prediction on this CPU; only a linear SVM stored as weights (LinearSVC) is truly fast at prediction. RBF SVM training time grows steeply with data.
- On 30 digits drawn in code, proper preprocessing raised correct answers from **0/30** (naive resize) to **21/30**. The failures (e.g. a plain "1" read as 3 or 2, a bold "4" read as 6) are styles

unlike the 1990s handwriting in the training set. The model is only as good as the match between new data and training data.

11 · Common errors and how to fix them

Error / symptom	Cause	Fix
`ValueError: Expected 2D array, got 1D array instead`	Passing a single image (shape `(64,)` or `(8, 8)`) to `predict`	`model.predict(img.reshape(1, -1))`: one row = one sample
`ValueError: X has 784 features, but SVC is expecting 64 features`	New image not resized to the training size	Preprocess to 8×8 (64 values) for `load_digits`, 28×28 (784) for MNIST
Every drawn digit predicted as the same class	Colours inverted or wrong value range (0–255 vs 0–16)	Invert, crop, centre, rescale exactly like the training data
100% accuracy that looks too good	Testing on training images (like the course's `digits.images[10]`)	Always evaluate on the held-out test split
`UndefinedMetricWarning` in the report	A class was never predicted (precision undefined)	Usually a sign of a broken model or tiny test set; check the data
RBF SVM accuracy ~10%	`gamma` far too large (or features not scaled)	Use `gamma='scale'`, tune with cross-validation, scale features
SVM training takes forever on big data	RBF SVM cost grows ~quadratically with samples	Use `LinearSVC`, a subset, or a CNN

12 · Try this yourself

1. Change `n_neighbors` to 1 and to 50 in the course kNN cell. How does the report change? Explain using the k-sweep plot.
2. Use `KNeighborsClassifier(n_neighbors=5, weights='distance')` so closer neighbours get a bigger vote. Better or worse?
3. Run `train_test_split` with `random_state=0, 1, 2, 3`: how much do the accuracies move? What does this say about comparing 98.5% with 98.9%?
4. Scale the features with `StandardScaler` before the SVM (`from sklearn.pipeline import make_pipeline`). Does the RBF SVM change?
5. Draw a digit in Paint (black on white), save it as `my_digit.png`, and run it through `preprocess_like_sklearn_digits` and `recognize_digit`.
6. Rotate a test digit by 20° (`scipy.ndimage.rotate`) and classify it. Why does a pixel-based classifier struggle?

13 · Viva questions (with short model answers)

1. **What is supervised learning?** Learning from examples that come with the correct answer (label), so the model can predict labels for new inputs. Digit recognition is supervised: image

→ digit 0–9.

2. **How does kNN classify a new image?** Compute its distance to every training image, take the k closest, and return the majority label among them.
3. **Why is kNN called a "lazy" learner?** `fit` only stores the data; all computation happens at prediction time. So training is instant but prediction cost grows with the size of the training set.
4. **What happens with k = 1 and with a very large k?** k = 1 memorises the training set and is sensitive to noise (overfitting); a very large k averages over far-away points and blurs classes (underfitting). Choose k by cross-validation; odd k avoids ties.
5. **What is a support vector?** A training point lying on or inside the margin; only these points determine the SVM's boundary.
6. **What do `C` and `gamma` control?** `C` trades margin width against training errors (high C = strict, risk of overfitting). `gamma` sets how far one point's influence reaches in the RBF kernel (high gamma = wiggly boundary, overfitting).
7. **What is the kernel trick?** Computing similarities as if the data were mapped into a higher-dimensional space, without actually building that space, so a linear separator there becomes a curved boundary in the original space.
8. **Why is `gamma='scale'` with `kernel='linear'` pointless?** A linear kernel is just a dot product; it has no gamma, so the setting is ignored.
9. **Define precision, recall, f1 and support.** Precision = TP/(TP+FP), recall = TP/(TP+FN), f1 = their harmonic mean, support = number of true samples of the class in the test set.
10. **Macro vs weighted average?** Macro averages classes equally; weighted averages by each class's support.
11. **Why must we test on held-out data?** The goal is performance on new images; testing on training images measures memory, not generalisation.
12. **Why do we need preprocessing for a new drawing?** The model only knows the training format (8×8, 0–16, bright ink on dark, centred); any mismatch moves the image to a region of feature space with no similar training examples.
13. **What is the difference between `load_digits` and MNIST?** load_digits: 1,797 images, 8×8, values 0–16. MNIST: 70,000 images, 28×28, values 0–255.
14. **Which is faster to predict with, kNN or SVM?** It depends: kNN's prediction cost grows with training size; an RBF SVM's grows with the number of support vectors (in our MNIST test it was even slower than kNN); a linear SVM stored as weights is the fastest.
15. **Why not use kNN/SVM on raw pixels for everything?** Raw pixels are not invariant to shifts, rotation, scale or lighting, and the cost grows with data; that is why we move to features (Day 12) and CNNs (Day 15).

Day 12 · Bag of Visual Words (BoVW): a texture classifier that actually works

Course lab: `CV-Week2-Lecture5.ipynb` (Day 12: Bag of Visual Words) **Notes chapter:** `week-2/notes/05_bag_of_visual_words.md` **Run time:** about 2–3 minutes on a laptop CPU (downloads about 12 MB of texture photos the first time).

What you will learn

- How to turn an image with *any number* of SIFT descriptors into **one fixed-length vector** (a histogram of "visual words").
- What k-means clustering does and what a "visual word" looks like.
- How vocabulary size `k` affects accuracy, and why histograms must be normalised.
- How to train and evaluate an SVM on BoVW histograms **properly** (train/test split, confusion matrix, an "unseen photo" test).
- Why the course notebook could never work (3 images, 1 per class) and how to fix it.

1 · The idea in plain language

BoVW borrows a trick from text classification. To decide whether an email is spam you can ignore word order and just **count words**: lots of "winner", "free", "prize" → spam. An image can be treated the same way:

Text	Image
document	image (here: a texture patch)
word	**visual word** = a typical small-patch pattern (a corner of a brick, a blade of grass…)
dictionary	**vocabulary** = the k cluster centres found by k-means
word-count histogram	**BoVW histogram**: how often each visual word appears in the image
spam classifier	SVM trained on the histograms

The pipeline (learn this order for the viva):

1. **Extract local features**: SIFT finds keypoints and describes the patch around each one with a **128-number descriptor** (gradient-direction histograms: 4×4 cells × 8 directions).
2. **Build the vocabulary**: run **k-means** on all *training* descriptors. Each of the k cluster centres is a visual word. (Analogy from the Key Concepts: grouping synonyms. Many slightly different "brick corner" descriptors all fall into one cluster.)
3. **Encode each image**: assign every descriptor to its nearest visual word and count → a histogram of length k. Divide by the total so it sums to 1.
4. **Classify**: feed the histograms to an SVM (or kNN).

Why bother? One image may have 40 descriptors and another 300. Classifiers like SVM need **the same number of features for every image**. The histogram always has exactly k numbers, whatever the number of keypoints.

What BoVW throws away: *where* the features are (it is a "bag"; order and position are lost). That is fine for textures (a brick wall is "brick-like" everywhere) and a weakness for objects whose layout matters.

2 · Setup and a small download helper

Install once: `pip install opencv-python scikit-learn scikit-image matplotlib`. SIFT is in the main `opencv-python` package since OpenCV 4.4 (the patent expired in 2020).

```
%matplotlib inline
import time
import urllib.request
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import cv2
from sklearn.cluster import KMeans
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay,
accuracy_score
from threadpoolctl import threadpool_limits

threadpool_limits(2)              # optional: use at most 2 CPU cores
cv2.setNumThreads(2)
plt.rcParams["figure.dpi"] = 80
print("OpenCV", cv2.__version__, "| SIFT available:", hasattr(cv2, "SIFT_create"))

DATA = Path("data/textures")
DATA.mkdir(parents=True, exist_ok=True)

def download(url, name=None):
    """Download url into data/textures/ once; later runs reuse the cached file."""
    path = DATA / (name or url.split("/")[-1])
    if not path.exists():
        try:
            urllib.request.urlretrieve(url, path)
        except Exception as e:
            raise RuntimeError(f"Could not download {url}: {e}\nAre you offline? Download the
file by hand into {DATA}/") from e
    return path
```

```
OpenCV 5.0.0 | SIFT available: True
```

3 · Why the course notebook cannot work

The course code loads **three images** (`'sample.jpg', 'left.jpg', 'right.jpg'`) and gives each one **its own class** (0, 1, 2). Then `train_test_split(..., test_size=0.3)` on 3 samples puts **2 images in training and 1 in testing**. The test image's class has *never been seen during training*, so the SVM **cannot** predict it: the course output shows 0% accuracy with "support 1.0". No setting of k or the kernel could fix this; it is a data problem, not a model problem.

Let's reproduce that failure with 3 texture photos (one per class), using the course's own code structure:

```
from skimage import data as skdata    # scikit-image ships a few real texture photos (brick, grass, gravel)

course_imgs = [skdata.brick(), skdata.grass(), skdata.gravel()]   # 3 images, 1 per class, as in the course
labels = [0, 1, 2]

sift = cv2.SIFT_create()
descriptors = []
for idx, img in enumerate(course_imgs):
    kp, des = sift.detectAndCompute(img, None)
    if des is not None:
        descriptors.extend(des)
descriptors = np.array(descriptors)
print("Collected descriptors shape:", descriptors.shape)

k = 20
kmeans = KMeans(n_clusters=k, random_state=42).fit(descriptors)
X = np.array([np.bincount(kmeans.predict(sift.detectAndCompute(im, None)[1]), minlength=k) for im in course_imgs])
y = np.array(labels)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print("train labels:", y_train, " test label:", y_test)
svm = SVC(kernel='linear').fit(X_train, y_train)
print("prediction:", svm.predict(X_test), "-> accuracy", accuracy_score(y_test, svm.predict(X_test)))
print("Can the SVM ever output the test class?", y_test[0] in svm.classes_)
```

```
Collected descriptors shape: (12499, 128)
train labels: [1 2]  test label: [0]
prediction: [1] -> accuracy 0.0
Can the SVM ever output the test class? False
```

Exactly the course's failure: the test image is class 0 (brick), but the SVM only knows classes 1 and 2, so it is impossible to be right.

The fix: a real dataset with **many images per class**, split so that every class appears in both training and testing. Large texture photos make this easy: we cut **many patches** from each photo, and we make sure training and test patches come from **different regions** (and, in a harder test, from **different photos**).

(About the `RuntimeWarning: divide by zero / overflow encountered in matmul` lines in the course output: they come from the course author's machine (numpy on macOS with Apple's Accelerate maths library is known to print these spurious warnings inside scikit-learn's k-means), not from the method. They do not appear here.)

4 · Building a real texture dataset

Seven texture classes from freely downloadable texture photos: example textures shipped with the **three.js** and **Babylon.js** 3D libraries (photographs of real materials prepared for 3D graphics) and the three texture photos inside **scikit-image**. Where possible a class has **two or three different photos**.

We convert everything to **grayscale** (SIFT only uses brightness, so colour cannot help or "cheat") and shrink large photos so the short side is at most 512 pixels, keeping texture scale roughly comparable.

```
TJ = "https://raw.githubusercontent.com/mrdoob/three.js/dev/examples/textures/"
BJ = "https://raw.githubusercontent.com/BabylonJS/Assets/master/textures/"

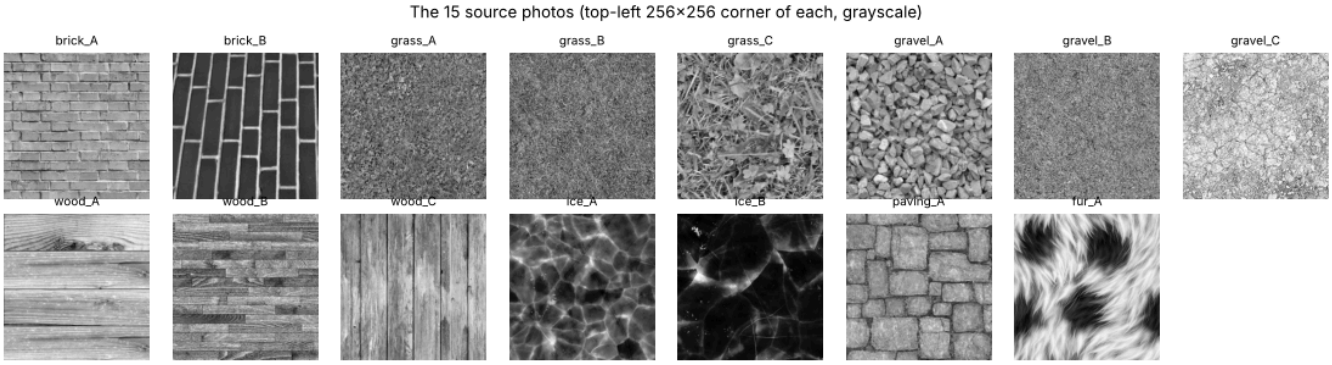
sources = {   # class -> list of (name, url or 'skimage:<name>')
    "brick":  [("brick_A", TJ + "brick_diffuse.jpg"), ("brick_B", "skimage:brick")],
    "grass":  [("grass_A", BJ + "grass.png"), ("grass_B", TJ + "terrain/grasslight-big.jpg"),
("grass_C", "skimage:grass")],
    "gravel": [("gravel_A", "skimage:gravel"), ("gravel_B", BJ + "sand.jpg"), ("gravel_C", BJ +
"ground.jpg")],
    "wood":   [("wood_A", TJ + "hardwood2_diffuse.jpg"), ("wood_B", BJ + "wood.jpg"), ("wood_C",
BJ + "crate.png")],
    "ice":    [("ice_A", TJ + "ambientcg/Ice002_1K-JPG_Color.jpg"), ("ice_B", TJ +
"ambientcg/Ice003_1K-JPG_Color.jpg")],
    "paving": [("paving_A", BJ + "floor.png")],
    "fur":    [("fur_A", BJ + "fur.jpg")],
}
class_names = list(sources)

def load_gray(src, name):
    if src.startswith("skimage:"):
        img = getattr(skdata, src.split(":")[1])()
    else:
        img = cv2.imread(str(download(src, name + Path(src).suffix)), cv2.IMREAD_GRAYSCALE)
    if name == "wood_C":
        img = img[40:216, 40:216]          # crate texture: keep only the planks, cut off the
frame
    s = 512 / min(img.shape)
    if s < 1:
        img = cv2.resize(img, None, fx=s, fy=s, interpolation=cv2.INTER_AREA)
    return img

photos = {c: [(n, load_gray(u, n)) for n, u in lst] for c, lst in sources.items()}
for c in class_names:
    print(f"{c:7s}", [(n, im.shape) for n, im in photos[c]])
```

```
brick   [('brick_A', (512, 512)), ('brick_B', (512, 512))]
grass   [('grass_A', (512, 512)), ('grass_B', (512, 512)), ('grass_C', (512, 512))]
gravel  [('gravel_A', (512, 512)), ('gravel_B', (512, 512)), ('gravel_C', (512, 512))]
wood    [('wood_A', (512, 1024)), ('wood_B', (256, 256)), ('wood_C', (176, 176))]
ice     [('ice_A', (512, 512)), ('ice_B', (512, 512))]
paving  [('paving_A', (512, 512))]
fur     [('fur_A', (512, 512))]
```

```
allp = [(c, n, im) for c in class_names for n, im in photos[c]]
fig, axes = plt.subplots(2, 8, figsize=(14, 3.9))
for a in axes.ravel(): a.axis("off")
for a, (c, n, im) in zip(axes.ravel(), allp):
    a.imshow(im[:256, :256], cmap="gray"); a.set_title(n, fontsize=9)
plt.suptitle("The 15 source photos (top-left 256x256 corner of each, grayscale)");
plt.tight_layout(); plt.show()
```



Cutting patches, with a clean train/test split

Each photo is split **vertically**: patches from the **left 60%** go to training, patches from the **right 40%** go to testing. Patches are 128×128 and step 64 pixels, so neighbouring training patches overlap, but **no test patch shares a single pixel with a training patch**. (If we just shuffled overlapping patches into train/test, near-copies would end up on both sides and the accuracy would be inflated.)

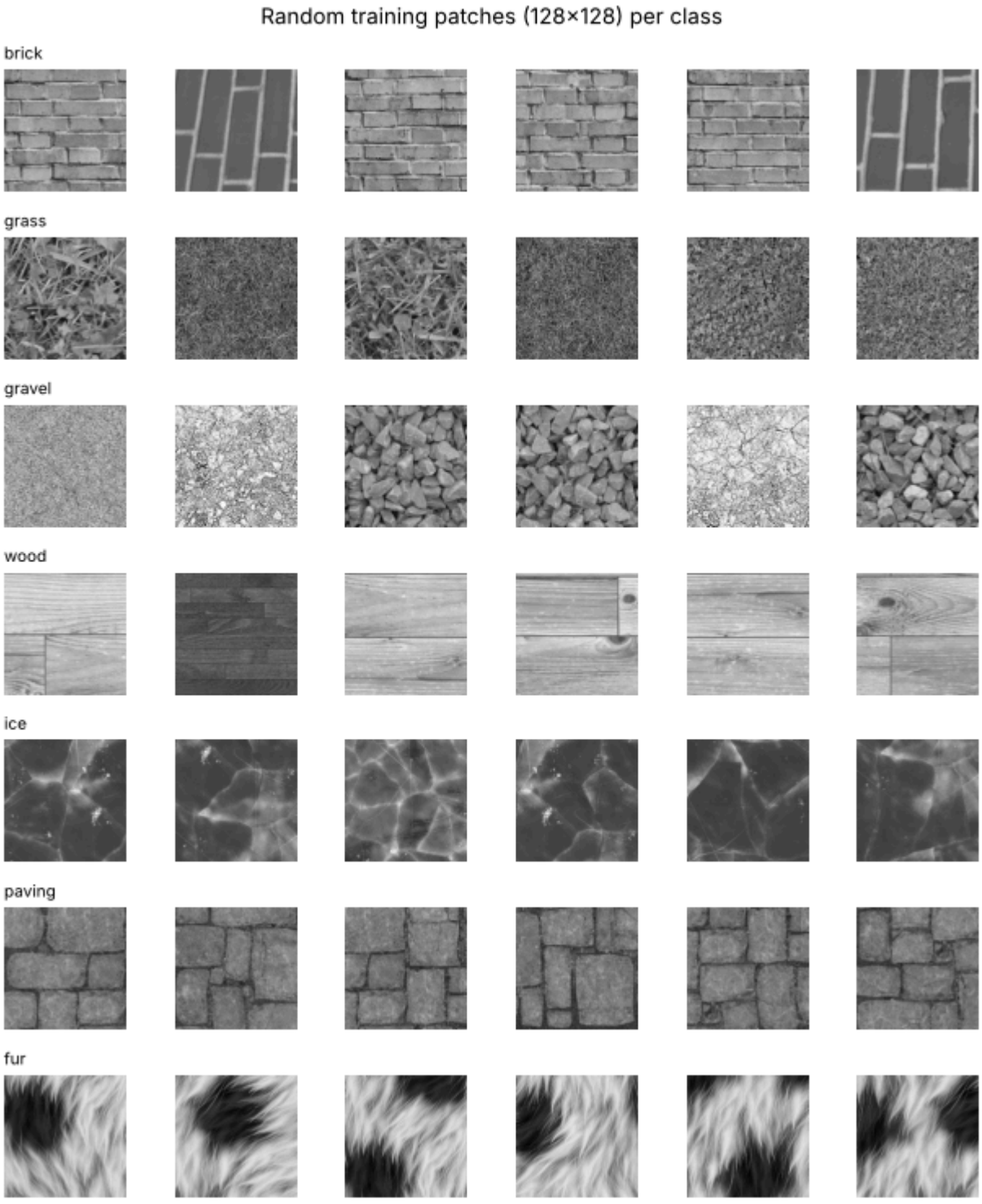
```
PATCH, STEP = 128, 64

def cut_patches(img, x0, x1, size=PATCH, with_pos=False):
    out = []
    for yy in range(0, img.shape[0] - size + 1, STEP):
        for xx in range(x0, x1 - size + 1, STEP):
            out.append((img[yy:yy + size, xx:xx + size], (xx, yy)) if with_pos else img[yy:yy +
size, xx:xx + size])
    return out

train_imgs, train_y, test_imgs, test_y, test_photo, train_photo, train_pos = [], [], [], [], [],
[], []
for ci, c in enumerate(class_names):
    for n, im in photos[c]:
        cut = int(im.shape[1] * 0.6)
        tr, te = cut_patches(im, 0, cut, with_pos=True), cut_patches(im, cut, im.shape[1])
        train_imgs += [p for p, _ in tr]; train_pos += [xy for _, xy in tr]
        train_y += [ci] * len(tr); train_photo += [n] * len(tr)
        test_imgs += te; test_y += [ci] * len(te); test_photo += [n] * len(te)
train_y, test_y = np.array(train_y), np.array(test_y)
print(f"training patches: {len(train_imgs)}, test patches: {len(test_imgs)}")
print("per class (train / test):", {c: (int((train_y == i).sum()), int((test_y == i).sum())) for
i, c in enumerate(class_names)})

fig, axes = plt.subplots(len(class_names), 6, figsize=(8, 9.5))
for ci, c in enumerate(class_names):
    idx = np.random.default_rng(ci).choice(np.where(train_y == ci)[0], 6, replace=False)
    for a, i in zip(axes[ci], idx):
        a.imshow(train_imgs[i], cmap="gray", vmin=0, vmax=255); a.axis("off")
    axes[ci, 0].set_title(c, fontsize=9, loc="left")
plt.suptitle("Random training patches (128x128) per class"); plt.tight_layout(); plt.show()
```

```
training patches: 311, test patches: 203
per class (train / test): {'brick': (42, 28), 'grass': (63, 42), 'gravel': (63, 42), 'wood': (59,
35), 'ice': (42, 28), 'paving': (21, 14), 'fur': (21, 14)}
```



5 · Step 1: SIFT descriptors (course code, adapted to patches)

Same calls as the course: `cv2.SIFT_create()` and `sift.detectAndCompute(img, None)`. `kp` is a list of keypoints (position, size, angle); `des` is an array with **one row of 128 numbers per keypoint**. `None` means "no mask: search the whole image". We keep descriptors **per patch**, because each patch needs its own histogram later.

```
sift = cv2.SIFT_create()

def sift_features(img):
    kp, des = sift.detectAndCompute(img, None)
    if des is None:                                    # a completely flat patch can have no
```

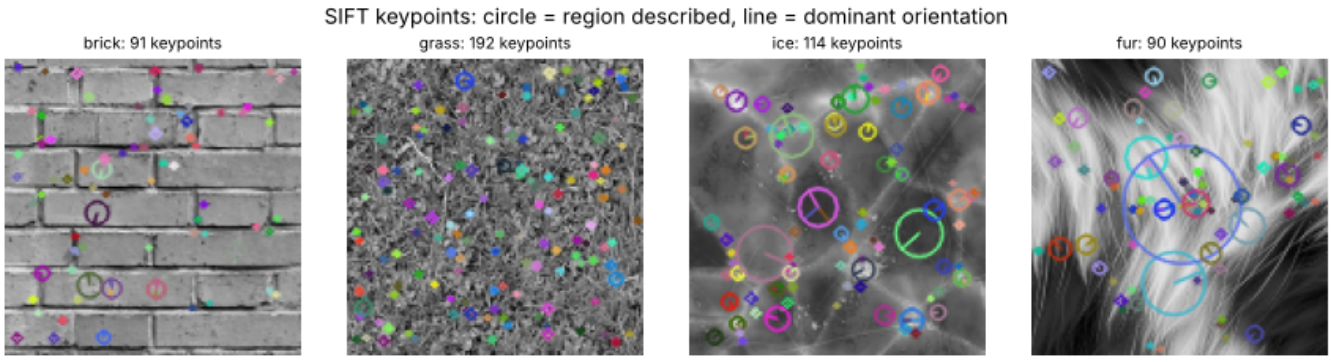
```
keypoints
        return [], np.zeros((0, 128), np.float32)
    return kp, des

t0 = time.perf_counter()
train_feats = [sift_features(p) for p in train_imgs]
test_feats = [sift_features(p) for p in test_imgs]
print(f"SIFT on {len(train_imgs) + len(test_imgs)} patches took {time.perf_counter() - t0:.1f}s")

descriptors = np.vstack([d for _, d in train_feats])     # ONLY training descriptors build the
vocabulary
n_kp = np.array([len(d) for _, d in train_feats])
print("Collected descriptors shape:", descriptors.shape)
print(f"keypoints per training patch: min {n_kp.min()}, mean {n_kp.mean():.0f}, max
{n_kp.max()}")
for ci, c in enumerate(class_names):
    print(f"   {c:7s} mean keypoints per patch: {n_kp[train_y == ci].mean():.0f}")

fig, axes = plt.subplots(1, 4, figsize=(11, 3))
for a, ci in zip(axes, [0, 1, 4, 6]):
    i = np.where(train_y == ci)[0][3]
    vis = cv2.drawKeypoints(train_imgs[i], train_feats[i][0], None,
flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)
    a.imshow(vis[..., ::-1]); a.set_title(f"{class_names[ci]}: {len(train_feats[i][0])}
keypoints", fontsize=9); a.axis("off")
plt.suptitle("SIFT keypoints: circle = region described, line = dominant orientation");
plt.tight_layout(); plt.show()
```

```
SIFT on 514 patches took 2.9s
Collected descriptors shape: (41952, 128)
keypoints per training patch: min 8, mean 135, max 387
   brick   mean keypoints per patch: 79
   grass   mean keypoints per patch: 213
   gravel  mean keypoints per patch: 198
   wood    mean keypoints per patch: 77
   ice     mean keypoints per patch: 74
   paving  mean keypoints per patch: 166
   fur     mean keypoints per patch: 73
```



Notice the **different number of keypoints per patch**: this is exactly why we need the histogram step. Also note smooth textures (ice) give fewer keypoints than busy ones (grass, gravel).

6 · Step 2: the visual vocabulary with k-means (course code)

`KMeans(n_clusters=k, random_state=42)` groups the 128-D descriptors into **k clusters**. Each cluster centre (`kmeans.cluster_centers_` , shape k×128) is one **visual word**.

How k-means works, in four lines: (1) pick k starting centres (scikit-learn uses the clever "k-means++" spread-out start), (2) assign every descriptor to its nearest centre, (3) move each

centre to the mean of its assigned descriptors, (4) repeat 2–3 until nothing changes. `random_state=42` makes the random start repeatable.

The course used **k = 20**. We start there, then sweep k in section 9. To keep it quick we fit k-means on a random sample of 20,000 training descriptors (a standard shortcut; the vocabulary barely changes).

```
rng = np.random.default_rng(0)
sample = descriptors[rng.choice(len(descriptors), min(20000, len(descriptors)), replace=False)]

k = 20  # number of visual words (vocabulary size)
t0 = time.perf_counter()
kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(sample)
print(f"k-means with k={k} on {len(sample)} descriptors: {time.perf_counter() - t0:.1f}s")
print("vocabulary (cluster centres) shape:", kmeans.cluster_centers_.shape)
```

```
k-means with k=20 on 20000 descriptors: 4.6s
vocabulary (cluster centres) shape: (20, 128)
```

7 · Step 3: one histogram per image (course `build_histogram`, fixed)

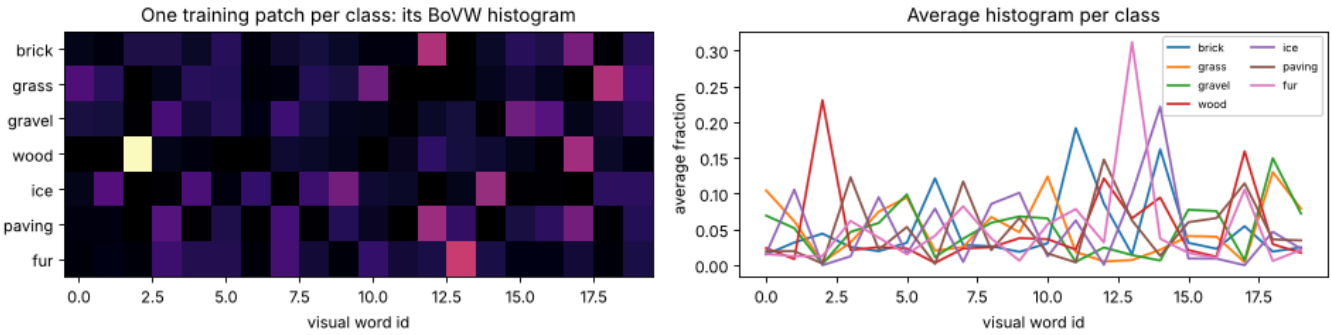
The course function counts words but does **not normalise**, so a patch with 300 keypoints gets a histogram 10× "taller" than one with 30 keypoints, even if the texture is the same. We divide by the total so each histogram sums to 1 (it becomes "what *fraction* of this patch's features are word w"). We also loop-free the counting with `np.bincount`.

```
def build_histogram(des, kmeans=kmeans, normalise=True):
    k = kmeans.n_clusters
    hist = np.zeros(k)
    if len(des):
        words = kmeans.predict(des)                    # nearest visual word for every descriptor
        hist = np.bincount(words, minlength=k).astype(float)   # count each word (same as the course's for-loop)
        if normalise:
            hist /= hist.sum()                         # FIX: fractions, not raw counts
    return hist

X_train = np.array([build_histogram(d) for _, d in train_feats])
X_test = np.array([build_histogram(d) for _, d in test_feats])
print("Feature matrix shapes:", X_train.shape, X_test.shape)

fig, ax = plt.subplots(1, 2, figsize=(12, 3.2))
ex = [np.where(train_y == ci)[0][0] for ci in range(len(class_names))]
ax[0].imshow(np.array([X_train[i] for i in ex]), cmap="magma", aspect="auto")
ax[0].set_yticks(range(len(class_names)), class_names); ax[0].set_xlabel("visual word id")
ax[0].set_title("One training patch per class: its BoVW histogram")
for ci, c in enumerate(class_names):
    ax[1].plot(X_train[train_y == ci].mean(0), label=c)
ax[1].set_xlabel("visual word id"); ax[1].set_ylabel("average fraction");
ax[1].legend(fontsize=7, ncol=2)
ax[1].set_title("Average histogram per class")
plt.tight_layout(); plt.show()
```

```
Feature matrix shapes: (311, 20) (203, 20)
```



Each class has its own "word frequency fingerprint". That fingerprint is what the SVM learns to recognise.

8 · Step 4: the texture classifier (course mini project, fixed)

The course split the 3 histograms with `train_test_split`. We already have a clean region-based split, so we train on the training patches and test on the test patches. Course model first (`SVC(kernel='linear')`), then the RBF kernel.

```
svm = SVC(kernel='linear')
svm.fit(X_train, train_y)
y_pred = svm.predict(X_test)
print("Classification Report (linear SVM, k=20):")
print(classification_report(test_y, y_pred, target_names=class_names, zero_division=0))

svm_rbf = SVC(kernel='rbf', C=10, gamma='scale').fit(X_train, train_y)
print(f"RBF SVM (C=10) accuracy with k=20: {accuracy_score(test_y,
svm_rbf.predict(X_test)):.3f}")
```

```
Classification Report (linear SVM, k=20):
              precision    recall  f1-score   support

       brick       1.00      0.50      0.67        28
       grass       0.77      0.98      0.86        42
      gravel       0.65      0.74      0.69        42
        wood       0.73      1.00      0.84        35
         ice       1.00      0.93      0.96        28
      paving       0.00      0.00      0.00        14
         fur       1.00      1.00      1.00        14

    accuracy                           0.79       203
   macro avg       0.74      0.73      0.72       203
weighted avg       0.76      0.79      0.76       203

RBF SVM (C=10) accuracy with k=20: 0.936
```

9 · Experiment: vocabulary size k vs accuracy

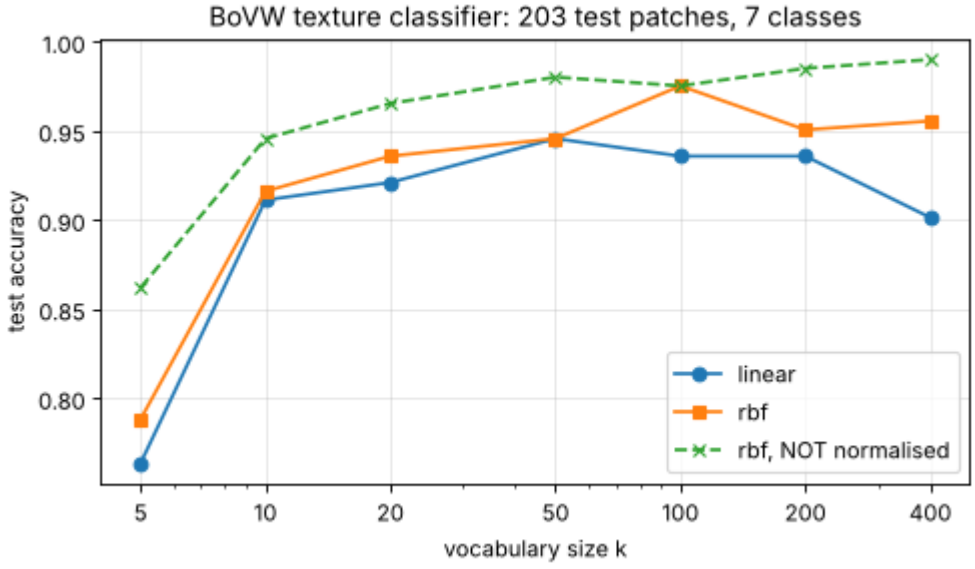
The Key Concepts' discussion point: *changing the number of clusters (k) alters the vocabulary size and affects classification accuracy.* Let's measure it, and also check what normalising the histograms buys us.

```
ks = [5, 10, 20, 50, 100, 200, 400]
res = {"linear": [], "rbf": [], "rbf, NOT normalised": []}
vocab = {}
t0 = time.perf_counter()
```

```
for kk in ks:
    km = KMeans(n_clusters=kk, random_state=42).fit(sample)
    vocab[kk] = km
    Xtr = np.array([build_histogram(d, km) for _, d in train_feats]); Xte =
np.array([build_histogram(d, km) for _, d in test_feats])
    Xtr_raw = np.array([build_histogram(d, km, normalise=False) for _, d in train_feats]);
Xte_raw = np.array([build_histogram(d, km, normalise=False) for _, d in test_feats])
    res["linear"].append(accuracy_score(test_y, SVC(kernel='linear', C=10).fit(Xtr,
train_y).predict(Xte)))
    res["rbf"].append(accuracy_score(test_y, SVC(kernel='rbf', C=10).fit(Xtr,
train_y).predict(Xte)))
    res["rbf, NOT normalised"].append(accuracy_score(test_y, SVC(kernel='rbf', C=10).fit(Xtr_raw,
train_y).predict(Xte_raw)))
    print(f"k={kk:>3}: linear {res['linear'][-1]:.3f} | rbf {res['rbf'][-1]:.3f} | rbf without
normalising {res['rbf, NOT normalised'][-1]:.3f}")
print(f"sweep took {time.perf_counter() - t0:.0f}s")

plt.figure(figsize=(7, 3.6))
for name, mk in zip(res, ["o-", "s-", "x--"]):
    plt.plot(ks, res[name], mk, label=name)
plt.xscale("log"); plt.xticks(ks, ks); plt.xlabel("vocabulary size k"); plt.ylabel("test
accuracy")
plt.title(f"BoVW texture classifier: {len(test_imgs)} test patches, {len(class_names)} classes");
plt.grid(alpha=.3); plt.legend(); plt.show()
```

```
k=  5: linear 0.764 | rbf 0.788 | rbf without normalising 0.862
k= 10: linear 0.911 | rbf 0.916 | rbf without normalising 0.946
k= 20: linear 0.921 | rbf 0.936 | rbf without normalising 0.966
k= 50: linear 0.946 | rbf 0.946 | rbf without normalising 0.980
k=100: linear 0.936 | rbf 0.975 | rbf without normalising 0.975
k=200: linear 0.936 | rbf 0.951 | rbf without normalising 0.985
k=400: linear 0.901 | rbf 0.956 | rbf without normalising 0.990
sweep took 144s
```



Reading the curve (numbers printed above):

- **Too few words (k = 5)**: the vocabulary is too coarse; very different patterns share a word, so classes look alike.
- **More words** helps up to a point, then levels off. With too many words each word becomes rare, histograms get sparse and noisy, and k-means and encoding get slower.
- The course's `SVC(kernel='linear')` (C = 1) only reached 79% at k = 20 (section 8; it never predicted "paving"), but with C = 10 the same linear SVM reaches 92%: `C` matters as much as `k`.
- The RBF kernel is best or joint best at every k ≥ 20.

- **Unexpected:** the un-normalised histograms scored higher. Section 9b explains why, and why you should still normalise.

We now take the best vocabulary (for normalised histograms) from the sweep and look at its mistakes.

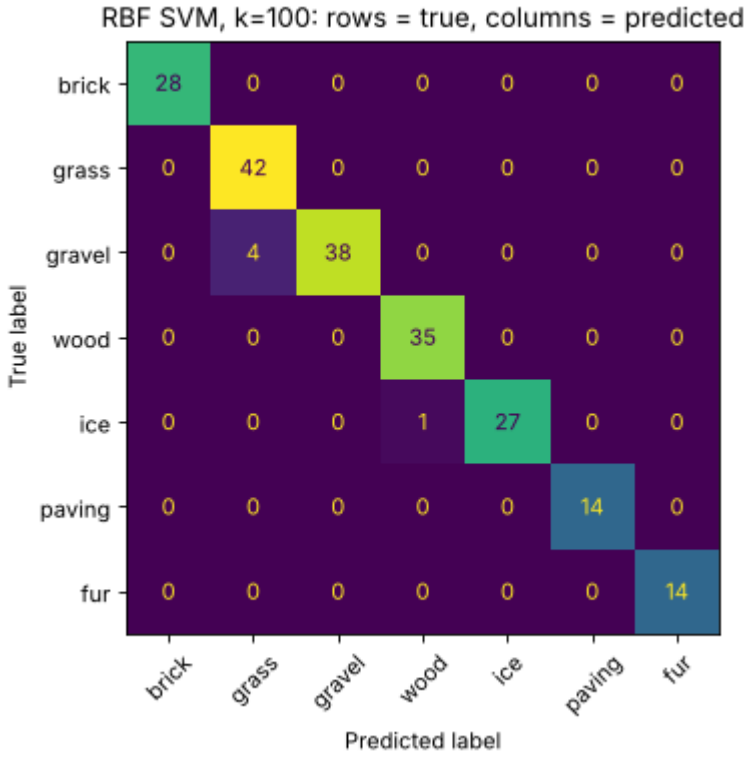
```
best_k = ks[int(np.argmax(res["rbf"]))]
km = vocab[best_k]
Xtr = np.array([build_histogram(d, km) for _, d in train_feats]); Xte =
np.array([build_histogram(d, km) for _, d in test_feats])
best_svm = SVC(kernel='rbf', C=10).fit(Xtr, train_y)
pred = best_svm.predict(Xte)
print(f"best k = {best_k}: accuracy {accuracy_score(test_y, pred):.3f}")
print(classification_report(test_y, pred, target_names=class_names, digits=3))
fig, ax = plt.subplots(figsize=(5.5, 5))
ConfusionMatrixDisplay(confusion_matrix(test_y, pred), display_labels=class_names).plot(ax=ax,
colorbar=False, xticks_rotation=45)
ax.set_title(f"RBF SVM, k={best_k}: rows = true, columns = predicted"); plt.tight_layout();
plt.show()

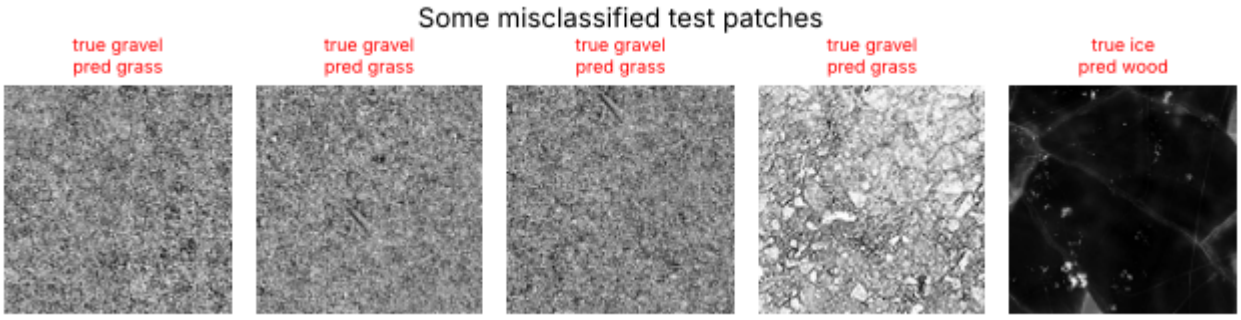
wrong = np.where(pred != test_y)[0]
if len(wrong):
    fig, axes = plt.subplots(1, min(8, len(wrong)), figsize=(1.6 * min(8, len(wrong)), 2.2))
    for a, i in zip(np.atleast_1d(axes), wrong[:8]):
        a.imshow(test_imgs[i], cmap="gray"); a.axis("off")
        a.set_title(f"true {class_names[test_y[i]]}\npred {class_names[pred[i]]}", fontsize=8,
color="red")
    plt.suptitle("Some misclassified test patches"); plt.tight_layout(); plt.show()
```

```
best k = 100: accuracy 0.975
              precision    recall  f1-score   support

       brick      1.000     1.000     1.000        28
       grass      0.913     1.000     0.955        42
      gravel      1.000     0.905     0.950        42
        wood      0.972     1.000     0.986        35
         ice      1.000     0.964     0.982        28
      paving      1.000     1.000     1.000        14
         fur      1.000     1.000     1.000        14

    accuracy                          0.975       203
   macro avg      0.984     0.981     0.982       203
weighted avg      0.977     0.975     0.975       203
```





9b · Normalising made things *worse* here. So why do it?

The sweep shows a surprise: on this dataset the **raw counts beat normalised histograms**. The reason is in section 5: busy textures (grass, gravel) have about 200 keypoints per patch and calm ones (wood, ice, fur) about 75. Since every patch is exactly 128×128, the **total count** is itself a useful clue, and normalising throws it away.

But that clue only works because all images are the same size. Real images are not. Test: keep the models trained on 128×128 patches, but classify **bigger 192×192 patches** cut from the same test regions (same textures, same scale, just more area, so more keypoints).

```
km = vocab[100]
big_test, big_y = [], []
for ci, c in enumerate(class_names):
    for n, im in photos[c]:
        cut = int(im.shape[1] * 0.6)
        b = cut_patches(im, cut, im.shape[1], size=192)
        big_test += b; big_y += [ci] * len(b)
big_y = np.array(big_y)
big_des = [sift_features(p)[1] for p in big_test]
for norm in [True, False]:
    Xtr = np.array([build_histogram(d, km, normalise=norm) for _, d in train_feats])
    Xbig = np.array([build_histogram(d, km, normalise=norm) for d in big_des])
    Xte = np.array([build_histogram(d, km, normalise=norm) for _, d in test_feats])
    m = SVC(kernel='rbf', C=10).fit(Xtr, train_y)
```

```
    print(f"normalise={norm!s:5}: 128x128 test patches {accuracy_score(test_y,
m.predict(Xte)):.3f} | "
          f"192x192 test patches ({len(big_test)}) {accuracy_score(big_y, m.predict(Xbig)):.3f}")
```

```
normalise=True : 128x128 test patches 0.975 | 192x192 test patches (96) 0.958
normalise=False: 128x128 test patches 0.975 | 192x192 test patches (96) 0.615
```

With raw counts, the bigger patches "look busier" than anything seen in training, and accuracy collapses; normalised histograms do not care about image size. **Lesson:** normalise whenever images (or numbers of keypoints) vary in size, which in practice is almost always. Our equal-size patches were a special case.

10 · What does a "visual word" look like?

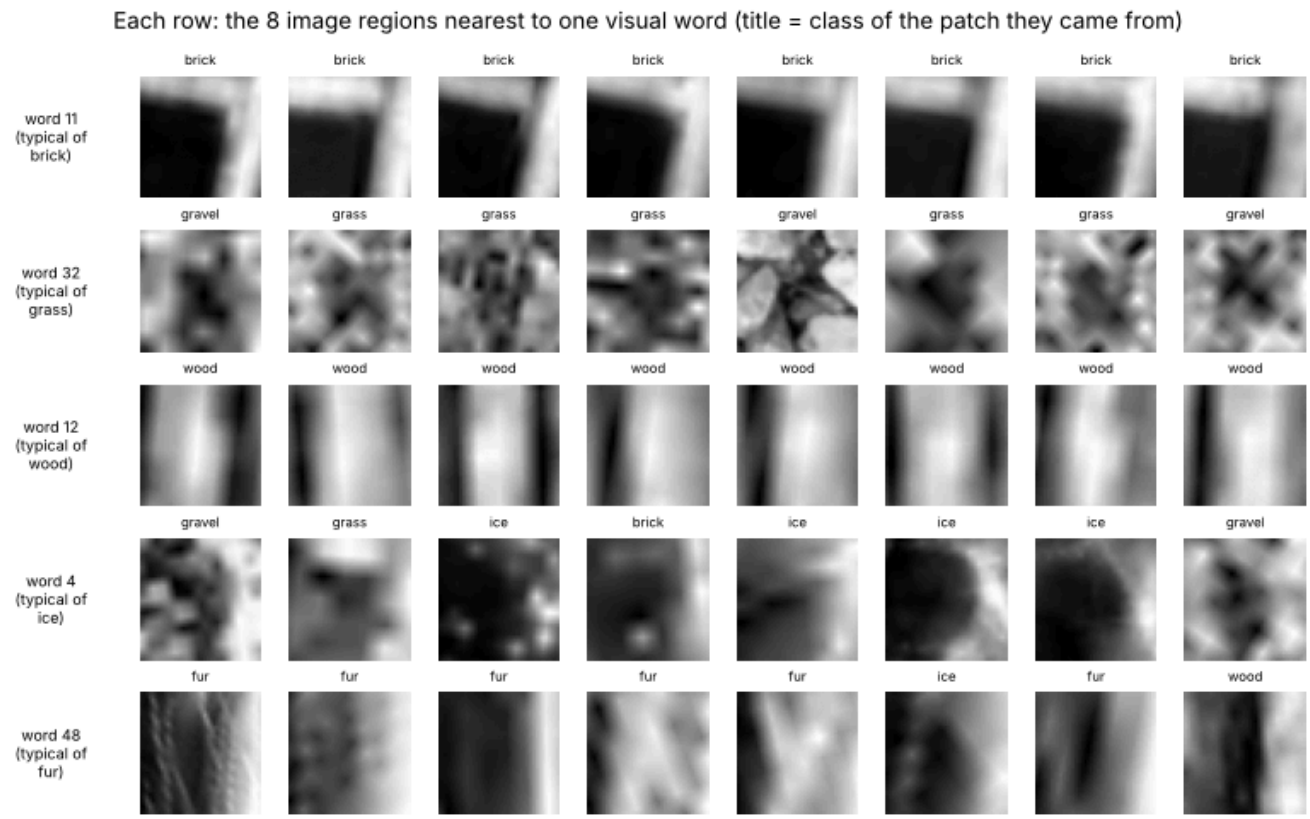
A visual word is just a point in 128-D space, which we cannot look at directly. But we can look at the **image regions whose descriptors are closest to that word's centre**. Below, for the k=50 vocabulary, we show, for 5 words, the 8 training regions nearest the centre (each region cut out around its keypoint, at the keypoint's scale and rotated to its orientation). We pick, for each of 5 classes, the word that is most typical of that class.

```
km50 = vocab[50]
all_kp = [(i, kp) for i, (kps, _) in enumerate(train_feats) for kp in kps]   # keypoints in the
same order as `descriptors`
dist = km50.transform(descriptors)                    # distance of every training descriptor to
every word centre
H50 = np.array([build_histogram(d, km50) for _, d in train_feats])
class_mean = np.array([H50[train_y == ci].mean(0) for ci in range(len(class_names))])

def region(patch_idx, kp, out=40):
    """Cut the region a SIFT keypoint describes, rotated upright, resized to out x out."""
    big = cv2.copyMakeBorder(train_imgs[patch_idx], 64, 64, 64, 64, cv2.BORDER_REFLECT)
    x, y = kp.pt[0] + 64, kp.pt[1] + 64
    M = cv2.getRotationMatrix2D((x, y), kp.angle, out / (3 * kp.size))   # scale so 3 x keypoint
size fills the tile
    M[:, 2] += (out / 2 - x, out / 2 - y)
    return cv2.warpAffine(big, M, (out, out))

show_classes = ["brick", "grass", "wood", "ice", "fur"]
fig, axes = plt.subplots(len(show_classes), 9, figsize=(10, 6.2))
for r, c in enumerate(show_classes):
    ci = class_names.index(c)
    others = np.delete(class_mean, ci, axis=0).max(0)
    w = int(np.argmax(class_mean[ci] - others))        # the word most over-represented in this
class
    nearest, seen = [], []
    for j in np.argsort(dist[:, w]):                # skip duplicates: overlapping patches contain
the same keypoint twice
        pi, kp = all_kp[j]
        gx, gy = train_pos[pi][0] + kp.pt[0], train_pos[pi][1] + kp.pt[1]
        if any(ph == train_photo[pi] and abs(gx - x0) < 4 and abs(gy - y0) < 4 for ph, x0, y0 in
seen):
            continue
        seen.append((train_photo[pi], gx, gy)); nearest.append(j)
        if len(nearest) == 8:
            break
    axes[r, 0].text(0.5, 0.5, f"word {w}\n(typical of\n{c})", ha="center", va="center",
fontsize=8); axes[r, 0].axis("off")
    for a, j in zip(axes[r, 1:], nearest):
        pi, kp = all_kp[j]
        a.imshow(region(pi, kp), cmap="gray"); a.axis("off")
        a.set_title(class_names[train_y[pi]], fontsize=7)
```

```
plt.suptitle("Each row: the 8 image regions nearest to one visual word (title = class of the
patch they came from)"); plt.tight_layout(); plt.show()
```



(The brick row looks almost identical because that brick photo is a *tileable* texture: the same brick corner repeats across the image.) Each row really is "one kind of small pattern": the same local structure (an edge, a blob, a line ending…) at different places and scales. Words are **shared** across classes (the region titles show which class each came from), and that is the point: it is the **mix** of word frequencies, not any single word, that identifies a texture.

11 · A harder, more honest test: a photo the model has never seen

Section 8 tested on unseen *regions* of the same photos. A real material recogniser must handle a **new photo** (different lighting, scale, camera). For the five classes that have 2 or 3 photos we hold out **one whole photo per class**, train on everything else (including the 1-photo classes), and test only on the held-out photos.

```
holdout = {"brick": "brick_B", "grass": "grass_C", "gravel": "gravel_A", "wood": "wood_A", "ice":
"ice_B"}
all_imgs = train_imgs + test_imgs
all_feats = train_feats + test_feats
all_y = np.concatenate([train_y, test_y])
all_photo = np.array(train_photo + test_photo)
is_test = np.isin(all_photo, list(holdout.values()))
print("held-out photos:", list(holdout.values()), f"-> {is_test.sum()} test patches,
{(~is_test).sum()} training patches")

desc_tr = np.vstack([all_feats[i][1] for i in np.where(~is_test)[0]])
km_h = KMeans(n_clusters=best_k, random_state=42).fit(desc_tr[rng.choice(len(desc_tr), min(20000,
len(desc_tr)), replace=False)])
Xa = np.array([build_histogram(d, km_h) for _, d in all_feats])
clf = SVC(kernel='rbf', C=10).fit(Xa[~is_test], all_y[~is_test])
pred_h = clf.predict(Xa[is_test])
```

```
print(f"accuracy on never-seen photos: {accuracy_score(all_y[is_test], pred_h):.3f}")
for c, n in holdout.items():
    m = all_photo[is_test] == n
    counts = np.bincount(pred_h[m], minlength=len(class_names))
    top = ", ".join(f"{class_names[j]} {counts[j]}" for j in np.argsort(counts)[::-1] if
counts[j])
    print(f"   {n:8s} (true {c:6s}): {(pred_h[m] == class_names.index(c)).mean():.2f} correct |
predicted as: {top}")
```

```
held-out photos: ['brick_B', 'grass_C', 'gravel_A', 'wood_A', 'ice_B'] -> 231 test patches, 283 t
raining patches
accuracy on never-seen photos: 0.333
   brick_B  (true brick ): 0.00 correct | predicted as: gravel 35
   grass_C  (true grass ): 0.94 correct | predicted as: grass 33, gravel 2
   gravel_A (true gravel): 0.57 correct | predicted as: gravel 20, paving 14, brick 1
   wood_A   (true wood  ): 0.00 correct | predicted as: brick 75, gravel 10, fur 6
   ice_B    (true ice   ): 0.69 correct | predicted as: ice 24, gravel 10, fur 1
```

```
fig, axes = plt.subplots(1, 5, figsize=(12, 2.6))
for a, (c, n) in zip(axes, holdout.items()):
    other = [p for p, _ in photos[c] if p != n]
    a.imshow(dict(photos[c])[n][:200, :200], cmap="gray"); a.axis("off")
    a.set_title(f"held out: {n}\n(trained on {', '.join(other)})", fontsize=8)
plt.suptitle("The photos the model never saw in this test"); plt.tight_layout(); plt.show()
```



This is the honest picture: only **33%** of patches from never-seen photos are classified correctly. Grass (94%) and ice (69%) transfer well because their photos look alike. The light pine planks (`wood_A`) are called **brick** (horizontal boards with joints really do look like a brick wall when you only count local patterns), and scikit-image's brick wall (`brick_B` : long, slanted dark bricks with bright mortar) is called gravel: it looks nothing like the brick photo it was trained on (short horizontal bricks). When the same material is photographed **at a very different scale or style**, the word frequencies change and BoVW fails. Classic fixes: more and more varied training photos per class, dense SIFT at several scales, colour descriptors, or a CNN that learns features from thousands of images.

12 · Extra experiment: detected keypoints vs a dense grid

`detectAndCompute` only describes "interesting" points, and smooth textures get few of them. A common BoVW improvement is **dense SIFT**: put keypoints on a regular grid (here every 8 pixels, size 16) and only *compute* descriptors there with `sift.compute` . Same vocabulary size, same SVM, same region split as section 8.

```
grid_kp = [cv2.KeyPoint(float(x), float(y), 16) for y in range(8, PATCH - 7, 8) for x in range(8,
PATCH - 7, 8)]
t0 = time.perf_counter()
dense_tr = [sift.compute(p, grid_kp)[1] for p in train_imgs]
dense_te = [sift.compute(p, grid_kp)[1] for p in test_imgs]
dd = np.vstack(dense_tr)
```

```
km_d = KMeans(n_clusters=best_k, random_state=42).fit(dd[rng.choice(len(dd), 20000,
replace=False)])
Xd_tr = np.array([build_histogram(d, km_d) for d in dense_tr]); Xd_te =
np.array([build_histogram(d, km_d) for d in dense_te])
acc_dense = accuracy_score(test_y, SVC(kernel='rbf', C=10).fit(Xd_tr, train_y).predict(Xd_te))
print(f"dense grid: {len(grid_kp)} descriptors per patch, accuracy {acc_dense:.3f}
({time.perf_counter() - t0:.0f}s)")
print(f"detected keypoints (section 9, same k={best_k}): accuracy {max(res['rbf']):.3f}")
```

```
dense grid: 225 descriptors per patch, accuracy 0.729  (22s)
detected keypoints (section 9, same k=100): accuracy 0.975
```

Result: dense SIFT was clearly worse here (about 73% vs 97.5%), the opposite of what many papers report. Two likely reasons, both visible in this notebook: (1) a single fixed scale (size 16) and no rotation normalisation (grid keypoints have no orientation), while detected keypoints adapt to each pattern's scale and angle; (2) with a fixed 225 descriptors per patch, the "number of keypoints" clue from section 9b disappears. Dense SIFT usually shines with several scales and much larger datasets. The lesson: **measure, don't assume.**

13 · 🔧 What we changed from the course notebook, and why

#	Course code	Problem	Fix
1	3 images (`sample.jpg` , `left.jpg` , `right.jpg`), one per class	**Fundamentally broken**: with one image per class, a train/test split leaves the test class unseen in training, so accuracy is 0% by construction (course output: accuracy 0.00, support 1). `left.jpg` / `right.jpg` are OpenCV's stereo-camera samples, not textures, and none of the files are provided.	Built a 7-class dataset from 15 real texture photos, many patches per class, region-based train/test split plus an "unseen photo" test.
2	`train_test_split` on 3 histograms, `test_size=0.3`	2 training samples, 1 test sample; the report and accuracy are meaningless.	Clean split with every class on both sides (hundreds of patches).
3	`build_histogram` returns raw counts	Histograms depend on how many keypoints an image happens to have.	Normalised to sum to 1 (L1); measured the effect.
4	k-means fitted on descriptors of *all* images	With a real split, the vocabulary must be learned from **training images only**, or test information leaks in.	Fitted k-means on training descriptors only.
5	`y.extend([idx]*len(des))` in the first cell	Builds a per-descriptor label list that is never used (dead code).	Removed.
6	Fixed k = 20, no evaluation beyond one report	The Key Concepts ask to explore k.	k sweep 5 → 400, confusion matrix, visual-word visualisation, dense SIFT comparison.
7	`RuntimeWarning ... matmul` in k-means	Comes from the course author's numpy/macOS maths library, not from BoVW.	Not reproducible here; nothing to fix in the method.

#	Course code	Problem	Fix
8	Images read from disk with `cv2.imread(path)` and no check	`cv2.imread` returns `None` for a missing file and SIFT then crashes with a cryptic error.	Images are downloaded and cached by a helper with a clear error message.

14 · Results, honestly

All numbers come from the run above.

- The course pipeline (3 images, one per class) scores **0%** by construction, reproduced in section 3.
- With a real dataset (7 classes, 15 photos, 311 training and 203 test patches from separate regions) the same SIFT → k-means → histogram → SVM pipeline reaches **79%** with the course's exact settings (k = 20, `SVC(kernel='linear')`, C = 1), **93.6%** with k = 20 and an RBF SVM, and **97.5%** with the best vocabulary (k = 100) and an RBF SVM.
- Vocabulary size matters: k = 5 gave 79% (RBF), k = 100 gave 97.5%, and larger vocabularies did not help further (95–96% at k = 200–400).
- Raw counts beat normalised histograms on equal-size patches (e.g. 99.0% vs 95.6% at k = 400), because the keypoint count itself separates busy from calm textures. On bigger 192×192 test patches the raw-count model fell to **61.5%**, while the normalised one kept **95.8%**: normalisation is what makes BoVW robust to image size.
- On whole **photos the model has never seen**, accuracy falls to **33%**. This is the most realistic number in this notebook: 2–3 photos per class are not enough variety for BoVW to generalise to a new photo of the same material in a different style or scale.
- Dense single-scale SIFT scored 72.9% vs 97.5% for detected keypoints on this data.
- Caveats: the texture photos come from 3D-graphics asset libraries (some are tiled/edited), the dataset is small, and texture scale was only roughly matched.

15 · Common errors and how to fix them

Error / symptom	Cause	Fix
`AttributeError: module 'cv2' has no attribute 'SIFT_create'`	Very old OpenCV (< 4.4)	`pip install --upgrade opencv-python` (or use `cv2.xfeatures2d.SIFT_create()` with old contrib builds)
`cv2.error ... (-215:Assertion failed) !_src.empty()`	`cv2.imread` returned `None` (wrong path)	Check the file exists; print `img is None`
`TypeError` / crash in `kmeans.predict(des)`	`des` is `None` for an image with no keypoints	Check `if des is not None` (or return an all-zero histogram, as here)
Accuracy 0% or a report with `support 1.0`	One image per class, test class unseen in training (the course bug)	Many images per class; stratified split

Error / symptom	Cause	Fix
`UndefinedMetricWarning: Precision is ill-defined`	Some class never predicted (or absent from the test set)	Usually a sign the dataset/split is broken; fix the data
Very high accuracy that does not hold on new photos	Train and test patches overlap or come from the same photo	Split by region or by photo, never shuffle overlapping patches
k-means very slow	Millions of descriptors, large k	Fit on a random sample (here 20,000) or use `MiniBatchKMeans`

16 · Try this yourself

1. Change `PATCH` to 64 or 192. More, smaller patches or fewer, bigger ones: which gives better accuracy, and why (keypoints per patch)?
2. Replace SIFT with ORB (`cv2.ORB_create()`); ORB descriptors are binary (32 bytes), so convert them with `des.astype(np.float32)` before k-means. Compare accuracy and speed.
3. Swap the SVM for `KNeighborsClassifier(n_neighbors=5)` on the histograms. Which classifier works better here?
4. Add another texture class: find a texture photo (e.g. another file from the three.js `examples/textures` folder), add it to `sources`, rerun.
5. Change the hold-out photos in section 11 (e.g. hold out `grass_A` instead of `grass_C`). Does the accuracy change a lot? What does that tell you?
6. Try `MiniBatchKMeans` from `sklearn.cluster` with k = 1000: faster? better?

17 · Viva questions (with short model answers)

1. **What is Bag of Visual Words?** Representing an image as a histogram of how often each "visual word" (a cluster of similar local descriptors) occurs, then classifying that histogram, like counting words in a document.
2. **Give the four steps of the pipeline.** Extract local descriptors (SIFT) → cluster training descriptors with k-means to make a vocabulary → assign each descriptor to its nearest word and count (histogram) → train a classifier (SVM) on the histograms.
3. **Why do we need the histogram step at all?** Images produce different numbers of descriptors; the histogram converts them into one fixed-length vector of k numbers that a classifier can use.
4. **What is a visual word?** A k-means cluster centre in descriptor space (a 128-D vector for SIFT): a typical local pattern.
5. **What does k control, and how do you choose it?** The vocabulary size. Too small: different patterns merge, low accuracy. Too large: sparse, noisy histograms and slower. Choose by validation (here we swept 5 to 400).
6. **Why normalise the histograms?** So that the representation does not depend on how many keypoints an image has (texture-rich vs smooth, large vs small).

7. **Why must the vocabulary be built from training images only?** Otherwise information from the test images leaks into the features and the test score is optimistic.
8. **Why did the course notebook get 0%?** One image per class; after the split the test image's class was never in training, so the SVM could not output it.
9. **What does a SIFT descriptor contain?** 128 numbers: histograms of gradient orientations (8 bins) in a 4×4 grid of cells around the keypoint, normalised; rotation is handled by measuring angles relative to the keypoint's dominant orientation.
10. **What information does BoVW lose?** Spatial layout: where the features are. (Spatial pyramid matching adds coarse layout back.)
11. **How does k-means work?** Choose k centres, assign each point to the nearest centre, move each centre to the mean of its points, repeat until stable.
12. **Is k-means supervised?** No, it never sees labels. Only the SVM at the end uses labels.
13. **Why is BoVW suited to textures?** A texture is a repeated statistical pattern; the frequency of local patterns describes it well, and position does not matter.
14. **Why might accuracy drop on a new photo of the same material?** Different scale, lighting and style shift the word frequencies; with few training photos the classifier has not seen that variety.
15. **What replaced BoVW in modern vision?** CNNs, which learn the features (and the "vocabulary") from data end to end instead of using hand-designed SIFT + k-means.

Day 13 · Face detection: Haar cascades and HOG

Course lab: `CV-Week2-Lecture6.ipynb` (Day 13: Face Detection) **Notes chapter:** `week-2/notes/06_face_detection_haar_hog.md` **Run time:** about 3–5 minutes on a laptop CPU (downloads about 11 MB of test images and a test video the first time).

What you will learn

- What **face detection** is (find *where* faces are) and how it differs from face *recognition* (*who* it is).
- How a **Haar cascade** works: rectangle features, the integral image, AdaBoost and the "reject early" cascade.
- How **HOG** (Histogram of Oriented Gradients) describes shape, with real HOG code (the course notebook had none).
- What `scaleFactor`, `minNeighbors` and `minSize` in `detectMultiScale` do, measured as **missed faces vs false alarms**.
- A fair **Haar vs HOG** comparison: speed, accuracy, rotation and lighting.
- A small **face detector app** for images, with webcam code you can switch on at home.

1 · The ideas in plain language

Detection = sliding window + a yes/no classifier. Imagine checking a big group photo with a small square cardboard frame: you slide it across every position, at every size, and at each spot ask "is there a face in this frame?". Every detector in this notebook works this way; they differ in *how* they answer the question.

Haar cascade (Viola–Jones, 2001)

- **Haar features** are black-and-white rectangle patterns. A feature's value = (sum of pixels under the white part) − (sum under the black part). Example: "the eye band is darker than the cheeks below it".
- The **integral image** makes every rectangle sum cost only 4 look-ups, whatever its size, so features are very cheap.
- **AdaBoost** picks the few thousand most useful features out of over 160,000 possible ones in a 24×24 window.
- The **cascade** arranges them in stages: stage 1 uses a couple of features and throws away most non-face windows immediately; only windows that pass go to the more expensive later stages. Analogy: airport security, where most people are waved through at the first check and only a few get the full search. That is why Haar is **fast**.
- Weakness: it learned *frontal, upright, evenly lit* faces; tilted heads, side views and strong shadows break the rectangle patterns.

HOG (Dalal & Triggs, 2005)

- Compute the **gradient** (direction and strength of brightness change) at every pixel.
- Split the window into small **cells** (8×8 pixels) and, in each, build a **histogram of gradient directions** (9 bins over 0–180°).
- **Normalise** groups of cells (blocks of 2×2 cells) so that overall brightness and contrast cancel out. This is why HOG is **robust to illumination**.
- A **linear SVM** (Day 11!) decides face / not face (or person / not person) from this descriptor.
- Analogy: describing a face by "which way the edges point, region by region", not by exact pixel brightness.

2 · Setup

Install once: `pip install opencv-python scikit-image matplotlib`. Optional, for the HOG **face** detector: `pip install dlib-bin` (a pre-built dlib; `pip install dlib` needs CMake and a C++ compiler).

> ⚠️ **OpenCV 5 note.** In OpenCV **5.x** the classic `CascadeClassifier` and `HOGDescriptor` were moved out of the main `opencv-python` package into **`opencv-contrib-python`**, and the wheels no longer ship the Haar `.xml` files. If `cv2.CascadeClassifier` does not exist, run `pip uninstall opencv-python` then `pip install opencv-contrib-python` (or stay on OpenCV 4: `pip install "opencv-python<5"`). The helper below downloads the `.xml` files when they are missing, so the notebook works on OpenCV 4 and 5.

```
%matplotlib inline
%config InlineBackend.figure_format = 'jpeg'
import time
import urllib.request
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt
import cv2

cv2.setNumThreads(2)                    # optional: use at most 2 CPU cores
cv2.utils.logging.setLogLevel(cv2.utils.logging.LOG_LEVEL_ERROR)   # hide OpenCV's informational warnings
plt.rcParams["figure.dpi"] = 80
print("OpenCV", cv2.__version__)
if not hasattr(cv2, "CascadeClassifier"):
    raise ImportError("This OpenCV build has no CascadeClassifier (OpenCV 5 moved it to opencv-contrib-python). "
                      "Run: pip uninstall -y opencv-python && pip install opencv-contrib-python")

DATA = Path("data/faces"); DATA.mkdir(parents=True, exist_ok=True)
OUTPUT = Path("output"); OUTPUT.mkdir(exist_ok=True)

def download(url, name=None):
    """Download url into data/faces/ once; later runs reuse the cached file."""
    path = DATA / (name or url.split("/")[-1])
    if not path.exists():
        try:
            urllib.request.urlretrieve(url, path)
        except Exception as e:
            raise RuntimeError(f"Could not download {url}: {e}\nAre you offline? Put the file in {DATA}/ by hand.") from e
    return path
```

```
CASCADE_URL = "https://raw.githubusercontent.com/opencv/opencv/4.x/data/haarcascades/"

def load_cascade(name):
    """Load a Haar cascade from OpenCV's own folder, or download it (OpenCV 5 wheels do not
include the .xml files)."""
    local = Path(cv2.data.haarcascades) / name
    path = local if local.exists() else download(CASCADE_URL + name)
    clf = cv2.CascadeClassifier(str(path))
    if clf.empty():                     # an empty classifier is what causes "(-215:Assertion
failed) !empty()"
        raise IOError(f"Could not load cascade {path}")
    print("loaded", name, "from", "OpenCV install" if local.exists() else "download")
    return clf

def show(img_bgr, title="", ax=None, size=6):
    """Replacement for cv2.imshow: notebooks cannot open OpenCV windows, so we draw with
matplotlib (BGR -> RGB)."""
    if ax is None:
        _, ax = plt.subplots(figsize=(size, size * img_bgr.shape[0] / img_bgr.shape[1]))
    ax.imshow(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB) if img_bgr.ndim == 3 else img_bgr,
cmap="gray")
    ax.set_title(title, fontsize=10); ax.axis("off")
    return ax
```

```
OpenCV 5.0.0
```

```
SAMPLES = "https://raw.githubusercontent.com/opencv/opencv/4.x/samples/data/"
CMU =
"https://raw.githubusercontent.com/opencv/opencv_extra/4.x/testdata/cv/cascadeandhog/images/"
urls = {
    "zidane.jpg": "https://github.com/ultralytics/assets/releases/download/v0.0.0/zidane.jpg",
    "lena.jpg": SAMPLES + "lena.jpg",
    "messi5.jpg": SAMPLES + "messi5.jpg",
    "class57.png": CMU + "class57.png",                 # CMU/MIT face test set images, via
OpenCV's test data
    "addams-family.png": CMU + "addams-family.png",
    "er.png": CMU + "er.png",
    "rehg-thanksgiving-1994.png": CMU + "rehg-thanksgiving-1994.png",
    "waynesworld2.png": CMU + "waynesworld2.png",
    "karen-and-rob.png": CMU + "karen-and-rob.png",
}
paths = {name: download(u, name) for name, u in urls.items()}
for n, p in paths.items():
    print(f"{n:28s} {cv2.imread(str(p)).shape}")
```

```
zidane.jpg                   (720, 1280, 3)
lena.jpg                     (512, 512, 3)
messi5.jpg                   (342, 548, 3)
class57.png                  (1024, 1280, 3)
addams-family.png            (890, 864, 3)
er.png                       (500, 500, 3)
rehg-thanksgiving-1994.png   (396, 580, 3)
waynesworld2.png             (599, 443, 3)
karen-and-rob.png            (484, 336, 3)
```

3 · The course code: Haar face detection on one image (fixed)

The course loads `basketball.jpg`, which was **not provided**, and shows the result with `cv2.imshow` + `cv2.waitKey(0)`, which **does not work in Jupyter** (no desktop window; on a remote/Colab kernel it crashes or hangs the kernel). We keep every other line and use a sports photo (`zidane.jpg`, two football coaches) plus our `show()` helper.

```
# Load Haar cascade (course: cv2.CascadeClassifier(cv2.data.haarcascades +
'haarcascade_frontalface_default.xml'))
```

```python
face_cascade = load_cascade('haarcascade_frontalface_default.xml')

# Load image
img = cv2.imread(str(paths['zidane.jpg']))       # course: cv2.imread('basketball.jpg') - file
not provided
if img is None:
    raise IOError("Image not found. Check the file path.")

# Convert to grayscale: Haar features only use brightness
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)
print("faces found:", len(faces), "| boxes (x, y, w, h):", [tuple(map(int, f)) for f in faces])

# Draw rectangles around detected faces ((255, 0, 0) is BLUE, because OpenCV uses BGR order)
for (x, y, w, h) in faces:
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Show result (course: cv2.imshow + cv2.waitKey, which do not work in a notebook)
show(img, "Course settings: scaleFactor=1.3, minNeighbors=5", size=8); plt.show()
```

```
loaded haarcascade_frontalface_default.xml from download
faces found: 1 | boxes (x, y, w, h): [(911, 107, 162, 162)]
```



Line by line

- `CascadeClassifier(...xml)` loads a **pre-trained** cascade: thousands of Haar features and thresholds learned by OpenCV's authors from labelled face images. We do not train anything today.
- `detectMultiScale(gray, scaleFactor, minNeighbors)` slides the 24×24 detector over the image at many sizes and returns a list of boxes `(x, y, w, h)`: top-left corner, width, height.
- Only **one of the two** faces is found with the course settings: the man on the left has his head **turned and tilted**, which a *frontal* cascade was never trained on.

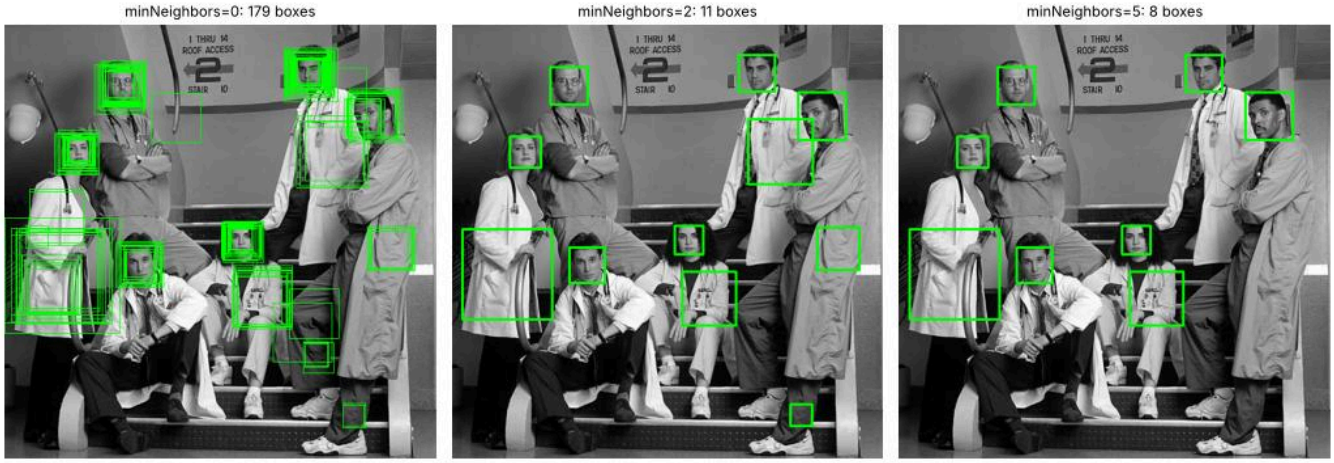
The three key parameters of `detectMultiScale`

Parameter	Meaning	Small value	Large value
`scaleFactor`	How much the image is shrunk between pyramid levels (1.3 =	1.05: many levels, finds faces of every	1.5: few levels, **fast**, can skip a face whose size

Parameter	Meaning	Small value	Large value
	30% smaller each step)	size, **slow**	falls between levels
`minNeighbors`	How many overlapping raw detections a box needs to be accepted	0–1: keeps everything, **many false alarms**	10+: very strict, **misses real faces**
`minSize` / `maxSize`	Smallest / largest face size to look for, e.g. `minSize=(30, 30)`	finds tiny faces (and tiny false alarms), slower	faster, ignores small faces

Why "neighbours"? A real face triggers the detector at many nearby positions and sizes, while a random false hit is usually isolated. `minNeighbors` groups overlapping raw boxes and keeps only groups with enough members.

```
# See the raw detections before grouping (minNeighbors=0) vs after
g = cv2.cvtColor(cv2.imread(str(paths['er.png'])), cv2.COLOR_BGR2GRAY)
fig, axes = plt.subplots(1, 3, figsize=(13, 4.6))
for a, mn in zip(axes, [0, 2, 5]):
    vis = cv2.cvtColor(g, cv2.COLOR_GRAY2BGR)
    det = face_cascade.detectMultiScale(g, scaleFactor=1.1, minNeighbors=mn)
    for (x, y, w, h) in det:
        cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 1 if mn == 0 else 2)
    show(vis, f"minNeighbors={mn}: {len(det)} boxes", ax=a)
plt.tight_layout(); plt.show()
```



With `minNeighbors=0` you see the raw sliding-window hits: clusters of boxes on every face plus scattered single hits elsewhere. Raising `minNeighbors` merges the clusters and drops the lonely hits.

What a Haar feature computes (the integral image trick)

```
lena = cv2.imread(str(paths['lena.jpg']), cv2.IMREAD_GRAYSCALE)
ii = cv2.integral(lena)                    # ii[y, x] = sum of all pixels above and left of (x, y)

def rect_sum(x, y, w, h):
    return int(ii[y + h, x + w] - ii[y, x + w] - ii[y + h, x] + ii[y, x])   # 4 look-ups, whatever the size

# Two-rectangle "eye band darker than cheeks" feature: value = cheeks (white) - eyes (black)
def eye_feature(x, y, w, h):
    return rect_sum(x, y + h // 2, w, h // 2) - rect_sum(x, y, w, h // 2)

print("check the 4-look-up sum against numpy:", rect_sum(250, 250, 40, 30), "==",
int(lena[250:280, 250:290].sum()))
```

```
eye_box, wall_box = (255, 255, 120, 70), (40, 40, 120, 70)
print("feature value on the eyes of the face:", eye_feature(*eye_box))
print("feature value on a background area   :", eye_feature(*wall_box))

vis = cv2.cvtColor(lena, cv2.COLOR_GRAY2BGR)
for (x, y, w, h), col in [(eye_box, (0, 0, 255)), (wall_box, (255, 0, 0))]:
    cv2.rectangle(vis, (x, y), (x + w, y + h // 2), (0, 0, 0), 2)
    cv2.rectangle(vis, (x, y + h // 2), (x + w, y + h), (255, 255, 255), 2)
    cv2.rectangle(vis, (x, y), (x + w, y + h), col, 1)
show(vis, "top half (black frame) vs bottom half (white frame)", size=4); plt.show()
```

```
check the 4-look-up sum against numpy: 131452 == 131452
feature value on the eyes of the face: 60631
feature value on a background area   : -15043
```

top half (black frame) vs bottom half (white frame)



On the eyes the feature gives a large positive value (cheeks much brighter than the eye band); on a background area it is small. One feature like this is a **weak** classifier. AdaBoost combines thousands of them into a strong one, and the cascade orders them so most windows are rejected after only a few.

4 · Measuring detection: reference faces, misses and false alarms

To compare settings and detectors fairly we need to know where the real faces are. We use 9 photos with 85 faces in total (mostly from the classic CMU/MIT face-detection test set). The reference boxes come from **YuNet**, a small modern CNN face detector built into OpenCV (score ≥ 0.8), and we **checked them by eye** on every image (one is shown below). A detection counts as correct if its centre lies inside a reference face and its size is similar; each reference face can be matched once.

- **Missed face** (false negative): a real face with no detection.
- **False alarm** (false positive): a box where there is no face.

```
yunet_path =
download("https://media.githubusercontent.com/media/opencv/opencv_zoo/main/models/face_detection_
yunet/face_detection_yunet_2023mar.onnx",
                         "face_detection_yunet_2023mar.onnx")
eval_names = ["class57.png", "addams-family.png", "er.png", "rehg-thanksgiving-1994.png",
"waynesworld2.png",
              "karen-and-rob.png", "zidane.jpg", "messi5.jpg", "lena.jpg"]
images = {n: cv2.imread(str(paths[n])) for n in eval_names}
grays = {n: cv2.cvtColor(im, cv2.COLOR_BGR2GRAY) for n, im in images.items()}
```

```python
yunet = cv2.FaceDetectorYN.create(str(yunet_path), "", (320, 320), 0.8, 0.3, 5000)   # score threshold 0.8

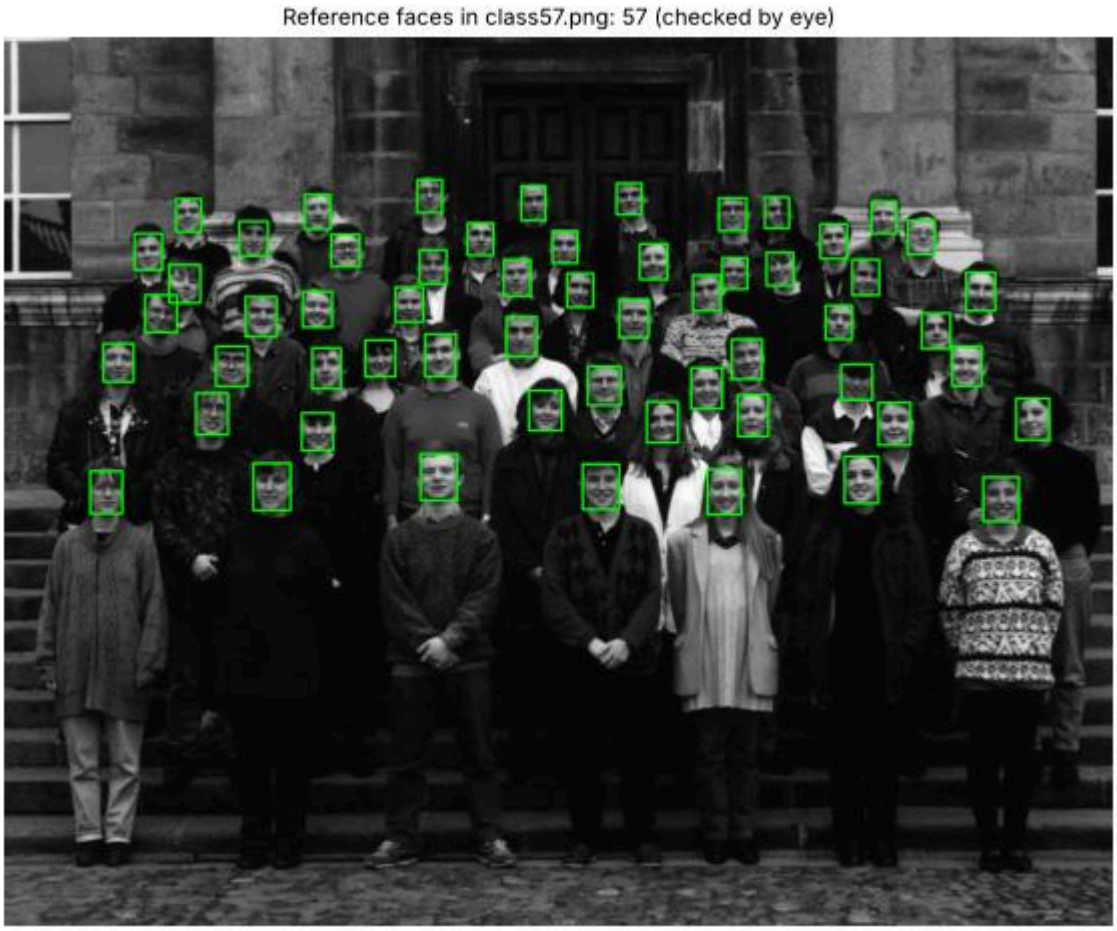
def yunet_faces(img):
    yunet.setInputSize((img.shape[1], img.shape[0]))
    _, f = yunet.detect(img)
    return [] if f is None else [tuple(map(int, r[:4])) for r in f]

refs = {n: yunet_faces(images[n]) for n in eval_names}
print({n: len(r) for n, r in refs.items()}, "-> total", sum(map(len, refs.values())), "reference faces")

def match(dets, ref):
    """Return (correct, false_alarms, missed)."""
    used, tp = set(), 0
    for (x, y, w, h) in dets:
        cx, cy = x + w / 2, y + h / 2
        for j, (rx, ry, rw, rh) in enumerate(ref):
            if j not in used and rx <= cx <= rx + rw and ry <= cy <= ry + rh and 0.5 < w / rw < 2.5:
                used.add(j); tp += 1; break
    return tp, len(dets) - tp, len(ref) - tp

vis = images["class57.png"].copy()
for (x, y, w, h) in refs["class57.png"]:
    cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 2)
show(vis, f"Reference faces in class57.png: {len(refs['class57.png'])} (checked by eye)", size=9); plt.show()
```

```
{'class57.png': 57, 'addams-family.png': 7, 'er.png': 6, 'rehg-thanksgiving-1994.png': 7, 'waynes world2.png': 2, 'karen-and-rob.png': 2, 'zidane.jpg': 2, 'messi5.jpg': 1, 'lena.jpg': 1} -> total 85 reference faces
```



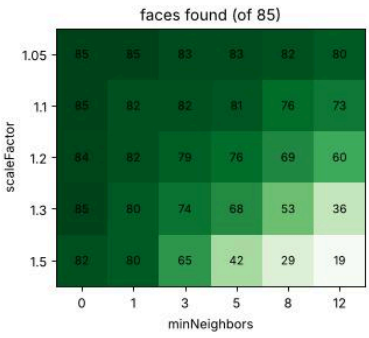
5 · Experiment: `scaleFactor` and `minNeighbors` (missed faces vs false alarms)

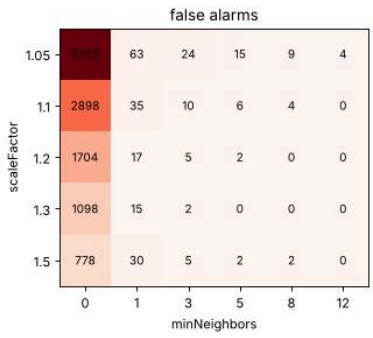
The Key Concepts ask you to *adjust scaleFactor and minNeighbors to improve accuracy*. Here is the full trade-off on all 9 photos.

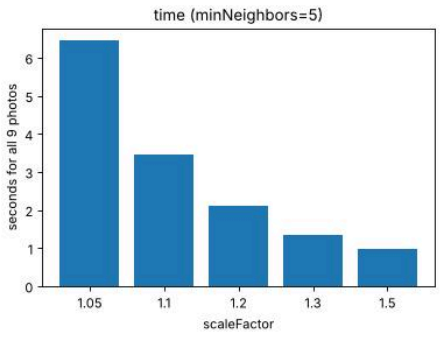
```
def run_haar(gray, sf, mn, min_size=(0, 0)):
    return face_cascade.detectMultiScale(gray, scaleFactor=sf, minNeighbors=mn, minSize=min_size)

sfs = [1.05, 1.1, 1.2, 1.3, 1.5]
mns = [0, 1, 3, 5, 8, 12]
total_ref = sum(map(len, refs.values()))
found = np.zeros((len(sfs), len(mns))); false = np.zeros_like(found); secs = np.zeros(len(sfs))
for i, sf in enumerate(sfs):
    for j, mn in enumerate(mns):
        t0 = time.perf_counter()
        for n in eval_names:
            tp, fp, miss = match(run_haar(grays[n], sf, mn), refs[n])
            found[i, j] += tp; false[i, j] += fp
        if mn == 5:
            secs[i] = time.perf_counter() - t0

fig, ax = plt.subplots(1, 3, figsize=(15, 3.8))
for a, M, t, cmap in [(ax[0], found, f"faces found (of {total_ref})", "Greens"), (ax[1], false,
"false alarms", "Reds")]:
    a.imshow(M, cmap=cmap); a.set_xticks(range(len(mns)), mns); a.set_yticks(range(len(sfs)),
sfs)
    a.set_xlabel("minNeighbors"); a.set_ylabel("scaleFactor"); a.set_title(t)
    for i in range(len(sfs)):
        for j in range(len(mns)):
            a.text(j, i, int(M[i, j]), ha="center", va="center", fontsize=9)
ax[2].bar([str(s) for s in sfs], secs); ax[2].set_xlabel("scaleFactor");
ax[2].set_ylabel("seconds for all 9 photos")
ax[2].set_title("time (minNeighbors=5)")
plt.tight_layout(); plt.show()
print("seconds for all 9 photos (minNeighbors=5):", {sf: round(t, 1) for sf, t in zip(sfs,
secs)})
for i, sf in enumerate(sfs):
    print(f"scaleFactor {sf}: " + " | ".join(f"mN={mn}: {int(found[i, j])} found, {int(false[i,
j])} false" for j, mn in enumerate(mns)))
```







```
seconds for all 9 photos (minNeighbors=5): {1.05: np.float64(6.5), 1.1: np.float64(3.5), 1.2: np.
float64(2.1), 1.3: np.float64(1.4), 1.5: np.float64(1.0)}
scaleFactor 1.05: mN=0: 85 found, 5752 false | mN=1: 85 found, 63 false | mN=3: 83 found, 24 fals
e | mN=5: 83 found, 15 false | mN=8: 82 found, 9 false | mN=12: 80 found, 4 false
scaleFactor 1.1: mN=0: 85 found, 2898 false | mN=1: 82 found, 35 false | mN=3: 82 found, 10 false
| mN=5: 81 found, 6 false | mN=8: 76 found, 4 false | mN=12: 73 found, 0 false
scaleFactor 1.2: mN=0: 84 found, 1704 false | mN=1: 82 found, 17 false | mN=3: 79 found, 5 false
| mN=5: 76 found, 2 false | mN=8: 69 found, 0 false | mN=12: 60 found, 0 false
scaleFactor 1.3: mN=0: 85 found, 1098 false | mN=1: 80 found, 15 false | mN=3: 74 found, 2 false
| mN=5: 68 found, 0 false | mN=8: 53 found, 0 false | mN=12: 36 found, 0 false
scaleFactor 1.5: mN=0: 82 found, 778 false | mN=1: 80 found, 30 false | mN=3: 65 found, 5 false |
mN=5: 42 found, 2 false | mN=8: 29 found, 2 false | mN=12: 19 found, 0 false
```

Reading the heatmaps (exact numbers printed above):

- Moving **right** (bigger `minNeighbors`) removes false alarms quickly, but eventually starts removing real faces too.
- Moving **down** (bigger `scaleFactor`) is much faster but checks fewer face sizes, so more faces are missed.
- There is no setting with zero misses and zero false alarms: that is the trade-off. A security camera might accept some false alarms to miss nothing; a photo app would rather miss a face than tag a lamp.
- The course setting (1.3, 5) is a fast, conservative choice; `scaleFactor=1.1, minNeighbors=5` is the most common recommendation and usually finds more faces for a small time cost.

6 · HOG: what the descriptor looks like (the course had no HOG code)

`skimage.feature.hog` computes the descriptor and an image to visualise it: each cell shows little lines whose length is the strength of each gradient direction.

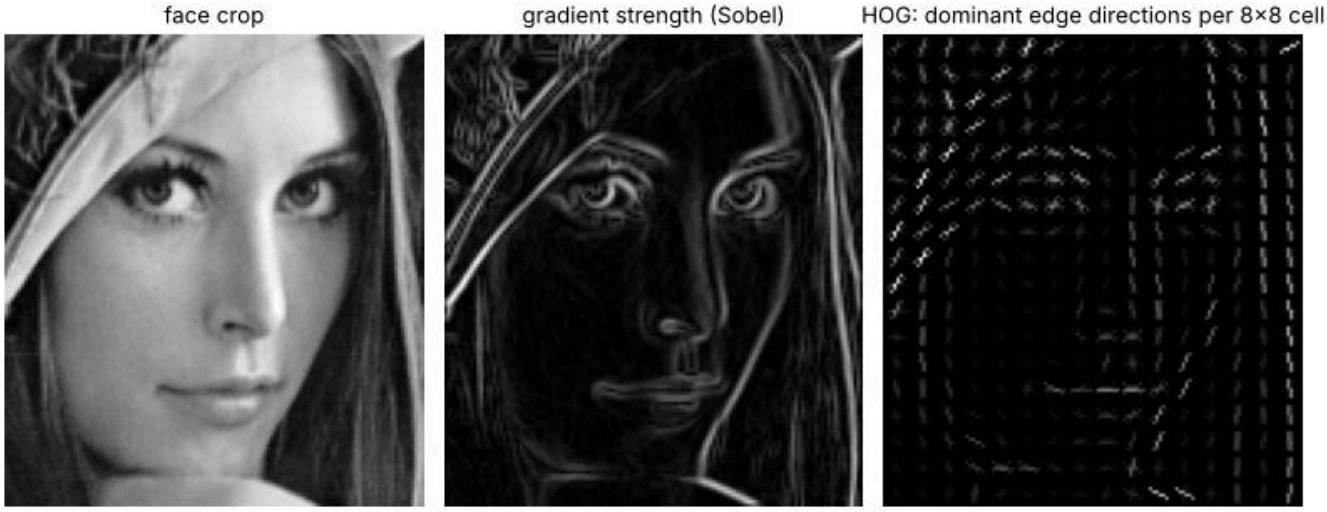
- `orientations=9`: 9 direction bins covering 0–180°.
- `pixels_per_cell=(8, 8)`: one histogram per 8×8 cell.
- `cells_per_block=(2, 2)`: normalise over 2×2 cells (this is the lighting-invariance step).

```
from skimage.feature import hog

face = cv2.resize(lena[200:400, 200:380], (128, 144))     # crop around the face
gx = cv2.Sobel(face.astype(np.float32), cv2.CV_32F, 1, 0); gy =
cv2.Sobel(face.astype(np.float32), cv2.CV_32F, 0, 1)
feat, hog_img = hog(face, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2),
visualize=True)
print("HOG descriptor length for a 128x144 crop:", feat.shape[0],
      "= (16-1) x (18-1) blocks x 2x2 cells x 9 bins =", 15 * 17 * 4 * 9)

fig, ax = plt.subplots(1, 3, figsize=(10, 4))
ax[0].imshow(face, cmap="gray"); ax[0].set_title("face crop")
ax[1].imshow(np.hypot(gx, gy), cmap="gray"); ax[1].set_title("gradient strength (Sobel)")
ax[2].imshow(hog_img, cmap="gray"); ax[2].set_title("HOG: dominant edge directions per 8x8 cell")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

```
HOG descriptor length for a 128x144 crop: 9180 = (16-1) x (18-1) blocks x 2x2 cells x 9 bins = 91
80
```




```
# Why HOG is "robust to illumination": compare the descriptor of the face with a darker, low-contrast copy
dark = np.clip(face.astype(np.float32) * 0.35 + 10, 0, 255).astype(np.uint8)
feat_dark = hog(dark, orientations=9, pixels_per_cell=(8, 8), cells_per_block=(2, 2))
cos = lambda a, b: float(a @ b / (np.linalg.norm(a) * np.linalg.norm(b)))
print(f"raw pixels:      mean absolute difference = {np.abs(face.astype(float) - dark).mean():.1f} grey levels")
print(f"HOG descriptors: cosine similarity = {cos(feat, feat_dark):.3f}  (1.0 = identical)")
fig, ax = plt.subplots(1, 2, figsize=(5, 3))
ax[0].imshow(face, cmap="gray", vmin=0, vmax=255); ax[0].set_title("original");
ax[1].imshow(dark, cmap="gray", vmin=0, vmax=255); ax[1].set_title("darker, low contrast")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

```
raw pixels:      mean absolute difference = 67.1 grey levels
HOG descriptors: cosine similarity = 0.978  (1.0 = identical)
```



The pixels changed a lot, but the HOG descriptor is almost unchanged: gradients *directions* stay the same when lighting is scaled, and block normalisation cancels the change in gradient *strength*.

HOG in action 1: OpenCV's HOG pedestrian detector

The classic use of HOG (Dalal & Triggs' paper) is finding **people**. OpenCV ships a ready-trained linear SVM for a 64×128 window. We run it on a frame of OpenCV's sample street video `vtest.avi`.

```
video = download(SAMPLES + "vtest.avi")
cap = cv2.VideoCapture(str(video))
```

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 100)
ok, frame = cap.read(); cap.release()
if not ok:
    raise IOError("Could not read vtest.avi (your OpenCV may lack a video backend)")

hog_people = cv2.HOGDescriptor()                    # default: 64x128 window, 8x8 cells, 9 bins,
2x2-cell blocks
hog_people.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
print("descriptor length per 64x128 window:", hog_people.getDescriptorSize(), "= 7 x 15 blocks x
4 cells x 9 bins")

t0 = time.perf_counter()
rects, weights = hog_people.detectMultiScale(frame, winStride=(8, 8), padding=(8, 8), scale=1.05)
dt = time.perf_counter() - t0
vis = frame.copy()
for (x, y, w, h), s in zip(rects, np.ravel(weights)):
    cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.putText(vis, f"{s:.1f}", (x, y - 4), cv2.FONT_HERSHEY_SIMPLEX, 0.45, (0, 255, 255), 1)
print(f"original frame: {len(rects)} people found in {dt * 1000:.0f} ms")

# The people here are only about 60-100 px tall, smaller than the 64x128 window. Upscale the
frame 2x and try again.
big = cv2.resize(frame, None, fx=2, fy=2)
t0 = time.perf_counter()
rects2, weights2 = hog_people.detectMultiScale(big, winStride=(8, 8), padding=(8, 8), scale=1.05)
dt2 = time.perf_counter() - t0
vis2 = frame.copy()
for (x, y, w, h) in rects2 // 2:                    # boxes back to original coordinates
    cv2.rectangle(vis2, (x, y), (x + w, y + h), (0, 255, 0), 2)
print(f"frame upscaled 2x: {len(rects2)} people found in {dt2 * 1000:.0f} ms")
fig, ax = plt.subplots(1, 2, figsize=(14, 5.3))
show(vis, f"HOG people detector, original size: {len(rects)} found", ax=ax[0])
show(vis2, f"same frame upscaled 2x first: {len(rects2)} found", ax=ax[1])
plt.tight_layout(); plt.show()
```

```
descriptor length per 64x128 window: 3780 = 7 x 15 blocks x 4 cells x 9 bins
original frame: 2 people found in 209 ms
frame upscaled 2x: 7 people found in 723 ms
```



- `winStride=(8, 8)` : move the 64×128 window 8 pixels at a time (smaller = more thorough, slower).
- `padding` : extra border so people at the edge can be found. `scale=1.05` : image pyramid step, like `scaleFactor` .
- The number on each box is the SVM score (how far on the "person" side of the boundary).
- **Window size matters:** the detector cannot find anyone smaller than its 64×128 window, because the pyramid only *shrinks* the image. Upscaling first lets it find the small, distant people, at a cost of several times the computation (4× as many pixels; 3–6× slower in our

runs). One of the upscaled boxes is a false alarm on the parked van. (Haar has the same rule with its 24×24 window, and dlib's `upsample` argument does exactly this.)

HOG in action 2: a HOG **face** detector (dlib)

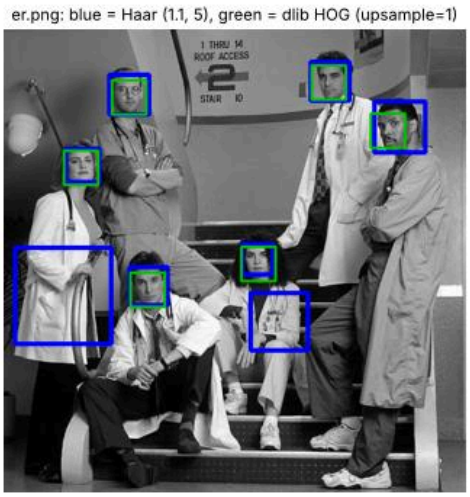
The most widely used HOG face detector is **dlib**'s `get_frontal_face_detector()`: HOG features + a linear SVM, sliding over an image pyramid, with an 80×80 window. Its argument `upsample` doubles the image size first (0 = no upsampling, finds faces ≥ about 80 px; 1 = finds faces ≥ about 40 px, slower).

```
try:
    import dlib
    hog_face = dlib.get_frontal_face_detector()
    HAVE_DLIB = True
    print("dlib", dlib.__version__, "loaded")
except ImportError:
    HAVE_DLIB = False
    print("dlib is not installed: run `pip install dlib-bin` to enable the HOG face detector.
Skipping HOG face cells.")

def run_hog(gray, upsample=1):
    return [(r.left(), r.top(), r.width(), r.height()) for r in hog_face(gray, upsample)]

if HAVE_DLIB:
    fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
    for a, n in zip(axes, ["er.png", "zidane.jpg"]):
        vis = images[n].copy()
        for (x, y, w, h) in run_haar(grays[n], 1.1, 5):
            cv2.rectangle(vis, (x, y), (x + w, y + h), (255, 0, 0), 3)
        for (x, y, w, h) in run_hog(grays[n], 1):
            cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 200, 0), 2)
        show(vis, f"{n}: blue = Haar (1.1, 5), green = dlib HOG (upsample=1)", ax=a)
    plt.tight_layout(); plt.show()
```

```
dlib 20.0.1 loaded
```





7 · A fair Haar vs HOG comparison

The Key Concepts say: *Haar = fast and lightweight but sensitive to lighting and pose; HOG = robust to illumination but computationally heavier.* Let's test each claim on the same 85 reference faces.

7a · Accuracy and speed

Times are the average of 3 runs over all 9 photos, both detectors on a single CPU core. For a fair size match, `Haar minSize=40` looks for roughly the same smallest face as `HOG upsample=1`.

```
detectors = {
    "Haar 1.3/5 (course)": lambda g: run_haar(g, 1.3, 5),
    "Haar 1.1/5": lambda g: run_haar(g, 1.1, 5),
    "Haar 1.1/5 minSize=40": lambda g: run_haar(g, 1.1, 5, (40, 40)),
}
if HAVE_DLIB:
    detectors["HOG upsample=0"] = lambda g: run_hog(g, 0)
    detectors["HOG upsample=1"] = lambda g: run_hog(g, 1)

cv2.setNumThreads(1)          # OpenCV's Haar search is multi-threaded, dlib is not: time both on
ONE core to be fair
table = {}
for name, fn in detectors.items():
    tp = fp = 0
    for n in eval_names:
        a, b, _ = match(fn(grays[n]), refs[n]); tp += a; fp += b
    t0 = time.perf_counter()
    for _ in range(3):
        for n in eval_names:
            fn(grays[n])
    table[name] = (tp, fp, (time.perf_counter() - t0) / 3)
print(f"{'detector':24s} {'found':>6s} {'missed':>7s} {'false':>6s} {'time (s)':>9s}")
for name, (tp, fp, t) in table.items():
    print(f"{name:24s} {tp:>6d} {total_ref - tp:>7d} {fp:>6d} {t:>9.2f}")
cv2.setNumThreads(2)
```

detector	found	missed	false	time (s)
Haar 1.3/5 (course)	68	17	0	1.41
Haar 1.1/5	81	4	6	3.33
Haar 1.1/5 minSize=40	67	18	6	2.20
HOG upsample=0	2	83	0	0.42
HOG upsample=1	79	6	0	1.97

7b · Robustness to rotation (head tilt)

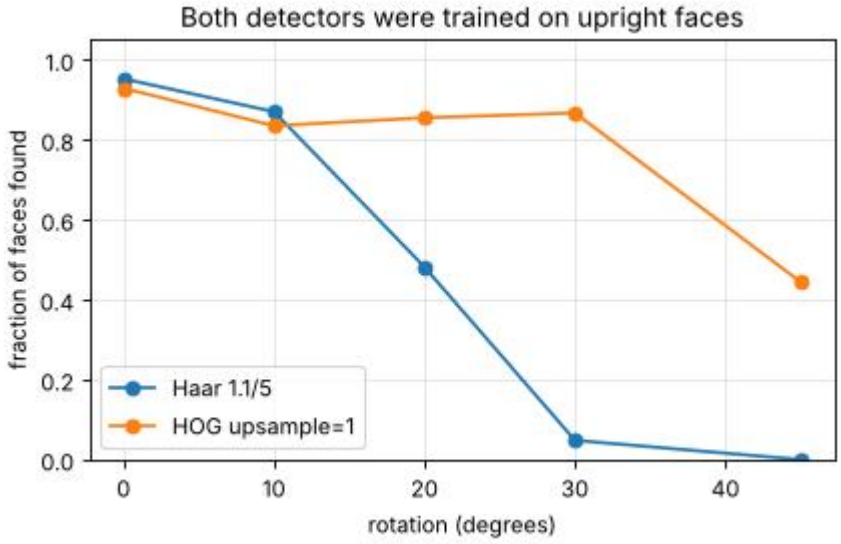
We rotate every photo by 0–45° (reference boxes are rotated with it) and count how many faces each detector still finds.

```
def rotate(img, refs_, angle):
    h, w = img.shape[:2]
    M = cv2.getRotationMatrix2D((w / 2, h / 2), angle, 1.0)
    out = cv2.warpAffine(img, M, (w, h), borderMode=cv2.BORDER_REPLICATE)
    new = []
    for (x, y, bw, bh) in refs_:
        cx, cy = M @ np.array([x + bw / 2, y + bh / 2, 1])
        if bw / 2 < cx < w - bw / 2 and bh / 2 < cy < h - bh / 2:      # face still fully inside
the frame
            new.append((int(cx - bw / 2), int(cy - bh / 2), bw, bh))
    return out, new

angles = [0, 10, 20, 30, 45]
rot_res = {name: [] for name in ["Haar 1.1/5", "HOG upsample=1"] if name in detectors}
for ang in angles:
    tot = 0; hits = {k: 0 for k in rot_res}
    for n in eval_names:
        g, r = rotate(grays[n], refs[n], ang); tot += len(r)
        for k in rot_res:
            hits[k] += match(detectors[k](g), r)[0]
    for k in rot_res:
        rot_res[k].append(hits[k] / tot)
    print(f"rotation {ang:>2}°: " + " | ".join(f"{k}: {hits[k]}/{tot} found" for k in rot_res))
```

```
plt.figure(figsize=(6, 3.4))
for k, v in rot_res.items():
    plt.plot(angles, v, "o-", label=k)
plt.xlabel("rotation (degrees)"); plt.ylabel("fraction of faces found"); plt.ylim(0, 1.05);
plt.grid(alpha=.3); plt.legend()
plt.title("Both detectors were trained on upright faces"); plt.show()
```

```
rotation  0°: Haar 1.1/5: 81/85 found | HOG upsample=1: 79/85 found
rotation 10°: Haar 1.1/5: 74/85 found | HOG upsample=1: 71/85 found
rotation 20°: Haar 1.1/5: 40/83 found | HOG upsample=1: 71/83 found
rotation 30°: Haar 1.1/5: 4/83 found | HOG upsample=1: 72/83 found
rotation 45°: Haar 1.1/5: 0/81 found | HOG upsample=1: 36/81 found
```



7c · Robustness to lighting

Four versions of every photo: original, **dark** (gamma 2.5, like an under-exposed photo), **low contrast** (washed out), and a **side shadow** (brightness falls to 15% across the image).

```
def lighting(img, kind):
    f = img.astype(np.float32) / 255
    if kind == "dark":
        f = f ** 2.5
    elif kind == "low contrast":
        f = 0.35 + 0.25 * f
    elif kind == "side shadow":
        f = f * np.linspace(1.0, 0.15, f.shape[1])[None, :]
    return (f * 255).clip(0, 255).astype(np.uint8)

kinds = ["original", "dark", "low contrast", "side shadow"]
fig, axes = plt.subplots(1, 4, figsize=(13, 2.8))
for a, kd in zip(axes, kinds):
    show(lighting(grays["er.png"], kd), kd, ax=a)
plt.tight_layout(); plt.show()

for kd in kinds:
    line = []
    for k in rot_res:
        tp = fp = 0
        for n in eval_names:
            a, b, _ = match(detectors[k](lighting(grays[n], kd)), refs[n]); tp += a; fp += b
        line.append(f"{k}: {tp}/{total_ref} found, {fp} false")
    print(f"{kd:13s} | " + " | ".join(line))
```




```
original      | Haar 1.1/5: 81/85 found, 6 false | HOG upsample=1: 79/85 found, 0 false
dark          | Haar 1.1/5: 73/85 found, 4 false | HOG upsample=1: 74/85 found, 0 false
low contrast  | Haar 1.1/5: 69/85 found, 3 false | HOG upsample=1: 72/85 found, 0 false
side shadow   | Haar 1.1/5: 77/85 found, 7 false | HOG upsample=1: 77/85 found, 0 false
```

7d · Turned faces (profile views)

Both detectors above are **frontal** detectors. OpenCV also ships `haarcascade_profileface.xml`, trained on faces seen from the side (it only detects faces looking one way, so we also run it on the mirrored image). Zidane's turned head is a good test.

```
profile_cascade = load_cascade('haarcascade_profileface.xml')
g = grays["zidane.jpg"]
prof = list(profile_cascade.detectMultiScale(g, 1.1, 5))
prof += [(g.shape[1] - x - w, y, w, h) for (x, y, w, h) in
profile_cascade.detectMultiScale(cv2.flip(g, 1), 1.1, 5)]
vis = images["zidane.jpg"].copy()
for (x, y, w, h) in run_haar(g, 1.1, 5):
    cv2.rectangle(vis, (x, y), (x + w, y + h), (255, 0, 0), 4)
for (x, y, w, h) in prof:
    cv2.rectangle(vis, (x, y), (x + w, y + h), (0, 0, 255), 2)
print("frontal Haar:", len(run_haar(g, 1.1, 5)), "| profile Haar (both directions):", len(prof),
      "| HOG:", len(run_hog(g, 1)) if HAVE_DLIB else "n/a")
show(vis, "blue = frontal cascade, red = profile cascade", size=8); plt.show()
```

```
loaded haarcascade_profileface.xml from download
frontal Haar: 2 | profile Haar (both directions): 0 | HOG: 1
```



Summary of the comparison (see the printed numbers; section 11 lists them):

- **Speed:** on one CPU core, over three runs of this notebook, Haar with the course settings (1.3/5) took 1.4–2.1 s for all 9 photos, Haar 1.1/5 took 3.3–5.0 s, and dlib HOG (upsample=1) took 2.0–2.2 s (times vary from run to run). So Haar is only faster with coarse settings; at

similar accuracy, dlib's HOG was **not** slower here. (Without upsampling HOG takes 0.5 s but finds just 2 of 85 faces, because most faces in these photos are smaller than its 80×80 window.)

- **Accuracy on normal photos:** Haar 1.1/5 found 81/85 faces with 6 false alarms; HOG found 79/85 with **0** false alarms. The course setting (Haar 1.3/5) found only 68/85.
- **Rotation:** Haar collapses beyond about 10° of tilt (40/83 at 20°, 4/83 at 30°, 0/81 at 45°). dlib's HOG kept 72/83 at 30° and 36/81 at 45°. Careful with the explanation: HOG features are *not* rotation invariant; dlib's detector is a set of **5 HOG filters** (frontal, looking left, looking right, and frontal tilted left/right), so it was *trained* to handle some tilt.
- **Lighting:** both lose some faces in dark and low-contrast versions (Haar 81 → 73 and 69; HOG 79 → 74 and 72). HOG lost fewer and never produced a false alarm; Haar kept producing 3–7. The side shadow cost both detectors a few faces (77 each).
- **Turned faces:** on `zidane.jpg` **nobody** finds Zidane's turned, shouting face: not the frontal cascade (its second box is a false alarm in the blurred crowd), not the profile cascade (0 detections), not HOG.

So the textbook summary is *partly* confirmed: Haar is more sensitive to lighting and pose, and HOG is more robust. But "HOG is computationally heavier" depends on the implementation and settings; dlib's optimised HOG was in the same speed range as Haar at similar accuracy. Modern systems use small CNN detectors (like YuNet, which we used to make the reference boxes) that beat both on accuracy and robustness.

8 · Exercise: detect faces in a webcam feed

The course webcam loop uses `cv2.imshow` and `cv2.waitKey`, which do not work in a notebook. Below is a notebook-friendly version that updates an image in place for a few seconds. It is **switched off** (`RUN_WEBCAM = False`) because the machine that built this notebook has no camera. Set it to `True` on your laptop. Also note the course loop never checks `ret`: if the camera fails, `frame` is `None` and `cvtColor` crashes, so we check it.

```
RUN_WEBCAM = False          # set to True on a laptop with a webcam

def webcam_face_detection(seconds=10, scaleFactor=1.3, minNeighbors=5, camera=0):
    from IPython.display import display, Image, clear_output
    cap = cv2.VideoCapture(camera)
    if not cap.isOpened():
        print("Could not open the webcam (is another program using it?)"); return
    t_end, n = time.time() + seconds, 0
    try:
        while time.time() < t_end:
            ret, frame = cap.read()
            if not ret:                         # the course loop skips this check
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            faces = face_cascade.detectMultiScale(gray, scaleFactor, minNeighbors)
            for (x, y, w, h) in faces:
                cv2.rectangle(frame, (x, y), (x + w, y + h), (255, 0, 0), 2)
            clear_output(wait=True)
            display(Image(data=cv2.imencode(".jpg", frame)[1].tobytes()))
            n += 1
    finally:
        cap.release()                           # always free the camera, even after an error / interrupt
    print(f"{n} frames in {seconds}s ({n / seconds:.1f} fps)")
```

```
if RUN_WEBCAM:
    webcam_face_detection(seconds=10)
else:
    print("Webcam demo skipped (RUN_WEBCAM = False).")
```

```
Webcam demo skipped (RUN_WEBCAM = False).
```

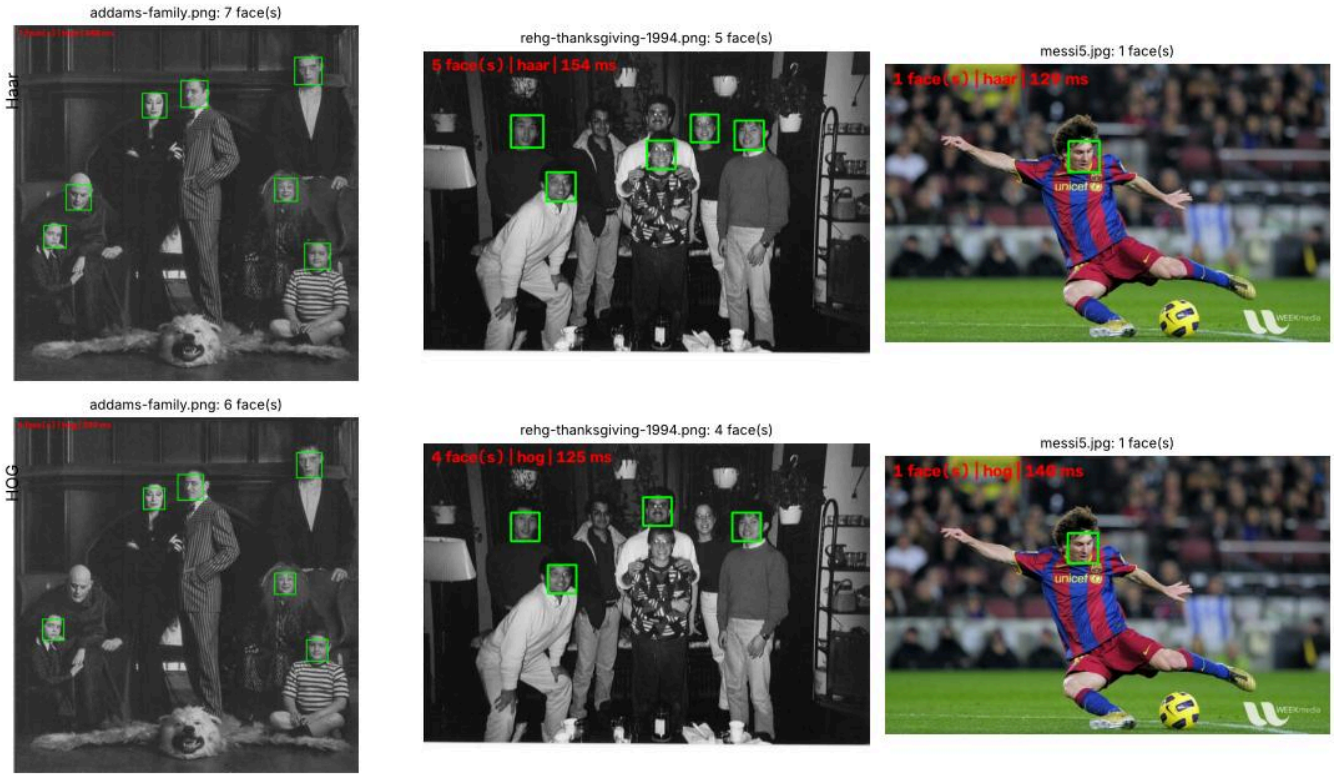
Things to try with the webcam (the Key Concepts' *learning points*): turn your head sideways, tilt it, cover half your face, switch off the light, move far from the camera. Then change `scaleFactor` and `minNeighbors` and watch false positives and missed detections change, exactly like the heatmaps in section 5. If you prefer a real OpenCV window, run the course loop in a plain `.py` script from a terminal, not in Jupyter.

9 · Mini project: a face detector app

One function that works on an image file or the webcam, with a choice of detector, that **returns** the boxes (so other code can use them) and saves the annotated image to `output/`.

```
def detect_faces(img_path=None, use_webcam=False, method="haar", scaleFactor=1.1, minNeighbors=5,
                 minSize=(30, 30), save=True, show_result=True, ax=None):
    """Detect faces in an image file (or start the webcam demo). Returns the list of (x, y, w, h)
boxes."""
    if use_webcam:
        webcam_face_detection(scaleFactor=scaleFactor, minNeighbors=minNeighbors)
        return []
    img = cv2.imread(str(img_path))
    if img is None:
        raise FileNotFoundError(f"Could not read {img_path}")
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    t0 = time.perf_counter()
    if method == "haar":
        faces = face_cascade.detectMultiScale(gray, scaleFactor=scaleFactor,
minNeighbors=minNeighbors, minSize=minSize)
    elif method == "hog" and HAVE_DLIB:
        faces = run_hog(gray, upsample=1)
    else:
        raise ValueError("method must be 'haar' or 'hog' (needs dlib)")
    ms = (time.perf_counter() - t0) * 1000
    for (x, y, w, h) in faces:
        cv2.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 2)
    cv2.putText(img, f"{len(faces)} face(s) | {method} | {ms:.0f} ms", (10, 25),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
    if save:
        cv2.imwrite(str(OUTPUT / f"faces_{method}_{Path(img_path).stem}.jpg"), img)
    if show_result:
        show(img, f"{Path(img_path).name}: {len(faces)} face(s)", ax=ax)
    return [tuple(map(int, f)) for f in faces]

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for col, n in enumerate(["addams-family.png", "rehg-thanksgiving-1994.png", "messi5.jpg"]):
    detect_faces(paths[n], method="haar", ax=axes[0, col])
    if HAVE_DLIB:
        detect_faces(paths[n], method="hog", ax=axes[1, col])
axes[0, 0].text(-30, 200, "Haar", rotation=90, fontsize=12); axes[1, 0].text(-30, 200, "HOG",
rotation=90, fontsize=12)
plt.tight_layout(); plt.show()
print("saved:", sorted(p.name for p in OUTPUT.glob("faces_*.jpg")))
```




```
saved: ['faces_haar_addams-family.jpg', 'faces_haar_messi5.jpg', 'faces_haar_rehg-thanksgiving-19
94.jpg', 'faces_hog_addams-family.jpg', 'faces_hog_messi5.jpg', 'faces_hog_rehg-thanksgiving-199
4.jpg']
```

10 · 🔧 What we changed from the course notebook, and why

#	Course code	Problem	Fix
1	`cv2.imread('basketball.jpg')`, `detect_faces('face_sample.jpg')`	Image files **not provided**; the cell raises `IOError`.	Download public test images (OpenCV samples, CMU/MIT face test images) with a cached helper.
2	`cv2.imshow(...)`, `cv2.waitKey(0)`, `cv2.destroyAllWindows()`	Do **not work in Jupyter** (no GUI window; crashes or hangs the kernel, impossible on Colab).	`show()` helper with matplotlib (BGR → RGB); webcam frames updated in place with `IPython.display`.
3	"HOG" section	Only theory: **no HOG code at all**.	HOG visualisation (scikit-image), lighting-invariance test, OpenCV HOG pedestrian detector, dlib HOG face detector, measured Haar vs HOG comparison.
4	Webcam loop	Never checks `ret`, so a missing camera crashes `cvtColor`; `cap.release()` is skipped on an error/interrupt; the infinite `while True` blocks the notebook.	Checks `ret`, uses `try/finally`, runs for a fixed time, switched off by `RUN_WEBCAM = False`.

#	Course code	Problem	Fix
5	`detect_faces()`	Returns nothing (other code cannot use the boxes), no check that the image loaded, fixed parameters.	Returns the boxes, checks `None`, exposes parameters, saves the output, Haar or HOG.
6	`cv2.data.haarcascades + '...xml'`	On **OpenCV 5** the wheel has no `.xml` files and no `CascadeClassifier` (moved to opencv-contrib), giving `AttributeError` or `(-215:Assertion failed) !empty()`.	`load_cascade()` falls back to downloading the file; version note in Setup.
7	Parameters only mentioned	No measurement of the effect of `scaleFactor` / `minNeighbors`.	Heatmaps of faces found vs false alarms over 30 settings.

11 · Results, honestly

All numbers come from the run above (85 reference faces in 9 photos; reference boxes from YuNet, checked by eye).

- The course code (Haar, scaleFactor 1.3, minNeighbors 5) on the football photo found 1 of 2 faces; on all 9 photos it found 68 of 85 faces with 0 false alarms.
- The sweep shows the trade-off clearly: `minNeighbors=0` finds nearly every face but with hundreds to thousands of false boxes (5,752 at scaleFactor 1.05); `scaleFactor=1.1, minNeighbors=5` found 81/85 with 6 false alarms; `scaleFactor=1.5, minNeighbors=12` found only 19/85. Smaller `scaleFactor` costs time: 1.05 was roughly 6–7× slower than 1.5 (see the printed seconds).
- dlib HOG (upsample=1): 79/85 faces, 0 false alarms, about 2 s on one core, vs Haar 1.1/5: 81/85, 6 false alarms, 3.3–5.0 s on one core (three runs).
- Rotation: at 30° Haar found 4/83 faces, HOG 72/83 (dlib includes filters trained on tilted faces).
- Lighting: dark / low contrast / side shadow: Haar 73, 69, 77 of 85; HOG 74, 72, 77 of 85.
- HOG people detector on a `vtest.avi` frame: 2 people at the original size, 7 boxes after upscaling 2× (most of them real people; one is on the parked van).
- Caveats: 9 photos is a small test set, many are old greyscale photos with frontal faces (Haar's best case), and the reference boxes come from another detector (we checked them visually, but a hidden/tiny face could be missing). Timings depend on the CPU.

12 · Common errors and how to fix them

Error / symptom	Cause	Fix
`error: (-215:Assertion failed) !empty() in function 'detectMultiScale'`	The cascade `.xml` was not found, so the classifier is empty (OpenCV 5 wheels have no `.xml` files; or a typo in the path)	Check `face_cascade.empty()`; download the `.xml` (see `load_cascade`)

Error / symptom	Cause	Fix
`AttributeError: module 'cv2' has no attribute 'CascadeClassifier'`	OpenCV 5 main package	`pip install opencv-contrib-python` (after uninstalling `opencv-python`), or `pip install "opencv-python<5"`
`error: (-215:Assertion failed) !_src.empty() in function 'cvtColor'`	`cv2.imread` returned `None` (wrong path) or the webcam returned no frame	Check `img is None` / `ret`
Kernel hangs / dies at `cv2.imshow`	No GUI in Jupyter	Use matplotlib (`plt.imshow`)
Colours look wrong in matplotlib (blue faces)	OpenCV images are BGR, matplotlib expects RGB	`cv2.cvtColor(img, cv2.COLOR_BGR2RGB)`
Many false boxes on clothes/background	`minNeighbors` too low, `minSize` too small	Raise `minNeighbors` (5–8), set `minSize`
Faces missed	Head turned/tilted, face too small, `scaleFactor` too large, `minNeighbors` too high	`scaleFactor=1.1`, lower `minNeighbors`, smaller `minSize`, try HOG or a CNN detector
Webcam "busy" after an error	`cap.release()` was never called	Use `try/finally`; restart the kernel
`ModuleNotFoundError: No module named 'dlib'`	dlib not installed	`pip install dlib-bin` (pre-built)

13 · Try this yourself

1. Run `detect_faces` on a photo of your own class or family. Which faces are missed, and why (turned, small, shadowed, glasses)?
2. Add eye detection: load `haarcascade_eye.xml` and search for eyes **only inside each face box** (a "region of interest"). Why is this faster and with fewer false alarms than searching the whole image?
3. Change `scaleFactor` to 1.01. How much slower is it? Do you find more faces?
4. Try `hog_people.detectMultiScale(frame, winStride=(4, 4), ...)` on several frames of `vtest.avi`. Count people found per frame and the time.
5. Rotate a photo by 20° before detection, then try detecting on several rotated copies (−20°, 0°, +20°) and merging the boxes. Does that fix tilted faces?
6. Use `cv2.equalizeHist(gray)` before Haar detection on the "dark" image from 7c. Does it help?

14 · Viva questions (with short model answers)

1. **Face detection vs face recognition?** Detection finds *where* faces are (boxes); recognition identifies *whose* face it is. Detection comes first.
2. **What is a Haar feature?** A rectangle pattern whose value is the pixel sum under its white part minus the sum under its black part; it responds to simple light/dark structures like the dark eye band above bright cheeks.

3. **What is the integral image and why use it?** A table where each entry is the sum of all pixels above and to the left; any rectangle sum then needs only 4 look-ups, so features are computed in constant time.
4. **What does AdaBoost do in Viola–Jones?** Selects a small set of the most useful weak (single-feature) classifiers and weights them into a strong classifier.
5. **Why a cascade?** Most windows are not faces; cheap early stages reject them quickly so expensive stages run only on promising windows. This makes Haar fast.
6. **Explain `scaleFactor` and `minNeighbors`.** scaleFactor is the image-pyramid step (smaller = more sizes checked, slower, fewer misses); minNeighbors is how many overlapping raw detections are needed to accept a face (higher = fewer false positives, more misses).
7. **How does HOG work?** Compute gradients, build 9-bin orientation histograms in 8×8-pixel cells, normalise over 2×2-cell blocks, concatenate, and classify each window with a linear SVM.
8. **Why is HOG robust to illumination?** It uses gradient directions, which do not change when brightness is scaled, and block normalisation removes changes in contrast. (We measured a cosine similarity close to 1 between a face and a much darker copy.)
9. **Haar vs HOG: which is faster, which is more accurate?** Classic answer: Haar faster, HOG more robust. In our test HOG (dlib) had similar recall (79 vs 81 of 85 faces), no false alarms (vs 6), better tolerance to tilt and lighting, and was not slower at matched settings; Haar was only faster with coarse settings that miss more faces.
10. **Why do both fail on tilted or profile faces?** They were trained on upright frontal faces; their features are not rotation invariant. Fixes: rotated copies, profile cascades, or CNN detectors trained on varied poses.
11. **What is a false positive / false negative in detection?** A box with no face / a face with no box.
12. **Why convert to grayscale?** Haar and HOG use brightness patterns only; one channel is 3× less data and faster.
13. **What window sizes do these detectors use?** OpenCV's frontal Haar cascade: 24×24; OpenCV's HOG people detector: 64×128; dlib's HOG face detector: 80×80. Larger faces are found by shrinking the image (pyramid).
14. **Why can't you use `cv2.imshow` in Jupyter?** It needs a desktop GUI window and an event loop (`waitKey`); notebooks render images in the browser, so use matplotlib.
15. **What replaced Haar and HOG in practice?** CNN-based detectors (e.g. MTCNN, RetinaFace, YuNet, SSD/YOLO-based), which handle pose, lighting and occlusion far better.

Day 14 · Pre-CNN foundations: from pixels to neurons

> **Not a course lab file — built from the Key Concepts "Pre-CNN Module" activities.** The course has no `CV-Week2-Lecture*.ipynb` for Day 14. This notebook works through every activity that module describes, so you have runnable code for each idea before the CNN lab (Day 15).

Notes chapter: `week-2/notes/07_pre_cnn_foundations.md` **Run time:** about 1 minute on a laptop CPU. No downloads (all images ship with scikit-image and scikit-learn).

What you will learn (the Key Concepts' order: *pixels → features → ML → ANN → CNN*)

1. NumPy arrays **are** images; grayscale vs RGB; the rotate/flip activity and why transformations hurt recognition.
2. The **dot product** as a similarity measure.
3. **Convolution = sliding dot product**, by hand with an edge kernel, then the same with `cv2.filter2D` and `nn.Conv2d`.
4. **Derivatives and gradients** ("how steep the hill is") and a tiny **gradient descent**.
5. **Mean, variance** and why **normalisation** matters.
6. **Supervised vs unsupervised** learning side by side: kNN with labels vs k-means without.
7. An **artificial neuron** computed by hand: $z = \Sigma\, w_i x_i + b \rightarrow f(z)$.
8. A small **feed-forward network (MLP)** trained on digits, with accuracy per epoch.
9. The **bridge to CNNs**: why dense layers do not scale to images.

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import cv2
import torch
import torch.nn as nn
from skimage import data as skdata
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from threadpoolctl import threadpool_limits

threadpool_limits(2); torch.set_num_threads(2)       # optional: use at most 2 CPU cores
np.random.seed(0); torch.manual_seed(0)
plt.rcParams["figure.dpi"] = 80
print("NumPy", np.__version__, "| OpenCV", cv2.__version__, "| PyTorch", torch.__version__)
```

```
NumPy 2.5.3 | OpenCV 5.0.0 | PyTorch 2.14.1+cu130
```

1 · Programming basics: an image is a NumPy array

> **Analogy (Key Concepts):** a NumPy array is like an **Excel sheet where each cell holds a number, with superpowers for maths**. A grayscale image is one sheet; an RGB image is three stacked sheets (red, green, blue). Pixels are **Lego blocks**: arrange the numbers and you build the picture.

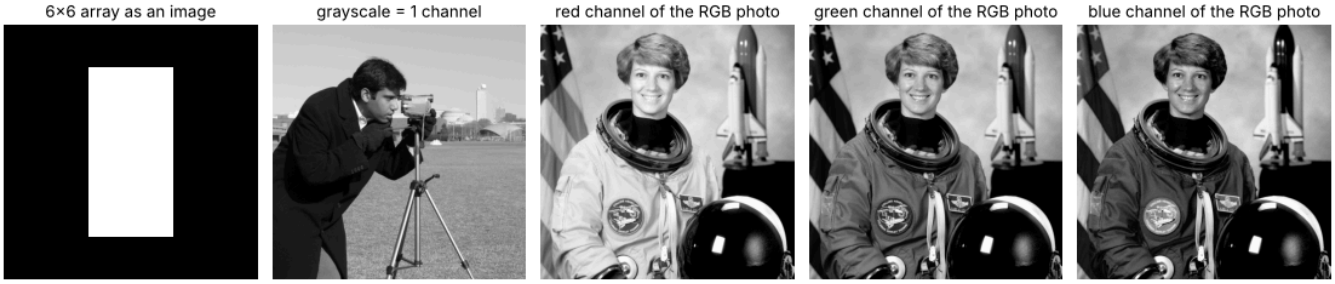
```
# Build a tiny image by hand: 0 = black, 255 = white
tiny = np.zeros((6, 6), dtype=np.uint8)
tiny[1:5, 2:4] = 255                        # a white vertical bar (rows 1-4, columns 2-3)
print("shape:", tiny.shape, "| dtype:", tiny.dtype)
print(tiny)

cam = skdata.camera()                       # a 512x512 grayscale photo (ships with scikit-image)
rgb = skdata.astronaut()                    # a 512x512 colour photo, shape (H, W, 3)
print("\ngrayscale photo:", cam.shape, cam.dtype, "| min/max:", cam.min(), cam.max())
print("colour photo   :", rgb.shape, "-> 3 channels; pixel (100, 200) =", rgb[100, 200], "(R, G, B)")

fig, ax = plt.subplots(1, 5, figsize=(14, 3))
ax[0].imshow(tiny, cmap="gray", vmin=0, vmax=255); ax[0].set_title("6x6 array as an image")
ax[1].imshow(cam, cmap="gray"); ax[1].set_title("grayscale = 1 channel")
for a, c, name in zip(ax[2:], range(3), ["red", "green", "blue"]):
    a.imshow(rgb[:, :, c], cmap="gray"); a.set_title(f"{name} channel of the RGB photo")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

```
shape: (6, 6) | dtype: uint8
[[  0   0   0   0   0   0]
 [  0   0 255 255   0   0]
 [  0   0 255 255   0   0]
 [  0   0 255 255   0   0]
 [  0   0 255 255   0   0]
 [  0   0   0   0   0   0]]

grayscale photo: (512, 512) uint8 | min/max: 0 255
colour photo   : (512, 512, 3) -> 3 channels; pixel (100, 200) = [81 57 17] (R, G, B)
```




```
# Everyday array operations = everyday image operations
crop = cam[50:250, 150:350]                 # slicing = cropping (rows first, then columns)
brighter = np.clip(cam.astype(np.int16) + 80, 0, 255).astype(np.uint8)   # add a number = brighten (clip to avoid wrap-around)
negative = 255 - cam                        # invert
small = cam[::4, ::4]                       # every 4th pixel = crude down-scaling
print("crop", crop.shape, "| small", small.shape)
print("Why clip? uint8 wraps around:", np.array([250], np.uint8) + np.array([10], np.uint8), "(250 + 10 = 4!)")

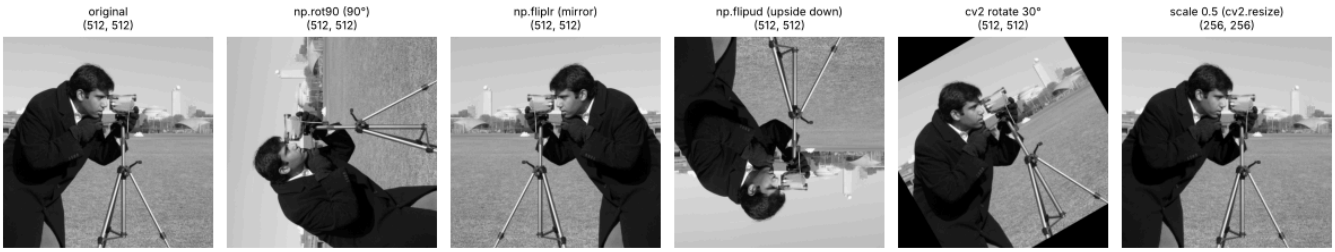
fig, ax = plt.subplots(1, 4, figsize=(12, 3))
for a, im, t in zip(ax, [crop, brighter, negative, small], ["crop (slicing)", "+80 brightness", "255 - image", "every 4th pixel"]):
    a.imshow(im, cmap="gray", vmin=0, vmax=255); a.set_title(t); a.axis("off")
plt.tight_layout(); plt.show()
```

```
crop (200, 200) | small (128, 128)
Why clip? uint8 wraps around: [4] (250 + 10 = 4!)
```



Activity: rotate and flip an image, then discuss how transformations affect recognition

```
h, w = cam.shape
M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 1.0)       # rotate 30 degrees about the centre
versions = {
    "original": cam,
    "np.rot90 (90°)": np.rot90(cam),
    "np.fliplr (mirror)": np.fliplr(cam),
    "np.flipud (upside down)": np.flipud(cam),
    "cv2 rotate 30°": cv2.warpAffine(cam, M, (w, h)),
    "scale 0.5 (cv2.resize)": cv2.resize(cam, None, fx=0.5, fy=0.5),
}
fig, ax = plt.subplots(1, 6, figsize=(15, 2.8))
for a, (t, im) in zip(ax, versions.items()):
    a.imshow(im, cmap="gray"); a.set_title(f"{t}\n{im.shape}", fontsize=9); a.axis("off")
plt.tight_layout(); plt.show()
```



For a person, all six are obviously the same scene. For a computer they are **completely different arrays**. Let's measure what that does to recognition with Day 11's kNN digit classifier: train on normal digits, then test on transformed copies of the test digits.

```
digits = datasets.load_digits()
X, y = digits.data / 16.0, digits.target                    # 64 features scaled to 0-1
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)

def transform_digits(Xs, fn):
    return np.array([fn(x.reshape(8, 8)).reshape(-1) for x in Xs])

def rot(angle):
    M = cv2.getRotationMatrix2D((3.5, 3.5), angle, 1.0)
    return lambda im: cv2.warpAffine(im, M, (8, 8))

tests = {"original": lambda im: im, "rotate 15°": rot(15), "rotate 30°": rot(30), "rotate 90°":
lambda im: np.rot90(im),
         "mirror": np.fliplr, "shift 1 px right": lambda im: np.roll(im, 1, axis=1)}
for name, fn in tests.items():
    acc = knn.score(transform_digits(X_test, fn), y_test)
    print(f"{name:18s} accuracy {acc:.3f}")
```

```
original           accuracy 0.989
rotate 15°         accuracy 0.902
rotate 30°         accuracy 0.609
rotate 90°         accuracy 0.165
mirror             accuracy 0.324
shift 1 px right   accuracy 0.617
```

A pixel-by-pixel classifier only recognises patterns **in the position and orientation it was trained on**: a 30° rotation or a mirror image destroys accuracy, and even a 1-pixel shift costs a lot. Two classic fixes: **data augmentation** (train on rotated/shifted copies too) and **architectures that are built to tolerate shifts**, which is exactly what convolution + pooling in a CNN provide.

2 · Linear algebra: the dot product is a similarity measure

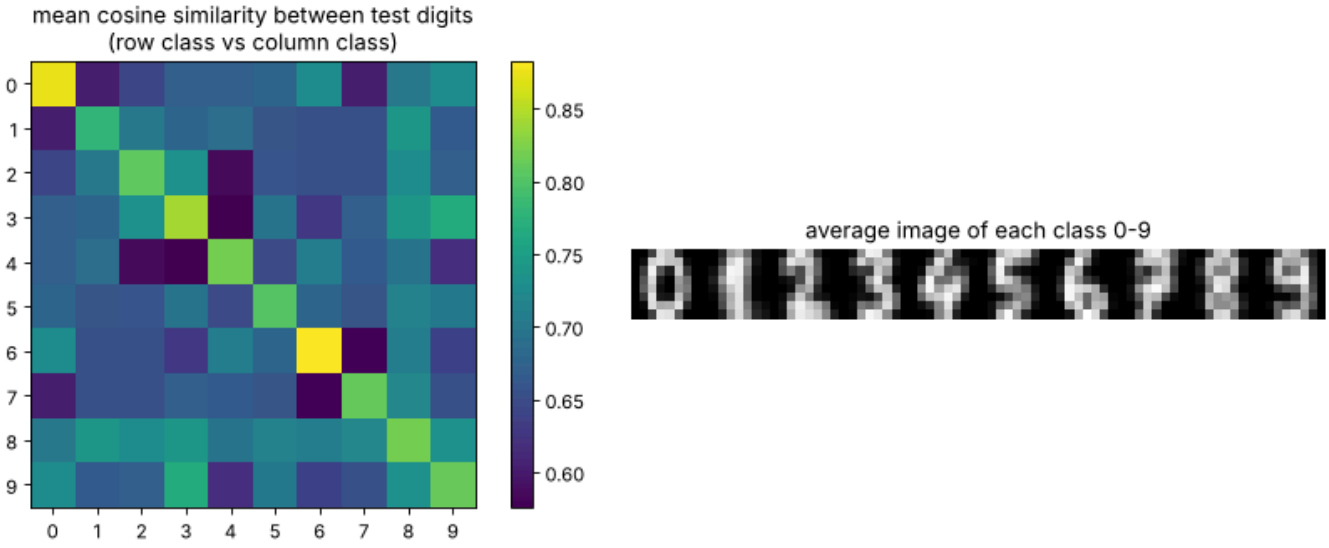
The dot product multiplies two vectors element by element and adds up: $\mathbf{a} \cdot \mathbf{b} = \Sigma\, a_i b_i$. If both vectors are large in the same places, the result is large. Dividing by their lengths gives the **cosine similarity** (1 = same direction, 0 = unrelated).

```
a = np.array([1, 2, 3]); b = np.array([2, 4, 6]); c = np.array([3, 0, -1])
print("a . b =", a @ b, "  (by hand: 1*2 + 2*4 + 3*6 = 28)")
cos = lambda u, v: float(u @ v / (np.linalg.norm(u) * np.linalg.norm(v)))
print(f"cos(a, b) = {cos(a, b):.2f}  -> same direction (b is just 2a)")
print(f"cos(a, c) = {cos(a, c):.2f}  -> perpendicular, 'nothing in common'")

# Images as vectors: average cosine similarity between digit classes
means = np.array([X_train[y_train == d].mean(0) for d in range(10)])
S = np.array([[np.mean([cos(u, v) for u in X_test[y_test == i][:30] for v in X_test[y_test == j]
[:30]]) for j in range(10)] for i in range(10)])
fig, ax = plt.subplots(1, 2, figsize=(11, 4.2))
im = ax[0].imshow(S, cmap="viridis"); ax[0].set_xticks(range(10)); ax[0].set_yticks(range(10))
ax[0].set_title("mean cosine similarity between test digits\n(row class vs column class)");
plt.colorbar(im, ax=ax[0])
ax[1].imshow(np.hstack([m.reshape(8, 8) for m in means]), cmap="gray"); ax[1].axis("off");
ax[1].set_title("average image of each class 0-9")
plt.tight_layout(); plt.show()
print("same-class similarity (diagonal) average:", S.diagonal().mean().round(3), "| different-
class average:", S[~np.eye(10, dtype=bool)].mean().round(3))

# A one-line classifier: pick the class whose average image has the biggest cosine similarity
pred = np.argmax([[cos(x, m) for m in means] for x in X_test], axis=1)
print(f"'nearest average image by dot product' classifier accuracy: {(pred ==
y_test).mean():.3f}")
```

```
a . b = 28   (by hand: 1*2 + 2*4 + 3*6 = 28)
cos(a, b) = 1.00  -> same direction (b is just 2a)
cos(a, c) = 0.00  -> perpendicular, 'nothing in common'
```




```
same-class similarity (diagonal) average: 0.825 | different-class average: 0.675
'nearest average image by dot product' classifier accuracy: 0.893
```

Digits of the same class have higher dot-product similarity than digits of different classes, and a classifier that just compares an image with each class's average image (10 dot products) already gets most digits right. **This is exactly what a neuron does: a dot product between its weights and its input.**

3 · Convolution = sliding dot product (by hand, then with OpenCV and PyTorch)

> **Analogy (Key Concepts):** a CNN is a detective with a **magnifying glass** (the kernel/filter) sliding over the image looking for clues (features).

At every position: put the 3×3 kernel on the image, multiply overlapping numbers, add them up (a dot product), write the result into the output, slide one step, repeat.

```python
img6 = np.array([[10, 10, 10, 200, 200, 200]] * 6, dtype=np.float32)   # dark left half, bright right half
edge_kernel = np.array([[-1, 0, 1],
                        [-2, 0, 2],
                        [-1, 0, 1]], dtype=np.float32)                  # Sobel-x: responds to left-to-right brightness change
print("image:\n", img6.astype(int))

def conv2d_by_hand(image, kernel, verbose_at=None):
    kh, kw = kernel.shape
    out = np.zeros((image.shape[0] - kh + 1, image.shape[1] - kw + 1), np.float32)    # 'valid': no padding
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            window = image[i:i + kh, j:j + kw]
            out[i, j] = np.sum(window * kernel)          # the dot product of window and kernel
            if (i, j) == verbose_at:
                print(f"\nat output position {(i, j)}: window =\n{window.astype(int)}")
                print("window * kernel =\n", (window * kernel).astype(int), "\nsum =", out[i, j])
    return out

out = conv2d_by_hand(img6, edge_kernel, verbose_at=(0, 1))
print("\nfull output (4x4, because a 3x3 kernel fits 6-3+1 = 4 times per row):\n",
out.astype(int))
```

```
image:
 [[ 10  10  10 200 200 200]
 [ 10  10  10 200 200 200]
 [ 10  10  10 200 200 200]
 [ 10  10  10 200 200 200]
 [ 10  10  10 200 200 200]
 [ 10  10  10 200 200 200]]

at output position (0, 1): window =
[[ 10  10 200]
 [ 10  10 200]
 [ 10  10 200]]
window * kernel =
 [[-10   0 200]
 [-20   0 400]
 [-10   0 200]]
sum = 760.0

full output (4x4, because a 3x3 kernel fits 6-3+1 = 4 times per row):
 [[  0 760 760   0]
 [  0 760 760   0]
 [  0 760 760   0]
 [  0 760 760   0]]
```

Big numbers appear only where the window straddles the dark→bright edge; flat regions give 0. That is an **edge detector**.

Now the same thing on a real photo, and checks that `cv2.filter2D` and a PyTorch `nn.Conv2d` layer compute exactly the same numbers. (Strictly, all three compute *cross-correlation*: the kernel is not flipped. Deep learning calls this "convolution" anyway; for symmetric kernels there is no difference.)

```
cam_f = cam.astype(np.float32)
t0 = time.perf_counter(); by_hand = conv2d_by_hand(cam_f, edge_kernel); t_hand =
time.perf_counter() - t0

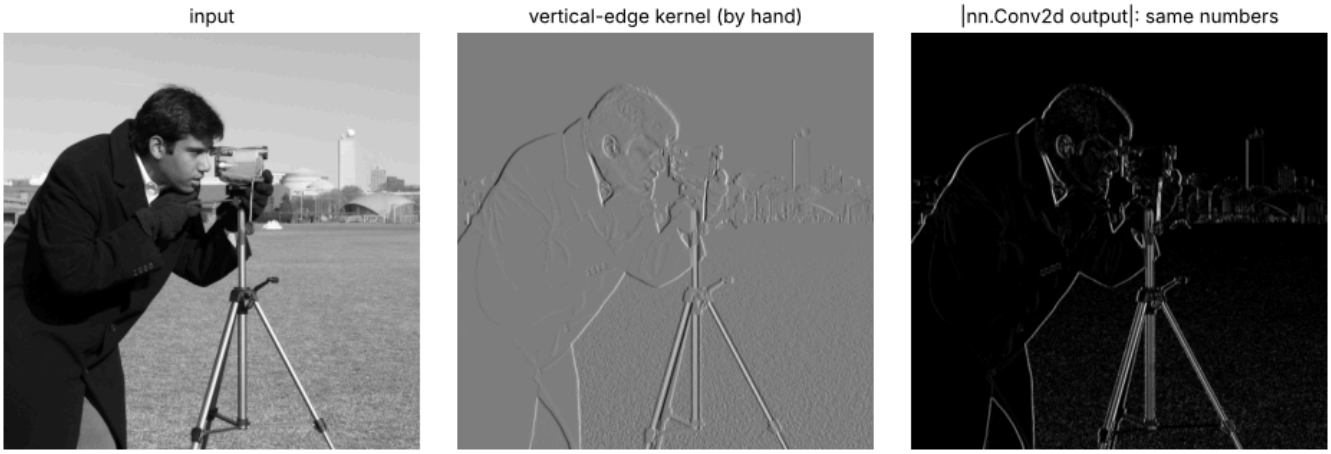
t0 = time.perf_counter()
cv_out = cv2.filter2D(cam_f, -1, edge_kernel)[1:-1, 1:-1]     # crop the border so the size
matches 'valid'
t_cv = time.perf_counter() - t0

conv = nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3, bias=False)   # padding=0 ->
'valid', like ours
with torch.no_grad():
    conv.weight[:] = torch.from_numpy(edge_kernel)                            # paste the kernel
in by hand
    torch_out = conv(torch.from_numpy(cam_f)[None, None])[0, 0].numpy()       # (H, W) -> (1, 1,
H, W)

print("output shapes:", by_hand.shape, cv_out.shape, torch_out.shape)
print("by hand == cv2.filter2D :", np.allclose(by_hand, cv_out, atol=1e-2))
print("by hand == nn.Conv2d    :", np.allclose(by_hand, torch_out, atol=1e-2))
print(f"time: python loops {t_hand:.2f}s vs cv2.filter2D {t_cv * 1000:.1f} ms")

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
ax[0].imshow(cam, cmap="gray"); ax[0].set_title("input")
ax[1].imshow(by_hand, cmap="gray"); ax[1].set_title("vertical-edge kernel (by hand)")
ax[2].imshow(np.abs(torch_out), cmap="gray"); ax[2].set_title("|nn.Conv2d output|: same numbers")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

```
output shapes: (510, 510) (510, 510) (510, 510)
by hand == cv2.filter2D : True
by hand == nn.Conv2d    : True
time: python loops 0.97s vs cv2.filter2D 0.8 ms
```



The bridge to CNNs in one sentence: an `nn.Conv2d` layer is this sliding dot product, but instead of us typing the 9 kernel numbers, the network **learns** them (and learns dozens of kernels per layer) from labelled examples.

4 · Calculus: derivatives and gradients ("how steep the hill is")

- The **derivative** f′(x) is the **slope** of f at x: how fast f changes if x moves a tiny bit. We can estimate it numerically: f′(x) ≈ (f(x+h) − f(x−h)) / 2h.
- The **gradient** is the same idea in several dimensions: a vector of slopes, one per input, pointing in the **direction of steepest increase**.

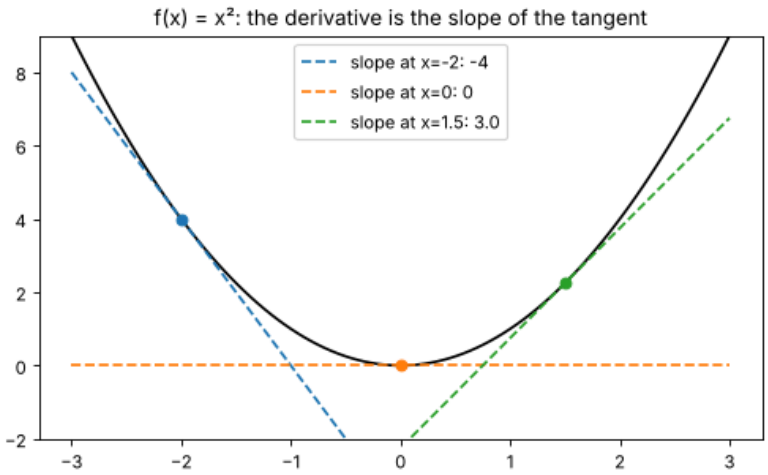
> **Analogy (Key Concepts):** gradient = "how steep the hill is". If you are climbing, the gradient tells you how hard it gets, and which way is straight up.

```
f = lambda x: x ** 2
df_exact = lambda x: 2 * x
num_deriv = lambda f, x, h=1e-4: (f(x + h) - f(x - h)) / (2 * h)
for x0 in [-2, 0, 1, 3]:
    print(f"x={x0:>2}: numerical slope {num_deriv(f, x0):7.4f} | exact 2x = {df_exact(x0)}")

xs = np.linspace(-3, 3, 100)
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(xs, f(xs), "k")
for x0, col in [(-2, "C0"), (0, "C1"), (1.5, "C2")]:
    s = df_exact(x0); ax[0].plot(xs, f(x0) + s * (xs - x0), "--", color=col, label=f"slope at x=
{x0}: {s}")
    ax[0].plot(x0, f(x0), "o", color=col)
ax[0].set_ylim(-2, 9); ax[0].legend(); ax[0].set_title("f(x) = x²: the derivative is the slope of
the tangent")

# 2-D: a hill z = exp(-(x²+y²)); arrows = gradient (points uphill, longer = steeper)
gx, gy = np.meshgrid(np.linspace(-2, 2, 15), np.linspace(-2, 2, 15))
z = np.exp(-(gx ** 2 + gy ** 2))
dzdx, dzdy = -2 * gx * z, -2 * gy * z
ax[1].contourf(gx, gy, z, 15, cmap="terrain"); ax[1].quiver(gx, gy, dzdx, dzdy, color="k")
ax[1].set_title("a hill: gradient arrows point uphill"); ax[1].set_aspect("equal")
plt.tight_layout(); plt.show()
```

```
x=-2: numerical slope -4.0000 | exact 2x = -4
x= 0: numerical slope  0.0000 | exact 2x = 0
x= 1: numerical slope  2.0000 | exact 2x = 2
x= 3: numerical slope  6.0000 | exact 2x = 6
```



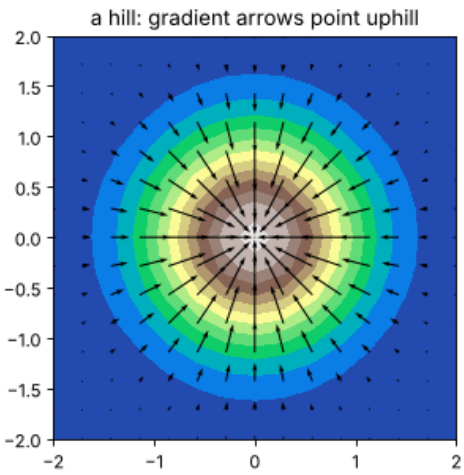


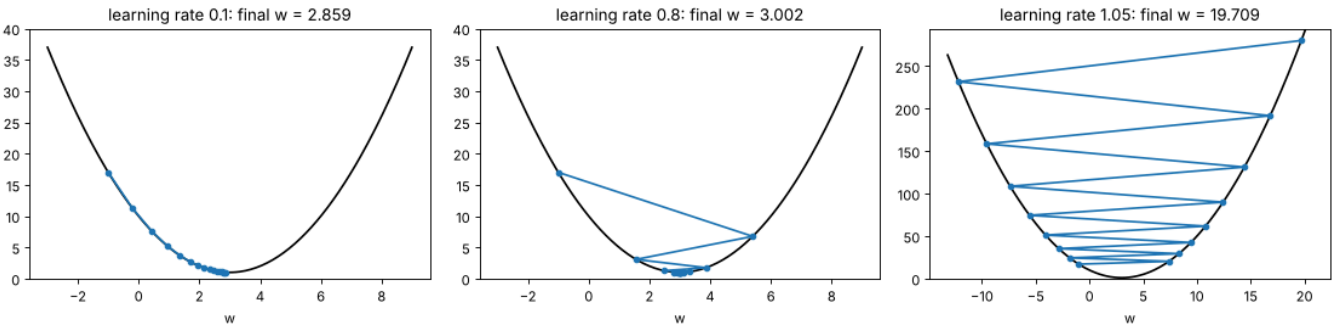
Image gradients are the same thing. The Sobel kernel in section 3 *is* a derivative estimate of brightness in the x direction. Edges are where the brightness "hill" is steepest. (Day 13's HOG is built from these gradients.)

A tiny gradient descent: how networks learn

To **minimise** a loss L(w), repeatedly step **downhill**: w ← w − learning_rate × L′(w). Below, L(w) = (w − 3)² + 1, whose minimum is at w = 3.

```
L = lambda w: (w - 3) ** 2 + 1
dL = lambda w: 2 * (w - 3)

fig, ax = plt.subplots(1, 3, figsize=(14, 3.6))
ws = np.linspace(-2, 8, 100)
for a, lr in zip(ax, [0.1, 0.8, 1.05]):
    w, path = -1.0, [-1.0]
    for _ in range(15):
        w = w - lr * dL(w)          # the whole learning rule
        path.append(w)
    wsx = np.linspace(min(path + [-2]) - 1, max(path + [8]) + 1, 200)
    a.plot(wsx, L(wsx), "k"); a.plot(path, [L(p) for p in path], "o-", ms=4)
    a.set_title(f"learning rate {lr}: final w = {path[-1]:.3f}"); a.set_xlabel("w")
    a.set_ylim(0, max(40, 1.05 * max(L(p) for p in path)))
plt.tight_layout(); plt.show()
print("lr=0.1 : small steady steps, converges | lr=0.8 : overshoots, zig-zags in | lr=1.05 :
every step overshoots more -> diverges")
```

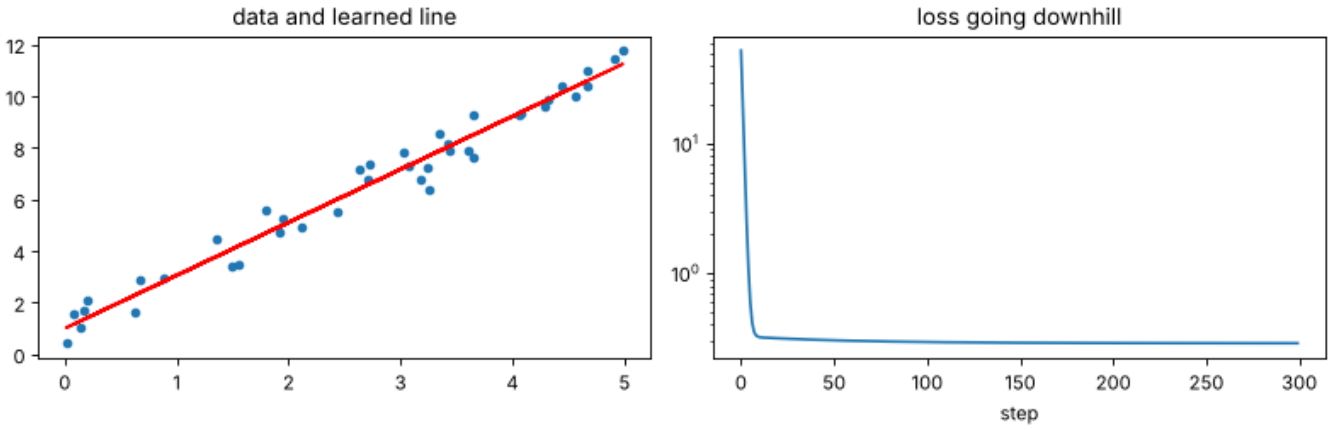



```
lr=0.1 : small steady steps, converges | lr=0.8 : overshoots, zig-zags in | lr=1.05 : every step
overshoots more -> diverges
```

```
# Gradient descent fitting a line y = m*x + c to noisy points (two parameters -> a 2-D gradient)
rng = np.random.default_rng(0)
xd = rng.uniform(0, 5, 40); yd = 2.0 * xd + 1.0 + rng.normal(0, 0.5, 40)
m, c, lr, hist = 0.0, 0.0, 0.02, []
for step in range(300):
    err = (m * xd + c) - yd
```

```
    loss = np.mean(err ** 2)                          # mean squared error
    dm, dc = 2 * np.mean(err * xd), 2 * np.mean(err)  # the gradient (partial derivatives)
    m, c = m - lr * dm, c - lr * dc
    hist.append(loss)
print(f"learned m = {m:.3f}, c = {c:.3f} (true values 2 and 1) | loss {hist[0]:.2f} ->
{hist[-1]:.3f}")
fig, ax = plt.subplots(1, 2, figsize=(10, 3.4))
ax[0].plot(xd, yd, "o", ms=4); ax[0].plot(xd, m * xd + c, "r"); ax[0].set_title("data and learned
line")
ax[1].plot(hist); ax[1].set_yscale("log"); ax[1].set_xlabel("step"); ax[1].set_title("loss going
downhill")
plt.tight_layout(); plt.show()
```

```
learned m = 2.055, c = 0.998 (true values 2 and 1) | loss 52.24 -> 0.288
```



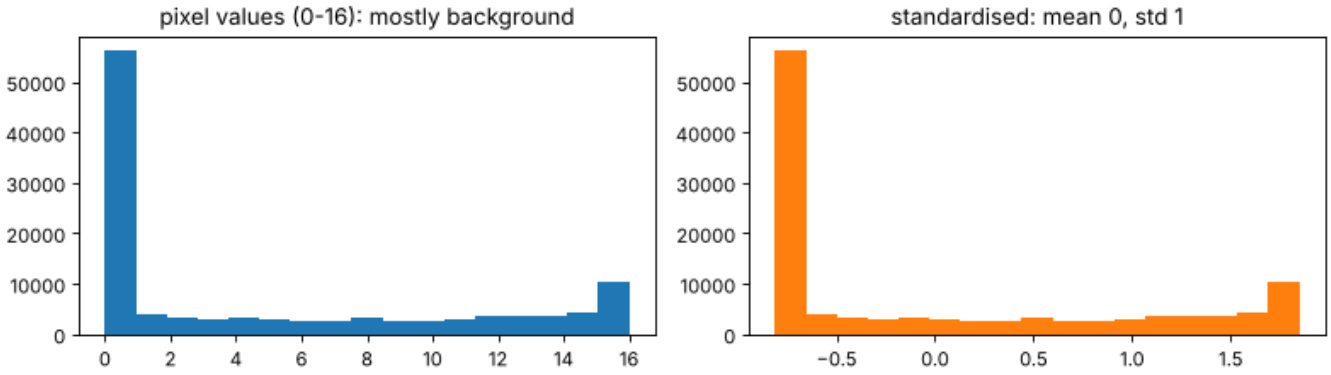
A neural network does exactly this, only with millions of parameters instead of two; `loss.backward()` in PyTorch computes all the partial derivatives (**backpropagation**) and `optimizer.step()` takes the downhill step.

5 · Probability and statistics: mean, variance, and why normalisation matters

- **Mean** μ = average value. **Variance** σ^2 = average squared distance from the mean; **standard deviation** σ = its square root (the typical spread).
- **Standardising** (z-score): $x' = (x - \mu) / \sigma$ gives mean 0 and standard deviation 1. Day 15's `transforms.Normalize((0.1307,), (0.3081,))` is exactly this, using MNIST's mean and std.

```
pix = digits.data.ravel()                 # every pixel value of every digit, 0-16
print(f"digits pixels: mean {pix.mean():.2f}, variance {pix.var():.2f}, std {pix.std():.2f}")
z = (pix - pix.mean()) / pix.std()
print(f"after standardising: mean {z.mean():.2f}, std {z.std():.2f}")
fig, ax = plt.subplots(1, 2, figsize=(10, 3))
ax[0].hist(pix, bins=17); ax[0].set_title("pixel values (0-16): mostly background")
ax[1].hist(z, bins=17, color="C1"); ax[1].set_title("standardised: mean 0, std 1")
plt.tight_layout(); plt.show()
```

```
digits pixels: mean 4.88, variance 36.20, std 6.02
after standardising: mean 0.00, std 1.00
```



Why does it matter? Distance-based methods (kNN, k-means, RBF SVM) and gradient descent treat all features as comparable. If one feature is measured in much bigger units, it dominates everything. Experiment: add one useless feature to the digits, measured in "big units" (random numbers from 0 to 1000), and run kNN with and without standardisation.

```
from sklearn.preprocessing import StandardScaler

noise_tr = rng.uniform(0, 1000, (len(X_train), 1)); noise_te = rng.uniform(0, 1000, (len(X_test),
1))
Xtr_bad, Xte_bad = np.hstack([X_train, noise_tr]), np.hstack([X_test, noise_te])
print(f"kNN on the 64 pixels only            : {KNeighborsClassifier(3).fit(X_train,
y_train).score(X_test, y_test):.3f}")
print(f"+ 1 useless feature in big units      : {KNeighborsClassifier(3).fit(Xtr_bad,
y_train).score(Xte_bad, y_test):.3f}")
sc = StandardScaler().fit(Xtr_bad)          # learn mean/std on the TRAINING data only
print(f"same, after StandardScaler            :
{KNeighborsClassifier(3).fit(sc.transform(Xtr_bad), y_train).score(sc.transform(Xte_bad),
y_test):.3f}")
```

```
kNN on the 64 pixels only            : 0.989
+ 1 useless feature in big units      : 0.224
same, after StandardScaler            : 0.972
```

One badly scaled feature drowned out all 64 pixels; standardising put them back on an equal footing. For neural networks, normalised inputs also make gradient descent faster and more stable (the "hill" is round instead of a long narrow valley).

6 · Machine learning fundamentals: supervised vs unsupervised, side by side

	Supervised (kNN)	Unsupervised (k-means)
Data given	images **and** labels	images **only**
Goal	predict the label of new images	discover groups by itself
Analogy (Key Concepts)	a teacher giving questions *with answers*	exploring a city *without a guide*

Same 1,797 digits. kNN learns from labels; k-means (k = 10) is never told which digit is which.

```
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
from sklearn.metrics import adjusted_rand_score

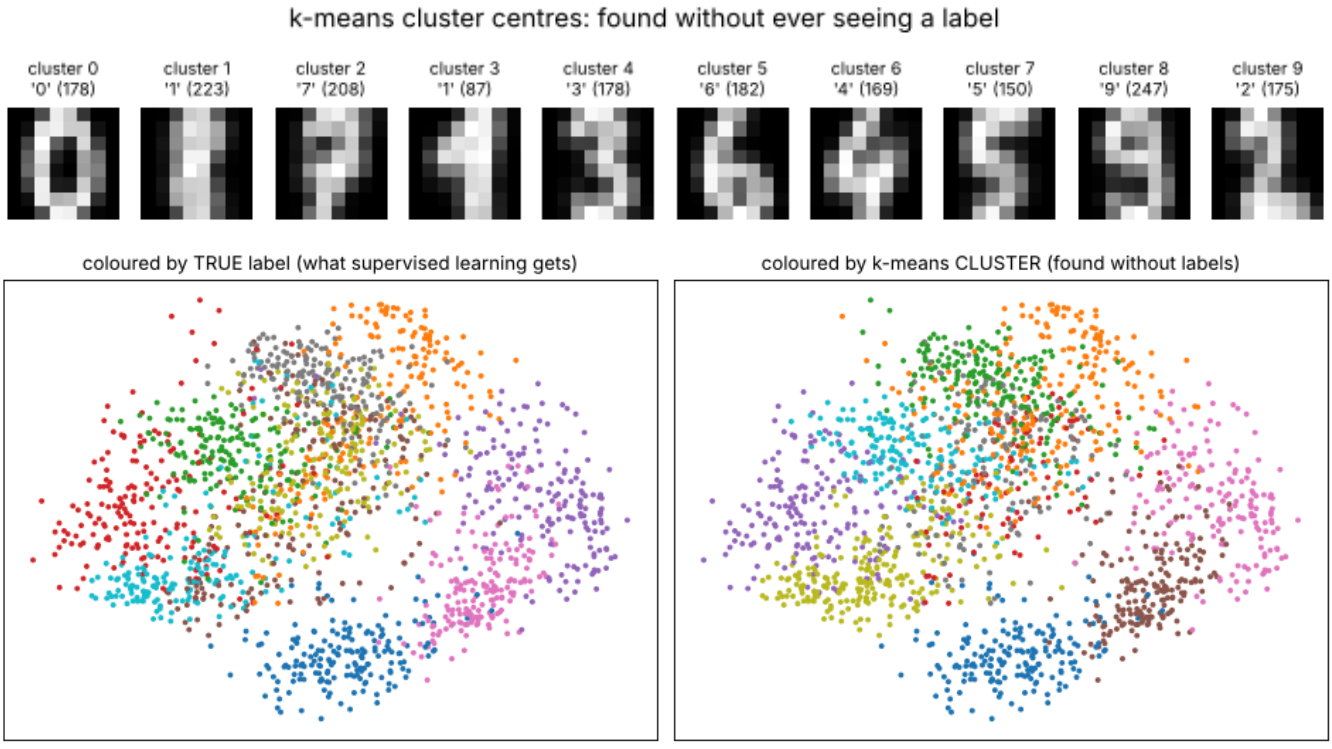
# Supervised
knn = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
print(f"SUPERVISED   kNN (with labels): test accuracy {knn.score(X_test, y_test):.3f}")
```

```
# Unsupervised: cluster ALL digits without labels
km = KMeans(n_clusters=10, random_state=0, n_init=10).fit(X)
clusters = km.labels_
# Only AFTER clustering do we peek at the labels to name each cluster by its most common digit
names = np.array([np.bincount(y[clusters == k], minlength=10).argmax() for k in range(10)])
acc = (names[clusters] == y).mean()
print(f"UNSUPERVISED k-means (no labels): if each cluster is named after its majority digit,
{acc:.3f} of digits land in the 'right' cluster")
print(f"   adjusted Rand index (0 = random grouping, 1 = perfect): {adjusted_rand_score(y,
clusters):.3f}")
print("   cluster names:", names, "-> digits never given their own cluster:",
sorted(set(range(10)) - set(names)))

fig, ax = plt.subplots(1, 10, figsize=(11, 1.6))
for k in range(10):
    ax[k].imshow(km.cluster_centers_[k].reshape(8, 8), cmap="gray"); ax[k].axis("off")
    ax[k].set_title(f"cluster {k}\n'{names[k]}' ({(clusters == k).sum()})", fontsize=8)
plt.suptitle("k-means cluster centres: found without ever seeing a label", y=1.15); plt.show()

P = PCA(n_components=2, random_state=0).fit_transform(X)
fig, ax = plt.subplots(1, 2, figsize=(12, 4.6))
ax[0].scatter(P[:, 0], P[:, 1], c=y, cmap="tab10", s=6); ax[0].set_title("coloured by TRUE label
(what supervised learning gets)")
ax[1].scatter(P[:, 0], P[:, 1], c=clusters, cmap="tab10", s=6); ax[1].set_title("coloured by k-
means CLUSTER (found without labels)")
for a in ax: a.set_xticks([]); a.set_yticks([])
plt.tight_layout(); plt.show()
```

```
SUPERVISED   kNN (with labels): test accuracy 0.989
UNSUPERVISED k-means (no labels): if each cluster is named after its majority digit, 0.792 of dig
its land in the 'right' cluster
   adjusted Rand index (0 = random grouping, 1 = perfect): 0.666
   cluster names: [0 1 7 1 3 6 4 5 9 2] -> digits never given their own cluster: [8]
```



k-means discovers most digit groups on its own (the cluster centres look like digits!), but some digits get merged or split, and it can never *name* a group without labels. That is the difference: unsupervised learning finds structure; supervised learning learns the specific answer we want. (The 2-D plots use **PCA**, an unsupervised dimensionality-reduction method from the Key Concepts list, to squash 64 numbers into 2 for drawing.)

7 · The artificial neuron, by hand

$z = \Sigma\, w_i x_i + b$, then **output = f(z)** where f is an activation function.

> **Analogy (Key Concepts):** inputs = evidence, weights = importance of each piece of evidence, bias = prior tendency, activation = the final decision.

Example: a neuron that should "fire" for a **vertical bar** in a 3×3 patch.

```
x = np.array([0, 1, 0,
              0, 1, 0,
              0, 1, 0], dtype=float)              # input: a vertical bar (9 pixel values)
wts = np.array([-1, 2, -1,
                -1, 2, -1,
                -1, 2, -1], dtype=float)          # weights: like the middle column, dislike the
sides
b = -3.0                                          # bias: needs strong evidence before firing

z = np.sum(wts * x) + b
print("step 1  products w_i * x_i :", (wts * x).astype(int))
print(f"step 2  sum = {np.sum(wts * x):.0f}, plus bias {b} -> z = {z:.1f}")
sigmoid = lambda t: 1 / (1 + np.exp(-t)); relu = lambda t: np.maximum(0, t)
print(f"step 3  sigmoid(z) = {sigmoid(z):.3f} | ReLU(z) = {relu(z):.1f} | tanh(z) =
{np.tanh(z):.3f}")

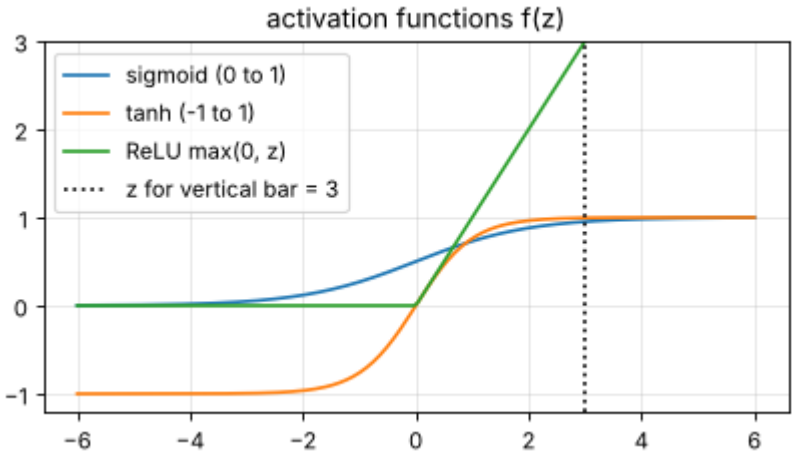
x_h = np.array([0, 0, 0, 1, 1, 1, 0, 0, 0], dtype=float)  # a HORIZONTAL bar
z_h = wts @ x_h + b
print(f"\nhorizontal bar: z = {z_h:.1f} -> sigmoid {sigmoid(z_h):.3f}, ReLU {relu(z_h):.1f}  (the
neuron stays quiet)")

lin = nn.Linear(9, 1)                              # the same neuron in PyTorch
with torch.no_grad():
    lin.weight[:] = torch.tensor(wts, dtype=torch.float32); lin.bias[:] = b
print("PyTorch nn.Linear gives the same z:", lin(torch.tensor(x, dtype=torch.float32)).item())

t = np.linspace(-6, 6, 200)
plt.figure(figsize=(6, 3))
for fn, name in [(sigmoid, "sigmoid (0 to 1)"), (np.tanh, "tanh (-1 to 1)"), (relu, "ReLU max(0,
z)")]:
    plt.plot(t, fn(t), label=name)
plt.axvline(z, color="k", ls=":", label=f"z for vertical bar = {z:.0f}"); plt.legend();
plt.grid(alpha=.3); plt.ylim(-1.2, 3)
plt.title("activation functions f(z)"); plt.show()
```

```
step 1  products w_i * x_i : [0 2 0 0 2 0 0 2 0]
step 2  sum = 6, plus bias -3.0 -> z = 3.0
step 3  sigmoid(z) = 0.953 | ReLU(z) = 3.0 | tanh(z) = 0.995

horizontal bar: z = -3.0 -> sigmoid 0.047, ReLU 0.0  (the neuron stays quiet)
PyTorch nn.Linear gives the same z: 3.0
```



Notice: this neuron's weights, laid out as 3×3, are an edge/line **kernel**. A convolution layer is just this neuron **re-used at every position** of the image.

Why activations? Without f, stacking layers is pointless: a linear function of a linear function is still linear. Non-linear activations let a network bend its decision boundaries.

8 · Neural network basics: a small feed-forward network (MLP) on digits

Layers: **input (64 pixels) → hidden (64 neurons, ReLU) → hidden (32, ReLU) → output (10 scores, one per digit)**.

- **Forward pass:** data flows forward and predictions are made.
- **Loss:** cross-entropy measures how wrong the predicted probabilities are.
- **Backpropagation:** `loss.backward()` sends the error backwards and computes every weight's gradient; `optimizer.step()` takes the downhill step (section 4).

> **Analogy (Key Concepts):** each neuron is a student giving a small opinion; the network combines them into a final decision.

```
Xtr_t = torch.tensor(X_train, dtype=torch.float32); ytr_t = torch.tensor(y_train)
Xte_t = torch.tensor(X_test, dtype=torch.float32); yte_t = torch.tensor(y_test)

def make_mlp():
    torch.manual_seed(0)
    return nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 32), nn.ReLU(),
nn.Linear(32, 10))

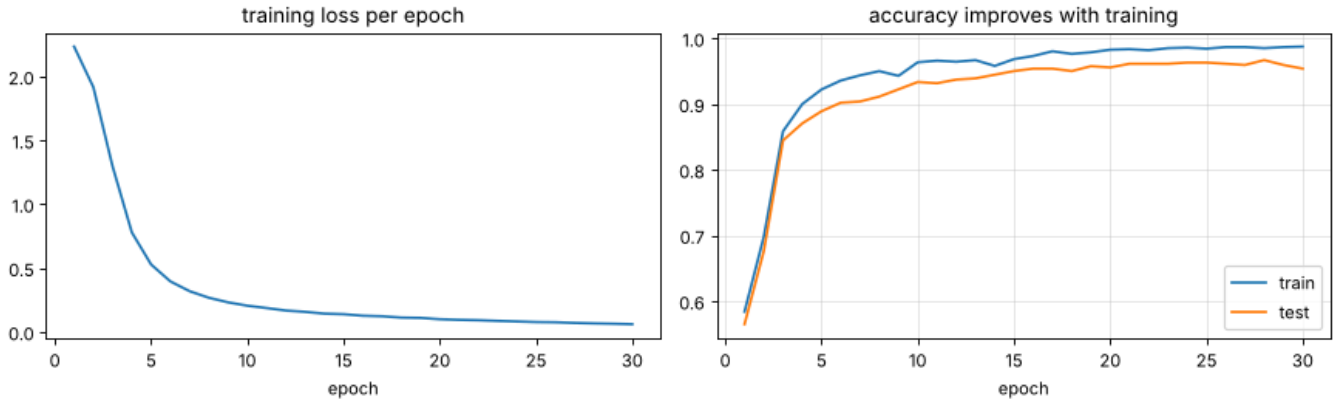
def train_mlp(model, Xtr, ytr, Xte, yte, epochs=30, lr=1e-3, batch=32):
    opt = torch.optim.Adam(model.parameters(), lr=lr)
    loss_fn = nn.CrossEntropyLoss()
    hist = {"train": [], "test": [], "loss": []}
    for ep in range(epochs):
        model.train()
        perm = torch.randperm(len(Xtr)); tot = 0.0
        for i in range(0, len(Xtr), batch):
            idx = perm[i:i + batch]
            out = model(Xtr[idx])                  # forward pass
            loss = loss_fn(out, ytr[idx])          # how wrong?
            opt.zero_grad(); loss.backward()       # backpropagation: gradients of every weight
            opt.step()                             # gradient-descent step
            tot += loss.item() * len(idx)
```

```
        model.eval()
        with torch.no_grad():
            hist["train"].append((model(Xtr).argmax(1) == ytr).float().mean().item())
            hist["test"].append((model(Xte).argmax(1) == yte).float().mean().item())
        hist["loss"].append(tot / len(Xtr))
    return hist

mlp = make_mlp()
print(mlp)
print("trainable parameters:", sum(p.numel() for p in mlp.parameters()), "= (64*64+64) +
(64*32+32) + (32*10+10)")
t0 = time.perf_counter()
hist = train_mlp(mlp, Xtr_t, ytr_t, Xte_t, yte_t)
print(f"trained 30 epochs in {time.perf_counter() - t0:.1f}s")
for ep in [0, 1, 4, 9, 19, 29]:
    print(f"epoch {ep + 1:>2}: loss {hist['loss'][ep]:.3f} | train acc {hist['train'][ep]:.3f} |
test acc {hist['test'][ep]:.3f}")

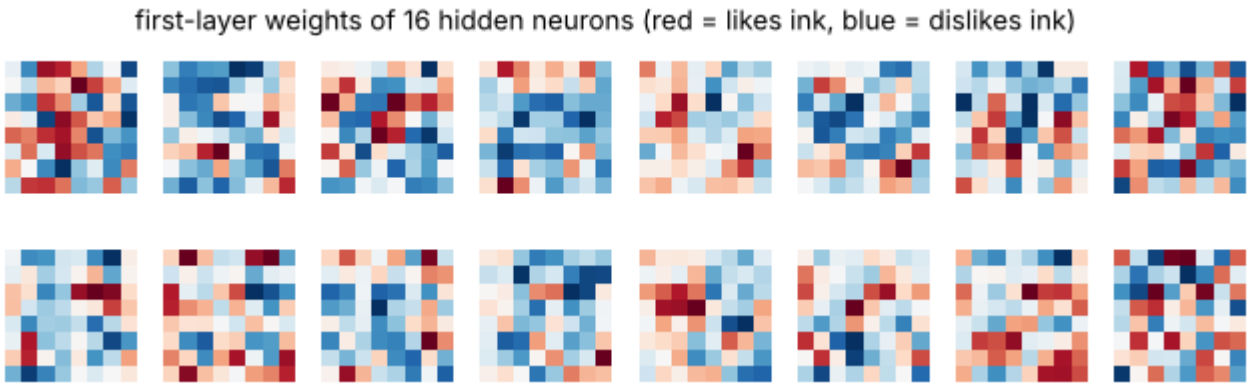
fig, ax = plt.subplots(1, 2, figsize=(11, 3.5))
ax[0].plot(range(1, 31), hist["loss"]); ax[0].set_title("training loss per epoch");
ax[0].set_xlabel("epoch")
ax[1].plot(range(1, 31), hist["train"], label="train"); ax[1].plot(range(1, 31), hist["test"],
label="test")
ax[1].set_title("accuracy improves with training"); ax[1].set_xlabel("epoch"); ax[1].legend();
ax[1].grid(alpha=.3)
plt.tight_layout(); plt.show()
```

```
Sequential(
  (0): Linear(in_features=64, out_features=64, bias=True)
  (1): ReLU()
  (2): Linear(in_features=64, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=10, bias=True)
)
trainable parameters: 6570 = (64*64+64) + (64*32+32) + (32*10+10)
trained 30 epochs in 15.4s
epoch  1: loss 2.232 | train acc 0.584 | test acc 0.565
epoch  2: loss 1.915 | train acc 0.698 | test acc 0.676
epoch  5: loss 0.527 | train acc 0.922 | test acc 0.889
epoch 10: loss 0.205 | train acc 0.963 | test acc 0.933
epoch 20: loss 0.099 | train acc 0.982 | test acc 0.956
epoch 30: loss 0.060 | train acc 0.987 | test acc 0.954
```



The Key Concepts activity, done: a small feed-forward network learns to classify digits and the accuracy climbs epoch by epoch (numbers printed above). The first-layer weights of each hidden neuron can even be viewed as 8×8 images:

```
W = mlp[0].weight.detach().numpy()             # (64 neurons, 64 inputs)
fig, ax = plt.subplots(2, 8, figsize=(10, 2.8))
for a, wv in zip(ax.ravel(), W[:16]):
    a.imshow(wv.reshape(8, 8), cmap="RdBu"); a.axis("off")
plt.suptitle("first-layer weights of 16 hidden neurons (red = likes ink, blue = dislikes ink)");
plt.show()
```



9 · Bridge to CNNs

The Key Concepts list two problems with fully connected (dense) networks on images:

Problem 1: too many parameters for large images.

```
def n_params(layer):
    return sum(p.numel() for p in layer.parameters())

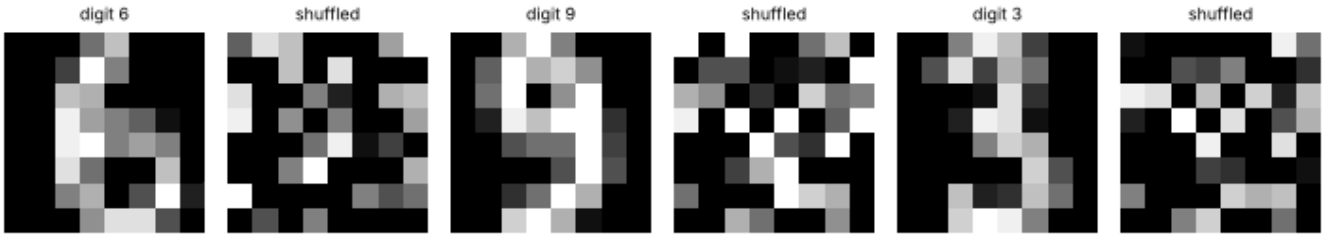
dense = nn.Linear(224 * 224 * 3, 1000)                    # every one of 1000 neurons sees all
150,528 input numbers
conv = nn.Conv2d(3, 64, kernel_size=3, padding=1)         # 64 filters, each 3x3x3, re-used at
every position
print(f"input 224x224x3 = {224 * 224 * 3:,} numbers")
print(f"dense layer, 1000 neurons : {n_params(dense):>12,} parameters  (= 150,528 x 1000 weights
+ 1000 biases)")
print(f"conv layer, 64 3x3 filters: {n_params(conv):>12,} parameters  (= 64 x (3x3x3) weights +
64 biases)")
print(f"-> the dense layer has {n_params(dense) / n_params(conv):,.0f}x more parameters")
print(f"   memory for the dense layer's weights alone (float32): {n_params(dense) * 4 / 1e6:.0f}
MB")
```

```
input 224x224x3 = 150,528 numbers
dense layer, 1000 neurons :  150,529,000 parameters  (= 150,528 x 1000 weights + 1000 biases)
conv layer, 64 3x3 filters:        1,792 parameters  (= 64 x (3x3x3) weights + 64 biases)
-> the dense layer has 84,001x more parameters
   memory for the dense layer's weights alone (float32): 602 MB
```

Problem 2: dense layers ignore spatial structure. A dense layer treats the image as a bag of independent numbers: it does not know which pixels are neighbours. Proof: **shuffle the pixel order** of every digit (the *same* random shuffle for all images) and retrain the MLP. To a human the images become noise; to the MLP nothing has changed.

```
perm = np.random.default_rng(42).permutation(64)
Xtr_s, Xte_s = Xtr_t[:, perm], Xte_t[:, perm]
hist_s = train_mlp(make_mlp(), Xtr_s, ytr_t, Xte_s, yte_t)
print(f"MLP test accuracy, normal pixel order  : {hist['test'][-1]:.3f}")
print(f"MLP test accuracy, pixels SHUFFLED     : {hist_s['test'][-1]:.3f}")
fig, ax = plt.subplots(1, 6, figsize=(9, 1.8))
for i in range(3):
    ax[2 * i].imshow(X_test[i].reshape(8, 8), cmap="gray"); ax[2 * i].set_title(f"digit
{y_test[i]}", fontsize=8)
    ax[2 * i + 1].imshow(X_test[i][perm].reshape(8, 8), cmap="gray"); ax[2 * i +
1].set_title("shuffled", fontsize=8)
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

```
MLP test accuracy, normal pixel order  : 0.954
MLP test accuracy, pixels SHUFFLED     : 0.965
```



The MLP does (about) equally well on scrambled images, so it was **never using the fact that neighbouring pixels belong together**. That is wasted knowledge: in real images, edges, corners and textures are *local* patterns. CNNs build this in:

- **Local connections:** each conv neuron looks at a small patch (3×3), like our edge kernel.
- **Weight sharing:** the same kernel slides over the whole image, so a pattern is recognised wherever it appears, with very few parameters.
- **Pooling:** summarises small regions, adding tolerance to small shifts (the 1-pixel shift that hurt kNN in section 1).

Section 3 applied an edge filter **manually**; on Day 15 a CNN **learns** its own filters from MNIST.

10 · Results, honestly

All numbers are from the run above.

- Transformations: kNN accuracy on digits fell from 98.9% (original) to 90.2% at 15°, 60.9% at 30°, 16.5% at 90°, 32.4% mirrored, and 61.7% with a mere 1-pixel shift: pixel-based recognition is fragile.
- Dot product as similarity: same-digit pairs averaged cosine 0.825 vs 0.675 for different digits; the "nearest average image" classifier reached 89.3% with just 10 dot products per image.
- Convolution by hand, `cv2.filter2D` and `nn.Conv2d` gave identical outputs; the Python loops took about a second, OpenCV well under a millisecond.
- Normalisation: one badly scaled useless feature dropped kNN from 98.9% to 22.4%; StandardScaler brought it back to 97.2%.
- Supervised kNN: 98.9% test accuracy. Unsupervised k-means: 79.2% of digits fall in a cluster whose majority label matches theirs (adjusted Rand index 0.666), without ever seeing a label; digit 8 never got a cluster of its own, while 1 was split into two clusters.
- MLP (6,570 parameters): test accuracy rose from 56.5% after epoch 1 to 95.4% after 30 epochs. That is *lower* than kNN's 98.9% on this tiny dataset (1,257 training images): neural networks need more data to shine. With shuffled pixels it scored 96.5%, the same within noise, which shows it ignores spatial layout.
- A dense layer on a 224×224×3 image with 1000 neurons has 150,529,000 parameters vs 1,792 for a 64-filter 3×3 conv layer.

11 · Common errors and how to fix them

Error / symptom	Cause	Fix
Image arithmetic gives strange dark/bright spots	`uint8` wraps around (250 + 10 = 4)	Convert to a bigger type (`astype(np.int16)` or `float32`), then `np.clip(..., 0, 255).astype(np.uint8)`
`IndexError` / wrong crop	NumPy indexes `[row, column]` = `[y, x]` , OpenCV points are `(x, y)`	Remember: arrays are rows first
`RuntimeError: Expected 4D input` in `nn.Conv2d`	PyTorch wants `(batch, channels, H, W)`	Add axes: `t[None, None]` or `t.unsqueeze(0).unsqueeze(0)`
`RuntimeError: mat1 and mat2 shapes cannot be multiplied`	Input size does not match `nn.Linear(in_features, ...)`	Flatten correctly; check `x.shape`
Loss becomes `nan` or grows	Learning rate too large (like lr = 1.05 in section 4)	Lower the learning rate; normalise inputs
kNN / k-means results dominated by one feature	Features on different scales	`StandardScaler` fitted on the training data
`Can't call numpy() on Tensor that requires grad`	Tensor is part of the autograd graph	`.detach().numpy()` or compute inside `torch.no_grad()`

12 · Try this yourself

1. Make your own 3×3 kernel that detects **horizontal** edges and run it with `conv2d_by_hand` on the camera image.
2. Change the bias of the "vertical bar" neuron to 0 and to −10. When does it fire for the horizontal bar? What is the bias doing?
3. In the gradient-descent cell, find the largest learning rate that still converges for L(w) = (w − 3)² + 1. (Hint: think about the slope 2(w − 3).)
4. Train the MLP with `nn.Sigmoid()` instead of `nn.ReLU()` . Is it slower to learn?
5. Run k-means with k = 20 instead of 10 and name clusters by majority label. Does the "accuracy" go up? Is that fair?
6. Augment the kNN training set with copies of each digit rotated by ±15° and re-test on the rotated test digits from section 1.

13 · Viva questions (with short model answers)

1. **How is an image stored in a computer?** As an array of numbers: H×W for grayscale (one channel), H×W×3 for colour (R, G, B channels), usually `uint8` values 0–255.
2. **Why does rotating or flipping an image hurt a pixel-based classifier?** The pixel values move to different positions, so the vector looks completely different, although the content is the same. Fixes: data augmentation, CNNs (convolution + pooling).

3. **What does the dot product measure?** How much two vectors point the same way (similarity); normalised, it is the cosine similarity.
4. **Why is convolution called a "sliding dot product"?** At each position the kernel is multiplied element-wise with the image patch under it and summed (a dot product), then it slides to the next position.
5. **What is the output size of a 3×3 convolution on a 6×6 image without padding?** 4×4 (6 − 3 + 1). In general (W − K + 2P)/S + 1.
6. **What is a derivative, and what is a gradient?** The derivative is the slope of a function at a point; the gradient is the vector of slopes for all inputs, pointing in the direction of steepest increase.
7. **What is gradient descent?** Repeatedly updating parameters by a small step against the gradient (downhill) to minimise the loss: $w \leftarrow w - lr \cdot \partial L/\partial w$.
8. **What happens if the learning rate is too large or too small?** Too large: overshoots and can diverge; too small: very slow convergence.
9. **What is normalisation (standardisation) and why does it matter?** Rescaling features to mean 0, std 1, so no feature dominates distances or gradients; it speeds up and stabilises training.
10. **Supervised vs unsupervised learning?** Supervised learns from labelled data to predict labels (kNN, SVM); unsupervised finds structure in unlabelled data (k-means, PCA).
11. **Write the equation of an artificial neuron.** $z = \Sigma\, w_i x_i + b$, output = f(z) with an activation function such as ReLU or sigmoid.
12. **Why do we need activation functions?** Without them, any stack of layers collapses into one linear function and cannot learn non-linear patterns.
13. **What are forward pass and backpropagation?** Forward: compute outputs and the loss from inputs. Backpropagation: compute the gradient of the loss with respect to every weight by the chain rule, going backwards through the layers.
14. **Why are fully connected layers a poor fit for images?** Too many parameters (150 million for one 1000-neuron layer on a 224×224×3 image) and they ignore spatial structure (our shuffled-pixel MLP did just as well).
15. **What do CNNs add?** Local connections, weight sharing and pooling: far fewer parameters, spatial patterns detected anywhere in the image, tolerance to small shifts.

Day 15 · CNN basics: a convolutional network for MNIST in PyTorch

Course lab: `CV-Week2-Lecture7.ipynb` (Day 15: CNN Basics) **Notes chapter:** `week-2/notes/08_cnn_basics.md` (deeper reading: `week-2-3/notes/00_deep_learning_basics.md` and `week-2-3/notes/01_neural_network_building_blocks.ipynb`, but this notebook stands on its own) **Run time:** about 5–8 minutes on a laptop CPU, almost all of it the one training epoch (much faster with a GPU). Downloads MNIST (about 12 MB) the first time.

What you will learn

- What convolution, **feature maps**, **stride**, **padding**, **pooling** and **flatten** do, with the shapes worked out step by step.
- Why the course's `fc1` has exactly **9216** inputs (the output-size formula).
- The course CNN, training loop and testing loop, line by line: `Normalize((0.1307,), (0.3081,))`, `log_softmax` + `nll_loss`, Adam, dropout, `model.train()` / `model.eval()`, `torch.no_grad()`.
- What the network learned: its first-layer filters and the feature maps it produces.
- The mini project, done for real: recognising digits **you draw**, and the preprocessing without which it fails.

1 · The ideas in plain language

On Day 14 we slid a hand-made edge kernel over an image (convolution = sliding dot product) and saw that a dense network ignores which pixels are neighbours. A **CNN** keeps the sliding kernels but **learns** their numbers.

> **Analogy (Key Concepts):** a CNN is a detective with magnifying glasses (filters) looking for clues (features) everywhere in the image. Early layers find edges and corners; deeper layers combine them into strokes, loops and whole digits.

The flow of data (Key Concepts tip): **Input → Convolution → (ReLU) → Pooling → Flatten → Fully connected → Output**

Word	Plain meaning
Kernel / filter	a small grid of learned weights (here 3×3) that slides over the image
Feature map	the output image of one filter: how strongly that pattern appears at each position
Stride	the step size of the slide (stride 2 = skip every other position → smaller output)
Padding	a border of zeros around the input so the output does not shrink and edge pixels are covered
Pooling	summarise each small region (max pooling keeps the strongest response) → smaller maps, tolerance to small shifts, less computation

Word	Plain meaning
Flatten	reshape the stack of feature maps into one long vector for the fully connected layers
Activation (ReLU)	max(0, x): adds non-linearity; without it stacked layers collapse into one linear map

Output size of a convolution or pooling layer (learn this formula):

output = (W − K + 2P) / S + 1 (W = input size, K = kernel size, P = padding, S = stride; round down)

```
%matplotlib inline
import time
import numpy as np
import matplotlib.pyplot as plt
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

torch.set_num_threads(2)                # optional: leave CPU cores free for the rest of the laptop
torch.manual_seed(0)                    # repeatable results (the course notebook had no seed)
plt.rcParams["figure.dpi"] = 80
print("PyTorch", torch.__version__, "| GPU available:", torch.cuda.is_available())
```

```
PyTorch 2.14.1+cu130 | GPU available: False
```

2 · The course code: data and normalisation

```
# Transform: normalize MNIST digits
transform = transforms.Compose([
    transforms.ToTensor(),                          # PIL image 0-255 -> float tensor 0.0-1.0, shape (1, 28, 28)
    transforms.Normalize((0.1307,), (0.3081,))     # (x - mean) / std, using MNIST's own mean and std
])

# Load MNIST dataset (downloads into ./data the first time)
train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, download=True, transform=transform)   # course omitted download=True here

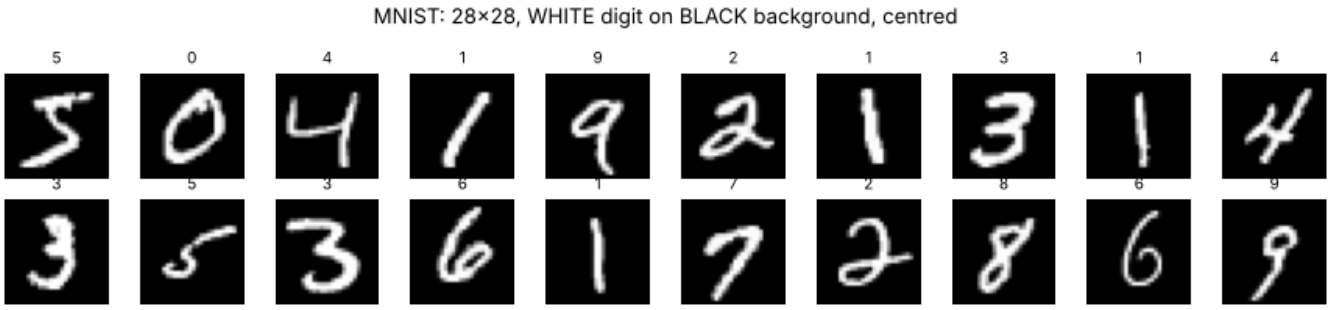
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)      # 64 images per training step, new order each epoch
test_loader = DataLoader(test_dataset, batch_size=1000, shuffle=False)     # bigger batches are fine when not training
print(len(train_dataset), "training images,", len(test_dataset), "test images,", len(train_loader), "batches per epoch")
```

```
60000 training images, 10000 test images, 938 batches per epoch
```

```
# Where do 0.1307 and 0.3081 come from? They are the mean and standard deviation of all MNIST training pixels (scaled 0-1).
raw = train_dataset.data.float() / 255
print(f"MNIST training pixels: mean = {raw.mean():.4f}, std = {raw.std():.4f}")
x0, y0 = train_dataset[0]
print(f"one normalised image: shape {tuple(x0.shape)}, min {x0.min():.3f}, max {x0.max():.3f}, label {y0}")
print("background pixels become (0 - 0.1307) / 0.3081 =", round((0 - 0.1307) / 0.3081, 4))
```

```
fig, axes = plt.subplots(2, 10, figsize=(11, 2.6))
for a, i in zip(axes.ravel(), range(20)):
    img, lab = train_dataset[i]
    a.imshow(img[0], cmap="gray"); a.set_title(str(lab), fontsize=9); a.axis("off")
plt.suptitle("MNIST: 28x28, WHITE digit on BLACK background, centred"); plt.tight_layout();
plt.show()
```

```
MNIST training pixels: mean = 0.1307, std = 0.3081
one normalised image: shape (1, 28, 28), min -0.424, max 2.821, label 5
background pixels become (0 - 0.1307) / 0.3081 = -0.4242
```



Why normalise? After `Normalize` the pixels have mean ≈ 0 and standard deviation ≈ 1. Inputs on this scale keep the early layers' outputs in a sensible range and make gradient descent faster and more stable (Day 14, section 5). The same `transform` must be used for **every** image the model ever sees, including your own drawings (section 9).

3 · Stride, padding, pooling and flatten, step by step

```
x = torch.arange(1, 37, dtype=torch.float32).reshape(1, 1, 6, 6)    # a 6x6 "image" in (batch,
channels, H, W) form
print("input shape:", tuple(x.shape))
for k, s, p in [(3, 1, 0), (3, 1, 1), (3, 2, 0), (3, 2, 1), (5, 1, 0)]:
    conv = nn.Conv2d(1, 1, kernel_size=k, stride=s, padding=p)
    out = conv(x)
    print(f"kernel {k}, stride {s}, padding {p}: output {tuple(out.shape[2:])}   formula (6 - {k}
+ 2*{p}) / {s} + 1 = {(6 - k + 2 * p) // s + 1}")

pool_in = torch.tensor([[1., 3, 2, 1], [4, 6, 5, 0], [7, 2, 9, 8], [1, 0, 3, 4]]).reshape(1, 1,
4, 4)
print("\nmax pooling 2x2 (stride 2) keeps the biggest number in each 2x2 block:")
print(pool_in[0, 0].int().numpy(), "\n->\n", F.max_pool2d(pool_in, 2)[0, 0].int().numpy())
print("average pooling instead:\n", F.avg_pool2d(pool_in, 2)[0, 0].numpy())

stack = torch.randn(1, 64, 12, 12)
print("\nflatten: a stack of 64 feature maps of 12x12", tuple(stack.shape), "->",
tuple(torch.flatten(stack, 1).shape),
      "(start_dim=1 keeps the batch dimension)")
```

```
input shape: (1, 1, 6, 6)
kernel 3, stride 1, padding 0: output (4, 4)   formula (6 - 3 + 2*0) / 1 + 1 = 4
kernel 3, stride 1, padding 1: output (6, 6)   formula (6 - 3 + 2*1) / 1 + 1 = 6
kernel 3, stride 2, padding 0: output (2, 2)   formula (6 - 3 + 2*0) / 2 + 1 = 2
kernel 3, stride 2, padding 1: output (3, 3)   formula (6 - 3 + 2*1) / 2 + 1 = 3
kernel 5, stride 1, padding 0: output (2, 2)   formula (6 - 5 + 2*0) / 1 + 1 = 2

max pooling 2x2 (stride 2) keeps the biggest number in each 2x2 block:
[[1 3 2 1]
 [4 6 5 0]
 [7 2 9 8]
 [1 0 3 4]]
->
 [[6 5]
 [7 9]]
average pooling instead:
 [[3.5 2. ]
 [2.5 6. ]]

flatten: a stack of 64 feature maps of 12x12 (1, 64, 12, 12) -> (1, 9216) (start_dim=1 keeps the
batch dimension)
```

```
# Padding demo: zeros around the border
padded = F.pad(x, (1, 1, 1, 1))
print("6x6 input with padding=1 becomes", tuple(padded.shape[2:]), "; first two rows:\n",
padded[0, 0, :2].int().numpy())

# Pooling gives tolerance to small shifts: shift a pattern by 1 pixel, compare after 2x2 max
pooling
a = torch.zeros(1, 1, 8, 8); a[0, 0, 2, 2] = 1
b = torch.zeros(1, 1, 8, 8); b[0, 0, 2, 3] = 1          # same dot, 1 pixel to the right
print("before pooling, identical?", torch.equal(a, b), "| after 2x2 max pooling, identical?",
torch.equal(F.max_pool2d(a, 2), F.max_pool2d(b, 2)))
```

```
6x6 input with padding=1 becomes (8, 8) ; first two rows:
 [[0 0 0 0 0 0 0 0]
 [0 1 2 3 4 5 6 0]]
before pooling, identical? False | after 2x2 max pooling, identical? True
```

That last line is the "translation invariance" the Key Concepts mention: after pooling, a small shift (within the pooling window) gives the same result. (A shift that crosses a pooling-block boundary still changes the output; pooling gives *some* tolerance, not full invariance.)

4 · The course CNN (unchanged), and why `fc1` has 9216 inputs

```
class CNN(nn.Module):
    def __init__(self):
        super(CNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1)      # 1 input channel (grey) -> 32 feature maps, 3x3
kernels, stride 1, no padding
        self.conv2 = nn.Conv2d(32, 64, 3, 1)     # 32 maps in -> 64 maps out; each kernel is
3x3x32
        self.dropout1 = nn.Dropout(0.25)         # during training, randomly zero 25% of values
        self.dropout2 = nn.Dropout(0.5)          # ... and 50% before the last layer
        self.fc1 = nn.Linear(9216, 128)          # 9216 = 64 maps x 12 x 12 (worked out below)
        self.fc2 = nn.Linear(128, 10)            # 10 scores, one per digit

    def forward(self, x):
        x = self.conv1(x)
        x = F.relu(x)
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, 2)
        x = self.dropout1(x)
```

```
        x = torch.flatten(x, 1)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.dropout2(x)
        x = self.fc2(x)
        output = F.log_softmax(x, dim=1)
        return output

model = CNN()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
# Follow one image through the network and print the shape after every step
model.eval()
x = train_dataset[0][0].unsqueeze(0)                    # (1, 1, 28, 28)
steps = [("input", lambda t: t), ("conv1 (3x3, 32 filters)", model.conv1), ("ReLU", F.relu),
         ("conv2 (3x3, 64 filters)", model.conv2), ("ReLU", F.relu), ("max_pool2d(2)", lambda t:
F.max_pool2d(t, 2)),
         ("dropout1", model.dropout1), ("flatten", lambda t: torch.flatten(t, 1)), ("fc1",
model.fc1), ("ReLU", F.relu),
         ("dropout2", model.dropout2), ("fc2", model.fc2), ("log_softmax", lambda t:
F.log_softmax(t, dim=1))]
with torch.no_grad():
    for name, fn in steps:
        x = fn(x)
        print(f"{name:26s} -> {str(tuple(x.shape)):18s} = {x.numel():>6,} numbers")

print("\nconv1: (28 - 3 + 2*0)/1 + 1 = 26  -> 26x26")
print("conv2: (26 - 3 + 2*0)/1 + 1 = 24  -> 24x24")
print("pool : (24 - 2)/2 + 1       = 12  -> 12x12")
print("flatten: 64 maps x 12 x 12  =", 64 * 12 * 12, "-> that is why fc1 = nn.Linear(9216, 128)")
```

```
input                      -> (1, 1, 28, 28)     =    784 numbers
conv1 (3x3, 32 filters)    -> (1, 32, 26, 26)    = 21,632 numbers
ReLU                       -> (1, 32, 26, 26)    = 21,632 numbers
conv2 (3x3, 64 filters)    -> (1, 64, 24, 24)    = 36,864 numbers
ReLU                       -> (1, 64, 24, 24)    = 36,864 numbers
max_pool2d(2)              -> (1, 64, 12, 12)    =  9,216 numbers
dropout1                   -> (1, 64, 12, 12)    =  9,216 numbers
flatten                    -> (1, 9216)          =  9,216 numbers
fc1                        -> (1, 128)           =    128 numbers
ReLU                       -> (1, 128)           =    128 numbers
dropout2                   -> (1, 128)           =    128 numbers
fc2                        -> (1, 10)            =     10 numbers
log_softmax                -> (1, 10)            =     10 numbers

conv1: (28 - 3 + 2*0)/1 + 1 = 26  -> 26x26
conv2: (26 - 3 + 2*0)/1 + 1 = 24  -> 24x24
pool : (24 - 2)/2 + 1       = 12  -> 12x12
flatten: 64 maps x 12 x 12  = 9216 -> that is why fc1 = nn.Linear(9216, 128)
```

```
print(f"{'layer':8s} {'weights shape':>18s} {'parameters':>11s}")
total = 0
for name, layer in [("conv1", model.conv1), ("conv2", model.conv2), ("fc1", model.fc1), ("fc2",
model.fc2)]:
    n = sum(p.numel() for p in layer.parameters()); total += n
    print(f"{name:8s} {str(tuple(layer.weight.shape)):>18s} {n:>11,}")
print(f"{'total':8s} {'':>18s} {total:>11,}")
print(f"fc1 alone holds {sum(p.numel() for p in model.fc1.parameters()) / total:.0%} of all
parameters")
```

```
layer         weights shape  parameters
conv1          (32, 1, 3, 3)         320
conv2         (64, 32, 3, 3)      18,496
fc1             (128, 9216)   1,179,776
fc2               (10, 128)       1,290
total                         1,199,882
fc1 alone holds 98% of all parameters
```

Things to notice

- The two conv layers together have under 20,000 parameters, yet they do all the "seeing". The single dense layer `fc1` has over 1.1 million: the Day 14 lesson again (dense layers are expensive). Modern CNNs replace big dense layers with global average pooling.
- No padding is used, so each 3×3 conv shrinks the image by 2 pixels (28 → 26 → 24).
- If you change the architecture (e.g. add `padding=1`, or another pooling layer), **9216 is no longer correct** and you get `mat1 and mat2 shapes cannot be multiplied`. Recompute with the formula, or print the shape after `flatten` as above.

Dropout randomly switches off neurons during training (25% after pooling, 50% before the last layer). The network cannot rely on any single neuron, which reduces overfitting. It is automatically **turned off** in `model.eval()` mode.

`log_softmax` + `nll_loss` = cross-entropy. `softmax` turns the 10 scores into probabilities that sum to 1; `log_softmax` is its logarithm (numerically safer). `nll_loss` (negative log-likelihood) picks −log(probability of the correct class): 0 if the model is certain and right, large if it gave the right answer a low probability. Together they are exactly `F.cross_entropy` applied to the raw scores:

```
scores = torch.tensor([[2.0, 0.5, -1.0, 0.1, 0.0, 0.3, -0.5, 1.0, 0.2, -2.0]])   # raw fc2 output
for one image
target = torch.tensor([0])                                                          # the true
class is 0
print("softmax probabilities:", F.softmax(scores, dim=1).numpy().round(3), "sum =",
F.softmax(scores, dim=1).sum().item())
print("nll_loss(log_softmax) =", F.nll_loss(F.log_softmax(scores, dim=1), target).item())
print("cross_entropy(scores) =", F.cross_entropy(scores, target).item())
print("-log(p of class 0)    =", -torch.log(F.softmax(scores, dim=1)[0, 0]).item())
```

```
softmax probabilities: [[0.421 0.094 0.021 0.063 0.057 0.077 0.035 0.155 0.07  0.008]] sum = 1.0
nll_loss(log_softmax) = 0.8646115064620972
cross_entropy(scores) = 0.8646115064620972
-log(p of class 0)    = 0.8646114468574524
```

5 · Training loop (course code, plus a loss record)

One **training step** = the 5 lines every PyTorch training loop has:

1. `optimizer.zero_grad()` : clear the gradients from the previous step (PyTorch *adds* gradients up otherwise).
2. `output = model(data)` : **forward pass**.
3. `loss = F.nll_loss(output, target)` : how wrong were we on this batch?
4. `loss.backward()` : **backpropagation**, computing the gradient of the loss for every weight.
5. `optimizer.step()` : move every weight a little downhill.

Adam is gradient descent with two improvements: each weight gets its own adaptive step size, and it keeps a running average of past gradients ("momentum") to smooth the path. `lr=0.001` is its standard starting learning rate.

`model.train()` switches dropout **on**. The only change from the course: we also store every batch's loss so we can plot the training curve. The course trained for 2 epochs; to keep this notebook's run time reasonable on a CPU we train **1 epoch** (`EPOCHS = 2` gives the course's setting).

```
losses = []                                     # added: one loss value per batch, for the
plot

def train(model, device, train_loader, optimizer, epoch):
    model.train()                               # dropout ON
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()
        losses.append(loss.item())              # added
        if batch_idx % 100 == 0:
            print(f'Train Epoch: {epoch} [{batch_idx*len(data)}/{len(train_loader.dataset)}]
Loss: {loss.item():.6f}')
```

6 · Testing loop (course code)

- `model.eval()` : dropout **off** (use the whole network).
- `with torch.no_grad()` : do not record operations for backpropagation: faster and uses less memory, and we are not learning here.
- `reduction='sum'` adds up the loss over the batch so we can average over all 10,000 images at the end.
- `output.argmax(dim=1)` = the class with the highest score = the prediction.

```
def test(model, device, test_loader):
    model.eval()                                # dropout OFF
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
```

```
            test_loss += F.nll_loss(output, target, reduction='sum').item()
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()
    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct / len(test_loader.dataset)
    print(f'Test set: Average loss: {test_loss:.4f}, Accuracy:
{correct}/{len(test_loader.dataset)} ({accuracy:.2f}%)')
    return accuracy

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
print("Before training (random weights):")
acc0 = test(model, device, test_loader)
```

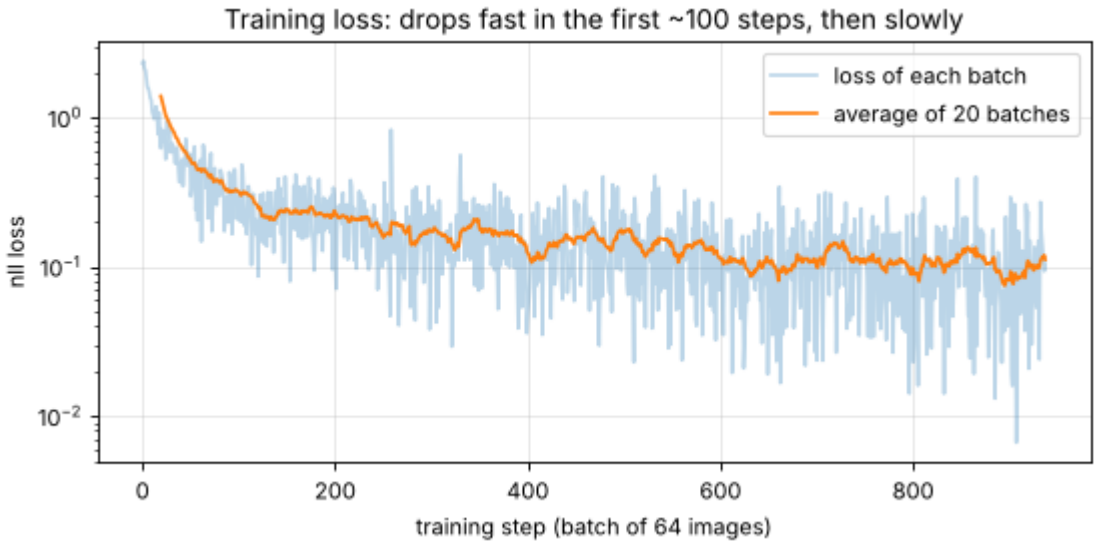
```
Before training (random weights):
Test set: Average loss: 2.3034, Accuracy: 1042/10000 (10.42%)
```

```
EPOCHS = 1                                          # the course used 2: range(1, 3)
t0 = time.perf_counter()
for epoch in range(1, EPOCHS + 1):
    train(model, device, train_loader, optimizer, epoch)
    acc = test(model, device, test_loader)
train_time = time.perf_counter() - t0
print(f"\ntraining + testing took {train_time / 60:.1f} minutes on {device}")
torch.save(model.state_dict(), "data/mnist_cnn.pt")          # keep the trained weights (data/ is
not committed)
```

```
Train Epoch: 1 [0/60000] Loss: 2.309885
Train Epoch: 1 [6400/60000] Loss: 0.277482
Train Epoch: 1 [12800/60000] Loss: 0.111392
Train Epoch: 1 [19200/60000] Loss: 0.171644
Train Epoch: 1 [25600/60000] Loss: 0.048270
Train Epoch: 1 [32000/60000] Loss: 0.119304
Train Epoch: 1 [38400/60000] Loss: 0.111809
Train Epoch: 1 [44800/60000] Loss: 0.023685
Train Epoch: 1 [51200/60000] Loss: 0.023061
Train Epoch: 1 [57600/60000] Loss: 0.081401
Test set: Average loss: 0.0458, Accuracy: 9856/10000 (98.56%)

training + testing took 5.4 minutes on cpu
```

```
smooth = np.convolve(losses, np.ones(20) / 20, mode="valid")
plt.figure(figsize=(8, 3.4))
plt.plot(losses, alpha=.3, label="loss of each batch")
plt.plot(range(19, len(losses)), smooth, label="average of 20 batches")
plt.yscale("log"); plt.xlabel("training step (batch of 64 images)"); plt.ylabel("nll loss")
plt.title("Training loss: drops fast in the first ~100 steps, then slowly"); plt.legend();
plt.grid(alpha=.3); plt.show()
print(f"loss: first batch {losses[0]:.3f} (about -log(1/10) = {np.log(10):.3f}, a random guess) -
> mean of last 100 batches {np.mean(losses[-100:]):.3f}")
```




```
loss: first batch 2.310 (about -log(1/10) = 2.303, a random guess) -> mean of last 100 batches 0.
106
```

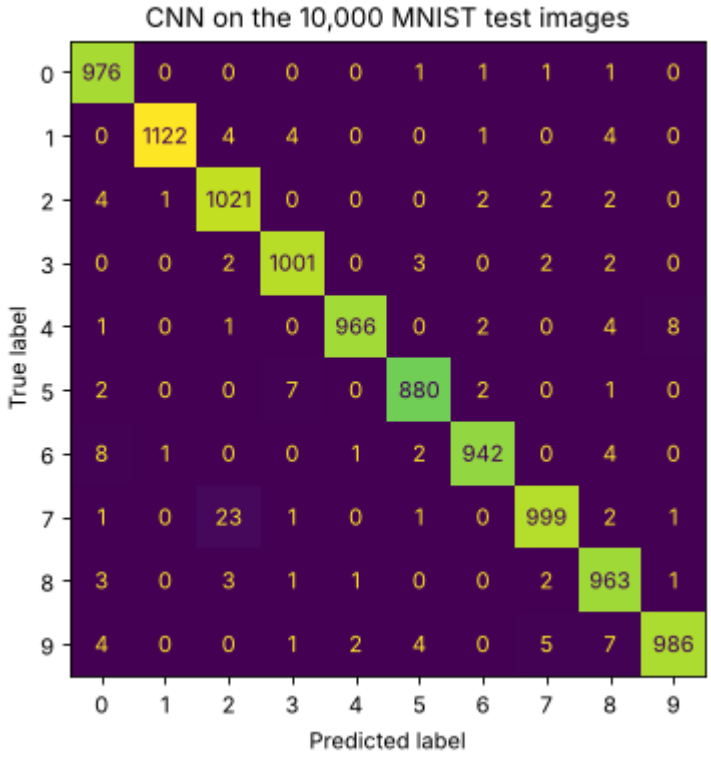
The very first loss is about 2.30 = −log(1/10): a random network gives each digit a probability of about 1/10. The noisy curve is normal: each batch contains different images.

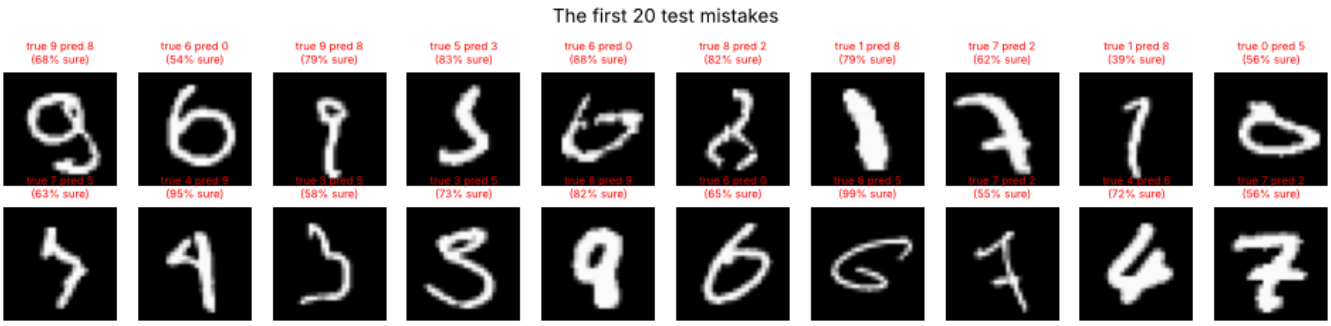
7 · Which digits does it get wrong?

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
model.eval()
preds, targets, probs = [], [], []
with torch.no_grad():
    for data, target in test_loader:
        out = model(data.to(device))
        preds.append(out.argmax(1).cpu()); targets.append(target);
probs.append(out.exp().max(1).values.cpu())
preds, targets, probs = torch.cat(preds).numpy(), torch.cat(targets).numpy(),
torch.cat(probs).numpy()
wrong = np.where(preds != targets)[0]
print(f"{len(wrong)} mistakes out of {len(targets)}")
fig, ax = plt.subplots(figsize=(5.5, 5))
ConfusionMatrixDisplay(confusion_matrix(targets, preds)).plot(ax=ax, colorbar=False)
ax.set_title("CNN on the 10,000 MNIST test images"); plt.tight_layout(); plt.show()

fig, axes = plt.subplots(2, 10, figsize=(12, 3))
for a, i in zip(axes.ravel(), wrong[:20]):
    a.imshow(test_dataset[i][0][0], cmap="gray"); a.axis("off")
    a.set_title(f"true {targets[i]} pred {preds[i]}\n({probs[i]:.0%} sure)", fontsize=7,
color="red")
plt.suptitle("The first 20 test mistakes"); plt.tight_layout(); plt.show()
```

```
144 mistakes out of 10000
```





Many of the mistakes are genuinely ambiguous or badly written digits. Note that the model is sometimes **confidently wrong**: a high softmax probability is not a guarantee.

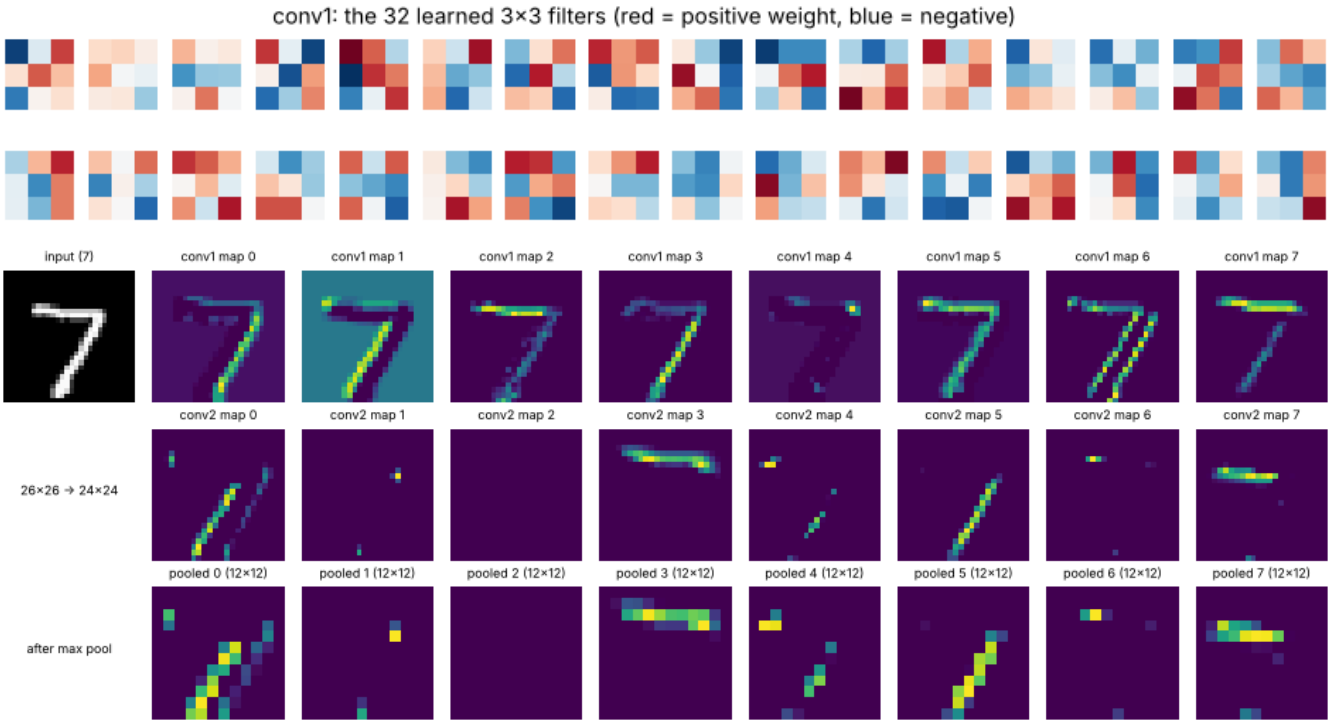
8 · What did the network learn? Filters and feature maps

The 32 first-layer filters are 3×3 grids of learned numbers. Compare them with Day 14's hand-made Sobel kernel: many are edge or stroke detectors in different directions, **learned from data**.

```
W1 = model.conv1.weight.detach().cpu()[:, 0]          # (32, 3, 3)
fig, axes = plt.subplots(2, 16, figsize=(13, 2))
for a, w in zip(axes.ravel(), W1):
    a.imshow(w, cmap="RdBu", vmin=-W1.abs().max(), vmax=W1.abs().max()); a.axis("off")
plt.suptitle("conv1: the 32 learned 3x3 filters (red = positive weight, blue = negative)");
plt.show()

img, lab = test_dataset[0]
with torch.no_grad():
    f1 = F.relu(model.conv1(img[None].to(device)))[0].cpu()
    f2 = F.relu(model.conv2(f1[None].to(device)))[0].cpu()
    p2 = F.max_pool2d(f2[None], 2)[0]
fig, axes = plt.subplots(3, 9, figsize=(12, 4.4))
axes[0, 0].imshow(img[0], cmap="gray"); axes[0, 0].set_title(f"input ({lab})", fontsize=8)
for j in range(8):
    axes[0, j + 1].imshow(f1[j], cmap="viridis"); axes[0, j + 1].set_title(f"conv1 map {j}",
fontsize=8)
    axes[1, j + 1].imshow(f2[j], cmap="viridis"); axes[1, j + 1].set_title(f"conv2 map {j}",
```

```
fontsize=8)
    axes[2, j + 1].imshow(p2[j], cmap="viridis"); axes[2, j + 1].set_title(f"pooled {j} (12x12)",
fontsize=8)
for a in axes.ravel(): a.axis("off")
axes[1, 0].text(0.5, 0.5, "26x26 -> 24x24", ha="center", fontsize=8); axes[2, 0].text(0.5, 0.5,
"after max pool", ha="center", fontsize=8)
plt.tight_layout(); plt.show()
```



Each **feature map** is bright where its filter's pattern appears: some light up on left edges of strokes, some on bottom edges, some on the whole stroke. conv2 maps combine conv1's maps into more complex patterns, and pooling halves their size while keeping the strong responses.

9 · Mini project: a digit classifier for digits *you* draw

The course stops at *"Try feeding custom digit images or drawing digits"* with no code. Here it is. We draw 30 digits in code (pen-stroke style and two fonts), deliberately **dark ink on white paper, small and off-centre**, like a photo of a page. Then we compare four preprocessing pipelines.

What MNIST images look like, so what our input must look like:

- 28×28 pixels, **white digit on black** background,
- the digit is scaled to fit a **20×20 box** and **centred** (by its centre of mass) in the 28×28 frame,
- pixel values scaled to 0–1 and then **normalised** with mean 0.1307 and std 0.3081.

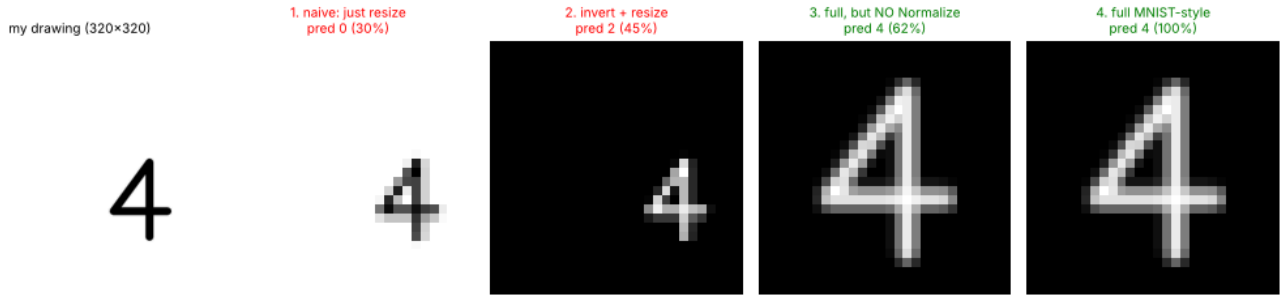
```
def draw_digit(d, style, size=320, rng=None):
    """Dark digit on white paper, smaller than the page and not centred."""
    rng = rng or np.random.default_rng(d)
    tile = np.full((200, 200), 255, np.uint8)
    if style == "pen":
        strokes = {0: [(100, 30), (60, 60), (55, 120), (80, 170), (120, 170), (145, 120), (140,
60), (100, 30)],
                   1: [(80, 60), (110, 30), (110, 175)], 2: [(55, 65), (90, 30), (135, 45), (140,
85), (60, 170), (150, 170)],
                   3: [(55, 45), (130, 35), (100, 95), (145, 130), (115, 172), (55, 165)], 4:
[(120, 175), (120, 30), (50, 125), (155, 125)],
                   5: [(140, 30), (70, 30), (62, 95), (125, 95), (140, 140), (105, 172), (55,
```

```
160)],
                  6: [(130, 30), (75, 80), (60, 140), (95, 172), (138, 145), (118, 105), (65,
125)], 7: [(50, 35), (150, 35), (90, 175)],
                  8: [(100, 100), (60, 60), (100, 28), (140, 60), (100, 100), (55, 140), (100,
175), (145, 140), (100, 100)],
                  9: [(135, 70), (100, 30), (60, 65), (95, 100), (135, 70), (125, 175)]}
        cv2.polylines(tile, [np.array(strokes[d], np.int32)], False, 0, 14, cv2.LINE_AA)
    else:
        font = cv2.FONT_HERSHEY_SIMPLEX if style == "font A" else cv2.FONT_HERSHEY_SCRIPT_SIMPLEX
        cv2.putText(tile, str(d), (40, 165), font, 6, 0, 14, cv2.LINE_AA)
    page = np.full((size, size), 255, np.uint8)
    tile = cv2.resize(tile, (130, 130), interpolation=cv2.INTER_AREA)
    oy, ox = rng.integers(10, size - 140, 2)
    page[oy:oy + 130, ox:ox + 130] = tile
    return page

def to_mnist(img, invert=True, crop_centre=True, normalise=True):
    """Grey image of one digit -> normalised (1, 1, 28, 28) tensor like MNIST. Flags let us
switch steps off."""
    g = 255 - img if invert else img.copy()                         # 1) MNIST is white-on-black
    if crop_centre:
        ys, xs = np.nonzero(g > 50)                                 # 2) crop to the digit
        g = g[ys.min():ys.max() + 1, xs.min():xs.max() + 1]
        h, w = g.shape; s = 20 / max(h, w)                          # 3) fit the longest side into
20 pixels (keep aspect)
        g = cv2.resize(g, (max(1, round(w * s)), max(1, round(h * s))),
interpolation=cv2.INTER_AREA)
        canvas = np.zeros((28, 28), np.uint8)                       # 4) paste into 28x28 ...
        h, w = g.shape; y0, x0 = (28 - h) // 2, (28 - w) // 2
        canvas[y0:y0 + h, x0:x0 + w] = g
        m = cv2.moments(canvas)                                     # ... and shift so the centre
of mass is in the middle
        dx, dy = 14 - m["m10"] / m["m00"], 14 - m["m01"] / m["m00"]
        g = cv2.warpAffine(canvas, np.float32([[1, 0, dx], [0, 1, dy]]), (28, 28))
    else:
        g = cv2.resize(g, (28, 28), interpolation=cv2.INTER_AREA)  # naive: squash the whole page
to 28x28
    t = torch.tensor(g, dtype=torch.float32)[None, None] / 255.0    # 5) 0-255 -> 0-1 (what
ToTensor does)
    if normalise:
        t = (t - 0.1307) / 0.3081                                   # 6) the same Normalize as in
training
    return t

def predict(t):
    model.eval()
    with torch.no_grad():
        p = model(t.to(device)).exp()[0].cpu()
    return int(p.argmax()), float(p.max())

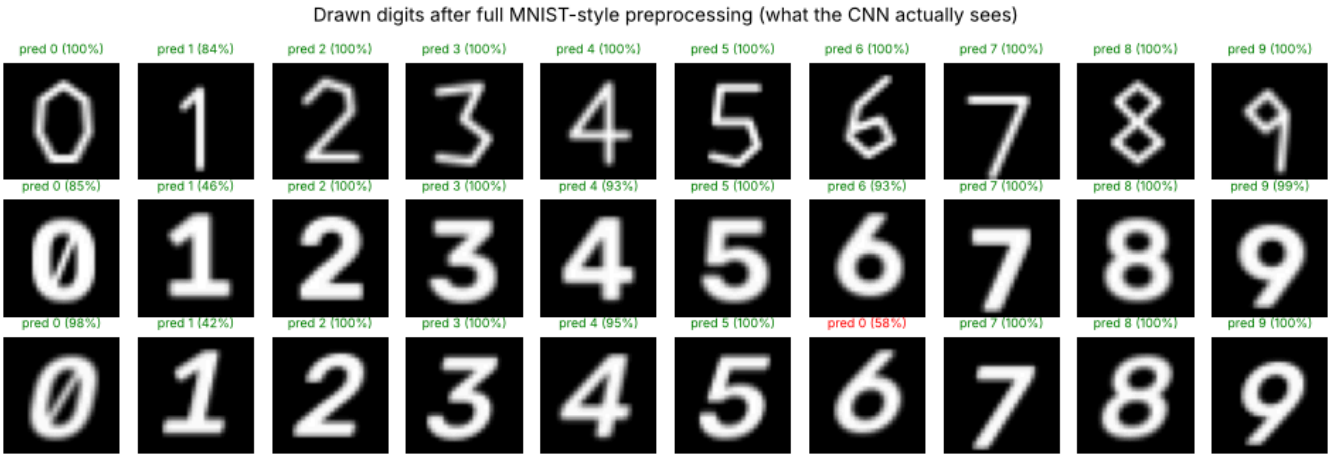
page = draw_digit(4, "pen")
variants = {
    "1. naive: just resize": dict(invert=False, crop_centre=False, normalise=True),
    "2. invert + resize": dict(invert=True, crop_centre=False, normalise=True),
    "3. full, but NO Normalize": dict(invert=True, crop_centre=True, normalise=False),
    "4. full MNIST-style": dict(invert=True, crop_centre=True, normalise=True),
}
fig, axes = plt.subplots(1, 5, figsize=(13, 2.9))
axes[0].imshow(page, cmap="gray"); axes[0].set_title("my drawing (320x320)", fontsize=9)
for a, (name, kw) in zip(axes[1:], variants.items()):
    t = to_mnist(page, **kw); p, c = predict(t)
    a.imshow(t[0, 0], cmap="gray"); a.set_title(f"{name}\npred {p} ({c:.0%})", fontsize=9,
color="green" if p == 4 else "red")
for a in axes: a.axis("off")
plt.tight_layout(); plt.show()
```



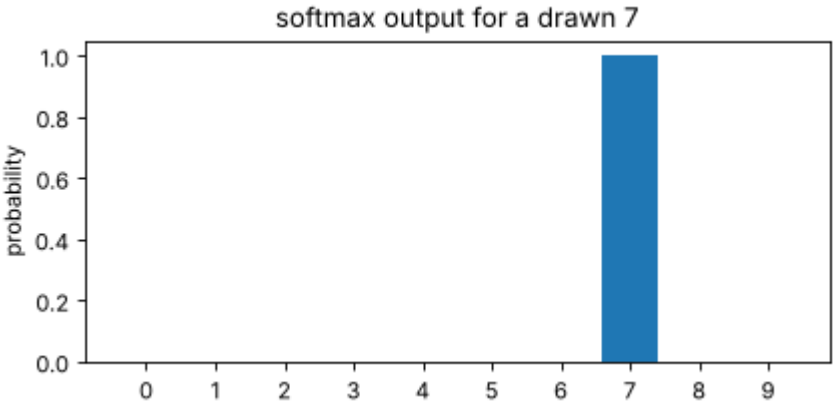

```
styles = ["pen", "font A", "font B"]
pages = {(s, d): draw_digit(d, s, rng=np.random.default_rng(100 * i + d)) for i, s in
enumerate(styles) for d in range(10)}
for name, kw in variants.items():
    res = [predict(to_mnist(pg, **kw)) + (d,) for (s, d), pg in pages.items()]
    ok = sum(p == d for p, c, d in res)
    print(f"{name:28s}: {ok:>2}/30 correct | mean confidence {np.mean([c for p, c, d in
res]):.0%}")

fig, axes = plt.subplots(3, 10, figsize=(12, 4.2))
for r, s in enumerate(styles):
    for d in range(10):
        t = to_mnist(pages[(s, d)]); p, c = predict(t)
        axes[r, d].imshow(t[0, 0], cmap="gray"); axes[r, d].axis("off")
        axes[r, d].set_title(f"pred {p} ({c:.0%})", fontsize=8, color="green" if p == d else
"red")
plt.suptitle("Drawn digits after full MNIST-style preprocessing (what the CNN actually sees)");
plt.tight_layout(); plt.show()
```

```
1. naive: just resize       :  2/30 correct | mean confidence 35%
2. invert + resize          :  3/30 correct | mean confidence 47%
3. full, but NO Normalize   : 29/30 correct | mean confidence 58%
4. full MNIST-style         : 29/30 correct | mean confidence 93%
```




```
# The model's full opinion for one drawing: softmax probabilities of all 10 classes
d = 7
t = to_mnist(pages[("pen", d)])
with torch.no_grad():
    p = model(t.to(device)).exp()[0].cpu().numpy()
plt.figure(figsize=(6, 2.6)); plt.bar(range(10), p); plt.xticks(range(10))
plt.title(f"softmax output for a drawn {d}"); plt.ylabel("probability"); plt.show()
```



What this shows (counts printed above): the network itself is the same in all four rows; only the preprocessing changes. Feeding a black-on-white page squashed to 28×28 gives nonsense (2 of 30 right); inverting alone barely helps (3 of 30) because the digit is still tiny and off-centre. Once the digit is **inverted, cropped, scaled to 20×20 and centred**, 29 of 30 are right. Forgetting `Normalize` did *not* change the number of correct answers in this run (29/30 either way), but the network became much less sure of itself (mean confidence 58% instead of 93%; the "4" above drops from 100% to 62%): its inputs are on a scale it never saw in training, so it is working "out of distribution" and could easily tip over on harder digits. Always apply the same transform as in training. The one remaining mistake (a script-font 6 read as 0) is a drawing style unlike MNIST's handwriting.

To use a photo of your own handwriting: `img = cv2.imread("my_digit.jpg", cv2.IMREAD_GRAYSCALE)`, crop to one digit, threshold with Otsu if the lighting is uneven (`cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)`), then `predict(to_mnist(img))`.

10 · 🔧 What we changed from the course notebook, and why

The course CNN, training loop and testing loop are correct and run as written; we kept them unchanged (apart from recording the loss). What was missing or fragile:

#	Course code	Problem	What we did
1	Mini project: "Try feeding custom digit images or drawing digits"	**No code at all**: the mini project is not implemented.	Drew 30 digits, wrote MNIST-style preprocessing, and measured what goes wrong without each step.
2	`datasets.MNIST('./data', train=False, transform=transform)`	No `download=True` for the test set. It only works because the training-set call happens to download all four MNIST files first.	Added `download=True`.
3	`Normalize((0.1307,), (0.3081,))` unexplained	Magic numbers; students often forget to apply them to new images.	Computed them from the data; showed the effect of forgetting them.
4	`nn.Linear(9216, 128)` unexplained	Breaks as soon as the architecture changes.	Printed the shape after every layer; output-size formula.

#	Course code	Problem	What we did
5	No seed, no loss curve, no confusion matrix, model not saved	Results differ every run; no insight into learning or errors.	`torch.manual_seed(0)`, loss plot, confusion matrix, misclassified examples, `torch.save`.
6	2 epochs	About 5 minutes per epoch on a 2-thread laptop CPU (4.7–5.4 min in our runs, including testing).	`EPOCHS = 1` by default (change to 2 to match the course).
7	Topics in the Key Concepts (feature maps, stride, padding, pooling, flatten)	Not demonstrated in the lab.	Section 3 and section 8.

11 · Results, honestly

All numbers come from the run above.

- Before training the random network scored 10.42% (chance level).
- After **1 epoch** (938 batches, about 5 minutes including testing, on this CPU with 2 threads) the course CNN reached **98.56% on the 10,000 test images** (144 mistakes). The course notebook reports 98.79% after 2 epochs.
- The training loss fell from 2.310 on the first batch (= −log(1/10), a random guess) to an average of 0.106 over the last 100 batches.
- Drawn digits (30): naive resize 2/30, invert + resize 3/30, full preprocessing without Normalize 29/30 (but mean confidence 58%), full MNIST-style preprocessing **29/30** (mean confidence 93%).
- For comparison (Day 11), an RBF SVM on raw MNIST pixels reached 95.9% with 20,000 training images; the CNN beats it after one pass over the data.
- Parameter count: 1,199,882 in total, 98% of them in `fc1`.
- Caveat: one run, one seed; a second epoch, data augmentation or a learning-rate schedule would push test accuracy higher (99%+ is typical for this architecture).

12 · Common errors and how to fix them

Error / symptom	Cause	Fix
`RuntimeError: mat1 and mat2 shapes cannot be multiplied (64x... and 9216x128)`	Architecture or input size changed, so the flattened size is no longer 9216	Print `x.shape` after `flatten`; recompute with (W − K + 2P)/S + 1
`RuntimeError: Expected 4D input` / `Given groups=1, weight of size [32, 1, 3, 3], expected input[1, 28, 28]...`	A single image without the batch dimension	`img.unsqueeze(0)` → (1, 1, 28, 28)
`Expected all tensors to be on the same device`	Model on GPU, data on CPU (or the reverse)	`data.to(device)` and `model.to(device)`

Error / symptom	Cause	Fix
Loss stays at about 2.30	Not learning: forgot `optimizer.step()`, learning rate far too small/large, or labels wrong	Check the 5-line training step
Test accuracy changes between runs of `test()`	Forgot `model.eval()`: dropout still active	Always `model.eval()` before testing / predicting
Out of memory / slow testing	Gradients recorded during testing	Wrap evaluation in `with torch.no_grad():`
Your own digit is always predicted wrong	Not preprocessed like MNIST (colours, size, centring, Normalize)	Use `to_mnist()` from section 9
`nll_loss` gives strange (negative) values	`nll_loss` fed raw scores or probabilities, not log-probabilities	Use `log_softmax` first, or `F.cross_entropy` on raw scores
Download fails	Offline or blocked	Download the four MNIST files by hand into `data/MNIST/raw/`

13 · Try this yourself

1. Set `EPOCHS = 2` (the course setting). How much does test accuracy improve, and how much longer does it take?
2. Add `padding=1` to both conv layers. What must `fc1`'s input size become? (Work it out with the formula first, then check by printing shapes.)
3. Remove the two dropout layers and train again. Compare training loss and test accuracy. Is there more overfitting?
4. Replace `optim.Adam(..., lr=0.001)` with `optim.SGD(model.parameters(), lr=0.01)`. Faster or slower to learn?
5. Make the network smaller: `conv1` with 8 filters and `conv2` with 16. How many parameters? How accurate?
6. Draw a digit in Paint (black on white), save it, and run `predict(to_mnist(cv2.imread(path, 0)))`.
7. Compare with Day 11: the RBF SVM on raw MNIST pixels reached about 96% with 20,000 training images. Why does the CNN do better?

14 · Viva questions (with short model answers)

1. **What is a CNN and why is it suited to images?** A neural network whose main layers are convolutions: small learned filters slid over the image. Local connections and weight sharing exploit spatial structure with far fewer parameters than dense layers.
2. **What is a feature map?** The output of one filter over the whole input: a 2-D map of how strongly that pattern appears at each position.
3. **Give the output-size formula and apply it to conv1.** (W − K + 2P)/S + 1 = (28 − 3 + 0)/1 + 1 = 26.

4. **Why does `fc1` have 9216 inputs?** 28 → conv1 → 26 → conv2 → 24 → 2×2 max pool → 12; 64 channels × 12 × 12 = 9216.
5. **What do stride and padding do?** Stride is the step of the sliding window (larger → smaller output); padding adds a border (usually zeros) so the output does not shrink and edge pixels are covered.
6. **What is max pooling for?** Down-samples feature maps by keeping the maximum of each region: less computation, some tolerance to small shifts, helps against overfitting.
7. **What does flatten do?** Reshapes the (channels, H, W) feature maps into one vector so fully connected layers can use it.
8. **Why ReLU?** It adds non-linearity cheaply (max(0, x)) and does not saturate for positive inputs, so gradients flow well.
9. **What is `Normalize((0.1307,), (0.3081,))`?** Subtract MNIST's mean pixel value and divide by its standard deviation, so inputs have mean 0 and std 1.
10. **Why `log_softmax` with `nll_loss`?** Together they compute the cross-entropy loss: −log of the probability given to the correct class.
11. **What does Adam do?** A gradient-descent optimiser with a per-parameter adaptive learning rate and momentum.
12. **What is dropout and when is it active?** Randomly zeroing a fraction of activations during training to reduce overfitting; active in `model.train()`, disabled in `model.eval()`.
13. **Why `model.eval()` and `torch.no_grad()` during testing?** eval() turns off dropout (and uses fixed batch-norm statistics); no_grad() stops gradient tracking to save memory and time.
14. **What are the five lines of a training step?** zero_grad, forward pass, loss, backward, optimizer.step.
15. **Why did my own drawn digit fail until I preprocessed it?** The model only knows MNIST's format: 28×28, white on black, size-normalised, centred, normalised values. Any mismatch moves the input outside what it learned.

CV Week 2 · Cheat Sheet

Features (Day 8)

Term	One-liner
Feature	distinctive, informative pattern that can be detected, described, compared · good = **unique, repeatable, compact**
Keypoint vs descriptor	keypoint = **where** (x, y, size, angle, response) · descriptor = **what it looks like** (vector)
Harris	corner = window changes in **every** direction · gradients → corner response · `cornerHarris(gray, 2, 3, 0.04)` · simple, good for buildings · **not scale/rotation invariant**, no descriptor
SIFT	scale space → **DoG** extrema (scale inv.) → orientation (rotation inv.) → **128-D** descriptor · very robust · slow · patent expired, free in OpenCV ≥ 4.4
ORB	**FAST** detector + **BRIEF** binary descriptor + orientation · 256 bits · Hamming distance · free, fast, rotation inv. · weaker on big scale changes
Homography	3×3 matrix mapping one image plane to another (rotating camera or flat scene) · needs ≥ 4 pairs
RANSAC	fit to random minimal sets, keep the model with most **inliers** → ignores wrong matches
Panorama	ORB → match → homography (RANSAC) → warpPerspective + paste

Matching (Day 9)

- **Image matching** = finding correspondences between images → logos, object recognition, image retrieval, AR.
- **BFMatcher**: compare all pairs · `NORM_HAMMING` for ORB, `NORM_L2` for SIFT · `crossCheck=True` = mutual best.
- **FLANN**: approximate nearest neighbours via an index (KD-tree for SIFT, **LSH** for ORB) · fast, scales · approximate, needs tuning.
- **Lowe's ratio test**: keep if best < **0.7** × second best → removes ambiguous matches.
- **Logo detector**: detect → FLANN → ratio test → homography (RANSAC, > 10 good) → `perspectiveTransform` the logo corners → `polylines`.
- **SURF**: faster than SIFT, patented.

Tracking (Day 10)

- **Tracking** = following a target across frames · challenges: appearance change, **occlusion**, fast motion.
- **Optical flow** = pixel motion between frames (brightness constancy) · **sparse** (selected keypoints, fast; Lucas–Kanade) vs **dense** (every pixel, detailed; **Farneback**).
- **KLT** = `goodFeaturesToTrack` (Shi–Tomasi corners) + `calcOpticalFlowPyrLK` · efficient · fails on occlusion and large jumps → re-initialise · `winSize` = search window, `maxLevel` = pyramid levels (big motion), `st == 1` = found.

- **Dense-flow visualisation**: `cartToPolar` → hue = **direction**, value = **speed** → HSV→BGR.

ML in CV (Day 11)

	kNN	SVM
Idea	majority vote of k nearest	max-margin hyperplane
Training	none (store data)	optimisation (slow on big data)
Prediction	slow (all distances)	fast (support vectors)
Pros	simple, small data	high-dimensional data, robust to overfitting
Cons	slow on large data, sensitive to irrelevant features and the metric	expensive, kernel/C/gamma tuning
- **Support vectors** = closest points, they define the boundary · **RBF kernel** = curved boundary (kernel trick).		
- **precision** = of predicted X, % really X · **recall** = of real X, % found · **f1** = balance · **support** = count.		
- `load_digits` = 8×8, values 0–16 (not real MNIST 28×28).		

Bag of Visual Words (Day 12)

- descriptors → **k-means** → k **visual words** → per-image **histogram** (fixed length k) → SVM/kNN.
- text analogy: word ↔ visual word · dictionary ↔ cluster centres · document ↔ image · word counts ↔ histogram.
- fixed-length vector from a variable number of features · ignores spatial layout · k too small = vague, too big = sparse.
- uses: textures, materials, scenes · build the vocabulary from **training** images only.

Face detection (Day 13)

- **detection** (where) before **recognition** (who).
- **Haar** = white − black rectangle sums · **integral image** · **AdaBoost** picks features · **cascade** rejects early → fast, lightweight · frontal, controlled · sensitive to lighting, pose, occlusion.
- `detectMultiScale(gray, scaleFactor=1.3, minNeighbors=5)` · lower scaleFactor = more sizes/slower · higher minNeighbors = fewer false positives.
- **HOG** = gradients → per-cell 9-bin direction histograms → block normalisation → SVM · robust to illumination, shape-aware · heavier · pedestrians and faces.
- Haar = **fast, lightweight** · HOG = **robust, heavier**.

Pre-CNN foundations (Day 14)

- dot product = similarity · **convolution = sliding dot product** · gradient = steepest direction ("how steep the hill") · normalise inputs.
- **Supervised** = labelled, predict (classification / regression) · **Unsupervised** = unlabelled, discover (clustering, PCA/t-SNE, association).
- **Neuron**: $z = \Sigma w_i x_i + b \rightarrow f(z)$ · inputs = evidence, weights = importance, bias = prior tendency, activation = decision.
- **ANN**: input → hidden → output · forward pass · **backpropagation** updates the weights.
- FC on images: too many parameters, loses spatial structure → **CNN** (local filters, shared weights).

CNN basics (Day 15)

- **Output size = (W − K + 2P)/S + 1** · stride = step · padding = zero border (keep size, cover edges).
- **Feature map** = one filter's response grid · **pooling** (max/avg) = downsample, translation invariance, less computation and overfitting · **flatten** = maps → vector for FC.
- **ReLU** fast default · **sigmoid** 0–1 probabilities · **tanh** −1…1.
- Lab CNN: 1×28×28 → conv 32×26×26 → conv 64×24×24 → pool 64×12×12 → flatten **9216** → 128 → 10 → log_softmax.
- `log_softmax` + `nll_loss` = **cross-entropy** · Adam lr 1e-3 · `zero_grad → forward → loss → backward → step`.
- `model.train()` dropout on · `model.eval()` + `torch.no_grad()` for testing · Normalize(0.1307, 0.3081) = MNIST mean/std.
- Flow: **Input → Conv → Pool → Flatten → FC → Output**.